



UNIVERSIDAD DE VALLADOLID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE
TELECOMUNICACIÓN

TALLER DE PROYECTOS II: VEHÍCULO CONECTADO

MÁSTER EN INGENIERÍA DE TELECOMUNICACIÓN

Integrantes del equipo “*The GOAT Projects*”:

Ing. Guillermo Chico Delgado

Ing. Adolfo José Dopico González

Ing. Alejandro Martínez de Abajo

Ing. Álvaro Pascual Cabrero

TÍTULO: **Proyecto Coche Conectado**

ASIGNATURA: **Taller de proyectos II**

PROFESORADO: **Dr. Ing. Alfonso Bahillo**
Dr. Ing. Javier Aguiar
Dr. Ing. Juan Carlos Aguado

DEPARTAMENTO: **Teoría de la Señal y Comunicaciones**
Ingeniería Telemática

MIEMBROS DEL EQUIPO:

NOMBRE DEL EQUIPO: **The GOAT Projects**

INTEGRANTE 1: **Ing. Guillermo Chico Delgado**

INTEGRANTE 2: **Ing. Adolfo José Dopico González**

INTEGRANTE 3: **Ing. Alejandro Martínez de Abajo**

INTEGRANTE 4: **Ing. Álvaro Pascual Cabrero**

FECHA: **Junio 2024**

CUATRIMESTRE: **2do Cuatrimestre**

INDICE

1	Introducción.....	18
1.1	Contexto y motivación.....	18
1.2	Conceptos generales	18
1.3	Hipótesis y objetivos.....	20
1.4	Descripción del documento	21
2	Red LTE	23
2.1	Características 4G	23
2.2	Arquitectura de red 4G	24
2.2.1	Red de acceso E-UTRAN.....	24
2.2.2	Red troncal EPC	26
2.2.3	Equipo de usuario	27
2.2.4	IP Multimedia System (IMS)	27
2.2.5	Interfaz radio	28
2.3	Rendimiento de la red 4G	29
3	Inteligencia Artificial.....	30
3.1	Conceptos teóricos y objetivos	30
3.2	Herramientas elegidas.....	31
3.3	Generación de un dataset	33
3.4	Entrenamiento de la red	36
3.5	Implementación de la red.....	37
3.6	Análisis de resultados	40
3.7	Legislación.....	42
4	Despliegue infraestructura	43
4.1	Estudio del entorno	45
4.2	Cálculo de cobertura	50
4.3	Infraestructura Propuesta	61
4.3.1	Despliegue E-UTRAN.....	61
4.3.2	Despliegue EPC.....	76
5	Estudio Económico	83
6	Desarrollo de un prototipo.....	86
6.1	Descripción del prototipo.....	86
6.2	Descripción de los componentes.....	87
6.3	Organización del proyecto	90
6.4	Infraestructura 4G	96
6.5	Inteligencia Artificial	102
6.5.1	Desarrollo de los datasets	103

6.5.2	Entrenamiento del modelo.....	105
6.5.3	Resultados obtenidos	107
6.6	Implementación del modelo.....	115
6.6.1	Implementación en Roboflow	115
6.6.2	Implementación en un servidor	117
6.6.3	Implementación en una Jetson.....	119
6.7	Control del vehículo.....	121
6.8	Análisis de rendimiento	124
6.8.1	Rendimiento de la red.....	124
6.8.2	Rendimiento del PC.....	129
6.9	Incorporación de la Jetson	132
6.10	Legislación.....	135
7	Conclusiones y líneas futuras	136
8	Bibliografía	138
	ANEXO A-SCRIPTS	143
	ANEXO B-TUTORIAL XIRIO	173
	ANEXO C-TUTORIAL ROBOFLOW.....	180
	ANEXO D-IMÁGENES EN GRAN ESCALA.....	184
	ANEXO E-TUTORIAL JETSON	186
	ANEXO F-DIAGRAMAS GANTT.....	191
	ANEXO G-DOCUMENTOS LEGISLACIÓN.....	193

ÍNDICE DE FIGURAS

FIGURA 1. NIVELES DE CONDUCCIÓN AUTÓNOMA.....	19
FIGURA 2. ARQUITECTURA DE UNA RED 4G	24
FIGURA 3. ARQUITECTURA DE UNA RED DE ACCESO E-UTRAN	25
FIGURA 4. ENTIDADES DEL EPC	26
FIGURA 5. DIAGRAMA DE VENN SOBRE LOS CONCEPTOS DE INTELIGENCIA ARTIFICIAL, DEEP LEARNING Y MACHINE LEARNING (KELLEHER, 2019).....	31
FIGURA 6. EJEMPLO DE PREPROCESADO Y SEPARACIÓN DE IMÁGENES DE UN DATASET CON ROBOFLOW.	32
FIGURA 7. EJEMPLO DE PREPROCESADO Y AUGMENTACIÓN DE DATOS DE UN DATASET.	34
FIGURA 8. EJEMPLO DE SEPARACIÓN DE LOS DATOS EN DISTINTOS CONJUNTOS.	34
FIGURA 9. CLASES EMPLEADAS PARA ETIQUETAR CON LA HERRAMIENTA ROBOFLOW.	35
FIGURA 10. EJEMPLO DE IMAGEN ETIQUETADA.	35
FIGURA 11. ENTIDADES POR CLASE ETIQUETADAS EN LAS IMÁGENES DEL DATASET.	36
FIGURA 12. EJEMPLO MAPA DE CALOR DE LA CLASE VELOCIDAD-MAX-30.	36
FIGURA 13. ESQUEMA DE UNA RED NEURONAL CONVOLUCIONAL (CNN) (MARTÍNEZ ZARZUELA, 2021)	37
FIGURA 14. PRODUCTOS DE LA FAMILIA JETSON (NVIDIA, 2024).	40
FIGURA 15. ESQUEMA MATRIZ DE CONFUSIÓN (DE MIGUEL & DURÁN , 2023-2024).	41
FIGURA 16. EJEMPLO DE FUNCIÓN ESCALÓN CON UMBRAL 0.5 EN FORMATO SVG GENERADA CON MATLAB.	41
FIGURA 17. INFORMACIÓN BANDA 7	43
FIGURA 18. FDD VS TDD	43
FIGURA 19. DISTANCIA CUBIERTA POR BANDA	44
FIGURA 20. VARIACIÓN IPC DESDE 2011.....	45
FIGURA 21. EJEMPLO DE DASHBOARD CAMERA DE MIFLEET.....	47
FIGURA 22. PARÁMETROS DE VIDEO & AUDIO DEL DATASHEET.	47
FIGURA 23. GRÁFICAS CON LOS RESULTADOS DEL TRÁFICO.....	49
FIGURA 24. PREDICCIÓN VALORES DE TRÁFICO 2025.	50
FIGURA 25. INFORMACIÓN OFRECIDA EN INFOANTENAS.....	51
FIGURA 26. EMPLAZAMIENTOS DEFINIDOS EN XIRIO.....	51
FIGURA 27. CONFIGURACIÓN BANDA DE FRECUENCIA.	52
FIGURA 28. CONFIGURACIÓN DIAGRAMA DE RADIACIÓN.	53
FIGURA 29. DIAGRAMA DE RADIACIÓN HORIZONTAL.	53
FIGURA 30. DIAGRAMA DE RADIACIÓN VERTICAL.....	53
FIGURA 31. CONFIGURACIÓN DE TERMINAL.....	54
FIGURA 32. ESTUDIO INICIAL XIRIO-ONLINE.	55
FIGURA 33. TRAMO 1: MORISCOS-PAJARES.	56
FIGURA 34. TRAMO 2: PAJARES-PARADA DE RUBIALES.	56
FIGURA 35. TRAMO 3: PARADA DE RUBIALES- CASTRILLO.	56
FIGURA 36. TRAMO 4: CASTRILLO-ALAEJOS.	57
FIGURA 37. TRAMO 5: ALAEJOS-SIETE IGLESIAS.	57
FIGURA 38. TRAMO 6: SIETE IGLESIAS-POLLOS.	57
FIGURA 39. TRAMO 7: POLLOS-TORDESILLAS.....	58
FIGURA 40. ESTUDIO FINAL XIRIO-ONLINE.....	58
FIGURA 41. TRAMOS 1 Y 2 CON LAS NUEVAS ESTACIONES.	59
FIGURA 42. TRAMOS 3 Y 4 CON LAS NUEVAS ESTACIONES.	59
FIGURA 43. TRAMOS 5 Y 6 CON LAS NUEVAS ESTACIONES.	60
FIGURA 44. TRAMO 7 CON LAS NUEVAS ESTACIONES.	60
FIGURA 45. EJEMPLO DE UNA RED E-UTRAN COMPLETA.....	61

FIGURA 46. DIMENSIONES ANTENA KATHREIN.	62
FIGURA 47. ESPECIFICACIONES ANTENA KATHREIN.	62
FIGURA 48. UNIDAD DE RADIO.	63
FIGURA 49. ESPECIFICACIONES RRU.	63
FIGURA 50. UNIDAD BANDA BASE.	64
FIGURA 51. ESPECIFICACIONES BBU.	64
FIGURA 52. ESQUEMA E-UTRAN SIMPLIFICADO.	65
FIGURA 53. FUENTE DE ALIMENTACIÓN.	66
FIGURA 54. ESPECIFICACIONES DE LA FUENTE DE ALIMENTACIÓN.	66
FIGURA 55. IMAGEN DEL ARMARIO RACK 600X600.	67
FIGURA 56. SISTEMA DE AIRE ACONDICIONADO Y SUS DIMENSIONES.	67
FIGURA 57. ESPECIFICACIONES AC.	68
FIGURA 58. GENERADOR DE EMERGENCIA.	68
FIGURA 59. DEPOSITO RESERVA.	69
FIGURA 60. MAPA DE TORMENTAS DE LA PENÍNSULA IBÉRICA.	69
FIGURA 61. PRECIO TERRENO COSTE/m ² EN 2022.	71
FIGURA 62. PLANO DE LA TORRE Y EJEMPLO.	71
FIGURA 63. CIMENTACIÓN TORRE.	72
FIGURA 64. CONSTRUCCIÓN DE LAS ARQUETAS.	73
FIGURA 65. IMAGEN DE LA CASETA CON SUS DIMENSIONES.	73
FIGURA 66. EMPLAZAMIENTOS NUEVA OBRA (PUNTOS EN ROJO).	74
FIGURA 67. EMPLAZAMIENTOS TOTALES DE TODAS LAS ESTACIONES NUEVAS (ROJO) Y EXISTENTES (AZUL).	75
FIGURA 68. CISCO ASR5000-CHSSYS-K9.	76
FIGURA 69. FIBRA OSCURA LYNTIA.	77
FIGURA 70. FIBRA OSCURA REINTEL.	77
FIGURA 71. ESQUEMA RED GPON.	78
FIGURA 72. UBIQUITI UFIBER OLT 8-PORTS GPON.	78
FIGURA 73. DISTRIBUCIÓN DE ESTACIONES AÑADIENDO OLTS EN NARANJA.	79
FIGURA 74. DATASHEET OLT.	79
FIGURA 75. UBIQUITI UFIBER ONT LOCO.	80
FIGURA 76. ESPECIFICACIONES ONT.	80
FIGURA 77. ESQUEMA OLTS.	81
FIGURA 78. PÉRDIDAS EN FIBRA.	81
FIGURA 79. DELL POWEREDGE R750XS.	82
FIGURA 80. NVIDIA QUADRO RTX A6000.	82
FIGURA 81. DIBUJO DEL FUNCIONAMIENTO DEL SISTEMA.	86
FIGURA 82. PC DEL LABORATORIO.	87
FIGURA 83. VEHÍCULO AWS DEEPRACER (IZQUIERDA Y CENTRO) Y CÁMARA EMPLEADA (DERECHA).	87
FIGURA 84. MÓDEM HUAWEI E3372 LTE USB STICK.	88
FIGURA 85. BLADERF MICRO XA9.	88
FIGURA 86. KIT DE DESARROLLADOR JETSON ORIN NANO.	89
FIGURA 87. ROUTER LINKSYS EMPLEADO.	89
FIGURA 88. CABLEADO VARIADO Y BATERÍA DEL MOTOR DEL VEHÍCULO.	89
FIGURA 89. DIAGRAMA GANTT EN MINIATURA.	95
FIGURA 90. IMAGEN DEL PANEL JIRA (IZQUIERDA) Y UN COMENTARIO DE EJEMPLO DE UNA DE LAS TAREAS (DERECHA).	95
FIGURA 91. INFORMACIÓN DEL BLADERF CONECTADO AL PC.	98
FIGURA 92. CONEXIÓN REALIZADA ENTRE SRSENB Y SRSEPC.	99
FIGURA 93. AUTENTICACIÓN EXITOSA DEL UE CON LA RED 4G.	101
FIGURA 94. EJEMPLO DE FOTOGRAMA CAPTURADO CON EL PROPIO VEHÍCULO.	104

FIGURA 95. CLASES CREADAS PARA EL DESARROLLO DEL DATASET CON SEÑALES EN MINIATURA.	104
FIGURA 96. ESTADO DEL DATASET GENERADO CON IMÁGENES DE SEÑALES EN MINIATURA. ...	105
FIGURA 97. EMPLEO DE LA API DE ROBOFLOW PARA PODER EXPORTAR EL DATASET A UN ENTORNO JUPYTER NOTEBOOK.	105
FIGURA 98. MATRICES DE CONFUSIÓN DEL DATASET DE SEÑALES DE TRÁFICO REALES (IZQUIERDA) Y DEL DATASET DE SEÑALES EN MINIATURA (DERECHA).	108
FIGURA 99. GRÁFICAS F1-CONFIANZA DEL DATASET DE SEÑALES DE TRÁFICO REALES (IZQUIERDA) Y DEL DATASET DE SEÑALES EN MINIATURA (DERECHA).	109
FIGURA 100. CURVAS PRECISIÓN-CONFIANZA Y RECALL-CONFIANZA DEL DATASET GENERADO CON SEÑALES EN MINIATURA.	110
FIGURA 101. CURVAS PRECISIÓN-CONFIANZA Y RECALL-CONFIANZA DEL DATASET GENERADO CON SEÑALES REALES.	111
FIGURA 102. GRÁFICAS PRECISIÓN-RECALL DEL DATASET DE SEÑALES DE TRÁFICO REALES (IZQUIERDA) Y DEL DATASET DE SEÑALES EN MINIATURA (DERECHA).	112
FIGURA 103. EJEMPLO DE RESULTADOS MODELO CON PROBLEMAS DE CLSS_LOSS (IZQUIERDA), SIN PROBLEMAS (CENTRO) Y CON PROBLEMAS DE BOX_LOSS (DERECHA) (AUTORÍA PROPIA).	112
FIGURA 104. GRÁFICAS BOX_LOSS Y CLS_LOSS DEL DATASET DE SEÑALES DE TRÁFICO REALES.	113
FIGURA 105. GRÁFICAS BOX_LOSS Y CLS_LOSS DEL DATASET DE SEÑALES DE TRÁFICO EN MINIATURA.	113
FIGURA 106. COLLAGE CON LAS PREDICCIONES SOBRE LAS IMÁGENES (IZQUIERDA) Y LAS ETIQUETAS REALES (DERECHA) DEL DATASET DE SEÑALES EN MINIATURA.	114
FIGURA 107. COLLAGE CON LAS PREDICCIONES SOBRE LAS IMÁGENES (IZQUIERDA) Y LAS ETIQUETAS REALES (DERECHA) DEL DATASET DE SEÑALES REALES.	114
FIGURA 108. OPCIÓN DE ENTRENAR EL MODELO CON ROBOFLOW.	115
FIGURA 109. DESPLEGAR MODELO ENTRENADO CON ROBOFLOW CON SU SERVICIO WEB.	116
FIGURA 110. CARGAR UNA IMAGEN PARA OBTENER SUS RESULTADOS.	116
FIGURA 111. FRAME RESULTADO EXTRAÍDO DE UN VÍDEO SOMETIDO A LA INFERENCIA DEL MODELO (PRIMER MÉTODO).	118
FIGURA 112. FRAME RESULTADO EXTRAÍDO DE UN VÍDEO SOMETIDO A LA INFERENCIA DEL MODELO (SEGUNDO MÉTODO).	118
FIGURA 113. MICROORDENADOR JETSON ORIN NANO DE LA FAMILIA NVIDIA CON SUS PRINCIPALES PERIFÉRICOS (NVIDIA).	119
FIGURA 114. FRAGMENTO DE CÓDIGO DEL SCRIPT PARA DESPLEGAR LA INFERENCIA DE FORMA LOCAL.	120
FIGURA 115. RESULTADO EJECUCIÓN SCRIPT DESPLIEGUE MODELO LOCAL.	121
FIGURA 116. RESULTADO DE TEST DE VELOCIDAD ONLINE EN EL COCHE.	124
FIGURA 117. MEDIDAS DE RENDIMIENTO EN EL PC CON IPERF3 (SERVIDOR).	125
FIGURA 118. RESULTADO DE PING EN EL COCHE SIN NINGUNA CONEXIÓN.	126
FIGURA 119. RESULTADO DE PING EN EL PC.	126
FIGURA 120. MEDIDAS DE RENDIMIENTO EN EL COCHE CON IPERF3 (CLIENTE).	127
FIGURA 121. RESULTADO DE PING EN EL COCHE CON UNA CONEXIÓN TCP.	127
FIGURA 122. RESULTADO DE PING EN EL COCHE CON UNA CONEXIÓN UDP.	128
FIGURA 123. RESULTADO DE PING EN EL COCHE MIENTRAS SE TRANSMITE VÍDEO HACIA EL PC.	128
FIGURA 124. RENDIMIENTO DEL PC SIN NINGUNA CONEXIÓN.	129
FIGURA 125. RENDIMIENTO DEL PC CON UNA CONEXIÓN TCP.	129
FIGURA 126. RENDIMIENTO DEL PC MIENTRAS EL COCHE TRANSMITE VÍDEO AL PC.	129
FIGURA 127. RENDIMIENTO DEL PC MIENTRAS SE EJECUTA EL SCRIPT DE CONTROL MANUAL DEL COCHE.	130

FIGURA 128. EJECUCIÓN Y TIEMPOS DEL SCRIPT DE DETECCIÓN Y CLASIFICACIÓN DE SEÑALES.	130
FIGURA 129. RENDIMIENTO DEL PC MIENTRAS SE EJECUTA EL SCRIPT DE DETECCIÓN Y CLASIFICACIÓN DE SEÑALES.	131
FIGURA 130. EJECUCIÓN Y TIEMPOS DEL SCRIPT DE DETECCIÓN Y CLASIFICACIÓN DE SEÑALES.	131
FIGURA 131. RENDIMIENTO DEL PC MIENTRAS SE EJECUTA EL SCRIPT DE CONTROL DE VEHÍCULO Y EL DE DETECCIÓN Y CLASIFICACIÓN DE SEÑALES.	131
FIGURA 132. MONTAJE DEL ROUTER.	132
FIGURA 133. RESULTADOS CONFIGURACIÓN IP EN EL PC.	133
FIGURA 134. RESULTADOS DE LATENCIA EN LA INFERENCIA COMPARATIVA CON VALORES ANTIGUOS (SUPERIOR) Y NUEVOS (INFERIOR).	134
FIGURA 135. USO DE LA GPU AL EJECUTAR NUESTRO SCRIPT.	135
FIGURA 136. SELECCIÓN DEL ESTUDIO.	173
FIGURA 137. SELECCIÓN DEL SERVICIO.	173
FIGURA 138. DESCRIPCIÓN DEL ESTUDIO.	174
FIGURA 139. CREACIÓN DE UN SECTOR.	174
FIGURA 140. SECTORIZACIÓN DEL TRANSMISOR.	175
FIGURA 141. CONFIGURACIÓN SECTOR.	175
FIGURA 142. CONFIGURACIÓN DE FRECUENCIAS DE UN SECTOR.	176
FIGURA 143. PARÁMETROS TERMINAL.	176
FIGURA 144. MÉTODO DE CÁLCULO.	177
FIGURA 145. OTRAS PROPIEDADES DE CÁLCULO.	177
FIGURA 146. DEFINICIÓN RANGO COLORES.	178
FIGURA 147. RESULTADO DEL ESTUDIO SIMPLE.	178
FIGURA 148. SELECCIÓN TIPO ESTUDIO.	178
FIGURA 149. IMPORTACIÓN DE COBERTURA.	179
FIGURA 150. ESTUDIO MULTITRANSMISOR.	179
FIGURA 151. CREACIÓN DE UN NUEVO PROYECTO CON ROBOFLOW.	180
FIGURA 152. CARACTERÍSTICAS DEL PROYECTO.	180
FIGURA 153. CREACIÓN DE NUEVAS CLASES.	181
FIGURA 154. AÑADIMOS LAS IMÁGENES AL DATASET.	181
FIGURA 155. ELEGIMOS LA OPCIÓN DE ANOTAR IMÁGENES.	181
FIGURA 156. ENTORNO DE ETIQUETADO CON DESCRIPCIÓN DE SUS COMPONENTES.	182
FIGURA 157. AÑADIMOS LAS IMÁGENES AL DATASET.	182
FIGURA 158. COMPROBACIÓN DEL DATASET.	182
FIGURA 159. ELECCIÓN DE LOS PORCENTAJES DE CADA GRUPO DE IMÁGENES.	183
FIGURA 160. HEALTH CHECK DEL DATASET.	183
FIGURA 161. OPCIONES DE PREPROCESADO Y AUGMENTACIÓN.	183
FIGURA 162. COLLAGE CON LAS PREDICCIONES SOBRE LAS IMÁGENES (IZQUIERDA) Y LAS ETIQUETAS REALES (DERECHA) DEL DATASET DE SEÑALES EN MINIATURA A GRAN ESCALA Y MAYOR RESOLUCIÓN.	184
FIGURA 163. COLLAGE CON LAS PREDICCIONES SOBRE LAS IMÁGENES (IZQUIERDA) Y LAS ETIQUETAS REALES (DERECHA) DEL DATASET DE SEÑALES REALES A GRAN ESCALA Y MAYOR RESOLUCIÓN.	185
FIGURA 164. FRAGMENTO DE TABLA DE COMPATIBILIDADES DE LOS DISTINTOS SOFTWARES SDK DE NVIDIA (NVIDIA).	186
FIGURA 165. PASO 1 SDK MANAGER NVIDIA.	187
FIGURA 166. PASO 2 SDK MANAGER NVIDIA.	187
FIGURA 167. PASO 3 PRIMERA CONFIGURACIÓN SDK MANAGER NVIDIA.	188
FIGURA 168. VISTAS SUPERIOR Y LATERAL DE LA COLOCACIÓN DEL JUMPER EN LA JETSON. ...	188
FIGURA 169. PASO 3 FLASHEO DE LA MICROSD CON SDK MANAGER NVIDIA.	189
FIGURA 170. PASO 3 COMPLEMENTOS DE SDK MANAGER DE NVIDIA.	189

FIGURA 171. PASO 4 RESUMEN DE SDK MANAGER NVIDIA. 190

FIGURA 172. ESCRITORIO DE LA JETSON ORIN NANO..... 190

FIGURA 173. DIAGRAMA GANTT CON LA PLANIFICACIÓN INICIAL. 191

FIGURA 174. DIAGRAMA GANTT CON LA PLANIFICACIÓN FINAL. 192

ÍNDICE DE TABLAS

TABLA 1. CÁLCULO DE DISTANCIAS Y TIEMPO ENTRE SEÑALES DE TRÁFICO DE UNA AUTOVÍA....	46
TABLA 2. INTENSIDAD MEDIA DIARIA (IMD) DE VEHÍCULOS LIGEROS EN LA AUTOVÍA A-62.	49
TABLA 3. INTENSIDAD MEDIA DIARIA (IMD) DE VEHÍCULOS PESADOS EN LA AUTOVÍA A-62.	49
TABLA 4. TABLA CON PRESUPUESTO DE INSTALACIÓN Y MATERIALES POR E-UTRAN COMPLETA.	83
TABLA 5. TABLA CON PRESUPUESTO DE INSTALACIÓN Y MATERIALES POR E-UTRAN ALQUILER. .	84
TABLA 6. COSTE DE DESPLIEGUE E-UTRAN TOTAL.	84
TABLA 7. DESPLIEGUE TOTAL AÑADIENDO EL ESPECTRO.....	84
TABLA 8. CONSUMO ESTIMADO E-UTRAN.	85
TABLA 9. CONSUMO ESTIMADO EPC.....	85
TABLA 10. COSTE DE OPERACIÓN DESGLOSADO.	85
TABLA 11. COSTE DE OPERACIÓN Y DESPLIEGUE COMPLETO.....	85
TABLA 12. IPS ASOCIADAS PARA EL MONTAJE.	132

ÍNDICE DE SIGLAS Y ACRÓNIMOS

3GPP	<i>3rd Generation Partnership Project</i>
ADAS	<i>Sistemas de Asistencia al Conductor</i>
AEPD	<i>Agencia Española de Protección de Datos</i>
AESA	<i>Agencia Estatal de Seguridad Aérea</i>
API	<i>Application Programming Interface</i>
APN	<i>Access Point Name</i>
BLER	<i>Block Error Rate</i>
CNN	<i>Convolutional Neural Network</i>
CPU	<i>Central Processing Unit</i>
CSV	<i>Comma Separated Values</i>
CUDA	<i>Computer Unified Device Architecture</i>
DAFO	<i>Debilidades, Amenazas, Fortalezas y Oportunidades</i>
EARFCN	<i>E-UTRA Absolute Radio Frequency Channel Number</i>
ENB	<i>Evolved Node B</i>
EPC	<i>Evolved Packet Core</i>
E-UTRAN	<i>Evolved UTRAN</i>
FDD	<i>Frecuency-division duplex</i>
GPU	<i>Graphics Processing Unit</i>
HDMI	<i>High-Definition Multimedia Interface</i>
HSS	<i>Home Subscriber Server</i>
IA	<i>Inteligencia Artificial</i>
IaaS	<i>Infrastructure as a Service</i>
ICMP	<i>Internet Control Message Protocol</i>
IMD	<i>Intensidad Media Diaria de Tráfico</i>
IMS	<i>IP Multimedia Subsystem</i>
IMSI	<i>International Mobile Subscriber Identity</i>
LED	<i>Light Emitting Diode</i>
LiDAR	<i>Ligth Detection and Ranging</i>
LTE	<i>Long Term Evolution</i>
MB	<i>MegaBytes</i>
Mbps	<i>Megabits por Segundo</i>
MCC	<i>Mobile Country Code</i>
ME	<i>Mobile Equipment</i>
MIMO	<i>multiple-input multiple-output</i>
MME	<i>Mobility Management Entity</i>
MNC	<i>Mobile Network Code</i>
MQTT	<i>Message Queuing Telemetry Transport</i>
MT	<i>Mobile Terminal</i>
NFS	<i>Network File System</i>
NLP	<i>Natural Language Processing</i>
OACI	<i>Organización de Aviación Civil Internacional</i>
OFDMA	<i>Orthogonal Frequency Division Multiple Access</i>
OLT	<i>Optical Line Terminal</i>
ONT	<i>Optical Network Terminal</i>
PC	<i>Personal Computer</i>
PCRF	<i>Policy Control and Charing Rules Function</i>
P-GW	<i>Packet Data Network Gateway</i>
PRB	<i>Physical Resource Block</i>

QAM	<i>Quadrature Amplitude Modulation</i>
RGPD	<i>Reglamento General de Protección de Datos</i>
RNN	<i>Recurrent Neural Network</i>
RSRP	<i>Reference Signal Received Power</i>
SC-FDMA	<i>Single-Carrier Frequency Division Multiple Access</i>
SDR	<i>Software Defined Radio</i>
S-GW	<i>Serving Gateway</i>
SIM	<i>Subscribe Identity Module</i>
SINR	<i>Signal to Interference plus Noise Ratio</i>
SIP	<i>Session Initiation Protocol</i>
SO	<i>Sistema Operativo</i>
srsRAN	<i>Software Radio Systems Radio Access Network</i>
TDD	<i>Time-division duplex</i>
TE	<i>Terminal Equipment</i>
UE	<i>User Equipment</i>
URI	<i>Uniform Resource Identifier</i>
USB	<i>Universal Serial Bus</i>
USIM	<i>Universal Subscriber Identity Module</i>
UTRAN	<i>UMTS Terrestrial Radio Access Networks</i>
UVa	<i>Universidad de Valladolid</i>
VGA	<i>Video Graphics Array</i>
YOLO	<i>You Online Look Once</i>

Agradecimientos

Nos gustaría agradecer al coordinador del máster Ignacio de Miguel profesor de la UVa por el esfuerzo y la dedicación con la que ha coordinado y organizado este Máster de Ingeniería de las Telecomunicaciones durante el curso 2023-2024.

Agradecer también, a todo el profesorado que nos ha guiado y ayudado a lo largo del desarrollo de este proyecto, en especial a los profesores Dr. Ing. Javier Aguiar, Dr. Ing. Alfonso Bahillo y el Dr. Ing. Juan Carlos Aguado por sus continuas aportaciones y consejos y por todo el trabajo realizado para que el proyecto se pudiera realizar de la forma más exitosa.

Por último, agradecer a todos los miembros del equipo The GOAT Projects por el increíble desempeño realizado a lo largo del curso ya que sin ellos hubiese sido imposible realizar este proyecto, a todos vosotros, gracias.

Resumen

A lo largo de estos últimos años, la tecnología ha evolucionado de manera exponencial, expandiéndose a nuevos ámbitos y disciplinas. Todo ello, con el objetivo de maximizar la eficiencia tanto en términos de tiempo como de recursos. Los avances en Inteligencia Artificial, electrónica, computación y telecomunicaciones han permitido el desarrollo de nuevos sistemas y estrategias que permiten mejorar distintos ámbitos de la sociedad, como el entorno industrial, la vida cotidiana, el ocio, el transporte o incluso en la medicina.

Uno de los factores clave de este proceso de digitalización de la sociedad, es el uso cada vez más frecuente de sensores, o cámaras para la recogida y procesamiento de datos en entornos donde antiguamente no era viable su despliegue. En esta nueva era, los datos se han convertido en un recurso fundamental, permitiendo desde la creación de modelos de regresión hasta la actuación en tiempo real de manera remota sobre sistemas sensorizados.

Con estas premisas, se ha desarrollado este proyecto focalizado en la aplicación en conjunto de elementos como técnicas de Inteligencia Artificial, redes de telecomunicaciones móviles 4G y vehículos conectados. El objetivo, consiste en desarrollar un sistema que integre todos estos elementos y permita crear una red de tráfico más segura, permitiendo al vehículo actuar en ciertas ocasiones y así reducir la cantidad de accidentes producidos en carretera.

Palabras clave:

Inteligencia Artificial, vehículo conectado, red 4G y detección de objetos.

Abstract

Over the last few years, technology has evolved exponentially, expanding into new fields and disciplines, with the aim of maximizing efficiency in terms of both time and resources. Advances in Artificial Intelligence, electronics, computing and telecommunications have enabled the development of new systems and strategies to improve different areas of society, such as the industrial environment, daily life, leisure, transportation and even medicine.

One of the key factors in this process of digitization of society is the increasingly frequent use of sensors or cameras to collect and process data in environments where their deployment was not feasible in the past. In this new era, data has become a fundamental resource, allowing from the creation of regression models to real-time remote action on sensorized systems.

With these premises, this project has been developed focused on the joint application of elements such as Artificial Intelligence techniques, 4G mobile telecommunications networks and connected vehicles. The objective is to develop a system that integrates all these elements and allows to create a safer traffic network, allowing the vehicle to act on certain occasions and thus reduce the number of accidents on the road.

Keywords:

Artificial Intelligence, connected vehicle, 4G network, and object detection.

1 Introducción

En el presente informe, desarrollado por el equipo de ingenieros de telecomunicaciones *The_GOAT_Projects*, se pretende recoger todos los puntos referentes al diseño de un sistema basado en Inteligencia Artificial y redes de telecomunicaciones 4G. La idea del sistema a desarrollar consiste en conseguir implementar una serie de medidas de seguridad adicionales en vehículos ligeros, con el fin de reducir la cantidad de accidentes en las carreteras. Para ello, se han evaluado y diseñado todos los preparativos simulando el desarrollo del sistema real, además de un prototipado a baja escala del mismo.

1.1 Contexto y motivación

En 2022, se matricularon casi 1.300.000 nuevos vehículos en España, elevando el total de vehículos matriculados a 33.404.417 (DGT, Anuario Estadístico General DGT, 2022). Este dato subraya la importancia crucial de contar con un vehículo para desplazarse de manera rápida y segura en la sociedad actual. Sin embargo, la otra cara de la moneda la representan los accidentes de tráfico, que continúan siendo un problema significativo y una de las principales causas de mortalidad en España, ya que en tan solo 2023, 1145 personas fallecieron en accidentes de tráfico y 4.495 resultaron heridas de gravedad durante trayectos en coche (DGT, cifras tráfico, 2023).

Es por ello, que los fabricantes de vehículos cada vez tienen un mayor compromiso con el desarrollo de Sistemas de Asistencia al Conductor (ADAS, por sus siglas en inglés) que son tecnologías avanzadas, diseñadas para mejorar la seguridad y la experiencia de conducción mediante la automatización, adaptación y mejora de los sistemas de los vehículos. Estos sistemas, engloban desde mecanismos de frenada de emergencia hasta modelos que hacen uso de últimas tecnologías como la Inteligencia Artificial, permitiendo el desarrollo de sistemas de control de vehículo capaces de evitar o reducir posibles accidentes en carretera.

Por lo tanto, el objetivo de este proyecto es diseñar un sistema aplicable a diferentes vehículos, capaz de reconocer y clasificar las señales de tráfico presentes en la carretera, utilizando un algoritmo basado en *Deep Learning* y, en concordancia con el conductor del vehículo, actuar sobre el control del coche. El objetivo de esto es crear un vehículo conectado y reducir la cantidad de accidentes de tráfico.

De manera paralela al sistema de detección de señales, diseñaremos un sistema de comunicaciones móviles que sea capaz de comunicarse con nuestro vehículo a través de una SIM y una red de telecomunicaciones 4G. Con el fin de comprobar la viabilidad de nuestro sistema, lo aplicaremos a un supuesto caso real a lo largo de la autovía A-62 entre Salamanca y Tordesillas.

1.2 Conceptos generales

Un vehículo autónomo, es capaz de operar sin intervención humana, utilizando una combinación de sensores, cámaras, radares, y algoritmos de Inteligencia Artificial para navegar y tomar decisiones en tiempo real en función del entorno y situación percibida (ACM, 2022). Sin embargo, cualquier vehículo no será considerado realmente autónomo y esto dependerá del control de vehículo y atención que se requiera del conductor. A continuación, se va a describir cuales son los niveles de conducción autónoma (Higuera, 2024):

Nivel 0: Este es el nivel más básico, el que tienen todos los vehículos por defecto, en este nivel no existe ninguna ayuda a la conducción por parte del vehículo y todo depende completamente del conductor, por lo que es el menos seguro.

Nivel 1: En este nivel el vehículo ofrece asistencia al conductor para reducir la carga de tareas. Ejemplos de esto son, el control de velocidad y el asistente de mantenimiento de carril. Estos sistemas, gestionan el movimiento lateral o longitudinal, pero nunca ambos simultáneamente y lo hacen de forma independiente. Este nivel también incluye las asistencias de aparcamiento en las que el conductor maneja solo los pedales, además de sistemas de aviso de colisión o detección de peatones con frenado automático. Aunque estas ayudas facilitan la conducción, el conductor debe estar siempre atento.

Nivel 2: Aquí, el coche ya puede controlar simultáneamente los movimientos laterales y longitudinales. Incluyendo asistencia al aparcamiento completamente automática y sistemas de ayuda a la conducción que reducen el estrés en atascos, permitiendo acelerar, frenar o mantener la distancia con el vehículo al que sigue, además de mantenerse en el carril por sí mismo. Sin embargo, el papel del conductor sigue siendo un requisito. Aquí se encuentra la aplicación propuesta en este informe.

Nivel 3: Este supone un punto de inflexión, ya que el coche puede encargarse de casi toda la conducción. El vehículo percibe su entorno y actúa en consecuencia tomando decisiones propias, permitiendo al conductor soltar el volante y los pedales. La supervisión humana en caso de que el software falle o cometa errores sigue siendo necesaria.

Nivel 4: A partir de este nivel, el vehículo es totalmente capaz de conducir por sí solo sin intervención humana alguna. Incluso puede decidir si sobreponerse al control humano en situaciones concretas.

Nivel 5: Define la automatización total, donde los coches no tendrían ni volante ni pedales. Estos vehículos se controlan de forma completamente autónoma, sin necesidad de intervención o supervisión humana.

SOCIETY OF AUTOMOTIVE ENGINEERS (SAE) AUTOMATION LEVELS

Full Automation

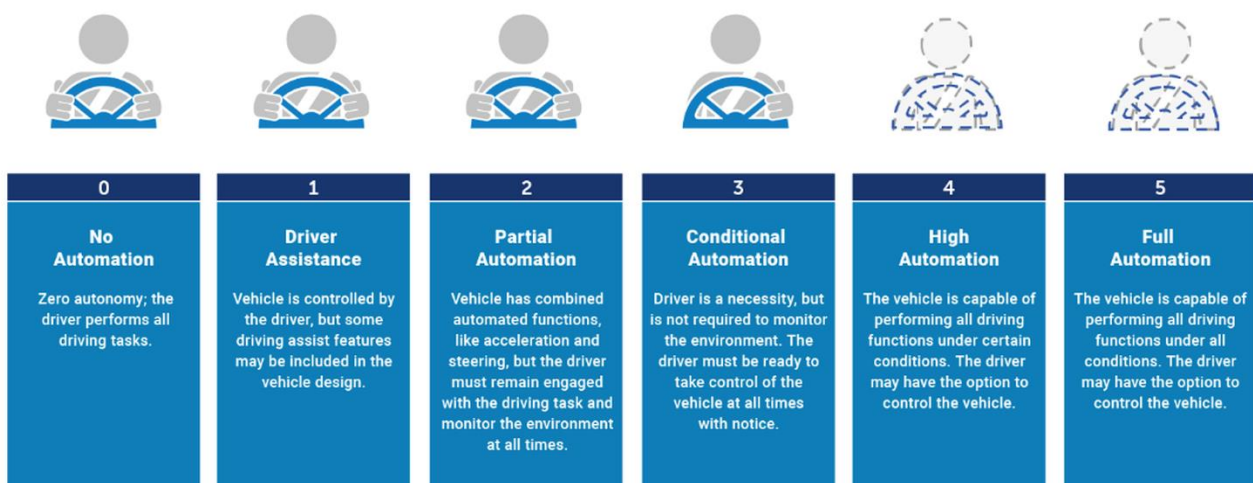


Figura 1. Niveles de conducción autónoma.

No obstante, existe un inconveniente para esta tecnología, ya que hay que definir sobre quien recaería la responsabilidad en caso de accidente, lo que presenta una situación comprometida para la legislación vigente. La Unión Europea ha intentado anticiparse a los conflictos que presentará la llegada de esta tecnología cuando cohabiten vehículos tradicionales con aquellos de mayor capacidad autónoma, y adoptó una estrategia para la movilización automatizada en 2018 (UE, 2023).

Los vehículos autónomos serán una realidad indiscutible en las carreteras del futuro próximo. Sus beneficios parecen innegables: uso eficiente del combustible, mejora en la fluidez del tráfico, reducción de la congestión y acceso a la movilidad para personas que actualmente no pueden conducir. Sin embargo, el mayor beneficio supuesto de esta tecnología será su capacidad para reducir significativamente las muertes causadas por errores humanos en la carretera. (IOP, 2018)

1.3 Hipótesis y objetivos

La hipótesis inicial del proyecto consiste en desarrollar un sistema basado en una red LTE 4G y un modelo de Inteligencia Artificial, para poder conectar un vehículo durante un tramo de autovía entre Salamanca y Tordesillas (A-62). Para lograr este objetivo, se listan a continuación una serie de objetivos más concretos a cumplir, desde la perspectiva de tener que desplegar el modelo diseñado en este informe:

- Realización de listados con todas las tareas a realizar, los tiempos a cumplir, la asignación a cada miembro de una serie de tareas y sus correspondientes descripciones.
- Investigación de los distintos softwares a emplear para desplegar la red LTE y el modelo de Inteligencia Artificial.
- Configurar correctamente tanto el EPC como los eNodeB de la red con sus respectivos parámetros.
- Configurar y dar de alta la SIM dentro de la red a desplegar.
- Recopilación de imágenes, filtrado y creación del dataset (incluyendo el preprocesado y la augmentación).
- Creación y entrenamiento del modelo encargado de detectar las señales en las imágenes con el dataset previamente creado.
- Despliegue del modelo en distintas plataformas y posterior valoración de la viabilidad de cada una de ellas (*edge computing*, *cloud computing*...).
- Estudio y valoración económica del despliegue necesario para llevar a cabo del proyecto en el tramo de autovía asignado.
- Estudio del tráfico y densidad media de usuarios para estimar la capacidad del sistema.
- Implementación de la inferencia del modelo en la red 4G LTE.
- Medición del rendimiento tanto de la red como del modelo.
- Corrección de imprevistos y otros problemas.
- Elaboración de documentación pertinente y presupuestos.

1.4 Descripción del documento

Durante el transcurso del presente documento, vamos a tratar una serie de puntos distribuidos por apartados en los que se analizarán diferentes aspectos y puntos del proyecto desarrollado. El documento consta de 9 apartados diferentes, en los que veremos una organización inicial del proyecto, un breve análisis de las redes móviles de telecomunicaciones, explicaciones de las posibles soluciones empleando Inteligencia Artificial, el diseño del despliegue necesario con su correspondiente estudio económico, los pasos realizados para el desarrollo de un prototipo, una serie de conclusiones, líneas futuras, la correspondiente bibliografía y finalmente los Anexos. A continuación, veremos en más detalle cada uno de estos puntos:

- **Capítulo 1. Introducción:** En este primer capítulo, realizaremos una breve introducción al caso de estudio con su correspondiente contextualización social, seguida de una definición de los conceptos más importantes a tener en cuenta. Más adelante, extraeremos una hipótesis inicial con sus respectivos pasos y puntos a realizar para tratar de demostrarla y finalmente una descripción en profundidad de las distintas secciones del documento.
- **Capítulo 2. Red LTE:** Iniciaremos este capítulo, comentando brevemente alguno de los aspectos teóricos y características más relevantes de las redes de telecomunicaciones LTE. Continuaremos con un análisis en mayor profundidad de los distintos componentes de estas redes, considerando aquellos que pueden ser de utilidad para desarrollar nuestro sistema, y contemplaremos distintos sistemas de medida de rendimiento de las prestaciones de la red que pueden ser empleados en un futuro sistema.
- **Capítulo 3. Inteligencia Artificial:** En este tercer apartado, trataremos todos los conceptos referentes a la parte del sistema de detección de señales, para ello iniciaremos con una explicación muy concisa de términos importantes a conocer en el campo de la Inteligencia Artificial. Continuaremos con una explicación de todos los procesos necesarios para la creación de un dataset adecuado para nuestro caso de estudio, y complementaremos dicha explicación con la creación de un prototipo de dataset aplicable al caso real. Finalmente, trataremos de sintetizar las distintas métricas que pueden ser empleadas para medir el rendimiento de este tipo de sistemas.
- **Capítulo 4. Despliegue infraestructura:** En el cuarto apartado, realizaremos un estudio en profundidad de todos los elementos necesarios para poder realizar el despliegue requerido de nuestro sistema. Para ello, se recogen todos los puntos referentes a los materiales necesarios, la localización de las antenas, el alquiler de equipos o los permisos necesarios entre otros.
- **Capítulo 5. Estudio económico:** Una vez vistos todos los puntos necesarios para el desarrollo del proyecto, en este apartado recogeremos todos los costes y presupuestos de diseño, desarrollo y despliegue de cada uno de los apartados del proyecto.
- **Capítulo 6. Desarrollo de un prototipo:** A modo de refuerzo del estudio realizado, en este capítulo se recogen todos los pasos que se han seguido para el desarrollo de un prototipo a pequeña escala del caso real en un entorno más reducido y controlado. Estos pasos recogen el desarrollo de un dataset adaptado al entorno reducido, el diseño e implementación de dicho modelo en distintos dispositivos y la creación de una red de telecomunicaciones móviles para poder controlar un vehículo en miniatura.

- **Capítulo 7. Conclusiones y líneas futuras:** A modo de broche final, en este capítulo se recogen las conclusiones realizadas a lo largo del desarrollo del proyecto y una serie de posibles líneas futuras o caminos a valorar de cara a mejorar el sistema.
- **Bibliografía:** De manera adicional, se aporta la bibliografía consultada durante el transcurso del caso de estudio y redacción de este informe, en formato APA.
- **Anexos:** Finalmente, se recogen todos los puntos adicionales al proyecto, enfocados como tutoriales en caso de que se requiera replicar los procesos realizados para desarrollar nuestro sistema. Además, algunas de las imágenes de mayor interés se han añadido con una mayor resolución y los códigos empleados de los cuales se han extraído fragmentos a lo largo de todo el informe.

El proyecto será llevado a cabo por un equipo de ingenieros de la Universidad de Valladolid bajo el nombre de *The_GOAT_Projects*, encargados del diseño y desarrollo de cada uno de los puntos del informe y la redacción de cada uno de ellos en este informe de forma conjunta.

2 Red LTE

El término LTE (*Long Term Evolution*) hace referencia al conjunto de especificaciones del 3GPP que definen los pasos para la evolución de las redes de acceso UTRAN (*Universal Terrestrial Radio Access Network*). La finalidad del LTE es otorgar acceso móvil de banda ancha que permita altas velocidades para los usuarios y reduzca el coste por bit transmitido.

El 3GPP divide la red del sistema LTE en dos componentes. El primero es la red de acceso radio E-UTRAN y el segundo es la red troncal EPC. Las redes LTE poseen una arquitectura basada en la conmutación de paquetes. La red UTRAN emplea OFDMA en el enlace descendente y SC-FDMA en el enlace ascendente para la transmisión en la capa física (3GPP, 2006).

2.1 Características 4G

La tecnología LTE representa la cuarta generación de comunicaciones móviles que ofrece una serie de características que la han hecho destacar. En este apartado se van a explorar las principales características de LTE 4G (Pertíñez López, 2015; Dahlman, Parkvall, & Sköld, 2011):

- **Alta velocidad de datos:** LTE proporciona velocidades de datos rápidas, lo que permite descargas y cargas de contenido multimedia de alta calidad de manera rápida y fluida. Las velocidades típicas de 4G suelen oscilar entre 40 y 300 Mbps.
- **Baja latencia:** LTE ofrece una latencia significativamente reducida (entre 30 y 50 milisegundos) en comparación con tecnologías anteriores, lo que se traduce en tiempos de respuesta más rápidos para aplicaciones sensibles al retardo, como la transmisión de vídeo en tiempo real, los juegos en línea y las comunicaciones en tiempo real.
- **Mayor eficiencia espectral:** LTE utiliza técnicas avanzadas de modulación, acceso múltiple y gestión de recursos para optimizar el uso del espectro radioeléctrico, permitiendo una mayor capacidad y eficiencia en la transmisión de datos.
- **Compatibilidad con IP:** LTE está diseñado desde su concepción para ser compatible con el protocolo de Internet (IP), lo que facilita la integración con servicios y aplicaciones basados en IP, incluyendo voz sobre IP (VoIP), videoconferencia, mensajería multimedia y servicios en la nube.
- **Soporte para movilidad:** LTE ofrece un rendimiento robusto y confiable incluso cuando los usuarios se desplazan a altas velocidades, lo que lo hace ideal para aplicaciones y servicios en entornos de movilidad, como vehículos en movimiento rápido y dispositivos portátiles.
- **Mejora de la cobertura y penetración en interiores:** LTE proporciona una mejor cobertura y penetración en interiores en comparación con tecnologías anteriores, lo que garantiza una experiencia de usuario consistente y de alta calidad en diversos entornos y ubicaciones.

Estas son solo algunas de las características más importante, existen muchas más, pero debemos resumir para captar los puntos más interesantes de cara al proyecto.

2.2 Arquitectura de red 4G

La arquitectura de un sistema LTE se puede dividir en tres componentes principales (Agusti-Comes, y otros):

- **Equipo de usuario:** Dispositivo que permite al usuario conectarse a la red. Este equipo dispondrá normalmente de una tarjeta SIM que tendrá la información para identificarse y conectarse a la red.
- **Red de acceso:** componente encargado de establecer la comunicación a los equipos de usuario mediante transmisión radio. Se encarga de gestionar los recursos radio y está compuesta por estaciones base.
- **Red troncal:** parte del sistema cuya función es controlar el acceso a la red móvil, esto es, la autenticación de los usuarios, gestión de la movilidad de los usuarios o la gestión de interconexión con otras redes.

Por lo tanto, podemos dividir un sistema LTE en los tres elementos citados anteriormente. El equipo de usuario, la red de acceso que denominaremos E-UTRAN y la red troncal que llamaremos EPC. Asimismo, podemos considerar el subsistema IMS que facilita los servicios de comunicación multimedia. En la Figura 2 se puede observar un esquema de la arquitectura de una red 4G.

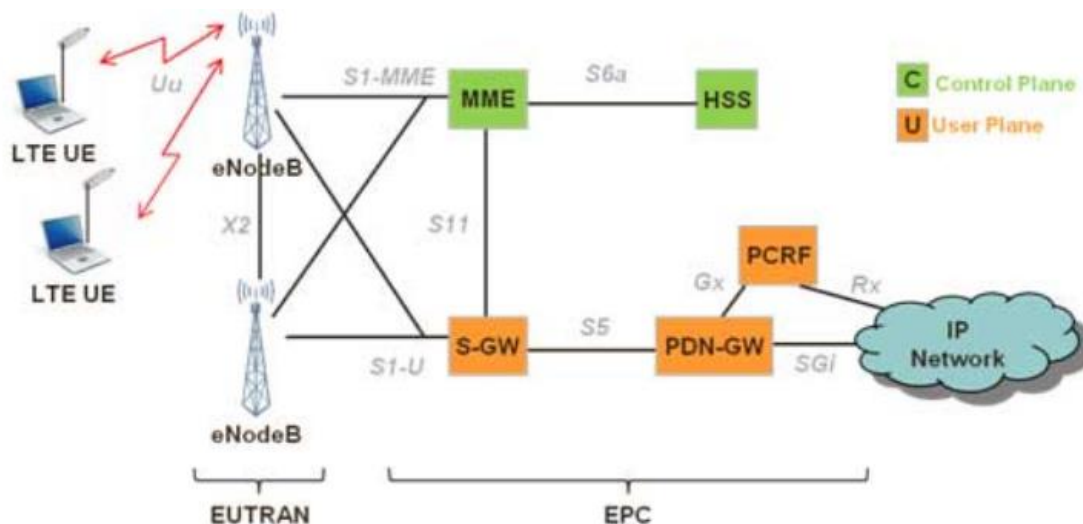


Figura 2. Arquitectura de una red 4G

Una vez introducidos estos elementos principales, en los siguientes subapartados, trataremos de realizar una explicación con un mayor grado de detalle de las características y descripciones de cada uno de ellos.

2.2.1 Red de acceso E-UTRAN

La red de acceso E-UTRAN (*Evolved Universal Terrestrial Radio Access Network*) está únicamente compuesta de estaciones base que denominaremos eNB (*evolved NodeB*). Estas estaciones base integran todas las funcionalidades de la red de acceso y se ocupan de proporcionar la conexión entre los usuarios y la red troncal EPC (Agusti-Comes, y otros).

El eNB actúa como punto final para los protocolos de la interfaz radio, facilitando la transmisión de paquetes IP y los mensajes de señalización necesarios para la comunicación con los equipos de usuario. Además, realiza las siguientes funciones:

- **Control de admisión de servicios:** evalúa si los recursos disponibles en la red son suficientes para admitir una nueva solicitud de servicio. Esto asegura que no se sobrecargue la red y que se mantenga la calidad del servicio para los usuarios existentes.
- **Gestión de la movilidad:** supervisa y gestiona el traspaso (*handover*) de las conexiones de los usuarios entre diferentes celdas o eNBs mientras los usuarios se desplazan. Esto es esencial para mantener la continuidad de las llamadas y las sesiones de datos sin interrupciones.
- **Asignación dinámica de recursos:** distribuye de manera eficiente los recursos de la red (como el espectro de frecuencia y la potencia de transmisión) entre los usuarios en función de la demanda y las condiciones de la red. Esto ayuda a optimizar el uso de los recursos y maximizar el rendimiento de la red.
- **Control de interferencias:** implementa técnicas para minimizar la interferencia entre las señales de diferentes celdas o usuarios. Esto es crucial para mejorar la calidad de la señal y aumentar la capacidad de la red.
- **Encaminamiento del tráfico:** gestiona el direccionamiento de los paquetes de datos hacia y desde los equipos de usuario a través de la red. Esto incluye la selección de las mejores rutas para el tráfico de datos, asegurando una transmisión eficiente y rápida.

Los eNBs cuentan con 3 interfaces para comunicarse entre ellos, con los equipos de usuario y con la red troncal. En la Figura 3 pueden verse estas interfaces.

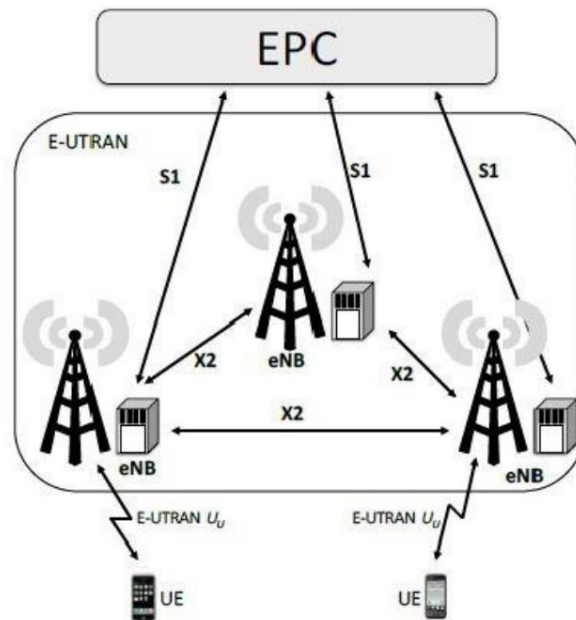


Figura 3. Arquitectura de una red de acceso E-UTRAN

La interfaz X2 es la interfaz utilizada para conectar los eNBs entre sí. Permite la comunicación y coordinación entre estaciones base vecinas para facilitar funciones como la transferencia de usuarios entre celdas, la sincronización de datos o la gestión eficiente de los recursos radio.

Para comunicar los eNBs con la red troncal EPC se emplea la interfaz S1. Interfaz que a su vez se divide en otras dos, la S1-U y la S1-MME:

- **Interfaz S1-U:** utilizada para el plano de usuario que gestiona el tráfico de los usuarios.
- **Interfaz S1-MME:** empleada para el plano de control, esto es, el manejo de funciones y protocolos necesarios para administrar la interfaz.

La tercera y última de las interfaces es la interfaz Uu, esta interfaz radio se encarga de comunicar al eNB con el equipo de usuario, y permite la transmisión de datos, voz y señales de control entre el dispositivo móvil y la estación base, facilitando la comunicación inalámbrica dentro del sistema LTE.

2.2.2 Red troncal EPC

El EPC (*Evolved Packet Core*) constituye el núcleo de la red y es el encargado de ofrecer un servicio de conectividad IP a los usuarios. Permite el acceso a redes externas y plataformas de servicios multimedia. A su vez, el núcleo de la red EPC está compuesto por varias entidades que se van a explicar a continuación. En la Figura 4 pueden verse dichas entidades.

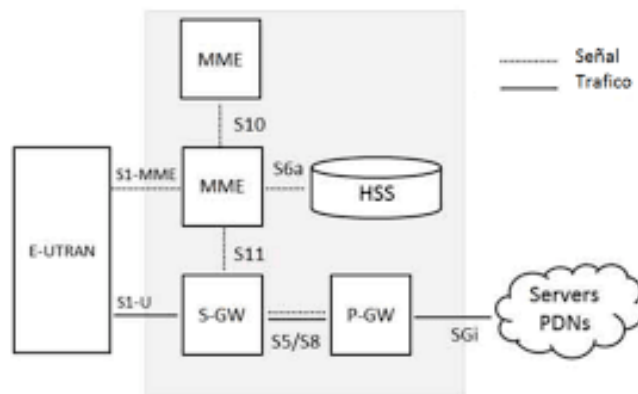


Figura 4. Entidades del EPC

Las entidades de red que constituyen el EPC son (Agusti-Comes, y otros):

- **MME:** elemento responsable de gestionar el acceso de los usuarios a través de E-UTRAN. Realiza funciones como autenticación y autorización de usuarios, gestión de servicios o gestión de movilidad y subportadoras.
- **S-GW:** elemento que actúa como punto de acceso al EPC y realiza funciones de enrutamiento de datos entre la estación base y el núcleo de la red o almacenamiento temporal de paquetes.
- **P-GW:** elemento que proporciona conectividad entre la red LTE y las redes externas y realiza funciones de enrutamiento, gestión de la movilidad y asignación de direcciones IP.
- **HSS:** base de datos de todos los usuarios de la red. Almacena información como identificadores de usuario, identificadores de servicio, información de seguridad y cifrado o la ubicación de un usuario en la red.
- **PCRF:** elemento que gestiona las políticas de calidad de servicio (QoS) y las reglas de tarificación para el tráfico de datos en la red LTE.

2.2.3 Equipo de usuario

El equipo de usuario en una red 4G, también conocido como UE (*User Equipment*), es el dispositivo que utiliza un usuario para conectarse a la red de LTE. Dicho equipo de usuario está compuesto por dos partes principales, el equipo móvil y el módulo de suscripción del usuario (Agusti-Comes, y otros):

- **Equipo móvil (ME):** incluye las funciones de comunicación con la red y las capacidades necesarias para la interacción del usuario con los servicios ofrecidos por la red. A su vez, el ME se divide en dos entidades:
 - Terminación móvil (MT): se encarga de las funciones de comunicación
 - Equipo terminal (TE): se ocupa de la interacción con el usuario
- **Módulo de suscripción del usuario (SIM/USIM):** identifica en la red al usuario al que esté asociada independientemente del equipo móvil utilizado.

2.2.4 IP Multimedia System (IMS)

El *IP Multimedia Subsystem* (IMS) es un subsistema que estandariza los mecanismos de control necesarios para ofrecer servicios multimedia sobre redes IP, como voz, vídeo, mensajería y datos. Es un componente clave en las redes de telecomunicaciones modernas, como LTE, proporcionando una plataforma común para la entrega de servicios integrados y ricos en multimedia (Agusti-Comes, y otros).

La infraestructura del IMS está formada por servidores, bases de datos y pasarelas que se comunican mediante protocolos. Esta red permite ofrecer servicios de voz y vídeo sobre IP, videoconferencia, mensajería instantánea, entre otros. Los terminales de usuario acceden a estos servicios a través de la red LTE. IMS pretende reemplazar los servicios de comunicación tradicionales basados en conmutación de circuitos a medio y largo plazo.

El modelo de servicio del IMS se divide en tres capas:

- **Capa de transporte:** proporciona la infraestructura de red IP necesaria para el encaminamiento de los flujos IP.
- **Capa de control:** incluye elementos responsables de la gestión de sesiones y la interacción con redes telefónicas tradicionales.
- **Capa de aplicación:** contiene los servidores de aplicación que gestionan la lógica y los datos de los servicios ofrecidos por IMS.

El IMS utiliza el Protocolo de Inicio de Sesión (*Session Initiation Protocol*, SIP), con extensiones adicionales, para gestionar el establecimiento y liberación de sesiones multimedia sobre redes IP.

2.2.5 Interfaz radio

En esta sección, se va a profundizar en la interfaz radio de las redes LTE. La interfaz radio de LTE 4G (también conocida como interfaz Uu) es el enlace de comunicación entre el equipo de usuario (UE) y la estación base (eNB). Esta interfaz es crucial para la transmisión de datos y señalización entre los dispositivos móviles y la red LTE (Erik Dahlman, 2011).

La arquitectura de protocolos radio de la red de nueva generación se basa en el modelo OSI, destacándose por su fácil comprensión y soporte de diversos servicios.

2.2.5.1 Capa física

La capa física de la interfaz radio se encarga de realizar la transmisión a través del canal radio. Para ello, se basa en las siguientes técnicas de acceso múltiple:

- OFDMA: tecnología utilizada en el enlace descendente en las redes LTE. Permite que diferentes usuarios compartan subportadoras, facilitando transmisiones simultáneas y eficientemente gestionadas. Esta técnica asigna dinámicamente subportadoras basándose en las condiciones del canal, mejorando la velocidad de transmisión y la eficiencia espectral. OFDMA ofrece ventajas como diversidad multiusuario, diversidad frecuencial, robustez frente al multitrayecto, flexibilidad en la banda asignada y eficiencia en la utilización del espectro.
- SC-FDMA: se emplea en el enlace ascendente. SC-FDMA utiliza una técnica de modulación que combina los beneficios de la transmisión de portadora única con las ventajas de la multiplexación por división de frecuencia. SC-FDMA es menos complejo y más eficiente en términos de consumo de energía, lo cual es crucial para extender la vida de la batería de los dispositivos móviles, al tiempo que mantiene una baja interferencia y alta calidad de señal en la red.

La capa física del sistema LTE está diseñada para operar en las bandas altas de UHF, que van desde los 450 MHz hasta los 3,5 GHz. El estándar ofrece hasta 40 bandas de operación posibles para funcionar ya sea por división de frecuencia (FDD) o por división de tiempo (TDD).

Los esquemas de modulación posibles para el enlace descendente son QPSK, 16-QAM y 64-QAM, mientras que para el enlace ascendente son QPSK y 16-QAM, con 64-QAM dependiendo de la capacidad del terminal móvil.

2.2.5.2 MIMO

El sistema MIMO (*multiple-input multiple-output*) es una tecnología clave en redes LTE 4G que mejora significativamente la capacidad y el rendimiento de la red. MIMO emplea múltiples antenas tanto en la transmisión como en la recepción de datos (Pertíñez López, 2015).

Los datos se dividen en tramas más pequeñas que se modulan y transmiten simultáneamente por diferentes antenas usando la misma frecuencia. Debido a las reflexiones por multitrayecto, la señal recibida en cada antena es una combinación de las tramas transmitidas. En el receptor, las tramas se separan mediante algoritmos basados en la estimación de los canales entre transmisor y receptor.

MIMO incrementa la capacidad de transmisión y el alcance sin necesitar más espectro, simplemente añadiendo más antenas. Sin embargo, requiere el mismo número de antenas tanto en recepción como en transmisión.

2.3 Rendimiento de la red 4G

En este apartado se procederá a describir teóricamente las métricas necesarias para poder analizar el rendimiento de cualquier red 4G. El análisis de dichos aspectos se realizará en apartados posteriores del informe. (Dahlman, Parkvall, & Sköld, 2011)

El **throughput**, también conocido como tasa de transferencia, se refiere a la cantidad de datos que pueden ser transferidos a través de una red en un período de tiempo determinado. Esta métrica es una medida de la capacidad efectiva de la red para transportar datos y es crucial para determinar la velocidad real de transferencia de datos entre dispositivos.

La **latencia** es el tiempo que tarda un paquete de datos en viajar desde el dispositivo del usuario hasta su destino y volver. Se mide en milisegundos (ms) y es crítica para aplicaciones en tiempo real como un vehículo autónomo.

El **jitter** se refiere a la variabilidad en el retardo de la transmisión de datos a través de una red. Se mide como la diferencia en el retardo entre los paquetes de datos sucesivos y se expresa típicamente en milisegundos (ms). El **jitter** es una medida de la consistencia y estabilidad de la latencia de red y puede tener un impacto significativo en la calidad de las aplicaciones en tiempo real.

La **cobertura** se refiere a la disponibilidad de la señal de red en diferentes ubicaciones geográficas. Una buena cobertura significa que los usuarios pueden acceder a la red en una amplia variedad de lugares, incluidas áreas rurales y urbanas, edificios y túneles.

La **capacidad de usuarios** se refiere a la cantidad máxima de usuarios que la red puede manejar simultáneamente sin degradar el rendimiento. Esto está influenciado por la cantidad de recursos de red disponibles y la eficiencia de la gestión del tráfico.

La **QoS** se refiere a la capacidad de la red para cumplir con los requisitos de rendimiento de diferentes tipos de aplicaciones y servicios. Esto incluye la priorización de tráfico, garantizando baja latencia y pérdida de paquetes para servicios críticos como voz sobre IP (VoIP) y videoconferencias.

La **SINR** consiste en la relación entre la potencia de la señal útil y el ruido más la interferencia. Una buena relación señal/interferencia es esencial para mantener una conexión de gran calidad y maximizar la tasa de transferencia de datos, por lo tanto, es necesaria para conseguir modulaciones más altas.

La **RSRP** es la potencia recibida desde la estación base. Esta influye en la calidad de la conexión, que está estrechamente relacionada con la modulación que puede ser utilizada. Cuanto más fuerte sea la señal, más alta será la modulación soportada.

La **Tasa de Error de Bloques (BLER)** mide la proporción de bloques de datos que no se reciben correctamente y deben ser retransmitidos. Un bloque se considera erróneo si no puede ser decodificado correctamente a pesar de los mecanismos de corrección de errores. Esta medida es muy importante ya que está estrechamente relacionada con el QoS, además de aumentar la latencia y reducir la tasa de transferencia efectiva si aumenta el número de retransmisiones.

Por último, vamos a hablar del **handover**. Es el proceso por el que un UE cambia su conexión de celda a la vez que mantiene una conexión activa con la red. El **handover** es crítico en redes 4G para mantener la continuidad del servicio, pero sobre todo en un escenario de alta movilidad como nuestro servicio.

3 Inteligencia Artificial

En este tercer capítulo trataremos otro de los pilares fundamentales del proyecto, la Inteligencia Artificial (también conocida como IA). Comenzaremos con unas breves pinceladas teóricas con el fin de situarnos en el contexto y trataremos de definir los objetivos a abordar en el desarrollo de este sistema empleando las posibilidades que nos brinda esta disciplina.

A continuación, explicaremos detalladamente los pasos que se han considerado a seguir para lograr conformar un dataset funcional, e implementarlo en distintos entornos y dispositivos como pueden ser un ordenador, una mini placa de computación conocida como Jetson Orin Nano de la familia NVIDIA o una plataforma de servicios tipo web online conocida como Roboflow entre otros.

Finalmente, puede resultar de interés evaluar cuantitativamente el sistema a desplegar mediante distintas métricas como la precisión o la matriz de confusión. Además, debido al carácter de este proyecto, deberemos solicitar los permisos referentes a la protección de datos en imágenes para poder desplegar nuestro sistema en un futuro.

3.1 Conceptos teóricos y objetivos

La Inteligencia Artificial puede definirse como la capacidad de la que puede dotarse a una máquina para que realice tareas o acciones hasta el momento destinadas al ser humano. Esta disciplina es realmente amplia y compleja, y las posibilidades son infinitas gracias a los nuevos algoritmos y avances tecnológicos en computación; elementos de vital importancia en el desarrollo de nuevas herramientas basadas en este concepto. Esto se debe, a que los sistemas de Inteligencia Artificial requieren de la capacidad de poder ajustar correctamente funciones matemáticas de una complejidad elevada, a la vez que se realiza una serie de cálculos sobre una gran cantidad de datos en un tiempo relativamente reducido.

No obstante, debemos de saber diferenciar claramente el término Inteligencia Artificial de otros conceptos relacionados como los son el *Deep Learning* o *Machine Learning*. Si bien estos conceptos están relacionados, no representan lo mismo, por lo que cabe resaltar las diferencias y relaciones entre ellos (Kelleher, 2019):

- **Inteligencia Artificial:** Se denomina Inteligencia Artificial al campo de la ciencia de la computación enfocada en la creación de sistemas capaces de realizar tareas que requieren de inteligencia o presencia humana. Esto implica el desarrollo de herramientas capaces de analizar una situación y el contexto que las rodea, tomar la mejor decisión en cada caso y actuar consecuentemente.
- **Machine Learning:** Se denomina *Machine Learning* (o Aprendizaje Automático), a un subcampo de la Inteligencia Artificial centrada en el desarrollo de algoritmos capaces de permitir a los ordenadores reconocer, aprender e identificar patrones presentes en datos, sin necesidad de estar programado explícitamente para ello.
- **Deep Learning:** Se denomina *Deep Learning* (o Aprendizaje Profundo), a una técnica dentro del campo *Machine Learning* basado en el uso de redes neuronales de gran cantidad de capas (es decir, gran profundidad) con las que se consigue que el ordenador aprenda relaciones y representaciones jerárquicas de los datos suministrados.

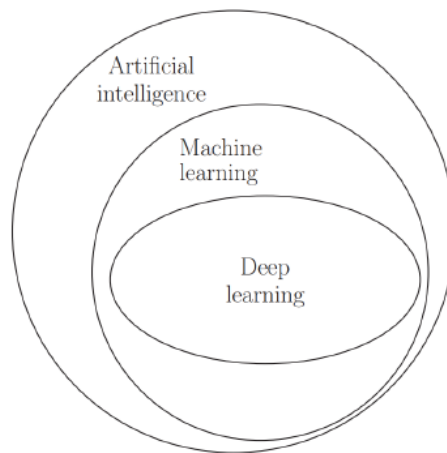


Figura 5. Diagrama de Venn sobre los conceptos de Inteligencia Artificial, Deep Learning y Machine Learning (Kelleher, 2019).

Como puede apreciarse en la figura previa y bien se ha mencionado anteriormente, los conceptos Inteligencia Artificial, *Machine Learning* y *Deep Learning* están relacionados entre sí debido a que unos engloban a otros, no obstante, no significan ni representan lo mismo. Podría definirse la Inteligencia Artificial como el concepto más abstracto y general, el *Machine Learning* como una técnica o subgrupo de estos y el *Deep Learning* como el más preciso y conciso dentro de este grupo de tres.

Finalmente, cabe destacar que existen una gran cantidad de herramientas, algoritmos y técnicas basados en estas disciplinas como lo son la visión por ordenador, análisis de documentos, procesamiento de lenguaje natural (NLP) o incluso sistemas de seguridad basados en reconocimiento facial y de voz.

El objetivo del proyecto, por lo tanto, es emplear técnicas de *Deep Learning* para procesar y clasificar distintas imágenes de señales de tráfico reales, y así lograr que un vehículo sea capaz de tomar decisiones para facilitar la conducción del usuario, en función de las señales identificadas en las imágenes capturadas con la propia cámara del vehículo. Por otra parte, la parte de procesamiento de todas las imágenes y la toma de decisiones puede ser implementada en el servicio en la nube basado en 4G, en cualquier dispositivo de cómputo en el borde (o similar) o en el ordenador del propio vehículo, más adelante se valorarán las posibles ventajas e inconvenientes presentes en cada una de las posibilidades y la decisión elegida.

3.2 Herramientas elegidas

Actualmente, existen una gran cantidad de herramientas y servicios online que tratan de facilitarnos el desarrollo de sistemas basados en Inteligencia Artificial, desde plataformas con una gran variedad de datasets ya generados y etiquetados, hasta servicios de Google para emplear sus recursos informáticos y poder entrenar los modelos con una gran velocidad. Por ello, puede resultar de interés realizar un proceso de estudio de las distintas herramientas existentes y elegir la que más se ajuste a nuestro caso de estudio.

En primer lugar, trataremos de emplear el servicio web Roboflow, este servicio ofrece una plataforma que facilita el flujo de trabajo de proyectos basados en *Computer Visión* (o Visión por Ordenador), es decir, nos brinda herramientas con las que poder analizar, etiquetar y generar imágenes para conformar un dataset con el que poder desarrollar un sistema de clasificación, entre otros. La elección de este servicio viene incentivada por la posibilidad de emplear las siguientes herramientas de la plataforma (Roboflow, Roboflow, 2024):

- **Etiquetado de imágenes:** Roboflow ofrece herramientas con las que poder etiquetar de una manera sencilla imágenes para focalizar las regiones de interés. Además, más adelante tenemos la posibilidad de separar todas las imágenes en grupos de entrenamiento (*train*), validación (*validation* o *val*) y testeo (*test*).
- **Preprocesado:** Roboflow ofrece una amplia variedad de opciones de preprocesado como la posibilidad de variar el contraste, la rotación, redimensionar imágenes según el tamaño deseado o convertirlas en escala de grises entre otras.
- **Compatibilidad con distintos formatos:** Roboflow ofrece la posibilidad de generar datos a partir del dataset en distintos formatos como .csv (Comma Separated Values), YOLOvX o TensorFlow entre otros.
- **Entrenamiento de modelos:** Roboflow ofrece la posibilidad de entrenar nuestros modelos con el dataset que generemos o importe, no obstante, al emplear los recursos del ordenador el tiempo se demora en exceso, por lo que usaremos otras herramientas que veremos más adelante para entrenar nuestro modelo.
- **Comprobación de modelos:** Roboflow ofrece un sistema capaz de comprobar e implementar un modelo, desplegando el modelo previamente entrenado y validar su correcto funcionamiento con las imágenes que queramos.

Por ende, aunque solo emplearemos las funciones de preprocesado y etiquetado de imágenes, optaremos por elegir este servicio dado su potencial y su accesibilidad (ya que es gratuito).

The screenshot displays two main sections of the Roboflow interface. The first section, titled '2 Train/Test Split', explains how images are split into three sets: Training Set (686 images, 50%), Validation Set (411 images, 30%), and Testing Set (274 images, 20%). It includes 'Continue' and 'Rebalance' buttons. The second section, titled '3 Preprocessing', provides instructions on how preprocessing can decrease training time and increase performance. It shows a list of applied transformations: 'Auto-Orient' and 'Resize' (Stretch to 640x640), each with an 'Edit' link. There is also an 'Add Preprocessing Step' button and a 'Continue' button at the bottom.

Figura 6. Ejemplo de preprocesado y separación de imágenes de un dataset con Roboflow.

Por otro lado, como se ha mencionado previamente, para la parte de entrenamiento del modelo el servicio de Roboflow no será una opción factible ya que emplea los propios recursos del ordenador. Debido a que los sistemas de Inteligencia Artificial requieren de gran capacidad de cómputo en paralelo (donde destacan las GPUs), emplear los recursos del propio ordenador personal puede suponer un cuello de botella, por ello, se optará por emplear los servicios gratuitos ofrecidos por Google Collab.

Google Collab (o Google Colaboratory), es un entorno en la nube basado en el uso de cuadernos Jupyter, donde puede ejecutarse código en Python, directamente en el navegador sin tener que configurar ningún entorno de trabajo. Además, permite disponer de un conjunto de recursos virtuales en la nube de Google con mucha mayor potencia de cálculo que nuestros ordenadores personales (servicio IaaS), sobre todo GPU. Otra de las ventajas es el entorno interactivo, este entorno dispone ya de una serie de librerías precargadas y permite conexión con Google Drive para poder sincronizar, guardar o descargar datos y resultados (Google, Google Collab, 2024; Google, Introducción a Google Collab, 2024).

3.3 Generación de un dataset

Una vez comentadas las distintas posibilidades de elección de las herramientas que trataremos de emplear, el siguiente paso será desarrollar un dataset con imágenes de señales de tráfico que pueden ser extraídas de bases de datos públicas o mediante la propia toma de fotografías por nosotros mismos. Una vez dispongamos de un conjunto de imágenes lo suficientemente completo, procederíamos a etiquetar todas las señales presentes en cada una de las fotografías mediante la herramienta de la plataforma Roboflow.

Para poder obtener este conjunto de imágenes podríamos recurrir a diferentes metodologías:

- **Bases de datos:** Emplear datos de señales de tráfico localizadas en bases de datos como Dataset Search¹ (de Google), Kaggle Datasets² (de Kaggle), Roboflow Universe³ (de Roboflow) o cualquier otro servicio que disponga de este tipo de bases de datos.
- **Imágenes públicas en Internet:** Para ello se podrían emplear algún *plugin* para el navegador o software externo que permita descargarse un grupo grande de imágenes de cualquier buscador de internet, bajarse las imágenes una a una (aunque se desaconseja ya que es muy ineficiente) o emplear un dataset generado por terceros.
- **Captura propia de imágenes:** A través de una cámara obteniendo las fotos directamente por nuestra propia mano. Teniendo en cuenta que esta opción puede suponer la más laboriosa y costosa de todas.

No obstante, debemos de tener en cuenta la existencia de ciertos parámetros que pueden afectar al desarrollo del sistema como: la cámara con la que se toman las imágenes, la iluminación, la resolución de las imágenes, el entorno en el que han sido tomadas las fotografías o la calidad de la imagen. Es un punto de vital importancia emplear las imágenes para entrenar un sistema basado de *Computer Vision*, lo más similares posibles a las imágenes de las condiciones reales en las que se va a desplegar el sistema.

Tras realizar todo el proceso de etiquetado de las imágenes, y suponiendo que se han tomado una cantidad razonablemente grande de datos. Realizaríamos un preprocesado con dichos datos para obtener una mayor variedad en las imágenes, y así evitar que el sistema “aprenda demasiado”, este, es un problema frecuente a la hora de desarrollar sistemas basados en Inteligencia Artificial conocido como “sobre entrenamiento” (o también llamado *Overfitting*) e implica que se entrena demasiado al modelo por lo que finalmente no es capaz de reconocer elementos que disten ligeramente de los datos con los que se ha entrenado.

¹ Google Dataset Search disponible en: <https://datasetsearch.research.google.com/>

² Ejemplo de Kaggle Dataset: <https://www.kaggle.com/datasets/andrewmvd/road-sign-detection>

³ Ejemplo de Roboflow: <https://universe.roboflow.com/search?q=object+detection%2C+traffic+signals>

Para la etapa de preprocesado y augmentación emplearemos algunas de las herramientas proporcionadas por Roboflow, entre ellas (Roboflow, Roboflow, 2024):

- **Redimensionar:** Ajusta todas las imágenes para que tengan las mismas dimensiones y sea más fácil para el sistema extraer las características y manejarlas.
- **Autoajuste de contraste y saturación:** Genera imágenes a partir de las imágenes ya existentes en el dataset aumentando o disminuyendo el contraste y la saturación.
- **Rotación:** Rota las imágenes una cierta cantidad de grados para no entrenar todo el sistema con imágenes de señales en un mismo ángulo.
- **Escala de grises:** Permite convertir imágenes RGB a imágenes en escala de grises, para evitar que el sistema se guíe demasiado por los colores de las señales.
- **Enfoque:** Permite desenfocar la imagen ligeramente para ajustarse más a condiciones reales, es frecuente que las cámaras puedan tardar en enfocar.



Figura 7. Ejemplo de preprocesado y augmentación de datos de un dataset.

Finalmente, dividiríamos las imágenes etiquetadas en tres conjuntos, un conjunto de entrenamiento para entrenar el modelo (denominado como *train*) que contendrá entorno al 70% de las imágenes, un conjunto de validación para validar el buen rendimiento del modelo tras cada ciclo de entrenamiento del modelo (denominado como *validation* o *val*) de entorno al 20% de las imágenes y un último conjunto de testeo, para realizar una última comprobación final del rendimiento del modelo entrenado de entorno al 10% de las imágenes. Cabe destacar que los datos de este último conjunto no deben verse nunca durante las etapas de validación y entrenamiento (esto se conoce como fisgonear los datos, y nos puede generar resultados sesgados).



Figura 8. Ejemplo de separación de los datos en distintos conjuntos.

En este caso, hemos optado por realizar un prototipo de dataset con el que realizar una primera aproximación de cara al proyecto. Para ello, hemos seguido los pasos establecidos y se ha generado un dataset con imágenes de señales de tráfico reales extraídas de un dataset de Internet⁴ después, se ha empleado la herramienta de etiquetado de Roboflow y se han generado 1547 imágenes que incluyen las siguientes clases (señales a reconocer a las imágenes) (Arcos, 2015):

	CEDA		STOP
	OBLIGATORIO-CONTINUAR-RECTO		VELOCIDAD-MAX-100
	OBLIGATORIO-GIRAR-DERECHA		VELOCIDAD-MAX-120
	OBLIGATORIO-GIRAR-IZQUIERDA		VELOCIDAD-MAX-30
	PROHIBIDO		VELOCIDAD-MAX-50
	PROHIBIDO-GIRAR-DERECHA		VELOCIDAD-MAX-70
	PROHIBIDO-GIRAR-IZQUIERDA		VELOCIDAD-MAX-90

Figura 9. Clases empleadas para etiquetar con la herramienta Roboflow.

Una vez se han etiquetado y revisado todas las imágenes, elegiremos alguno de las opciones de preprocesado y aumentaciones citadas previamente (ver Figura 7) con el fin de generar una mayor variación en las imágenes que se usarán de cara a entrenar el modelo final.



Figura 10. Ejemplo de imagen etiquetada.

Otra de las opciones que nos permite Roboflow, es consultar el estado de nuestro dataset, otorgándonos información acerca de la cantidad de elementos de cada clase en todo el dataset o un mapa de calor con las ubicaciones de las etiquetas asignadas a cada señal en las imágenes. A continuación, podemos observar el estado de nuestro dataset, si bien es cierto que tiene algunas imágenes infrarrepresentadas, debe tenerse en cuenta que solo es una versión inicial del prototipo. En caso de desarrollarse más una solución por esta rama, se trataría de nivelar la cantidad de entidades de cada clase para que el algoritmo sea más completo.

⁴ Dataset extraído de: <https://daus-lab.github.io/spanish-traffic-sign-dataset/>

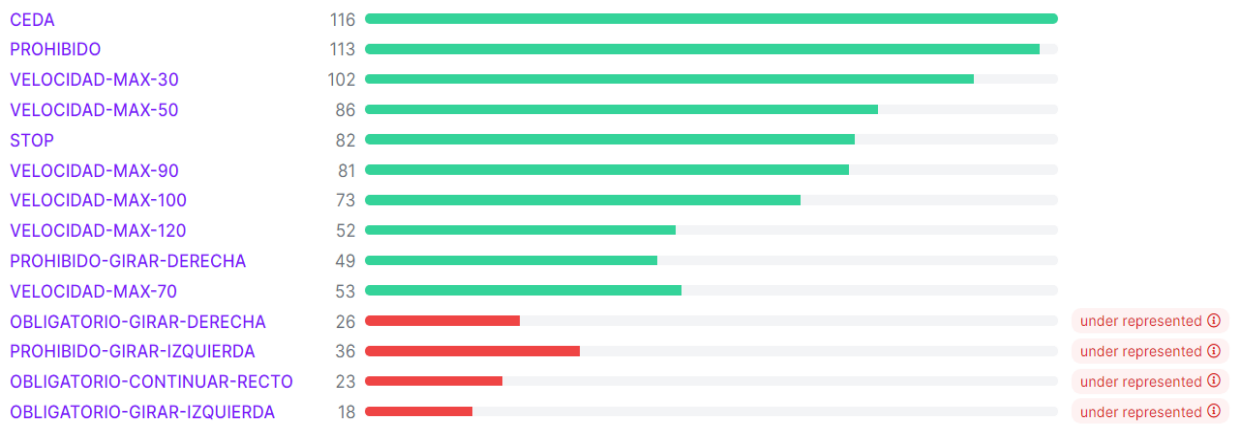


Figura 11. Entidades por clase etiquetadas en las imágenes del dataset.

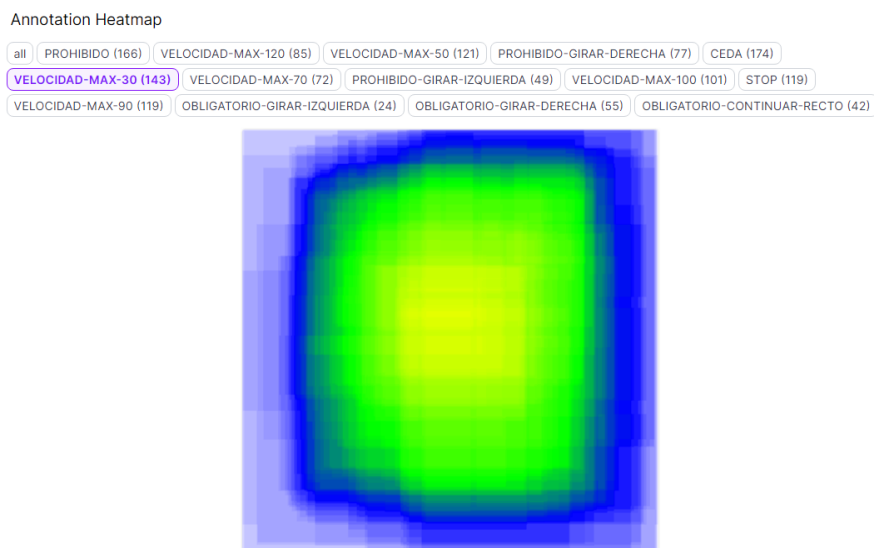


Figura 12. Ejemplo mapa de calor de la clase VELOCIDAD-MAX-30.

De cara a mejorar el posible modelo en una siguiente versión, se intentará lograr un mapa de calor más variado, empleando imágenes de señales de distintos tamaños y con una mayor variedad de posiciones y localizaciones. Todo con el fin de lograr un modelo completamente independiente de la posición y tamaño de la señal, ya que podría darse el caso de que estas dos características sesguen ligeramente el resultado obtenido.

3.4 Entrenamiento de la red

Una vez dispongamos de un dataset completo y preparado, la siguiente etapa sería la de entrenamiento del modelo con dichos datos, para realizar dicho entrenamiento se optará por emplear la plataforma de servicio web de Google Collab como se ha mencionado anteriormente. Además, se valora la posibilidad de emplear la API de Roboflow para transferir el dataset al entorno de Google Collab de manera sencilla con un sencillo par de códigos.

No obstante, debemos valorar las posibilidades de los distintos algoritmos aplicables al caso de estudio, en este caso, nos interesa un algoritmo eficaz y ligero capaz de identificar objetos dentro de imágenes. Algunos de los algoritmos valorados son:

- **YOLO:** YOLO o también conocido como *You Only Look Once* haciendo referencia precisamente a las capacidades del algoritmo que es capaz de detectar y clasificar objetos dentro de imágenes con solo mirar una vez la imagen, haciéndolo buen candidato para este proyecto por su eficiencia, velocidad, precisión y facilidad de implementación (Peiyuan Jiang, 2021).
- **SSD:** SSD o *Single Shot Multibox Detector* está basado en redes neuronales convolucionales (CNN) y al igual que YOLO es capaz de detectar objetos de una sola pasada. En contraposición es más complejo de implementar y desplegar que los modelos basados en YOLO (además de que no existen herramientas como Roboflow para este algoritmo) (Peiyuan Jiang, 2021).
- **Faster R-CNN:** Algoritmo que emplea dos etapas, una primera que extrae características de las imágenes y una segunda basada en regresión para generar regiones candidatas donde podrían estar localizados los objetos. Este sistema de dos capas lo hace más lento que otras opciones (Girshick, 2015).
- **TensorFlow:** Plataforma de aprendizaje automático desarrollada por Google basado en Redes Neuronales Convolucionales (CNN) y Redes Neuronales Concurrentes (RNN) entre otras. Destaca por su gran flexibilidad y variedad de herramientas posibles, no obstante, no es tan rápido y eficaz como lo son otras alternativas como YOLO (Hull, 2020).

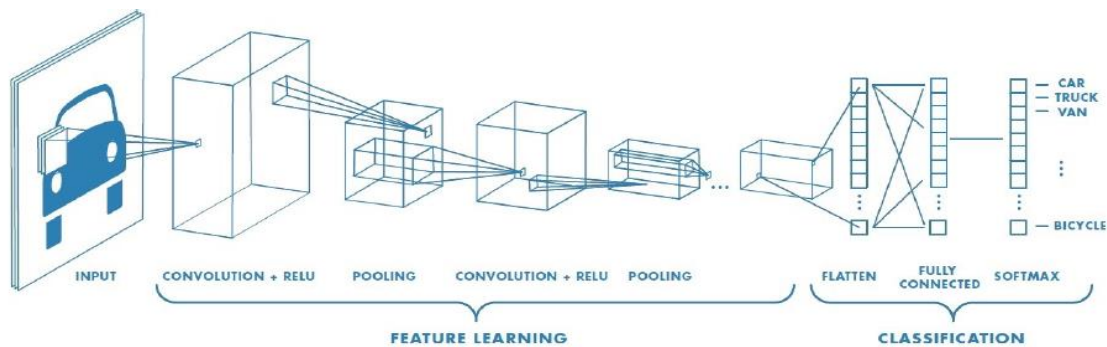


Figura 13. Esquema de una red neuronal convolucional (CNN) (Martínez Zarzuela, 2021)

Teniendo en cuenta que nuestro sistema debe fundamentarse en rapidez, precisión y facilidad de implementación se optará por emplear el algoritmo YOLOv9 (ya que es la última versión estable desplegada en 2024), debido a que es un algoritmo ampliamente utilizado en este tipo de tareas de detección de objetos a través de imágenes.

3.5 Implementación de la red

Una vez el dataset ha sido generado, se debe elegir un modelo para desarrollar y entrenar dicho modelo con los datos preparados, para más adelante, desplegar dicho modelo en un entorno de interés. En este apartado comentaremos los puntos referentes al entrenamiento e implementación de un modelo basado en el algoritmo YOLO.

El primer paso en cualquier modelo consiste en entrenar el modelo, es decir, introducir datos en un sistema que sea capaz de encontrar relaciones entre ellos. Para ello, emplearemos Google Collab como ya se ha mencionado previamente y el dataset que habremos generado. Además, se empleará YOLOv9 y someteremos al algoritmo a un proceso de entrenamiento de 20 *epochs* (o épocas, conocidas como la cantidad de veces que se hace pasar los datos por el modelo).

La elección del número de épocas viene condicionada por diversos factores como lo son el tamaño y complejidad del modelo a entrenar, la arquitectura del modelo (número de capas, número de neuronas o el método empleado para la convergencia entre otros). En nuestro caso de estudio, optaremos por un número de épocas relativamente bajo para entrenar nuestro modelo ya que, al no ser un modelo grande, si lo entrenamos demasiado puede desencadenar en acabar cometiendo *overfitting*.

Cuando el entrenamiento finalice, habremos obtenido un fichero de pesos (generalmente acabado con la extensión .pt). Con dicho fichero y un script con el que aplicar los pesos, podremos generar una inferencia. Es decir, preparar una instancia del modelo capaz de generar un resultado a la salida a partir de los datos nuevos que le metamos a la entrada, en otras palabras, un modelo entrenado capaz de responder ante nuevas entradas.

Finalmente, con el fichero de pesos y el script de ejecución, seremos capaces de trasladar nuestro modelo a cualquier dispositivo con los requerimientos mínimos de hardware y software. En nuestro caso, podemos explorar distintas posibilidades como lo son:

- **Edge computing:** También conocido como computación en el borde o en el extremo, esta filosofía se basa en procesar todos los datos lo más cerca posible del dispositivo que los genera, de esta manera, se consigue reducir la cantidad de datos a enviar al servidor y la carga de cómputo en él. En el caso de nuestro proyecto, si optásemos por emplear *edge computing*, los datos serían procesados en el propio vehículo o en algún dispositivo lo más cercano posible a este (Authors, 2022).
- **Cloud computing:** También conocido como computación en la nube, esta filosofía se basa en realizar la tarea de procesamiento de datos completamente opuesto al *edge computing*, es decir, procesar todos los datos en la propia nube. Esto permite no tener que disponer de elementos de cómputo muy potentes de forma local, pero incrementa la latencia y la capacidad necesaria del enlace para mandar y recibir todos los datos entre el elemento generador de ellos y el actuador pertinente (Cayetano, 2023-2024).

Es por ello, por lo que merece la pena valorar las posibles ventajas y desventajas de emplear un sistema u otro para el caso de estudio que nos concierne. En el caso de emplear el *edge computing*, podremos disponer de un sistema con las siguientes ventajas:

- Menor latencia (crucial para elementos como un vehículo conectado).
- Menor ancho de banda requerido.
- Independencia a fallos de conexión con la nube o en la red.
- Mayor seguridad (ya que los datos no se envían en ningún momento).

En cuando a las desventajas de emplear *edge computing*:

- Dificultad en la administración y gestión (en especial si el número de dispositivos es alto).
- Limitaciones en el procesamiento y almacenamiento de los datos.
- Dificultad en escalabilidad.

Por otro lado, si implementásemos un sistema basado en *cloud computing*, podríamos disponer de esta serie de ventajas frente al sistema basado en *edge computing*:

- Una gran escalabilidad (mejorar el sistema aumentando los recursos del servidor).
- Acceso remoto global (la posibilidad de acceder a los datos desde cualquier sitio).
- Facilidad de recuperación ante un desastre.

En cuanto a las desventajas de emplear *cloud computing*:

- Una mayor latencia (sobre todo con grandes volúmenes de datos).
- Un mayor ancho de banda requerido.
- Dependencia de conectividad a internet.
- Menor seguridad (ya que los datos pueden ser interceptados durante la comunicación).

Por ende, debido a que el caso de estudio presentado requiere de una latencia mínima y un consumo de ancho de banda lo más pequeño posible, se optará por diseñar un sistema basado en *edge computing* para poder procesar todos los datos del vehículo lo más rápido posible. No obstante, puede resultar de interés desarrollar parcialmente ambos sistemas y estudiar cual puede tener mejores prestaciones de cara al proyecto.

Debido a que el entrenamiento e implementación del modelo requiere de cierta carga computacional, puede resultar de interés valorar el uso de alguno de los productos de la empresa productora de tarjetas gráficas NVIDIA, esta empresa ofrece una amplia variedad de productos entre los que destacan los kits de desarrollador Jetson. La familia Jetson son un conjunto de miniordenadores de alto rendimiento con distintas características que incluyen CPU, GPU, memoria, administración de energía, interfaces de alta velocidad, puertos serie, conexión a internet y otras cosas más (NVIDIA, Familia Jetson , 2024).

Este tipo de dispositivos incluyen CPU basados en procesadores ARM de alto rendimiento, GPU de NVIDIA (que incluye arquitecturas avanzadas como la arquitectura Maxwell) y memoria configurable. Todos estos componentes bajo una misma placa son capaces de brindar una gran rapidez y precisión a la hora de realizar operaciones en paralelo, lo que las hace ideales para entrenar y desplegar modelos de IA y sistemas basados en *edge computing* (NVIDIA, Familia Jetson , 2024).

La familia Jetson dispone de una amplia variedad de modelos, cada una con sus propias características (NVIDIA, Developer Kits, 2024):

- **Jetson Nano:** Las más ligera y sencillas de la familia Jetson, destacan por su bajo consumo de energía y costo, ideal para implementar modelos de baja carga computacional.
- **Jetson TX2:** El balance entre consumo de energía y rendimiento, ideales para aplicaciones de tipo industrial tales como robots, drones o equipos médicos que requieran computación avanzada.
- **Jetson Xavier NX:** Las más potentes de la familia Jetson, ideada para las aplicaciones más exigentes y los cálculos más intensivos, no obstante, el consumo energético es bastante elevado.
- **Jetson Xavier AGX:** Destinadas a aplicaciones de alto rendimiento como la robótica avanzada y los vehículos autónomos, lo que la hace una gran candidata para nuestro caso de estudio.
- **Jetson Orin:** Las más recientes de la familia Jetson, destacando en su gran rendimiento sobre todo en aplicaciones de IA de reconocimiento de objetos y sistemas de *edge computing*.

Teniendo en cuenta el objetivo del proyecto, podría resultar de interés trabajar con una de estas tarjetas, especialmente con la Jetson Orin o la Jetson Xavier AGX, ya que ambas están especialmente diseñadas para las tareas que queremos abordar dentro del proyecto.



Figura 14. Productos de la familia Jetson (NVIDIA, 2024).

Otra de las ventajas de emplear este tipo de productos, es la cantidad de módulos y herramientas ofrecidas por la propia empresa para explotar aún más las posibilidades de estos productos. Por ejemplo, la posibilidad de instalar un kit de desarrollo de software de NVIDIA conocido como JetPack SDK (*Software Developer Kit*), este kit incluye un sistema operativo Linux, una serie de bibliotecas de aprendizaje profundo (como CUDA, TensorRT u OpenCV) o los propios controladores para poder manejar la placa.

3.6 Análisis de resultados

Otro de los pilares fundamentales de este caso de estudio, sería el análisis de resultados, es decir, evaluar si las decisiones tomadas y los sistemas diseñados e implementados cumplen con los requisitos mínimos establecidos y con qué rendimiento.

En el caso de la Inteligencia Artificial, disponemos de una gran cantidad de métricas para realizar una medición objetiva de nuestros modelos, en función del tipo de modelo y de su posible escenario de implementación (es decir, el uso que se le va a dar) nos pueden resultar de mayor interés unas métricas u otras. Queda a criterio del desarrollador valorar qué métricas son de mayor importancia y centrarse en obtener mejores puntuaciones en dichas métricas, para ello puede emplear puntuaciones, gráficas o porcentajes.

Algunas de las métricas más interesantes para emplear en nuestro modelo serían (de Miguel & Durán , 2023-2024; Yacoubi & Axman, 2020):

- **Precisión (*Precision*):** Se define como la cantidad de verdaderos positivos (TP) (es decir, los aciertos en las predicciones) frente a la suma de los verdaderos positivos (TP) y los falsos positivos (FP), es decir, proporción de verdaderos positivos entre todos los positivos detectados.

$$Precision = \frac{TP}{TP+FP} \quad (1)$$

- **Sensibilidad (*Recall*):** Se define como la cantidad de verdaderos positivos (TP) frente a la suma de los verdaderos positivos (TP) y los falsos negativos (FN), es decir, la proporción de verdaderos positivos entre todos los positivos reales existentes.

$$Recall = \frac{TP}{TP+FN} \quad (2)$$

- **Tiempo de procesamiento:** Se define como el tiempo entre la entrada de datos nuevos en la inferencia y la generación de un resultado en la salida.

$$tiempo_{proces} = tiempo_{resultado} - tiempo_{entrada-dato} \quad (3)$$

- **Matriz de confusión:** Consiste en una tabla de N dimensiones (en función del número de clases que se pueden identificar), empleada para organizar las etiquetas predichas y las reales de forma que se pueda apreciar de manera gráfica las etiquetas (o % de las etiquetas) que el modelo acierta y las que confunde con otras clases.

		Valor observado (f)	
		+1	-1
Valor predicho (h)	+1	TP	FP
	-1	FN	TN

Figura 15. Esquema matriz de confusión (de Miguel & Durán , 2023-2024).

- **Puntuación F1:** Puntuación asignada como la media proporcional entre la precisión y el *recall*, permite asignar un valor numérico al modelo con el que se puede valorar como de bueno es. La fórmula para obtener la puntuación F1 es:

$$F1 = 2 * \frac{Precision*Recall}{Precision+Recall} \quad (4)$$

- **Umbral de confianza:** El umbral de confianza consiste en un valor numérico modificable a partir del cual el sistema considera si un objeto detectado pertenece a una clase (supera dicho umbral) o no (no supera dicho umbral).

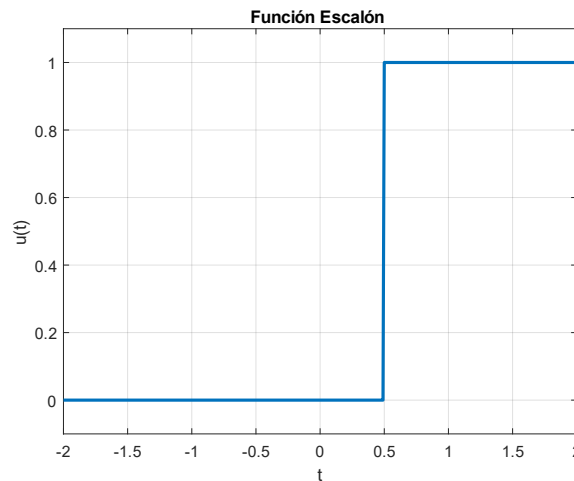


Figura 16. Ejemplo de función escalón con umbral 0.5 en formato svg generada con Matlab.

Por ende, tras el desarrollo y entrenamiento del modelo, el siguiente paso será aplicar estas métricas y analizar como de buenos y precisos son dichos resultados, y en función de ellos, optar por cambiar, mejorar o reestructurar tanto el modelo como el dataset.

3.7 Legislación

Debido a que nuestro proyecto necesita capturar imágenes del entorno para poder operar, debemos de tener en cuenta que se deben respetar los derechos de privacidad de los usuarios y cumplir los principios de una Inteligencia Artificial segura y sin riesgos. Es por ello, por lo que a lo largo de todo el proyecto nos aseguraremos de cumplir con todos los requisitos relacionados con permisos y leyes de protección de datos de usuarios.

Más concretamente, nos aseguraremos de que nuestro proyecto cumpla estrictamente con las directrices fijadas por el Reglamento General de Protección de Datos (RGPD) de la Unión Europea y la AEPD (Agencia Española de Protección de Datos). Para ello, trataremos de capturar, enviar y almacenar todos los datos obtenidos de la manera más segura posible, evitando que estos puedan ser filtrados o robados. Optaremos por emplear medidas de seguridad como la protección con algoritmos, contraseñas o la eliminación de los propios datos una vez procesados y empleados (AEPD Agencia Española de Protección de Datos, 2020).

Además, nuestro proyecto estará alineado con el Reglamento Europeo de Inteligencia Artificial de 2022, una normativa relativamente reciente que establece un marco legal para el tratamiento de datos referentes al desarrollo y despliegue de sistemas de IA en Europa. Este reglamento se centra en asegurar que las tecnologías de inteligencia artificial cumplen los principios de ética y buena conducta, minimizando los riesgos y garantizando la transparencia, seguridad y fiabilidad de los sistemas (MINISTERIO DE ASUNTOS ECONÓMICOS Y TRANSFORMACIÓN DIGITAL, 2022).

4 Despliegue infraestructura

Previo a la realización del estudio de cobertura con el que se dimensionará la red propuesta, se ha realizado una investigación de la viabilidad de uso de la banda escogida. Esta ha sido la banda 7 que cuenta con 10 MHz de ancho de banda de subida y 10 MHz de bajada. Se trata de una banda empleada para el 4G LTE de alta velocidad. Sin embargo, algunos de los operadores del país la emplearán en despliegues futuros de 5G (Netel, 2022). Para la licitación se cuenta con un rango de frecuencias de *Up-link* 2540-2550 MHz y de *Down-link* 2660-2670 MHz. Dicha banda cuenta con un modo dúplex tipo TDD de tipo IMT-E *Duplex Spacing*.

Band number	Duplex mode	Frequency	Uplink	Downlink	Duplex spacing	Channel bandwidths	Notes
7	FDD	2600	2500 – 2570	2620 – 2690	120	5, 10, 15, 20	

Figura 17. Información banda 7

Se puede observar cómo se emplea un espaciamiento considerablemente grande. De no ser así, el enlace ascendente y el enlace descendente podrían interferir entre sí, aumentando las tasas de error de la red LTE (MastDatabase, 2021). En 4G LTE, existen dos modos principales de operación para la comunicación entre la estación base y los dispositivos: FDD (*Frequency Division Duplexing*) y TDD (*Time Division Duplexing*).

LTE-TDD	LTE-FDD
✓ Allows for dynamically adjusting downlink slots For high demand, extra downlink slots can be added, increasing speeds at the cost of uplink throughput	✓ Overall higher throughput - Transmitting and receiving can happen at once, achieving max throughput of 2x TDD at same bandwidth - In TDD, the guard period (dead air time) increases with distance, reducing efficiency
✓ Less expensive - No need to buy expensive diplexer - Less spectrum needs to be licensed	✓ Lower latency No need to wait for the next period to send data like TDD
	✓ Less complex No complex time synchronisation requirements

Figura 18. FDD vs TDD

Es habitual emplear el LTE-FDD en escenarios rurales similares al objeto de estudio. Además, usando FDD se consigue una menor latencia. Mientras que el espectro TDD se utiliza como un búfer de desbordamiento de capacidad y para aplicaciones que requieren un enorme ancho de banda en el enlace descendente. Esto supone una capacidad nula o limitada en el enlace ascendente, esto es muy común para aplicaciones en núcleos urbanos (ADLittle, 2013).

El primer estudio realizado en Xirio-Online mostraba una distancia de cobertura más que suficiente. Sin embargo, aprovechando los seminarios con gente experta en la materia se realizaron una serie de preguntas en relación con la banda elegida. Clasificando la banda de frecuencias escogida como una banda *indoor* debido a las limitaciones de cobertura que presenta.

En consecuencia, se realizó una investigación más profunda de la materia donde se encontró la siguiente afirmación: “Accordingly, with such parameters, it is possible to transmit a signal at a distance of up to one kilometer in the case of the 2600 MHz band, up to 2.5 km in the 1800 MHz band and more than 3 km for the 900 MHz band.” - (IEEE, 2021). Respaldando la idea del ponente y evidenciando que la banda escogida no resultaría práctica en el escenario real.

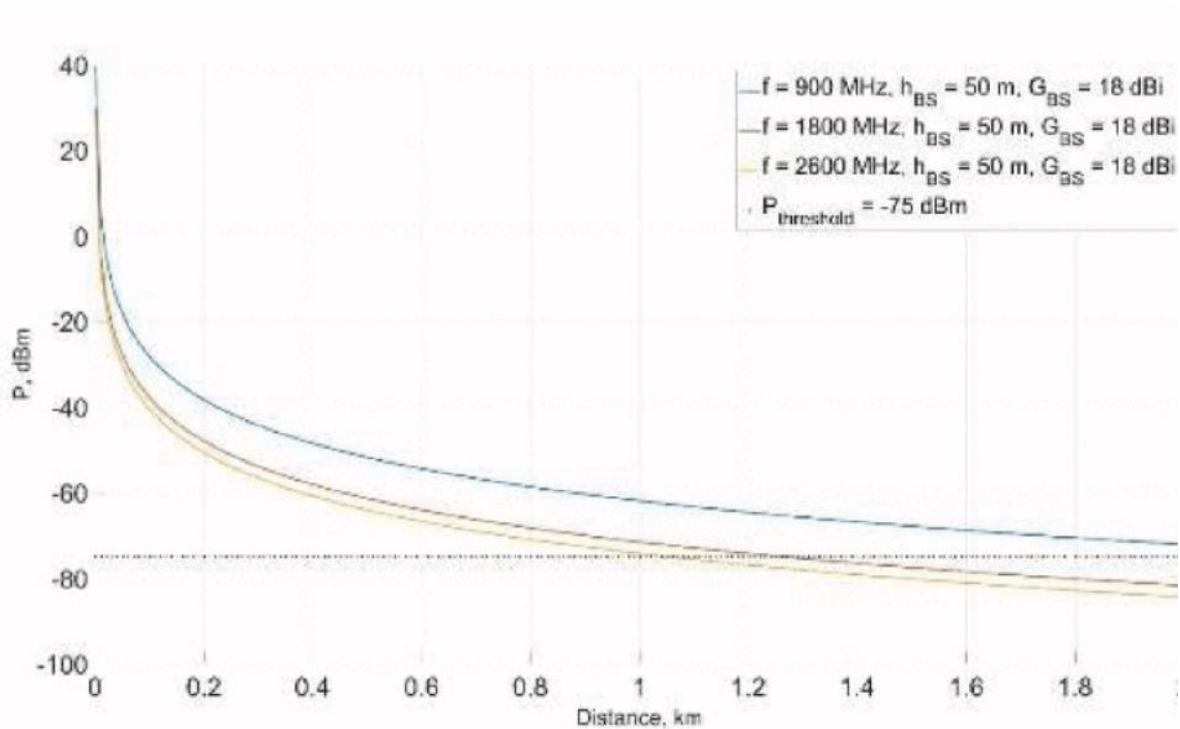


Figura 19. Distancia cubierta por banda

La principal diferencia radica en que, a 800 MHz o 1800 MHz, las ondas de radiofrecuencia pueden viajar mucho más lejos, resultando en un tamaño de celda mucho mayor para una misma potencia, frente a 2600 MHz. Esto implica que se necesitan menos eNodeBs, pero tendría una menor capacidad en la red. En cambio, si se trabaja a una frecuencia de 2600 MHz, resulta en tamaños de celda más pequeños y, en consecuencia, la necesidad de emplear un mayor número de celdas, consiguiendo una mayor capacidad de red.

Para el caso de este informe y de acuerdo con la gráfica anterior, podemos suponer un rango de distancia 1,8 kilómetros al emplear, a diferencia del estudio, antenas directivas para mejorar la distancia de cobertura a lo largo de la autovía. Esta cifra, resulta muy similar a la ofrecida por el experto de infraestructura LTE, cuando se le realizó esta consulta.

Definidos los inconvenientes de la banda escogida se ha supuesto la compra, y no cesión, de dicho espectro de frecuencias para su uso futuro. Se trata de un suceso no realista, por la posición ventajosa de las operadoras actualmente, pero ayudará a estimar los costes de la realización de un despliegue de infraestructura de estas características. De acuerdo, con una subasta realizada por el Ministerio de Industria, Turismo y Comercio (MITYC, 2011):

Bloque	Elegibilidad del bloque	Tamaño del bloque	Cobertura geográfica	Banda de frecuencias	Rango de frecuencias	Precio de salida	Precio del bloque	Ganador del bloque
B2	3.400	2x4,8MHz	Estatal	900 MHz	Bloque abstracto	169.000.000,00 €	169.000.000,00 €	Telefónica
D1_R11	4	2x10MHz	Extremadura	2,6 GHz	Bloque abstracto	235.473,36 €	SIN PUJA	N/A
E1_i	100	10MHz (TDD)	Estatal	2,6 GHz	Bloque abstracto	5.000.000,00 €	5.201.500,00 €	FranceTelecom
E1_ii	100	10MHz (TDD)	Estatal	2,6 GHz	Bloque abstracto	5.000.000,00 €	5.253.515,00 €	Vodafone
E1_iii	100	10MHz (TDD)	Estatal	2,6 GHz	Bloque abstracto	5.000.000,00 €	5.151.505,00 €	Vodafone

Figura 33. Precio salida Subasta Espectro 2011

Su precio de salida a subasta fue de 5.000.000 € y su precio de compra por la empresa Vodafone fue de 5,253 millones de €. De nuevo, se trata de un supuesto ficticio de venta del espectro y ofreceremos la misma cantidad de dinero, ajustada al IPC de España desde su compra (INE, 2024)

Indice	Porcentaje(%)
Nacional	28,9
Andalucía	28,7
Aragón	28,4
Asturias, Principado de	27,4
Balears, Illes	29,8
Canarias	25,9
Cantabria	28,9
Castilla y León	30,5

Figura 20. Variación IPC desde 2011

Realizado este ajuste, se ha estimado un escenario de venta ideal por un valor de 6.855.165€ para adquirir dicho espectro.

4.1 Estudio del entorno

En este apartado se van a analizar las características necesarias para ofrecer un servicio de red 4G para la detección de señales de tráfico en un tramo de carretera.

Este servicio a tiempo real necesita una latencia muy baja ya que no se puede permitir que el coche se salte ninguna señal de tráfico. La latencia es el tiempo que tarda un paquete de datos en ir desde el dispositivo móvil al servidor y volver.

En primer lugar, debemos evaluar cada cuanto distancia hay una señal de tráfico en un tramo de autovía, con el fin de poder realizar una estimación de los fotogramas necesarios a capturar por la cámara del coche para poder identificar todas las señales. Para ello, vamos a consultar la normativa oficial de la Norma 8.1-IC (Gobierno de España M. d., 2014).

En el apartado de “autovías”, podemos observar datos referentes a la distancia entre las señales, en tramos de autovía las señales están separadas de media 200 o 400 metros unas de otras. Calculando la distancia media entre las señales obtenemos:

$$\overline{distancia} = \frac{200+400}{2} = 300m \quad (5)$$

Por lo tanto, de media encontramos una señal cada 300m-200m. Además, consideraremos señales colocadas a 100m de forma ocasional como el peor de los casos, por lo que basaremos nuestro estudio en estos dos valores. Para ello elaboraremos la siguiente tabla en la que calcularemos el tiempo que tarda un coche en llegar de una señal a otra con distintos valores de velocidad (Nota, las cuentas se han truncado al valor superior):

VELOCIDAD	Distancia de 300m	Distancia de 200m	Distancia de 100m
80 km/h -> 23 m/s	13 seg	9 seg	5 seg
90 km/h -> 25 m/s	12 seg	8 seg	4 seg
100 km/h -> 28 m/s	11 seg	7 seg	4 seg
110 km/h -> 31 m/s	10 seg	7 seg	4 seg
120 km/h -> 34 m/s	9 seg	6 seg	3 seg
130 km/h -> 37 m/s	8 seg	6 seg	3 seg
140 km/h -> 39 m/s	8 seg	5 seg	3 seg

Tabla 1. Cálculo de distancias y tiempo entre señales de tráfico de una autovía.

Teniendo en cuenta una velocidad media de 120km/h y una señal cada 200 metros aproximadamente, tenemos 6 segundos de margen para actuar.

En el peor de los casos (pese a dar cobertura a todo el tramo de carretera), se va a considerar una latencia de red muy elevada (100-200ms). Si se suma el procesamiento de las imágenes por el servidor (~100ms), un tiempo de reacción del vehículo de 200ms y se establece un margen de seguridad de 50ms, se consigue detectar una señal y efectuar la acción correspondiente en algo más de medio segundo, consiguiendo así el objetivo marcado de detectar señales cada 6 segundos (o 3 segundos en el peor de los casos). Además, disponemos de un gran margen por si en algún tramo se encuentran señales más juntas o incluso señales nuevas de obra. Este valor calculado de latencia en el peor caso sería un retraso significativo pero manejable para señales que no requieran una reacción inmediata como es el caso del tramo de autovía.

Observando los resultados obtenidos podemos optar por coger el peor de los casos y considerar que las señales están separadas 3 segundos, teniendo mucho margen para poder mandar varios *frames* por segundo. Por ende, vamos a optar por mandar 5 *frames* por segundo desde el propio vehículo.

Optaremos por seleccionar una cámara genérica que suele ser empleada frecuentemente en vehículos para captura de imágenes, es decir, una *dashcam* (o *dashboard camera*), este tipo de dispositivos van generalmente montados sobre el salpicadero del vehículo y se encargan de capturar imágenes referentes a la carretera. A partir de sus parámetros podemos obtener una aproximación del tamaño de imágenes generadas.

En concreto las cámaras que emplearemos serán de la familia MiFleet, este tipo de cámaras están específicamente diseñadas para ser montadas sobre vehículos, a partir de sus *datasheets* podemos extraer los parámetros que nos interesen (MiFleet, 2024).



Figura 21. Ejemplo de dashboard camera de MiFleet.

Consultando el datasheet obtenemos los siguientes valores:

Video & Audio	Video & Audio Recording	2 Video Channels 2 Audio Channels
	Main stream resolution and frame rate	Driver facing camera = 720P @ 30fps Front facing camera = 1080P @ 30fps
	Largest Resource	Main stream 1080P @ 30 frame + 720P @ 30 frame, 2 way sub-stream VGA @ 30 frame
	Image Settings	Brightness, color, contrast, saturation, and sharpness adjustment
	Video Coding	H.264 Baseline Profile / H.264 Main Profile Baseline
	Video Compression Rate	Driver facing camera: 500Kbps ~ 6Mbps Front facing camera: 500Kbps ~ 6Mbps
	Audio Compression Standard	G.711U / ADPCM
	Audio Compression Rate	4:1
	General Function	Heartbeat mechanism and multi-level user management
	CBR / VBR	Support
	Audio Input	Support
	Built-in Speaker	Built-in speaker for broadcasting voice messages

Figura 22. Parámetros de Video & Audio del datasheet.

Una vez obtenido los valores, solamente debemos realizar una breve cuenta para poder realizar una estimación del tamaño promedio de las imágenes generadas con la cámara.

$$tamaño_{imagen} = \frac{6 \text{ Mb/s}}{30 \text{ frames/s}} = 200 \text{ kbit/frame} \quad (6)$$

Si queremos enviar una media de 5 frames por segundo, debemos multiplicar por 5 el valor y así obtener la cantidad de Bytes generados por usuario por segundo:

$$carga_{usuario} = 200 \text{ kbit/frame} * 5 \text{ frame/segundo} = 1 \text{ Mbit/segundo} \rightarrow 125 \text{ kB/s} \quad (7)$$

Se ha decidido situar una estación base cada 3 km ya que así se ofrece una cobertura continua y fiable, requisito mínimo para un servicio de conducción autónoma. Además, las celdas adyacentes se superponen ligeramente, garantizando una transición suave de señal (*handover*). Empleando antenas directivas se mejora la intensidad de la señal en la zona de interés (carretera), maximizando el rendimiento.

Una de las mejores modulaciones es 128-QAM, que posee 6 bits por símbolo y se emplea para maximizar la tasa de datos y dar servicio a un mayor número de usuarios. Intentar ofrecerla sería malgastar recursos ya que no necesitamos una alta tasa de datos y la cantidad de usuarios máxima a la que podemos dar servicio simultáneo está limitada por el RRU de las estaciones base. Por lo tanto, nuestro servicio se centrará en garantizar una modulación mínimo de 16-QAM en todo el territorio, que además es bastante más robusta. Para ello se consideran factores como la calidad de la señal o SINR y la intensidad de la señal o RSRP.

En el tramo de carretera estudiado apenas hay obstáculos que interfieran mucho con la señal, sumado a un despliegue pensado para tener estaciones base cada 3 km, resultan en buenas condiciones para que la calidad de la señal se mantenga en el rango deseado.

Una vez garantizados estos valores, la capacidad de usuarios total ofrecida se calcula primero multiplicando el ancho de banda disponible tanto en *Down-link* como en *Up-link* (10MHz cada uno) por la eficiencia espectral de la modulación 16-QAM (4 bps/Hz) que, considerándola real, será de un 70% aproximadamente (es decir que empleamos el 70% para mandar carga útil). Además, con MIMO 2x2 se puede duplicar esa tasa de bits efectiva.

La cantidad de datos del *Down-link* es tan baja que prácticamente siempre va a tener servicio, no obstante, para el *Up-link* tendremos que mandar imágenes que pesan mucho más, así que procedemos a calcular la capacidad de subida y el número de usuarios totales simultáneos a los que podríamos dar servicio. Por lo tanto, la tasa de transferencia (*throughput*) será:

$$throughput = 10MHz \times \frac{4bps}{Hz} \times \frac{70}{100} \times 2 = 56 Mbit/s \rightarrow \mathbf{7 MB/s} \quad (8)$$

Por lo que podremos dar servicio de forma simultánea:

$$usuarios = \frac{7000 kB/s}{125 kB/s} = \mathbf{56 usuarios} \quad (9)$$

Es decir, podemos dar servicio de forma simultánea a 56 usuarios en un tramo de 1.5Km que cubre cada antena directiva, por lo que la estimación de tráfico se puede calcular de forma aproximada como:

$$separación_{vehiculos} = \frac{1500m}{56 usuarios} = \mathbf{26.79 m/usuarios} \quad (10)$$

En otras palabras, en un tramo de autovía, si hubiera 566 usuarios o lo que es lo mismo, vehículos separados 27 metros, el sistema será capaz de dar servicio a todos ellos de manera simultánea. No obstante, debemos tener en cuenta otras limitaciones como el número máximo de usuarios que puede soportar la antena.

Dado que los vehículos se mueven a gran velocidad y atraviesan varias celdas gestionadas por diferentes eNodeBs, el tipo de *handover* escogido para el servicio es el Inter-eNodeB *Handover*. La capacidad de cambiar de celdas gestionadas por diferentes eNodeBs de forma eficiente va a permitir mantener una conexión estable y de gran calidad, crucial para servicios a tiempo real.

Los valores de BLER objetivos para canales de datos o canales de control son, respectivamente, 5-10% y 1-2%. Estos valores son tan bajos ya que se está ofreciendo una aplicación crítica como es la detección de señales de tráfico en tiempo real. Se tiene un BLER inferior para los canales de control debido a que es clave para garantizar la fiabilidad y la continuidad del servicio. Con esto se asegura que las señales de control y gestión de red sean precisas y robustas, evitando fallos de señalización que podrían desembocar en una pérdida de conexión, cosa que no es viable.

A continuación, realizaremos un breve estudio a mayores del tráfico medio presente en ese tramo de autovía durante los años 2019, 2020, 2021 y 2022, también conocido por el Ministerio de Transporte como el IMD (Intensidad Media Diaria de tráfico en carretera).

Para poder obtener los datos de la autovía de interés, podemos acceder a través de los servicios web del gobierno de España, los datos recogidos se muestran a continuación en dos tablas, una primera para vehículos ligeros como turismos y motos (IMD Ligeros), y una segunda para vehículos pesados como camiones (IMD Pesados). Aunque no daremos servicio a vehículos pesados, por lo que carecen de importancia estos datos, puede resultar de interés considerarlos para obtener una idea general del tráfico, para ello hemos dividido el tramo en 8 tramos. (Nota: debemos tener en cuenta que en los años 2020 y 2019 estuvo presente una pandemia por lo que los datos varían considerablemente) (Gobierno de España M. d., 2024):

AÑO/Tramo	Tramo 1	Tramo 2	Tramo 3	Tramo 4	Tramo 5	Tramo 6	Tramo 7	Tramo 8
2022	12839	10842	9867	9016	13451	10361	9545	11422
2021	11175	8905	8393	7524	8229	8907	8571	9363
2020	6363	5487	7605	5224	5116	5605	6365	7032
2019	10991	10134	8819	9035	8396	9479	10188	11000

Tabla 2. Intensidad Media Diaria (IMD) de vehículos ligeros en la autovía A-62.

AÑO/Tramo	Tramo 1	Tramo 2	Tramo 3	Tramo 4	Tramo 5	Tramo 6	Tramo 7	Tramo 8
2022	4220	4157	4338	4400	4717	4251	4151	4178
2021	3672	3866	3690	4167	3996	3997	3986	3835
2020	2691	2981	2345	3558	3687	3425	3336	3594
2019	3526	3878	3362	3991	3946	3995	3989	4054

Tabla 3. Intensidad Media Diaria (IMD) de vehículos pesados en la autovía A-62.

Graficando dichos valores obtenemos los siguientes resultados:

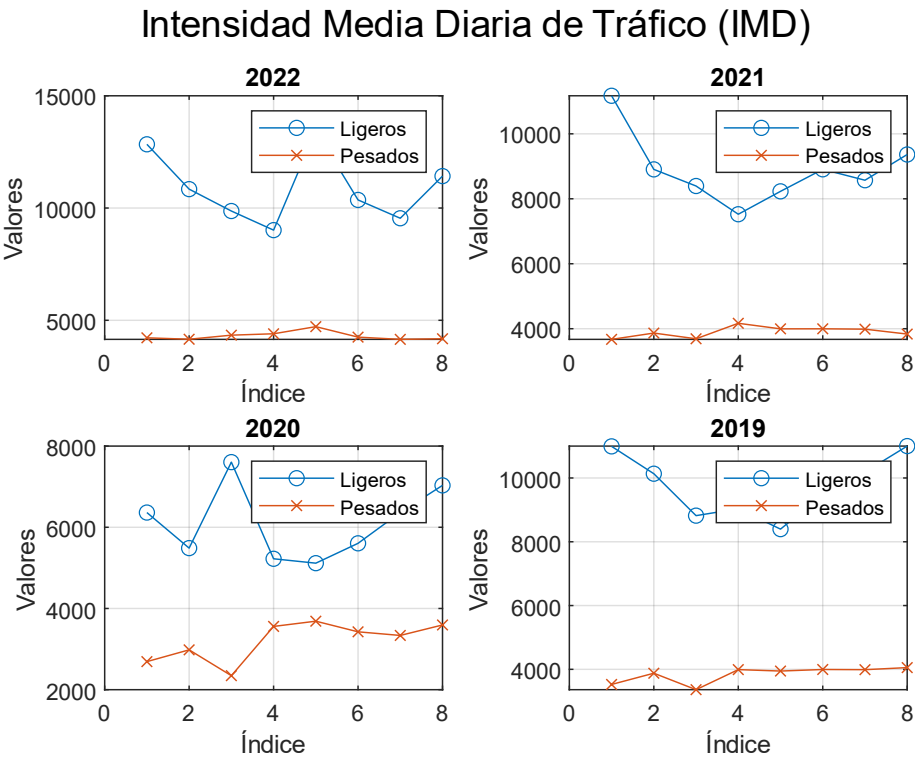


Figura 23. Gráficas con los resultados del tráfico.

Con estos datos, trataremos de realizar una pequeña regresión para intentar predecir los valores de tráfico en 2025 y así poder basar nuestro dimensionamiento en valores concretos (debemos tener en cuenta que la regresión se ha realizado con muy pocos datos y hay cierto sesgo debido a la presencia de la pandemia durante 2019-2020).

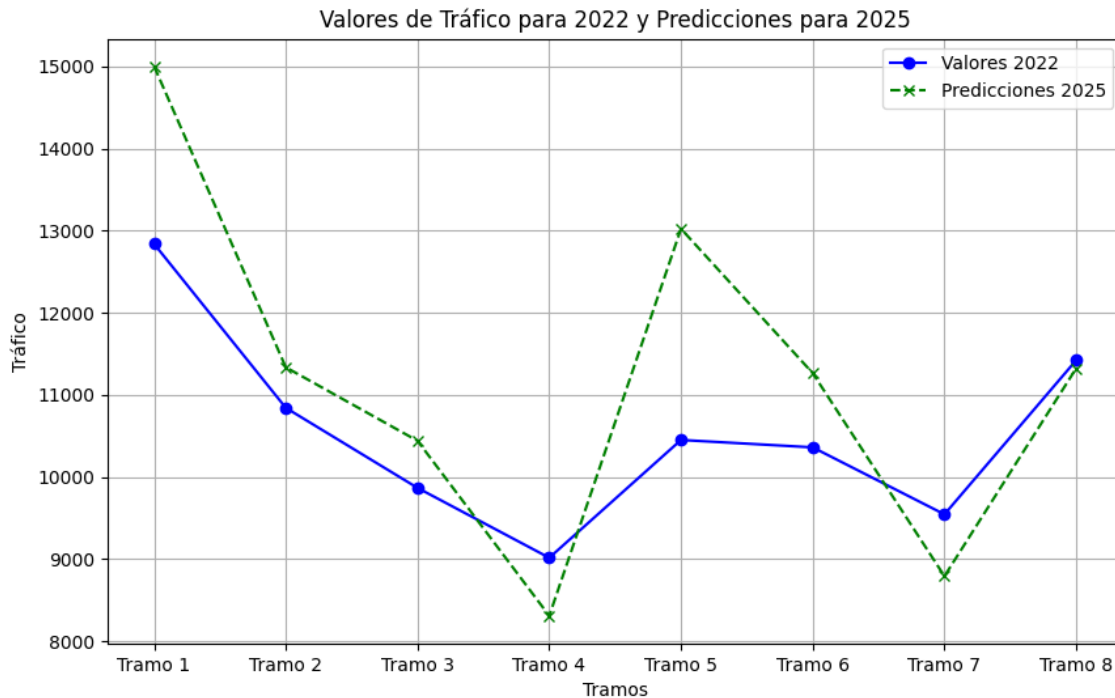


Figura 24. Predicción valores de tráfico 2025.

Como se puede apreciar los valores se mantendrán a excepción de dos tramos de carretera que aumentarán ligeramente la cantidad de tráfico.

4.2 Cálculo de cobertura

El software Xirio-Online se trata de una herramienta de simulación y planificación empleada en el ámbito de las telecomunicaciones. Esta, ofrece servicios gratuitos en los que encontramos diversas funcionalidades que permitirán facilitar la tarea de diseño y evaluación de redes inalámbricas, incluidos los servicios LTE. Para ello, esta herramienta incorpora de serie de datos topográficos como el relieve en mapa pudiendo incluso contemplar obstáculos como la edificación en ciudades y vegetación en el campo, si se tratase del servicio de pago.

La realización de estudios gratuitos ayuda a bocetar diseños de redes de telecomunicaciones al permitir simular la disponibilidad del servicio con una precisión aceptable. Si se quisiese profundizar en este estudio, se puede contemplar el uso de los servicios de pago debido a que estos permiten una mayor resolución ajustando el modelo simulado aún más a la realidad. Este software, además cuenta con una interfaz sencilla de usar para realizar dichas simulaciones sin precisar y unos conocimientos excesivamente elevados de radiofrecuencia. Proporcionando herramientas gráficas para la visualización de dichos estudios y análisis de los resultados. Otro punto fuerte de este programa es que al tratarse de una herramienta en línea permite el acceso desde cualquier lugar facilitando la colaboración entre equipos permitiendo compartir proyectos y resultados de forma sencilla. Por ello, se ha decidido emplear esta herramienta para llevar a cabo los estudios siguientes.

Previo a la realización del estudio es necesario conocer los emplazamientos de estaciones base existentes, lo cual nos permitiría por medio de un contrato ahorrar costes referentes a nuestra infraestructura, así como una puesta en servicio más rápida. Esta información se ha recogido en la página web del Gobierno⁵. En la cual se incluían cada una de las torres de comunicaciones GSM que alojan estaciones base LTE presentes en el territorio nacional. Para las cuales se definía su altura, proveedor de servicio, ubicación de la torre de celosía y códigos internos para definir dicha estación.

Para nuestra aplicación haremos uso de estas estaciones base ya construidas por las proveedoras de servicio como Vodafone o Telefónica, con el fin de ahorrar costes en la creación de infraestructura propia requerida para la aplicación de vehículo conectado.



Figura 25. Información ofrecida en InfoAntenas.

Para contemplar dichas estaciones en este estudio inicial del despliegue, ha sido necesario definir los emplazamientos en los cuales se encuentran cada una de ellas y, por tanto, sus torres de celosía o postes correspondientes. Este será el punto de partida a la hora de realizar el despliegue y contemplar dónde pueda ser necesario realizar la nueva construcción de torres, en el tramo definido para el uso de dicha aplicación.



Figura 26. Emplazamientos definidos en XIRIO.

⁵ Disponible en: <https://geoportal.minetur.gob.es/VCTEL/vcne.do>

Desde el gestor del catálogo se definirá la banda de frecuencia a emplear en nuestra aplicación, tratándose de la banda 7 para el servicio de comunicación por radiofrecuencia. Para ello, se abrirá el gestor de catálogo de la plataforma Xirio-Online. Dentro de este catálogo se pueden definir diferentes elementos entre ellos las bandas de frecuencia a emplear en los siguientes estudios. Sin embargo, en el desglose de frecuencias que vienen definidas de serie en el software, no se ha encontrado la banda prevista para este despliegue. Por ello, ha sido necesario añadir los parámetros de la banda de forma manual creando así una nueva banda de frecuencia de uso para realizar el correspondiente estudio.

Parámetros de la banda

Ancho de canal / Separación entre portadoras:
10
MHz

Ordinal del primer canal:
1

Tramo inferior:

Frecuencia inicial:
2540
MHz

Frecuencia final:
2550
MHz

Frecuencia primera portadora:
2545
MHz

☒ Tramo superior:

Frecuencia inicial:
2660
MHz

Frecuencia final:
2670
MHz

Frecuencia primera portadora:
2665
MHz

Canales prohibidos:

Canales prioritarios:

Introduzca una lista de canales separados por comas y/o intervalos de canales (Ejemplo: 2, 2', 5-7, 12'-21').

Figura 27. configuración Banda de Frecuencia.

Se ha definido para la aplicación un ancho de canal de 10 MHz. También se han establecido los rangos de frecuencia en los tramos inferiores, que hacen referencia a la *Up-link*, y en los tramos superiores, que se refieren al *Down-link*. Así como la frecuencia de la primera portadora en cada uno de estos dos casos.

La antena empleada dispone de una ganancia superior a la antena genérica ofrecida en la herramienta Xirio-Online. Por ello, se ha realizado alguna modificación en la configuración inicial de la antena que se define por defecto en este tipo de aplicaciones. Se trata de una antena también directiva, pero de una ganancia superior con un valor de 18.3 dBi. Ha sido necesario definir las frecuencias inicial y final a la que radiará tal antena, así como el *tilt* eléctrico. Se ha decidido emplear un *downtilt* eléctrico de 5° para el estudio debido a que esta antena se ajustará automáticamente en función de la inclinación eléctrica necesaria, pudiendo variar entre 3° y 13° y ajustándose de forma automática, consiguiendo así modificar el diagrama de radiación de forma dinámica (Eloy_Gonzalez, 2022).

Propiedades del Diagrama de Radiación

Propiedades	
Tipo de diagrama:	☒ Copolar ☐ Xpolar
Polarización:	Sin especificar ▼
Ganancia:	18.3 dBi
Frecuencia inicial:	2660 MHz ▼
Frecuencia final:	2670 MHz ▼
Tilt eléctrico:	5 °
XPD 90:	30 dB
Ancho de haz:	57 °
Rel. delante/atrás:	27.8 dB

Figura 28. configuración Diagrama de Radiación.

Podemos observar que el diagrama de radiación ofrecido por esta antena base es muy similar que encontramos en la antena elegida, por tanto, estos no se han visto modificados a la hora de realizar nuestro estudio.

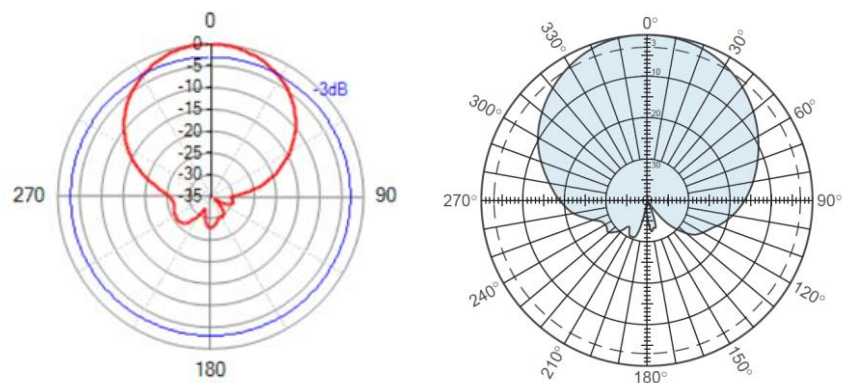


Figura 29. Diagrama de Radiación Horizontal.

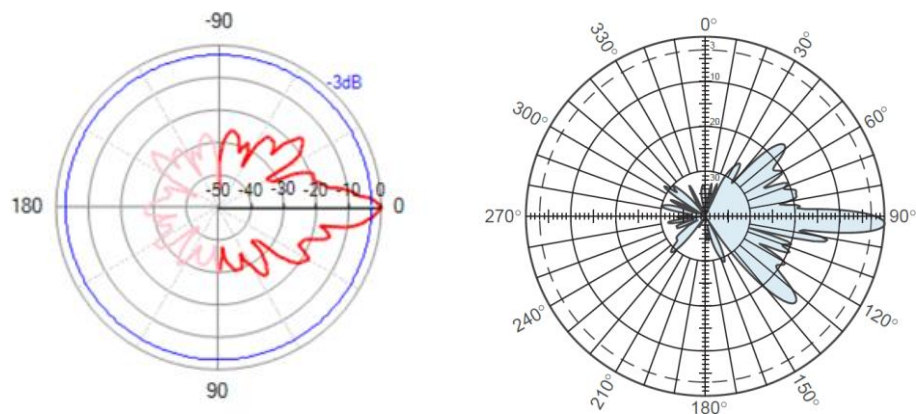


Figura 30 Diagrama de Radiación Vertical.

Una vez se han definido los diagramas de radiación y la configuración de las antenas que se van a emplear en las estaciones base del servicio, es necesario definir los parámetros de la antena UE que se encontrará en el terminal móvil, o sea en el vehículo conectado. Para ello, hay que emplear, de nuevo, el gestor de catálogo de Xirio-Online para definir los parámetros del equipo de usuario final. En los cuales habrá que asignar un nombre al terminal y definir los parámetros de radio. Se ha establecido el uso de una antena omnidireccional con 2dBi de potencia, una altura de 1,5m debido a que estará localizada en la parte superior del vehículo y una potencia de transmisión de 23dBm. También es necesario establecer cuál será su polarización, vertical en nuestro caso, y las frecuencias a las que realizará la transmisión dicha antena. Para esta aplicación, se ha decidido usar la banda 7 por lo que se ha elegido la frecuencia 2545 MHz de subida que aparece en la imagen.

Propiedades del terminal

Terminal ☆

Nombre: Antena Coche

Parámetros de radio

Antena: 3G/4G/5G 2 dBi Omni ↑

Altura antena: 1.5 m

Orientación: 0 °

Inclinación mecánica: 0 °

Frecuencias de transmisión:

Frecuencias
2545.000 MHz

Polarización: Vertical ▾

Feeder:

Longitud del feeder: 0 m

Pérdidas del feeder: 0.00 dB

Pérdidas pasivos: 3 dB

Potencia de transmisión: 23 dBm ▾

Figura 31. Configuración de terminal.

Se ha decidido emplear la sectorización para nuestra red 4G LTE, que se trata de una técnica utilizada para optimizar la cobertura y la capacidad de un número limitado de estaciones base. Esta resulta en la división de un área grande en tramos más pequeños que puedan cubrirse con antenas directivas, permitiendo un mayor alcance. La configuración típica de sectorización en entornos urbanos suele ser dividir el área a cubrir en 3 sectores de 120°. Sin embargo, para nuestra aplicación concreta se han definido 2 sectores, de 180° cada uno. Debido a que cada una de las antenas va a encontrarse en una situación de visión directa frente a alguno de los tramos de la autovía, por los cuales circulará el vehículo conectado de la aplicación propuesta.

El estudio de Xirio-Online proporciona datos precisos para una planificación eficiente, permitiendo maximizar la cobertura y la calidad del servicio. Las áreas con muy baja cobertura experimentan problemas como caídas y baja velocidad de transmisión en los datos, afectando enormemente a la experiencia del usuario. Las áreas rurales y remotas a menudo carecen de una cobertura adecuada debido a la falta de infraestructura existente.

En el primer estudio realizado por nuestro equipo (ver Figura 32), encontramos reflejadas todas estas deficiencias de la infraestructura actual, empleando los establecimientos definidos por el ministerio del gobierno. En el que se encuentran numerosos tramos con cobertura nula o deficiente a lo largo del tramo de autovía que se pretende abarcar.

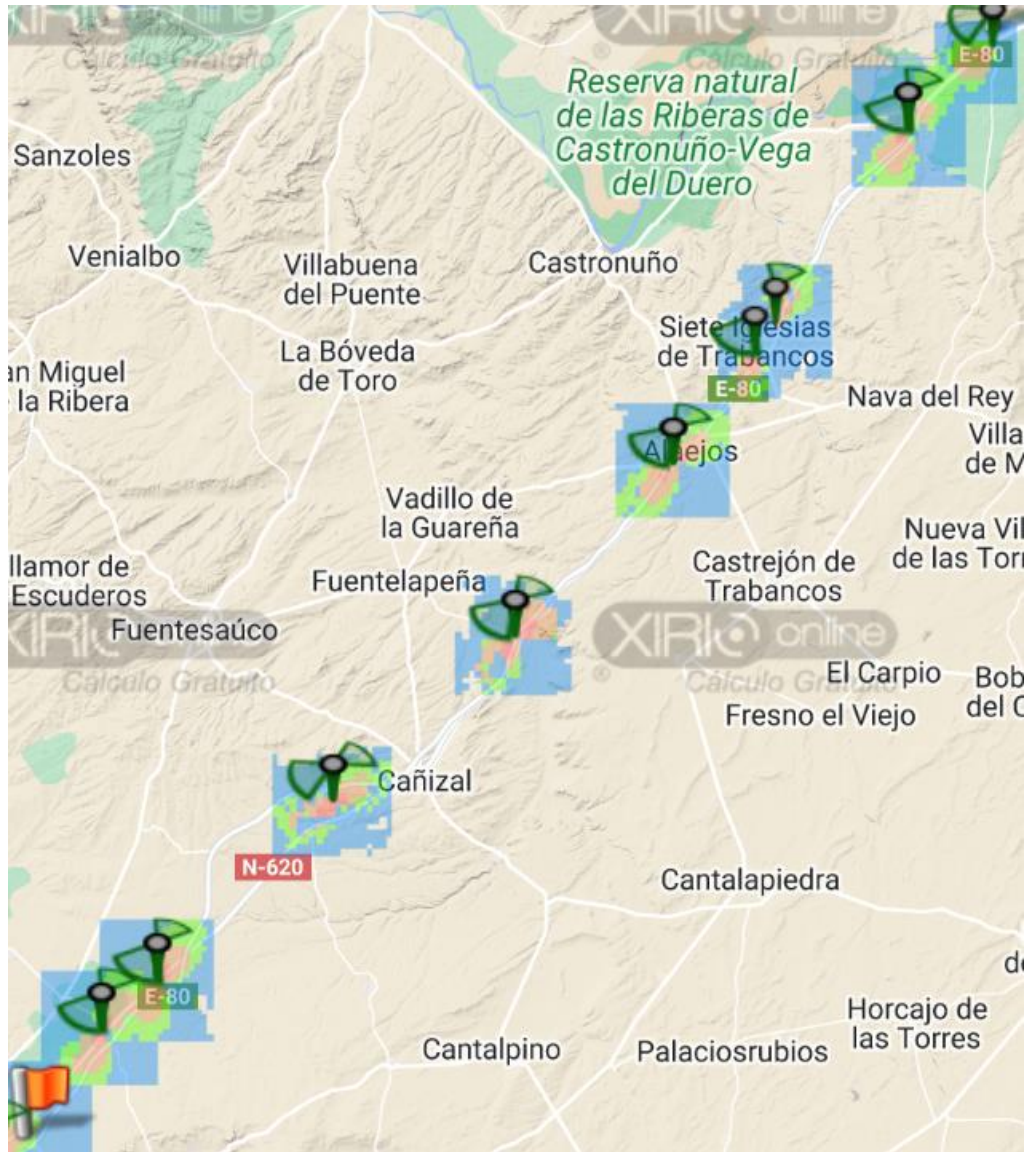


Figura 32. Estudio inicial Xirio-Online.

En base a los datos ofrecidos por el simulador, se manifiesta la necesidad de construir nuevas estaciones base, colocadas estratégicamente, para mejorar la calidad del servicio y garantizar una conectividad fiable a lo largo de toda la carretera. Este despliegue de infraestructura también será fundamental para soportar tecnologías emergentes en el ecosistema de vehículos conectados, como la comunicación vehículo a todo (V2X), que requiere una conectividad robusta para funciones avanzadas de conducción autónoma y sistemas de asistencia al conductor, mucho más avanzados que los contemplados en este informe. Para ello, se va a realizar un desglose más detallado de los tramos donde se encuentran tales deficiencias. Esto implica analizar la topografía y las características del área para garantizar que las ubicaciones seleccionadas sean adecuadas para soportar la infraestructura necesaria. Revisando cada uno de los tramos, en profundidad, mostrados en las siguientes imágenes. Se han determinado puntos en los que será necesario construir nuevas torres de telecomunicación debido a encontrar zonas sombrías a lo largo del recorrido.

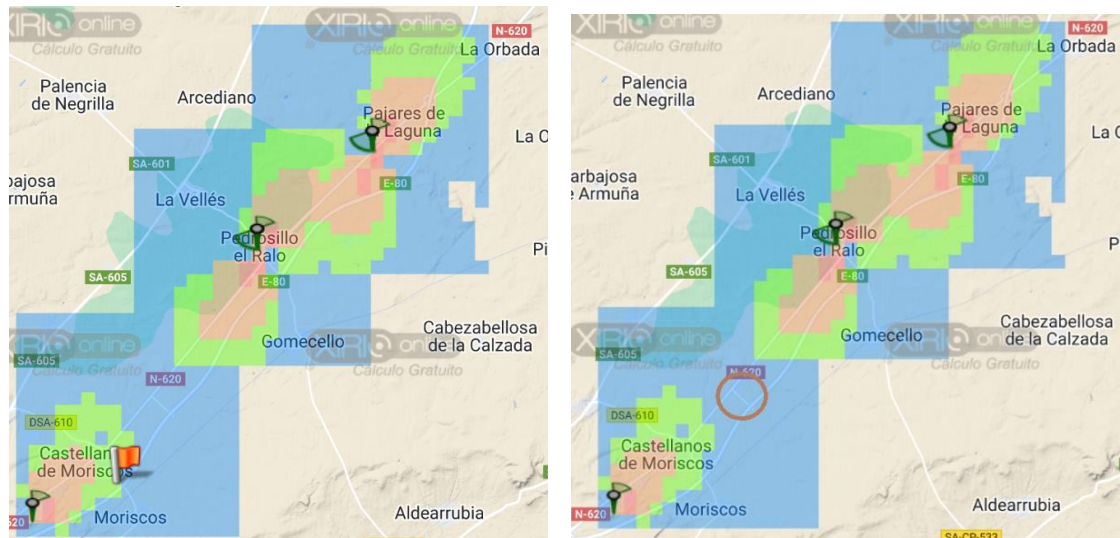


Figura 33. Tramo 1: Moriscos-Pajares.

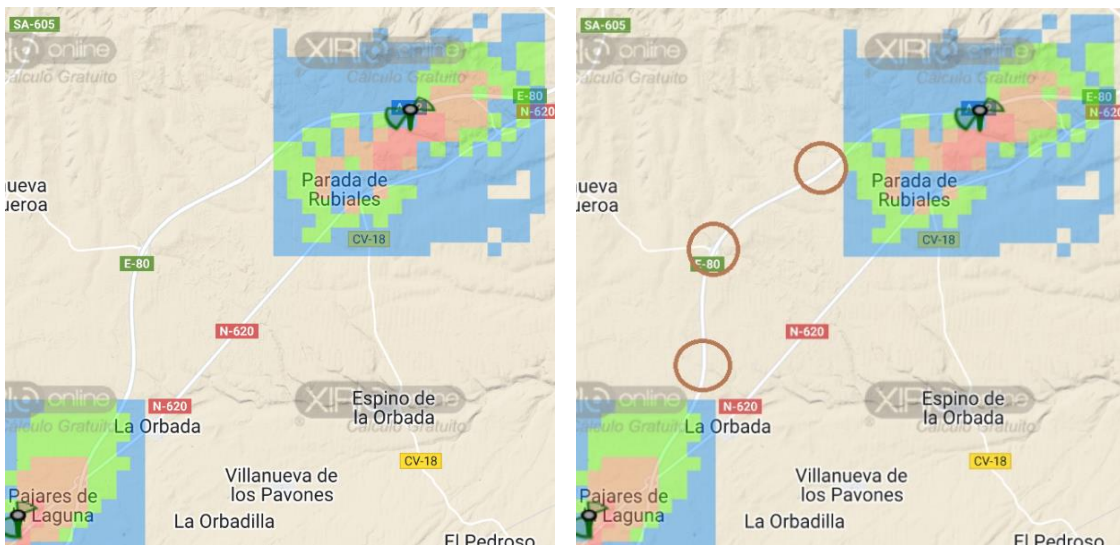


Figura 34. Tramo 2: Pajares-Parada de Rubiales.

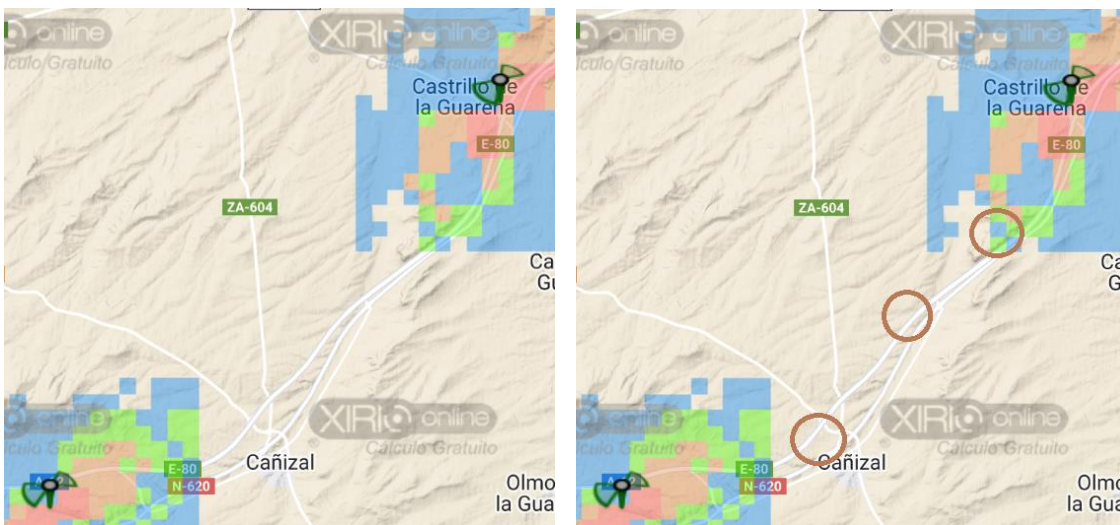


Figura 35. Tramo 3: Parada de Rubiales- Castrillo.

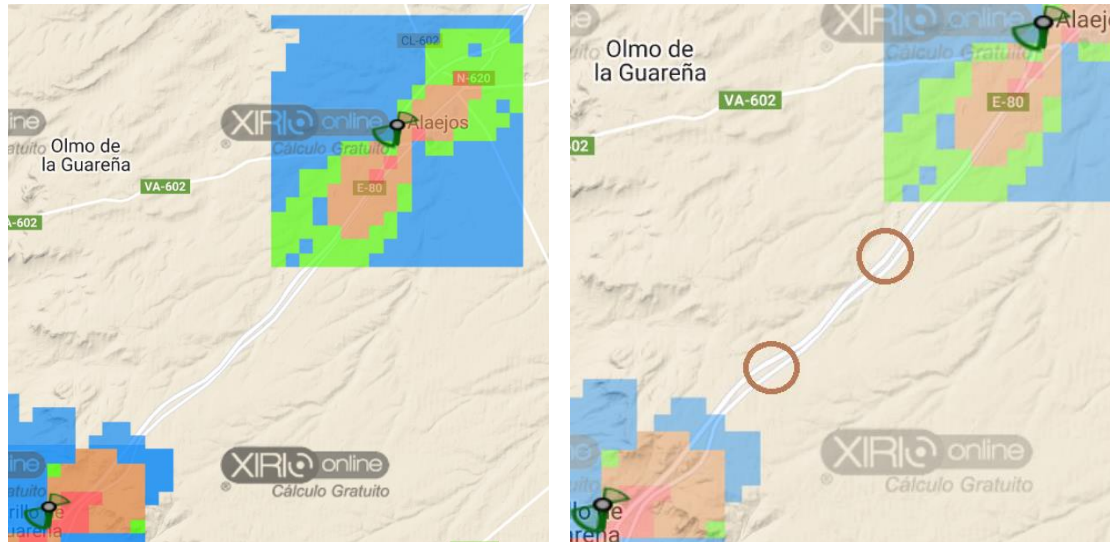


Figura 36. Tramo 4: Castrillo-Alaejos.

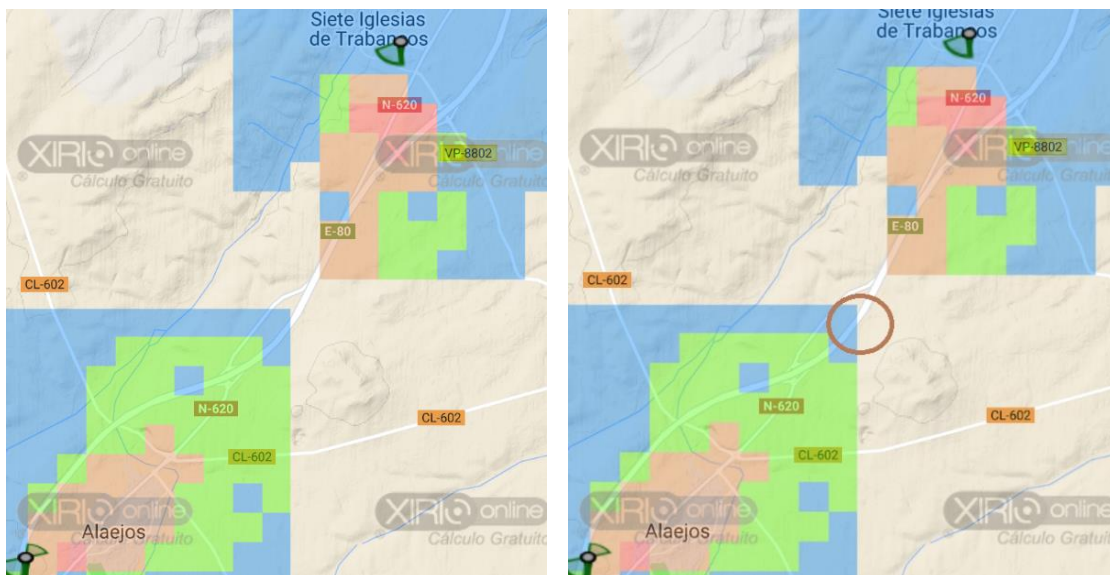


Figura 37. Tramo 5: Alaejos-Siete Iglesias.

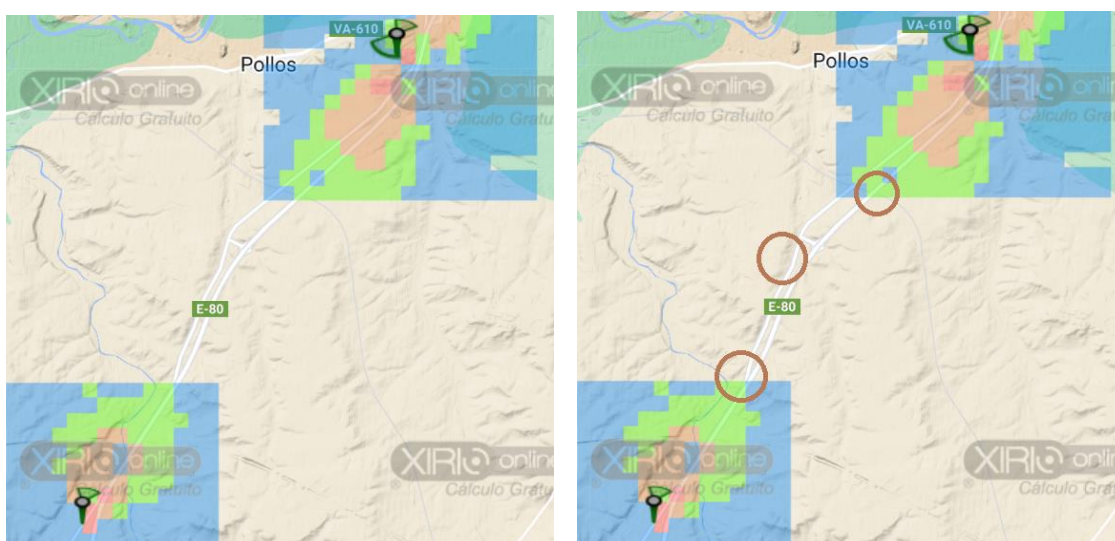


Figura 38. Tramo 6: Siete Iglesias-Pollos.

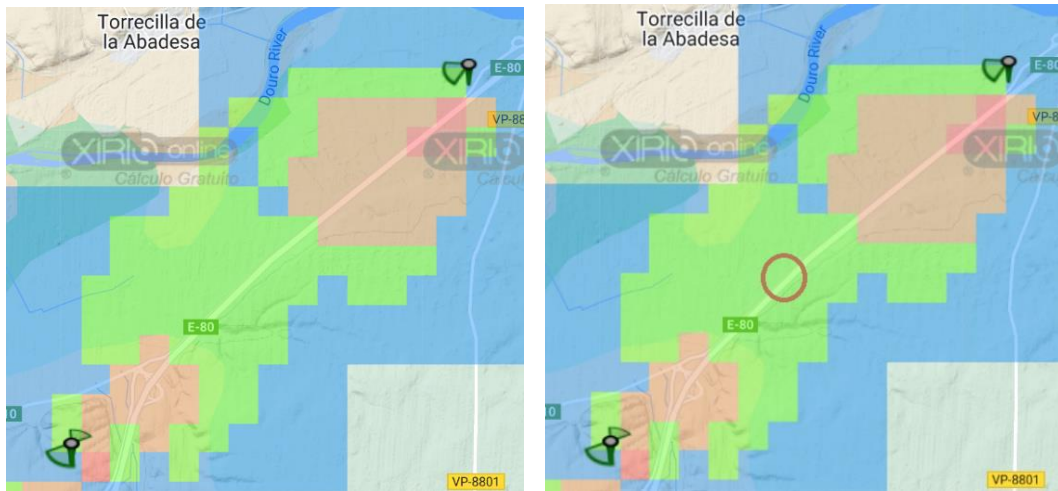


Figura 39. Tramo 7: Pollos-Tordesillas.

Una vez se ha comprendido la relevancia de construcción de nuevas torres de celosía, se ha calculado que sería necesarias 14 torres de telecomunicación de nueva obra. El estudio económico se contemplará más adelante en este informe. Definidos los lugares en los que se construirán dichas torres, se ha realizado un nuevo estudio gratuito en el que sí se tendrán en cuenta. Observamos una notoria mejoría de cobertura a lo largo de todo el recorrido.



Figura 40. Estudio Final Xirio-Online.

En las siguientes imágenes se realizará una ampliación sobre los diferentes tramos de estudio, mostrados con anterioridad en este informe. En ellas, se observa una calidad en la transmisión/recepción LTE elevada siendo el color naranja una buena cobertura y el color verde una cobertura suficiente para llevar a cabo la aplicación deseada.

Con la adición de las 14 nuevas torres, que cuentan con dos antenas directivas sectorizadas, se consigue cubrir todo el tramo propuesto en la autovía A-62 con una calidad de servicio adecuado.

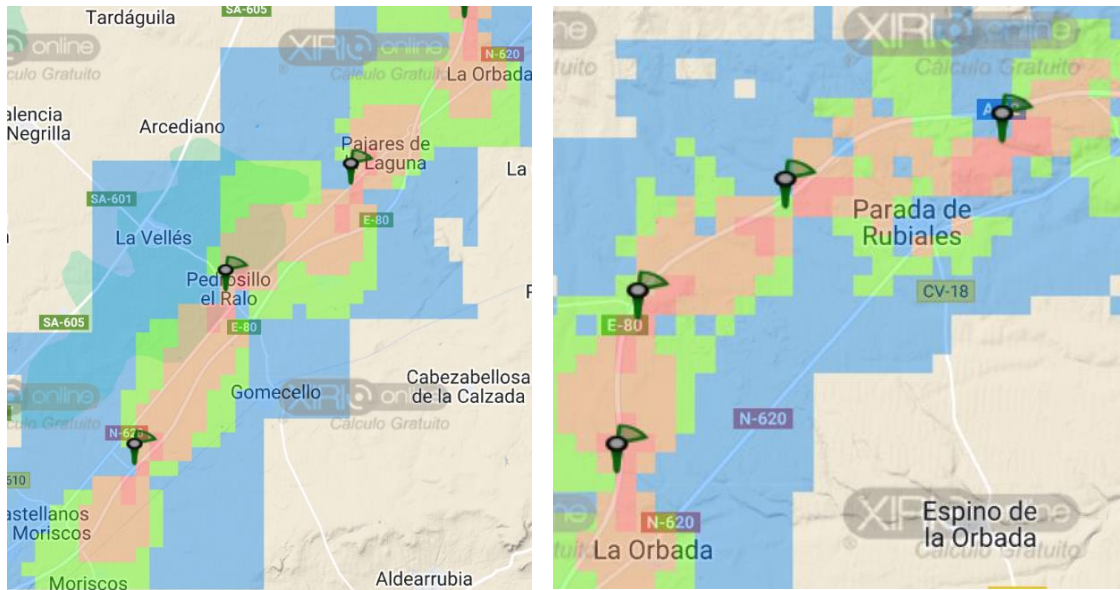


Figura 41. Tramos 1 y 2 con las nuevas estaciones.

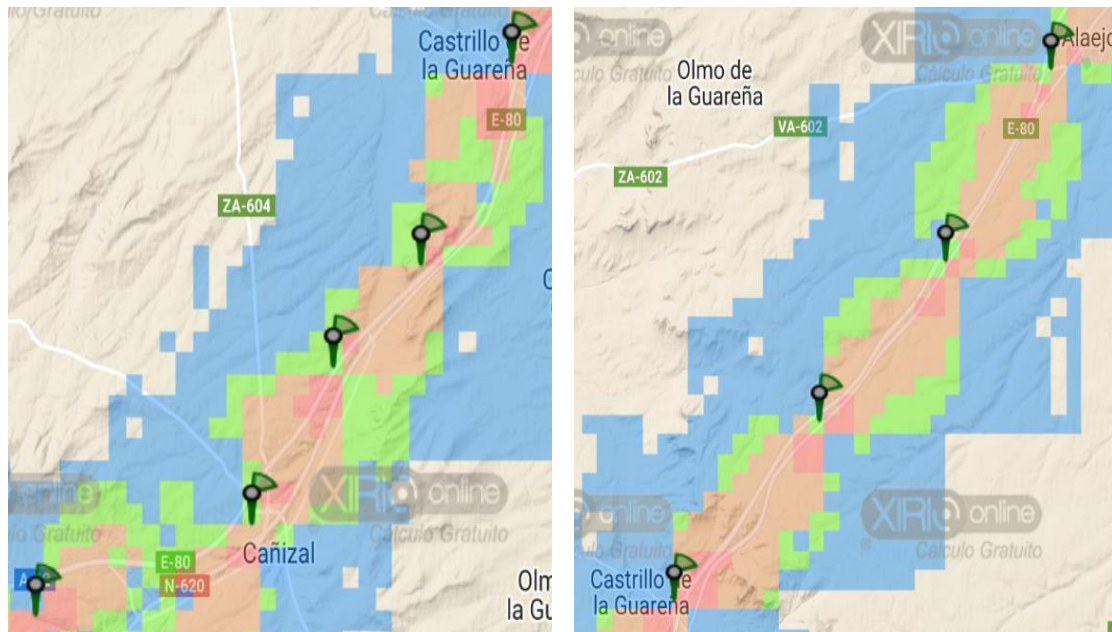


Figura 42. Tramos 3 y 4 con las nuevas estaciones.

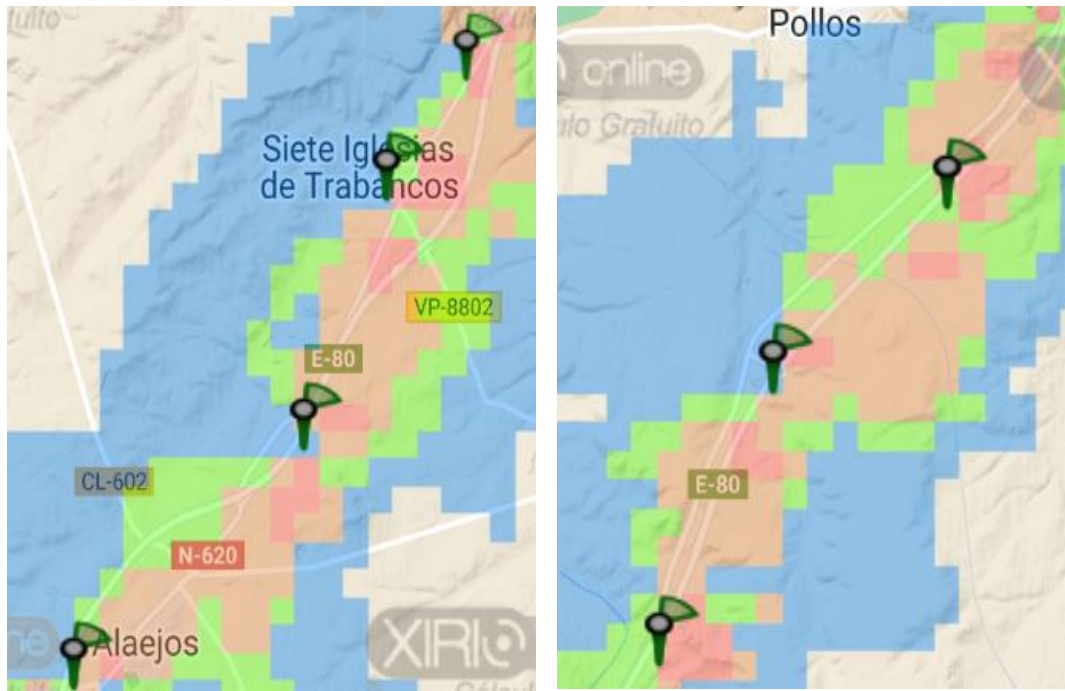


Figura 43. Tramos 5 y 6 con las nuevas estaciones.

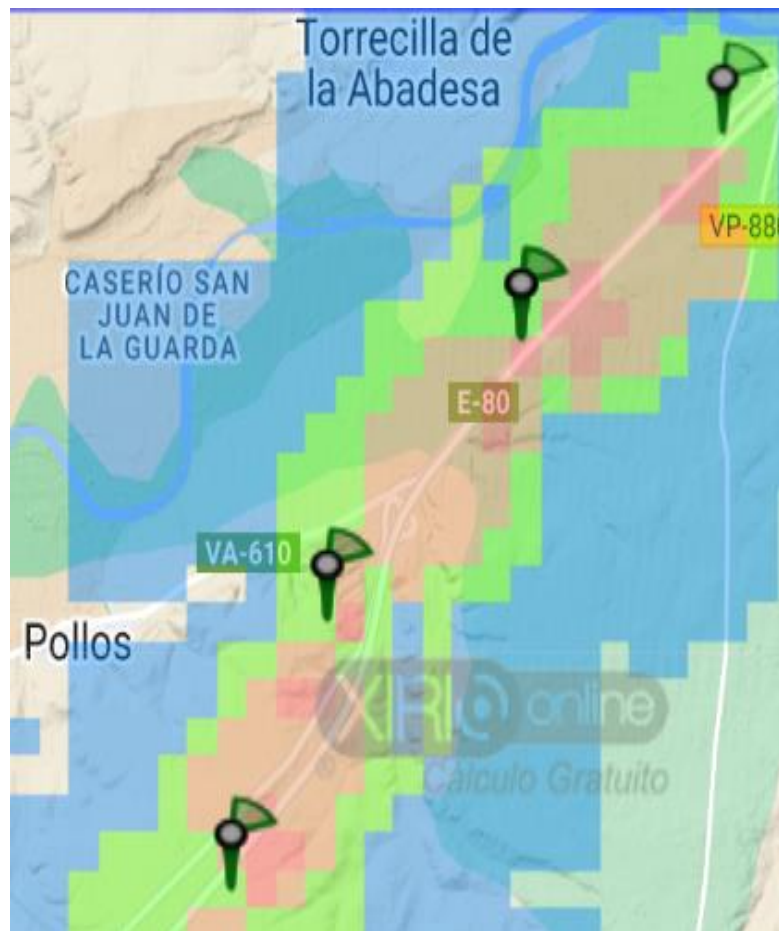


Figura 44. Tramo 7 con las nuevas estaciones.

4.3 Infraestructura Propuesta

Tras realizar el estudio de cobertura 4G LTE, mediante la herramienta Xirio-Online, se ha revelado que existen multitud de zonas con una cobertura de red insuficiente, afectando negativamente a la experiencia de usuario final, y por lo tanto no pudiendo dar servicio en el tramo de autovía definido. Por ello, se han definido una serie de ubicaciones exactas donde construir nuevas estaciones base con las que ampliar el servicio ofrecido. La implementación de nuevas estaciones base LTE requiere de un conjunto integral de equipos de telecomunicaciones diseñados para garantizar una cobertura óptima, alta capacidad de transmisión y una operación confiable. Para ello, es vital seleccionar proveedores confiables y que ofrezcan equipos de alta calidad. La instalación de nuevas estaciones base es un proyecto complejo que requiere una planificación detallada y una ejecución precisa.

A continuación, se presenta una descripción integral de la obra requerida, abarcando los emplazamientos identificados, los componentes de la infraestructura y los pasos necesarios para llevar a cabo la instalación. Dónde se pueden apreciar dos elementos diferenciados. Por un lado, se diferencia el EPC fundamental para la gestión de comunicación y conectividad de los dispositivos móviles en la red LTE, realizando labores de autenticación de usuarios y la interconexión con otras E-UTRAN. Por otro lado, la E-UTRAN encargada de la parte de acceso por radio de la red LTE, desempeñado un papel fundamental en la comunicación entre los dispositivos móviles y la red central, y diseñada para ofrecer mayores velocidades de datos y menor latencia.

4.3.1 Despliegue E-UTRAN

A diferencia de las estaciones base en tecnologías anteriores, los eNodeBs en LTE también realizan tareas de control como la gestión de la movilidad, manejando la conexión de los vehículos según se mueven entre diferentes celdas, y son capaces de realizar planificación de recursos de radio en el enlace. En esta sección, se va a realizar un listado y definición de los componentes y dispositivos que van a emplearse en cada uno de los *Cell Site* que han de construirse, para ser capaces dar cobertura en todo el tramo de autovía.

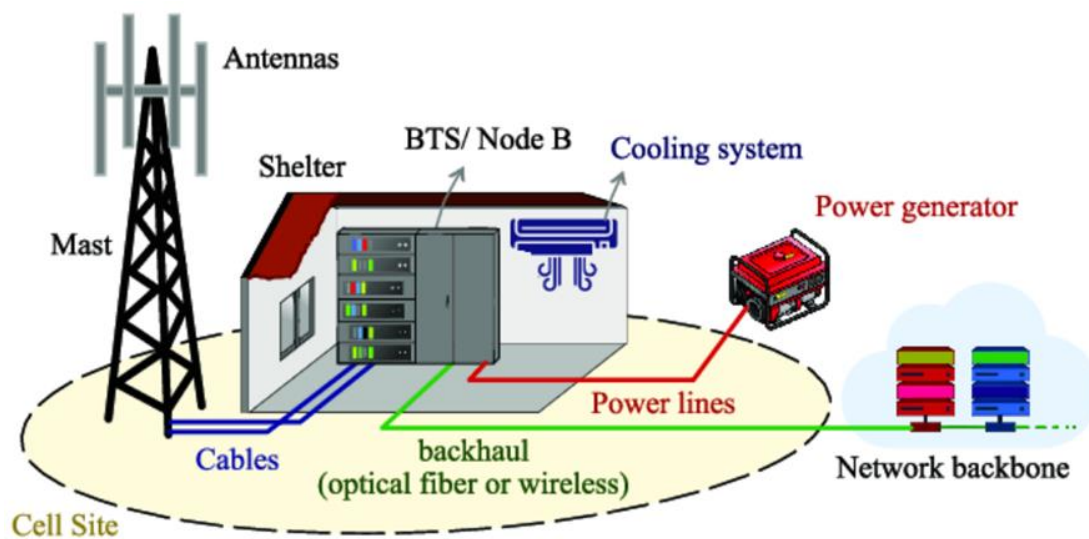


Figura 45. Ejemplo de una red E-UTRAN completa

4.3.1.1 Dispositivos de Radiofrecuencia

Las antenas son componentes fundamentales que permiten la transmisión y recepción de señales de radiofrecuencia. Para este despliegue, se utilizarán antenas sectoriales y capaces de soportar la tecnología MIMO (*Multiple Input Multiple Output*), que mejoran significativamente la capacidad y cobertura de la red a lo largo de la calzada. La antena escogida para la realización del despliegue ha sido el modelo 800-10622V01 de la marca Kathrein Scala Division. Esta se trata de una antena que resulta muy interesante debido a su capacidad para reducir la interferencia y mejorar la eficiencia de la transmisión en una dirección concreta, minimizando sus pérdidas y mejorando la experiencia del usuario.

Se trata de una antena valorada en un precio de 1.050 €, la cual serviría como antena principal en cada uno de los sectores de la estación base. Como se ha visto en apartados anteriores de este informe, se ha decidido realizar una sectorización de 180° resultando en dos sectores por estación base con un haz dirigido de unos 60°. Por lo que, cada E-UTRAN desplegada contaría con dos de estos elementos radiantes.

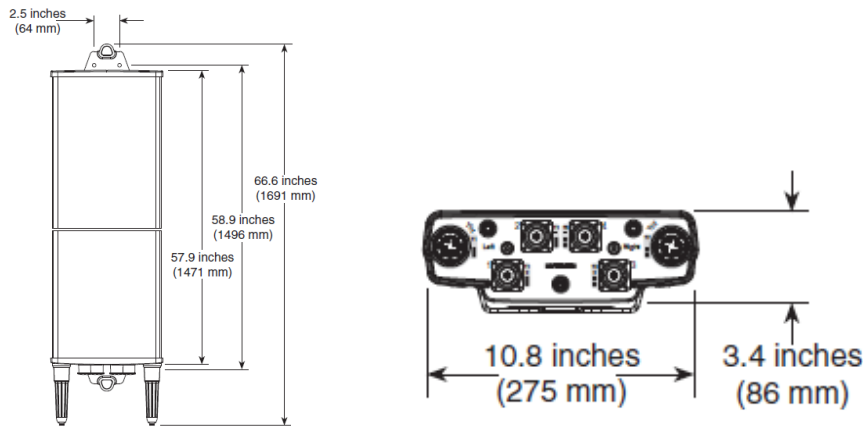


Figura 46. dimensiones Antena Kathrein.

Se trata de un modelo que resulta interesante para nuestra aplicación debido a sus capacidades en la frecuencia deseada para el despliegue futuro. Sus especificaciones permiten una entrada de potencia de hasta 300 vatios conjunto a una ganancia de 18dBi, lo que lo hace un modelo de antena idóneo para recorrer una larga distancia. Sin embargo, como hemos visto anteriormente, la frecuencia que se ha escogido para esta aplicación no es la más indicada cuando se está hablando de alcance de cobertura. Esta antena permite implementar un MIMO 2x2, tal y como se ha realizado en el estudio de Xirio-Online. Con su peso de 15 kg, ya montada, y los materiales de construcción diseñados para exteriores resulta muy interesante para implementar en nuestras torres.

General specifications:		Specifications: 2490–2690 MHz			
Frequency range	1710–2690 MHz	Gain	2°	8°	14° T
VSWR	<1.5:1		18.6	18.9	18.3 dBi
Impedance	50 ohms	+45° and -45° polarization horizontal beamwidth	57° (half-power)		
Intermodulation (2x20w)	IM3: <-150 dBc	+45° and -45° polarization vertical beamwidth	5° (half-power)		
Polarization	+45° and -45°	Front-to-back ratio (180°±30°)	>25 dB		
Connector	4 x 7-16 DIN female	Sidelobe suppression for:	2°	8°	14° T
Electrical downtilt	2°–14° (continuously adjustable) (manual or remote control)	First sidelobe above main beam	≥18	18	17 dB
Isolation, between ports	>30 dB	Within 0°–20° sector above horizon	≥17	17	14 dB
Maximum input power	300 watts (at 50°C) per input	Cross polar ratio 0°	22 dB (typical)		
		Sector ±60°	>10 dB		

Figura 47. Especificaciones Antena Kathrein.

Las RRU's son dispositivos capaces de amplificar y transmitir las señales de radiofrecuencia desde y hacia las antenas, por sí mismos. Estas unidades se instalan muy próximas a las antenas por medio de cableado coaxial. Puesto que es este incurre en perdidas elevadas (comparándolo con la fibra óptica), no es interesante que el elemento radioactivo y la unidad de radio se encuentren muy separados. Por lo que en el despliegue se encontrarían instalas en la misma torre, al estar estas conectadas a la unidad BBU mediante cables de fibra óptica libres de pérdidas.



Figura 48. Unidad de radio.

Se trata de un elemento de radiofrecuencia crucial para la eficiencia energética y la reducción de pérdida de señal, en nuestro sistema. El proveedor escogido ha sido Baicells y su producto elegido NOVA 436Q cuenta con un precio de 5.000 €, del cual se emplearían 1 unidad de radio por cada antena del sector, resultando en 2 RRU's por estación E-UTRAN desplegada. Estos dispositivos cuentan con una potencia de salida de 30dBm y 5,5kg por unidad (Baicells, 2024).

LTE Mode	TDD	
Frequency Bands	48 and partial bands 42, 43	
Channel Bandwidth	5/10/15/20 MHz per carrier	
Max Output Power	30 dBm / antenna	
Power Supply	+/- 48VDC, AC adaptor (multi-national standards)	
Power Consumption	~100W (peak)	
Interfaces	1 optical (SFP) and 1 RJ-45 Ethernet interface (1 GE)	
MIMO	DL: 2x2 on each carrier	
Installation	Pole or wall mount	
Antenna	eNB has N-Type connectors and supports external high-gain antenna(s), either (2) 2-port antennas or (1) 4-port antenna	
Dimensions (HxWxD)	12.2 x 9.4 x 4.1 inches 310 x 239 x 105 millimeters	
Weight	12.1 lbs / 5.5 kgs	
		User Capacity
		32 concurrent users 32+32 in DC mode Future software release: 64 and 96
		QoS Control
		3GPP standard Quality of Service Class Identifier (QCI)
		Modulation
		DL: QPSK, 16QAM, 64QAM, and future software release 256QAM UL: QPSK, 16QAM, 64QAM

Figura 49. Especificaciones RRU.

Por último, el corazón de la estación base el dispositivo Ranger Sierra 800 Fixed LTE Baseband Unit 2700MHz encargado de procesar las señales digitales Siendo necesario un solo dispositivo por estación con un precio de 9.000 € sirviendo como sistema *backhaul* responsable de conectar las unidades de radio de las E-UTRAN con el núcleo de la red, transmisión de datos y señales de control, fundamental para garantizar un servicio de red continuo y de alta calidad.



Figura 50. Unidad Banda Base.

Este dispositivo de la marca Cambium Networks cuenta con multitud de puertos de comunicación, una interfaz sencilla y un consumo reducido. Siendo el elemento final de la red de acceso de cada una de las estaciones base que se van a construir (Cambium_Network, 2019).

SPECTRUM	
Frequency Range	2300 - 2700 MHz, Bands 38, 40, 41 , software upgradeable to any TD-LTE band
Channel Width	5 MHz (future release), 10 MHz, 15 MHz (future release), 20 MHz
INTERFACE	
RRH Interface	Four SFP+ / CPR! v7.0
Ethernet Interface (Copper)	10/100/1000Base T, full duplex, rate auto negotiated, 802.3 compliant
Ethernet Interface (Optical)	1000Base -SX
PERFORMANCE	
Subscribers per Sector	64 (software upgradeable to 256')
Maximum Deployment Range	32 km (20 mi)
Latency	<30 msec typical
Throughput	• Config 1: 68 Mbps DL / 20 Mbps UL • Config 3: 95 Mbps DL / 13 Mbps UL.

Figura 51. Especificaciones BBu.

Todos los elementos de radio mencionados anteriormente resultarían en una distribución como la que se encuentra en el dibujo de la Figura 52. Las antenas se encontrarían colocadas en la parte superior de las torres de celosía, conectadas mediante un cable coaxial de 50 Ohm a las unidades de radio, sin necesidad de instalar un amplificador dedicado. Dichas unidades estarán comunicadas por cables de fibra óptica, de aproximadamente 25m, con el dispositivo BBU que servirá de interfaz frente al EPC desplegado.

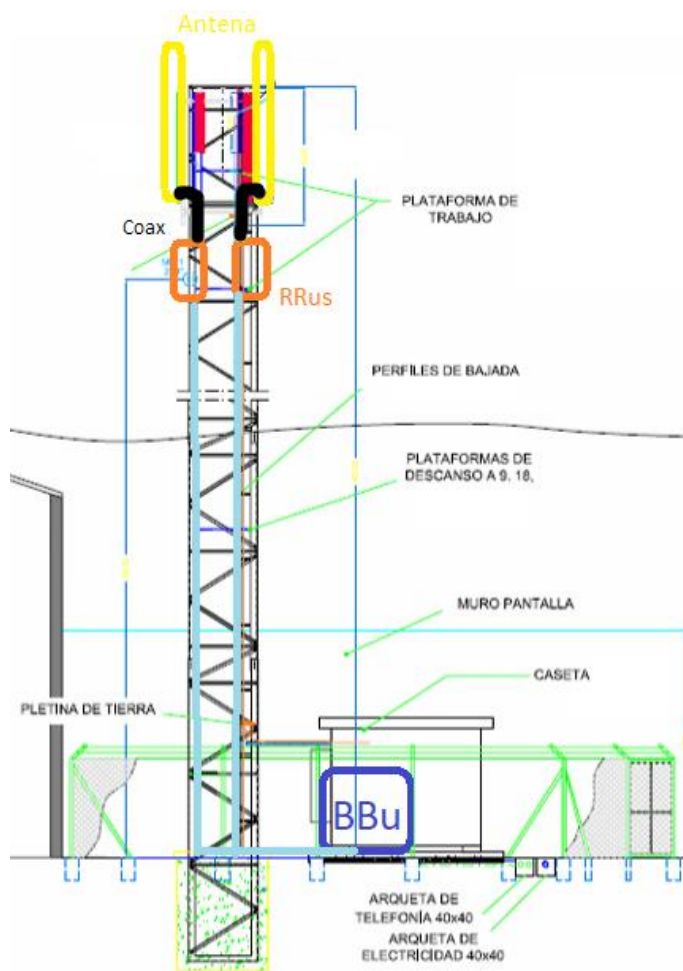


Figura 52. Esquema E-UTRAN simplificado.

4.3.1.2 Dispositivos de funcionamiento

Una vez se han establecido cual serán los elementos de radiocomunicación que van a emplearse en cada E-UTRAN desplegada, han de definirse el resto de los dispositivos que serán necesarios para su correcto funcionamiento. Para ello, es importante realizar una instalación eléctrica correcta siendo esencial para garantizar la seguridad y el buen funcionamiento de los dispositivos eléctricos. Una buena instalación también facilita futuras expansiones y modificaciones del sistema, permitiendo agregar nuevos dispositivos sin comprometer la seguridad ni el rendimiento.

Siendo necesario realizar la acometida eléctrica, que es el conjunto de elementos y dispositivos que conectan la red de distribución de una compañía eléctrica con la instalación eléctrica interna de nuestro emplazamiento. Su principal función será proporcionar un punto de conexión seguro y adecuado desde la red pública. Por lo que será necesario establecer dicho punto de conexión, se deberá implementar una caja general de protección para proteger al sistema de posibles sobrecargas. De igual forma, se ha de definir las líneas de acometida necesarias para llegar al punto de conexión y un contador eléctrico capaz de definir el consumo de la infraestructura. Será necesario, también, contar con un dispositivo que limite la potencia máxima de consumo de la instalación de forma que si se superase esté cortase el suministro eléctrico conocido por las siglas ICP – Interruptor de Control de Potencia. Para finalmente, realizar una derivación individual a la E-UTRAN.

Debido a la complicación de esta y la falta de conocimiento de la materia, no se ha profundizado más. Consultando licitaciones, obras y proyectos públicos se ha estimado un coste de la acometida subterránea de unos 8.000 €, adecuándose al Reglamento Electrotécnico para Baja Tensión e instrucciones Técnicas Complementarias, Real Decreto 842/2002, del 2 de agosto, del Ministerio de Ciencia y Tecnología. Dichos trabajos deberán llevarse a cabo por operarios especializados en conformidad con las normas de seguridad establecidas.

Dado que el sistema BBU requiere de una alimentación polarizada de -48V/48V, como también lo hace la RRU elegida, se ha seleccionado una fuente de alimentación capaz de proveer estos voltajes. La fuente de alimentación que se ha seleccionado es la de la marca UniPower con un precio de 1.500 € es la siguiente: (UniPower, 2022)



Figura 53. Fuente de alimentación.

En especificaciones se observa cómo con un voltaje nominal de 120/240 VAC es capaz de generar una potencia equivalente de 1600W por módulo. Siendo un único módulo suficiente para alimentar todos los equipos de radiofrecuencia del sistema. Sin embargo, es necesario alimentar los demás equipos de la infraestructura. Por tanto, se optaría por emplear un rack que contase con dos de estas fuentes de alimentación.

SYSTEM DESIGNATION		ASPIRO 1U - M00035	
OUTPUT			
System Voltage		-48VDC nominal 53.5V float	
Maximum Power @ 120VAC nom.		1600W	
Maximum Power @ 230/400VAC nom.		2400W	
Maximum Current @ 120VAC nominal	Load	23A @90-100VAC, 25A @ 100-180VAC	
	Battery	25A discharge s/w controlled (deducted from load capacity)	
Maximum Current @ 230/400VAC nominal	Load	45A	
	Battery	45A discharge s/w controlled (deducted from load capacity)	
No. Rectifier Slots		2	

Figura 54. Especificaciones de la fuente de alimentación.

Será también necesaria la incorporación de un ordenador encargado de las monitorización y supervisión del funcionamiento de los sistemas, así como de su inicial configuración. Se ha destinado un coste de alrededor de 1.500 € para montar dicha computadora y los periféricos necesarios para operar con ella. Por lo que será imprescindible contar con un dispositivo que almacene este ordenador, el BBU y la fuente de alimentación. Para ello, se ha escogido un armario de dimensiones reducidas capaz de albergar dichos dispositivos separados en racks por un precio un precio de unos 500 € en la tienda⁶.

⁶ Disponible en: <https://www.pccomponentes.com/armario-rack-19-22u-600x600>



Figura 55. Imagen del armario Rack 600x600.

Todos los componentes mencionados anteriormente implican un consumo eléctrico y por lo tanto una disipación de potencia, la cual puede generar grandes cantidades de calor. Por ello, es imperativo contemplar la adquisición de un sistema de refrigeración. Este resulta crucial para mantener temperaturas operativas adecuadas en entornos donde el calor excesivo puede dañar equipos o comprometer la calidad de servicio. Se ha elegido un sistema de refrigeración industrial de la marca GGMgastro con un precio de 4.500 €.



Figura 56. Sistema de aire acondicionado y sus dimensiones.

Se trata de un sistema de dimensiones reducidas y de un consumo bajo, debido a que la aplicación no requiere de una refrigeración excesiva y la caseta también es de dimensiones reducida. Este dispositivo cuenta con protección frente agua y polvo, ideal para uso externo.

Refrigerante	R452a / 700 g
Volumen máximo de la sala	15 m³
Temperatura	-10 °C hasta -22 °C
Compresor de rendimiento	373 Watt
Valores conectados	380V / 50 Hz
Protección contra el agua	IP 51 (protección contra el agua que cae verticalmente)
Protección contra el polvo	IP 51 (protección contra el polvo en cantidades perjudiciales)

Figura 57. Especificaciones AC.

Debido a que una caída de luz en la infraestructura podría suponer un coste enorme al causar un accidente en la calzada. Se ha decidido contemplar el uso de un generador de emergencia para garantizar el servicio ante un corte de suministro. El equipo que se ha elegido para llevar a cabo dicha función ha sido un generador de emergencia de aplicación industrial capaz de proveer suficiente energía para la infraestructura completa de la estación con una capacidad de hasta 6kWh. Este equipo se trata de un modelo de la marca Hyundai en su línea DHY8600SE-T con un coste de 2299 € que garantiza un voltaje correcto y estable al contar con autorregulación. Está protegido por el estándar IP23 que permite su uso en exteriores de forma segura. Presentando unos consumos de menos de 2,5L/h a máxima potencia (se mide en Litros al emplear Diesel).



Figura 58. Generador de emergencia.

⁷Se ha contemplado también la necesidad de disponer de un depósito de emergencia⁸ de 250L de Diesel con medidas de protección industrial valorado en 1.000 € para garantizar servicio durante un número de horas suficiente supuesta la caída de la red eléctrica. El precio de llenado de este depósito y generador supondría un coste de 505 €.

⁷ Disponible en: <https://megadescuento.es/1432-dhy8600se-t-generator-diesel-insonorizado-full-power>

⁸ Disponible en: <https://www.denios.es/tanque-movil-para-diesel-polymove-450-en-polietileno-gris-azul-10300450251450/214402>



Figura 59. Deposito reserva

Es importante dejar definida una tierra que pretende que cualquier elemento de material metálico y cualquier equipo se encuentre conectado a la red de tierras, tanto los dispositivos ubicados en el interior, como los situados en el exterior. Se definirá una única red de tierras donde todo se encuentre conectado a una barra equipotencial situada debajo del cuadro eléctrico de la infraestructura, unificando todas las masas de la infraestructura, con el fin de asegurar la protección de equipos. Para la instalación de tierra deberá cumplirse con el RBT. El coste de la red de tierras se estima en unos 500 €.

Debido a la altura que presentará cada una de las instalaciones E-UTRAN se ha de revisar el número de días de tormentas anual en España. Se observa que la zona de despliegue entre Salamanca-Valladolid presenta un número medio de once tormentas anuales. Eso obliga por normativa a la implementación de un pararrayos. Aunque no fuese obligatorio sería interesante contemplarlo puesto que un rayo puede causar sobretensiones eléctricas que dañen los equipos electrónicos sensibles de forma permanente. Los pararrayos se encargan de desviar esas sobretensiones de manera segura hacia el suelo, protegiendo los equipos de daños con un elevado coste (AEMET, Climatología de descargas eléctricas y de días de tormenta en España, 2019)

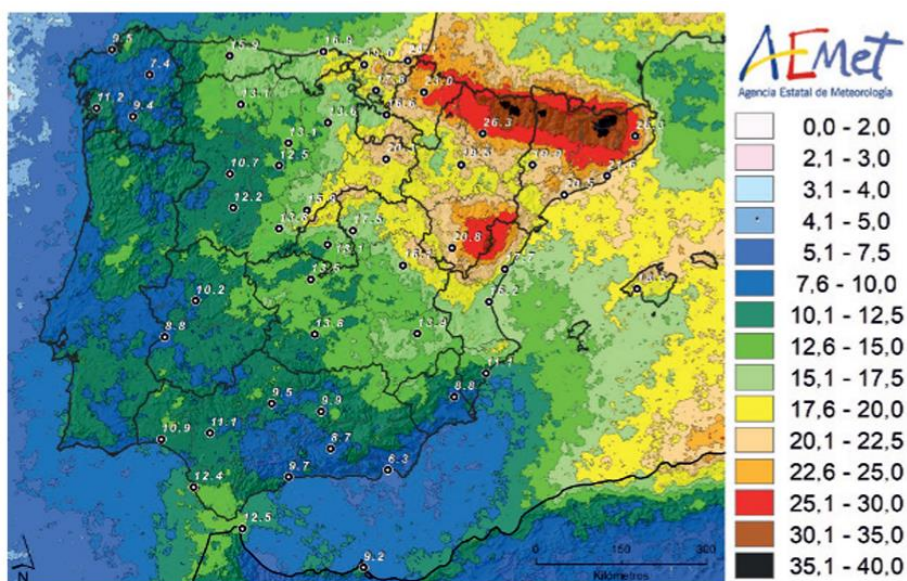


Figura 60. Mapa de tormentas de la Península Ibérica.

La instalación de un pararrayos no está regulada y se basa en la norma UNE 62305-4 “Protección contra el rayo”. “En la que se proporciona información para el diseño, instalación, inspección, mantenimiento, y ensayo de los sistemas de protección de los sistemas eléctricos y electrónicos con el fin de reducir el riesgo de fallos permanentes producidos por el impulso electromagnético del rayo en el interior de las estructuras” (Ingescio, 2020).

No se ha encontrado ninguna limitación en el diseño, pero si en las directrices de diseño de estos. Por ello, de acuerdo con el vigente reglamento en la norma UNE 62305 aseguraremos que los conductores de descarga se conecten directamente con el sistema de puesta a tierra, mencionado anteriormente. Se ha estimado un coste de 4.000 € para dotar a la infraestructura de protección frente a rayos con un pararrayos tipo punta Franklin, incluyendo su instalación y correcta conexión (AT3, 2012).

4.3.1.3 Instalación, acondicionamiento y cerramiento

Las infraestructuras de telecomunicaciones tienen como actividad principal la prestación de servicios básicos de telecomunicaciones. Estos servicios, considerados de interés general, se ofrecen en régimen de libre competencia, según lo establece el artículo 2.1 de la Ley General de Telecomunicaciones (BOE, Ley 11/2022, de 28 de junio, 2022). Además, conforme al apartado 34.2 de dicha Ley, las estaciones base, son consideradas equipamiento de carácter básico. Su instalación y despliegue se consideran, por tanto, obras de interés general. Esto implica que su implementación es prioritaria y necesaria para satisfacer las necesidades básicas de comunicación de la población, y pueden tener ciertos privilegios o consideraciones especiales dentro del marco regulatorio y administrativo.

Tras haber definido los dispositivos de radio a emplear, así como el equipamiento para una correcta alimentación y conservación de cada una de las estaciones base que se pretenden construir. Ha de definirse el procedimiento a seguir para la correcta preparación del terreno, siendo una fase crítica en la instalación de estaciones base LTE. Para ello, hay que asegurar que el sitio sea adecuado y seguro para soportar la infraestructura necesaria que se construirá en esos emplazamientos.

La mayoría de los emplazamientos seleccionados se ubican en entornos rurales, aunque generalmente más económicos, estos terrenos pueden presentar desafíos adicionales, como la necesidad de mayor infraestructura para acceder a servicios básicos o creación de caminos. Sin embargo, esto no se ha tenido en cuenta en este informe. El tamaño del terreno necesario varía en función de la infraestructura complementaria a la torre de telecomunicaciones, para cada una de las E-UTRAN se ha estimado un área de $150m^2$. El coste del terreno es una variable a tener en cuenta para establecer el presupuesto total de la instalación. Al considerar factores como la ubicación, tamaño, características del terreno y regulaciones locales.

Por simplificación, se ha contemplado únicamente el coste medio por m^2 según la siguiente Figura 61 en entorno urbano. A este se le ha añadido una subida del 9,2% realizando un ajuste con la situación económica española actual resultando en $156.23 €/m^2$. Lo que implicaría un coste por E-UTRAN de nueva construcción de 22.500 €. (EjePrime, 2022)

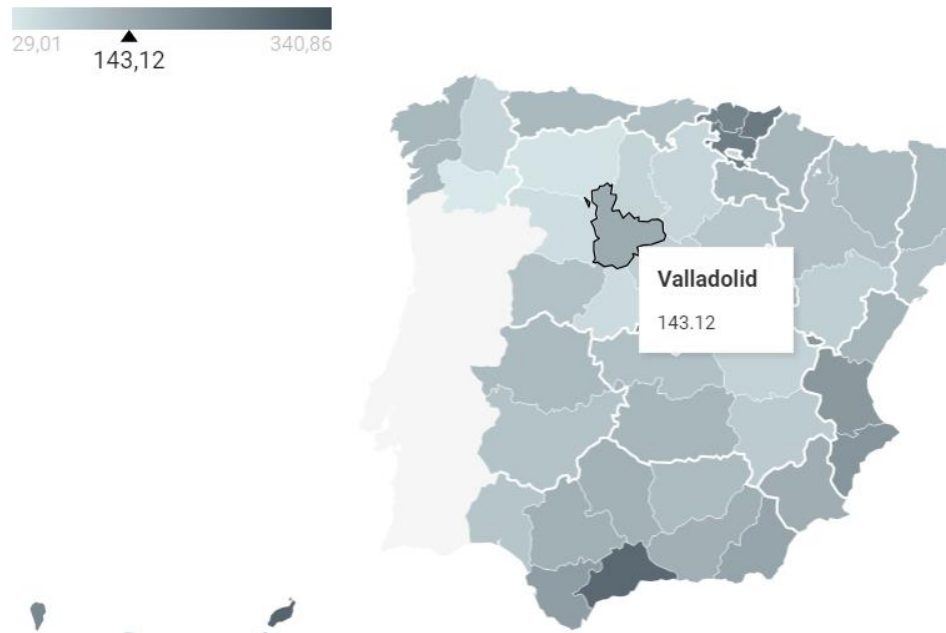


Figura 61. Precio terreno coste/m² en 2022.

Se ha decidido subir las antenas una distancia elevada buscando optimizar y maximizar el rango de cobertura propuesto por la infraestructura. Para ello, se ha elegido construir torres de celosía autosoportadas de acero con 24m de altura al tratarse de una opción robusta, económica y permiten una construcción más simple y una cimentación más económica frente a la opción de emplear un mástil. Como contrapartida los cables quedarán expuestos al clima y no estarán protegidos, por lo que el mantenimiento de estos ha de ser más frecuente para garantizar su correcto funcionamiento. De acuerdo con el siguiente plano se realizará la construcción de las diferentes torres donde cada una de ellas supondrá un coste incluida instalación de 16.500 €.

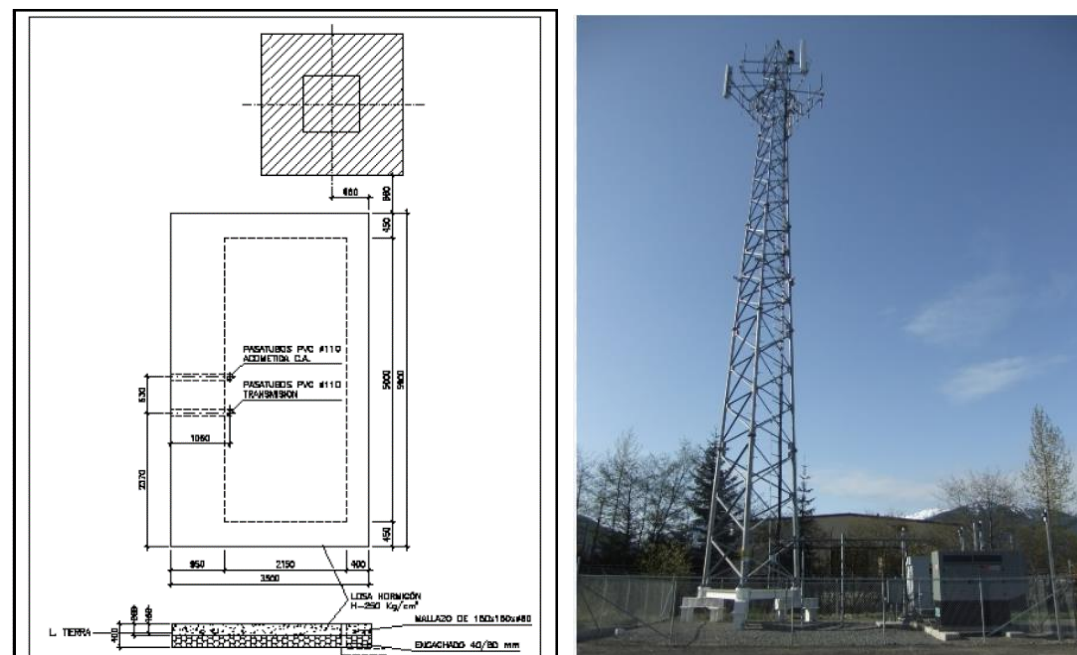


Figura 62. Plano de la torre y ejemplo.

Todos los componentes de acero que se encuentren expuestos al aire libre (perfiles, tornillos, arandelas, escaleras, plataformas o soporte de antenas entre otros) deben ser galvanizados mediante inmersión en un baño caliente de zinc. Esta capa actuará como una barrera física que impida que agentes corrosivos, como lo son el oxígeno y el agua, comprometan la estructura requiriendo un menor mantenimiento a lo largo del tiempo.

Previo a la instalación de la torre será necesaria una correcta preparación del terreno. Para ello, será importante nivelar el terreno cuya función será garantizar una superficie uniforme y estable para la construcción de la torre de celosía. En función de las condiciones exactas de cada uno de los emplazamientos seleccionados para construir las torres de telecomunicación deseadas serán necesarias tareas de compactación o relleno del terreno. Considerando la suma del coste para cada una de estas tareas se ha estimado un coste final de 13.500 €, para el correcto acondicionamiento del terreno.

Después será necesaria realizar una cimentación que sirva como base para la torre. Esta cimentación debe ser capaz de soportar tanto las cargas estáticas como las dinámicas a las que se pueda verse expuesta la torre por condiciones medioambientales. Se instalarán anclajes de acero que se extiendan desde la base de la cimentación hasta la parte superior de esta, donde se conectará la torre de celosía. Será necesario realizar una excavación de al menos 2 metros de profundidad y un ancho igual o mayor al de la propia torre. Contemplando una base de grava que permita mejorar la capacidad de drenaje del cimient. De acuerdo con proyectos públicos similares, se estima un coste final de unos 7.000 €.

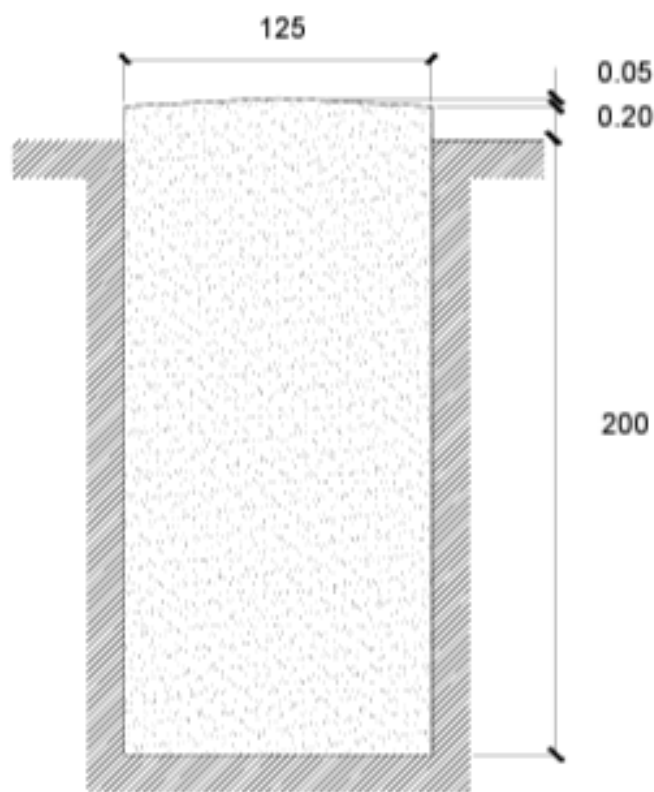


Figura 63. Cimentación Torre

Tras el correcto acondicionamiento del terreno será necesaria la construcción de dos arquetas. Serán las responsables de establecer el punto de entrada a la red de fibra y la red de alimentación. Se estima un coste individual de 800 €.

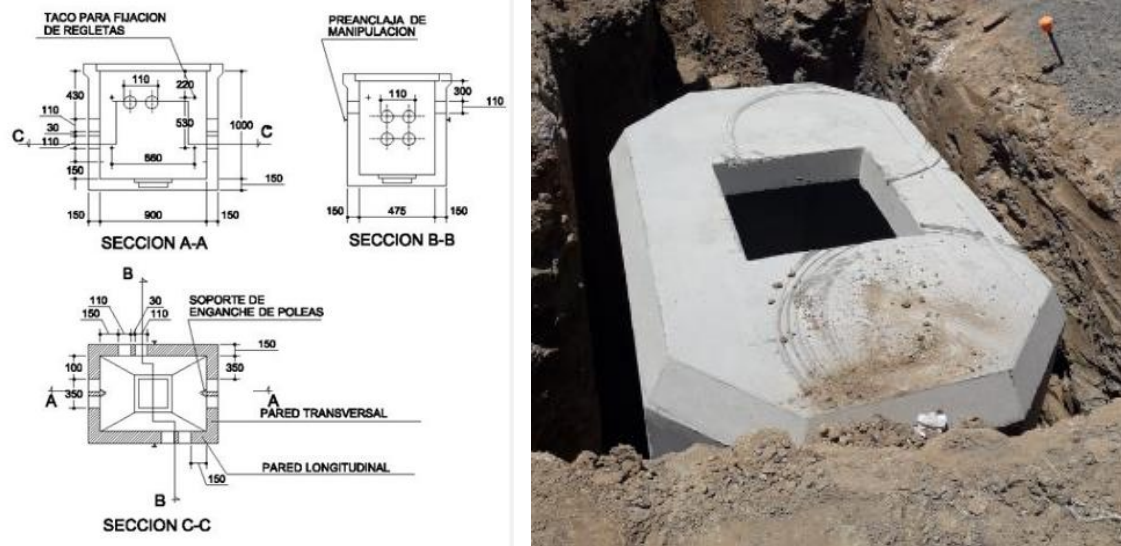


Figura 64. Construcción de las arquetas.

Para proteger los dispositivos necesarios para la implementación de la E-UTRAN se empleará una caseta de dimensiones reducidas. Esta será suficiente para albergar todos los dispositivos de interior descritos anteriormente. El coste de la caseta y su correspondiente instalación será de 800 €.

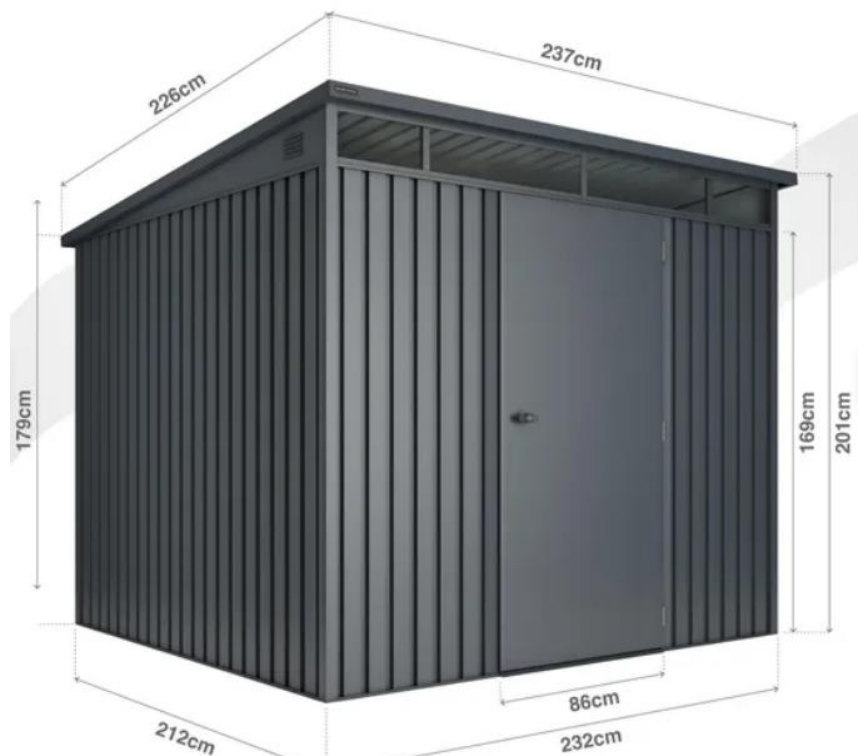


Figura 65. Imagen de la caseta con sus dimensiones.

Los bienes muebles o inmuebles vinculados a los centros, establecimientos y dependencias afectos a la explotación de las redes y a la prestación de los servicios de telecomunicaciones dispondrán de las medidas y sistemas de seguridad, vigilancia, difusión de información, prevención de riesgos y protección que se determinen por el Gobierno, a propuesta de los Ministerios de Defensa, del Interior o de Industria, Energía y Turismo, dentro del ámbito de sus respectivas competencias, según lo establece el artículo 4.5 (BOE, Ley 11/2022, de 28 de junio,

2022). Por ello, se va a realizar un vallado perimetral formado por un cerramiento metálico de alambre galvanizado de 5 mm de diámetro de 2,5 m de altura. Serán necesarios 55m de valla para realizar el cerramiento completo de la infraestructura. Se contemplará también la definición de una puerta, que permita la entrada a personas autorizadas, con lo que se estima un coste por metro de 120 € resultando en un total de 6.600 €⁹ por unidad E-UTRAN.

Debido a la considerable altura de las torres propuestas se requiere de una baliza para garantice la seguridad. De acuerdo con la guía de señalamiento e iluminación de obstáculos, regulado por la Agencia Estatal de Seguridad Aérea (AESA) y elaborado a partir del Anexo 14 de la Organización de Aviación Civil Internacional (OACI). Será necesario implementar una baliza de baja intensidad tipo A, siendo esta fija y roja con un precio de 300 € (AESA, 2018).

Todo lo descrito anteriormente aplicaría a todas las E-UTRAN de nueva obra. La localización de dichas infraestructuras se encuentra recogida en la siguiente imagen, señalizadas con un punto de color rojo en su ubicación.

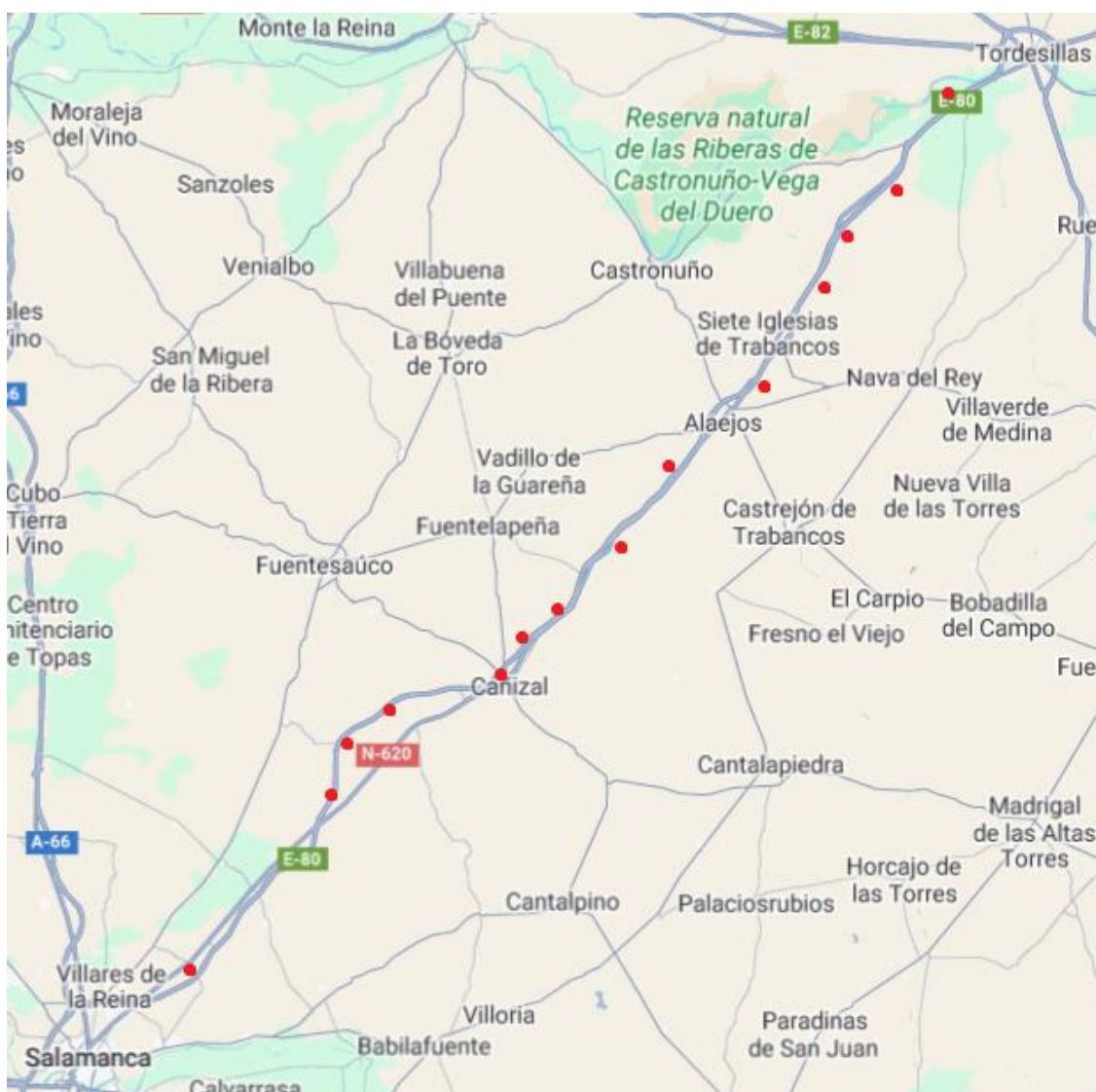


Figura 66. Emplazamientos nueva obra (puntos en rojo).

⁹ Disponible en: <https://www.habitissimo.es/presupuestos/instalar-valla>

Sin embargo, como se ha demostrado anteriormente no son suficientes para realizar el servicio de vehículo conectado para el cual se ha creado este informe. Por ello, se ofrece la siguiente imagen. Esta sí que recoge el total de E-UTRANs que se van a emplear para dar servicio. De nuevo, los puntos rojos indican los emplazamientos de nueva construcción mientras que los puntos azules recogen la ubicación de torres de telecomunicación existentes en la región, a fecha 26 de mayo de 2024. Los propietarios de estas torres, según lo recoge la web de del ministerio, son Vodafone o bien Movistar.



Figura 67. Emplazamientos totales de todas las estaciones nuevas (rojo) y existentes (azul).

Tales torres/mástiles, serán alquilados para colocar nuestros equipos, garantizando unas correctas condiciones de alimentación, protección, mantenimiento y operatividad. El cobro anual del contrato de arrendamiento se estima en 16.000 € por torre/mástil durante un periodo de 8 años. El coste de notaría para formalizar dicho contrato sería de 300 € contemplando cada torre de forma individual (Vigo, 2008).

4.3.2 Despliegue EPC

Se ha escogido un EPC que cumpla una serie de condiciones para el servicio de red 4G. Se ha priorizado la escalabilidad para poder dar servicio a más usuarios en el futuro, la compatibilidad con la banda 7 y MIMO 2x2, el coste económico, el rendimiento y que ofrezcan los servicios mínimos integrados de MME, S-GW, P-GW, HSS y PCRF. Podría ser interesante valorar la instalación de un segundo EPC en el futuro para mejorar la resiliencia y la redundancia.

Cada eNodeB posee dos antenas, cada una dando servicio como máximo a 32 usuarios simultáneos. Por lo tanto, el número de usuarios máximo al que podrá dar servicio el EPC será el siguiente:

$$\text{Número total de usuarios} = 64 \frac{\text{Usuarios}}{\text{Antena}} \times 2 \frac{\text{Antenas}}{\text{eNodeB}} \times 24 \text{ eNodeB} = \mathbf{3072 \text{ Usuarios}} \quad (11)$$

Teniendo todo lo anterior en cuenta, el EPC escogido será el Cisco ASR5000-CHSSYS-K9 (Cisco-GPL, 2024). Este ofrece alta capacidad, escalabilidad y compatibilidad con la banda 7 y MIMO 2x2, por un precio de 78.000€¹⁰. También está diseñado para combinar los servicios de HSS, P-GW, MME, S-GW y PCRF. Se ha decidido situar este EPC en una zona intermedia del tramo de carretera, concretamente en el pueblo de Cañizal. (CISCO, 2024)



Figura 68. Cisco ASR5000-CHSSYS-K9

Antes de comenzar con la propuesta de la red GPON, se va a considerar la fibra oscura. Se entiende por fibra oscura los circuitos de fibra óptica o redes que han sido desplegados por alguna empresa u operador de telecomunicaciones y que no están siendo utilizados, es decir, no están “iluminados”. Es una fibra sin luz, sin transmisiones a través de ella.

Los tubos que se instalan bajo el suelo para transportar fibra óptica y otros cables suelen tener una capacidad superior a la que la demanda de comunicaciones requiere. Es lógico que las operadoras, al realizar las obras pertinentes, opten por construir y enterrar conductos muy anchos para dejar instalados “hilos sin usar” y así evitar tener que volver a abrir zanjas cada vez que se incrementa la demanda de conexiones. Esto resulta en un sobredimensionamiento y muchas fibras quedan sin uso.

¹⁰ Disponible en: <https://itprice.com/cisco-gpl/>

Se buscaron tramos de fibra oscura próximos al tramo de carretera estudiado. Las compañías que poseen fibra oscura próxima son Lyntia (Lyntia, 2024) y Reintel (Reintel, 2024), como se puede observar en las figuras 69 y 70. Estas fibras no están lo suficientemente cercanas a la zona de interés, por lo que la decisión final es realizar un despliegue de fibra en el tramo de autovía.

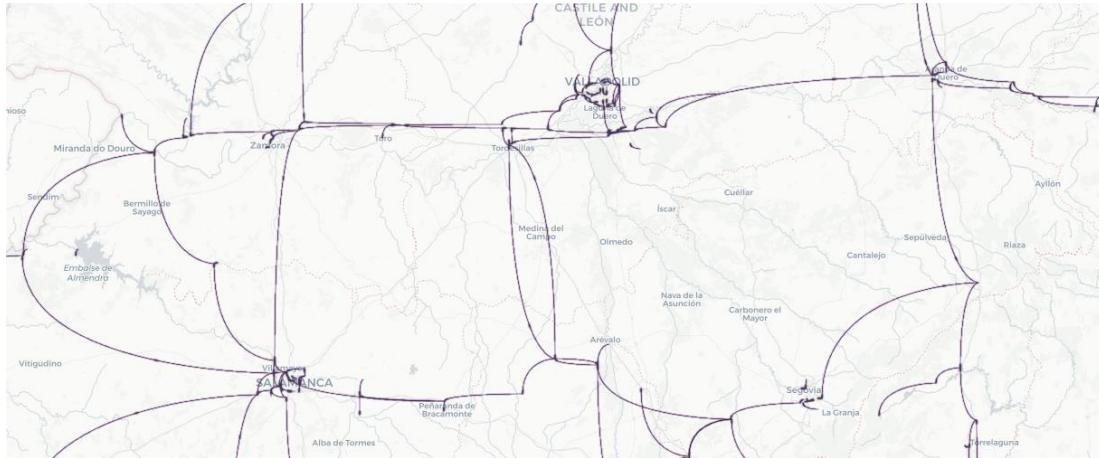


Figura 69. Fibra oscura Lyntia.

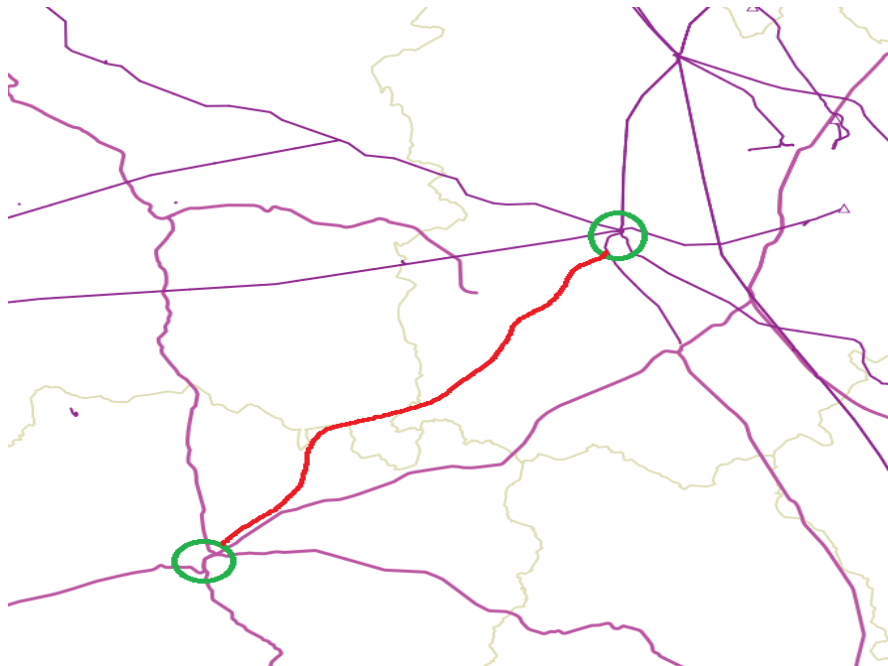


Figura 70. Fibra oscura Reintel.

GPON está definido por la ITU-T en el conjunto de recomendaciones G.984.X, donde se describen las técnicas para compartir un medio común (fibra óptica) entre varios usuarios, encapsular la información y gestionar los elementos de red, entre otros aspectos. El estándar GPON (*Gigabit Passive Optical Network*) es una mejora de las características de las redes con tecnología PON (*Passive Optical Network*). Consiste en la implementación de una red de fibra óptica con elementos pasivos, es decir, que no requieren alimentación externa para su funcionamiento, lo que resulta en una reducción de costos en los equipos utilizados para llegar hasta el usuario final. (Mauri Isorna, 2017)

Los sistemas GPON se componen principalmente de dos elementos: el OLT (*Optical Line Terminal*) que actúa como la cabecera del sistema, y las ONT (*Optical Network Terminal*), que

son los equipos terminales de usuario. Estos dispositivos están conectados a través de una red de distribución óptica pasiva.

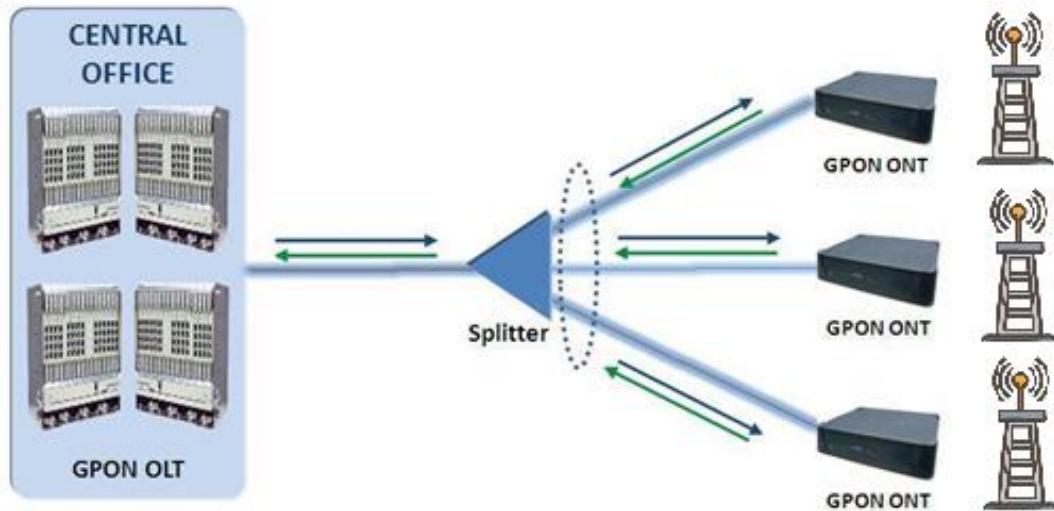


Figura 71. Esquema Red GPON.

El OLT se encuentra en el centro de operaciones del proveedor y actúa como el punto de partida del cable de fibra óptica hacia los usuarios. Sus funciones incluyen gestionar el tráfico de subida y bajada para múltiples suscriptores, realizar funciones de control de red y recolectar datos de diversas fuentes. Además, mediante el protocolo de rastreo, los OLT pueden ajustar la potencia óptica según la distancia a las ONT en la red, lo que resulta en una operación eficiente y una reducción de costos en los equipos utilizados para llegar hasta el usuario final.



Figura 72. Ubiquiti UFiber OLT 8-Ports GPON.

Tras analizar el tramo de carretera, pese a tener la opción de cubrir todo el territorio con 3 OLTs, se decide utilizar 4: Una situada en Salamanca, otra en Tordesillas, una tercera situada en Cañizal y la cuarta en Siete Iglesias. Con esta distribución no sólo ofrecemos cobertura en todo el tramo si no que mejoramos factores como la densidad de la cobertura, el balanceo de carga, la gestión del *handover* o redundancia.

La distribución de las OLT se puede ver en la siguiente figura, junto al mapa de las estaciones ya existentes y las que se van a construir:



Figura 73. Distribución de estaciones añadiendo OLTs en naranja.

Se ha decidido escoger una OLT de 8 puertos, ya que el planteamiento de la red va a consistir en un cable de fibra que se extienda tanto a un lado como al otro de la carretera, conectando directamente las ONTs a la fibra sin necesidad de pasar a través de un *splitter*. Por lo tanto, escoger más puertos sería sobredimensionar en exceso la red al estar la gran mayoría de puertos sin conectar, además de ser más costosa y complicada de gestionar. La OLT escogida es la Ubiquiti¹¹ UFiber OLT 8-Ports GPON.

Operating wavelength	
TX	1490 nm
RX	1310 nm
Line code	Scrambled NRZ
Optical interface nominal bit rate	
Downstream	2488.32 Mbit/s
Upstream	1244.16 Mbit/s
Physical reach	20 km

Figura 74. Datasheet OLT.

¹¹ Disponible en: <https://www.pccomponentes.com/ubiquiti-ufiber-olt-switch-gestionado-12-3-1u-8-puertos-sfp-2-sfp>

La ONT es el dispositivo ubicado en el hogar del usuario y está conectado a la toma óptica. Proporciona interfaces para diferentes servicios y funciona como un convertidor óptico-eléctrico. Su tarea principal es transformar la señal de luz recibida por la fibra óptica en señal eléctrica. Además, la ONT recibe y filtra la información del equipo OLT destinada a un usuario específico, y también encapsula y envía la información del usuario de regreso al equipo OLT para su posterior enrutamiento en la red.



Figura 75. Ubiquiti UFiber ONT Loco.

En concordancia con la OLT elegida, se va a utilizar el Ubiquiti UFiber ONT Loco, que además es robusta, de bajo consumo y de alto rendimiento. Localizaremos una ONT por cada estación base¹².

UFiber loco (Model: UF-LOCO)	
Dimensions	76.5 x 76.5 x 26.4 mm (3.01 x 3.01 x 1.04")
Weight	77 g (2.72 oz)
Networking Interfaces	(1) SC/APC, GPON WAN (1) Gigabit RJ45, Ethernet LAN
Networking Interface Speeds	2.488 Gbps Downstream 1.244 Gbps Upstream
GPON WAN, ITU G.984	
GbE LAN	10/100/1000 Mbps
Management Interface	In-Band Ethernet/PON
Normal Optical Power Range	TX (Class B+): 1.5 to 5 dBm RX: -28 to -8 dBm
Power Method	Micro-USB: 5V, 1A 24V Passive PoE (Pins +4, 5; -7, 8)
Power Supply	Micro-USB Power Adapter*: 5V, 1A
Max. Power Consumption	3.5W
Supported Voltage Range	4.7 to 5.3V
Processor Specs	MIPS32, 240 MHz
Memory Information	128 MB DDR3
Buttons	(1) Reset
Operating Temperature	-10 to 45° C (14 to 113° F)
Operating Humidity	10 to 90% Noncondensing
Certifications	CE, FCC, IC

Figura 76. Especificaciones ONT.

¹² Disponible en: <https://www.batna24.com/es/p/ubiquiti-ufloco5-ont-rmlmm>

Las OLTs tendrán 2 cables de fibra óptica para la conexión de las ONTs tanto a la izquierda como a la derecha del tramo de carretera, en el que estén ubicadas. Los cables de fibras se extenderán a lo largo de la carretera e irán conectados a las ONTs, estos cables cuentan con 24 fibras monomodo cada uno.¹³ Por lo que, debido al sobredimensionamiento previsto se empleará una fibra dedicada para cada ONT que conecte cada una de las OLTs.

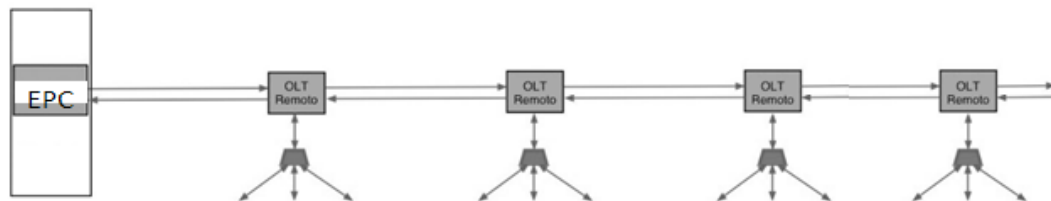


Figura 77. Esquema OLTs.

Este esquema evitara pérdidas innecesarias, al no hacer uso de *splitter* y simplifican la red al no ser necesario el uso de amplificadores intermedios, al garantizar las OLTs empleadas una distancia de 20km. Además, ofrece la posibilidad de ampliar la red en el futuro de forma más simple (Hayes, 2016).

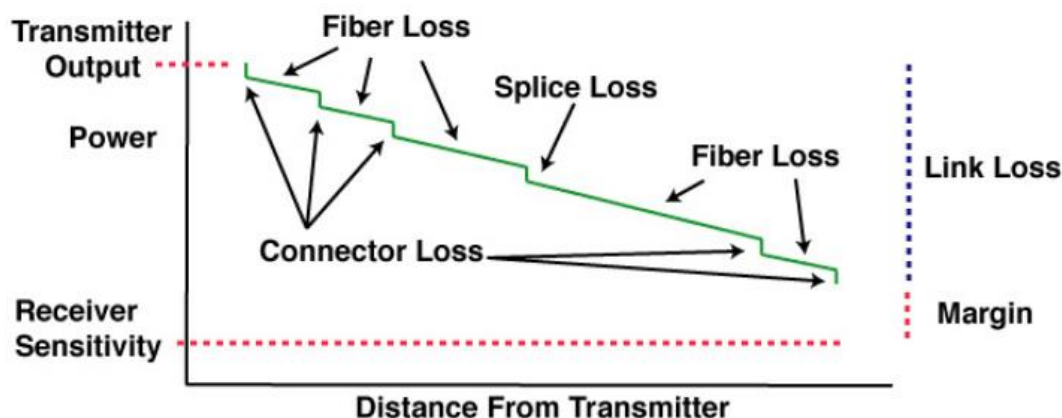


Figura 78. Pérdidas en fibra

Debido a las características de este despliegue y tratarse de una zona sin fibra desplegada previamente se ha decidió crear una zanja. Esta resulta esencial para la instalación de la fibra óptica escogida y, además, se empleará para llevar la acometida eléctrica, descrita anteriormente en el informe. Servirá para proteger los cables de posibles daños físicos y ambientales, como humedad, temperaturas extremas y actividad humana como robos. Mediante el uso de arquetas se va a facilitar el acceso a los cables para su mantenimiento o reemplazo futuro. Además, al enterrar los cables se reducen riesgos de accidentes eléctricos y evita la contaminación visual del entorno, eliminando cables aéreos. Se estima un coste de 30€/m para la construcción de dicha zanja (Mártin, 2015). Aprovechando el coste de la obra, se ha decidido instalar 2 cables de fibra óptica que cuentan con 24 fibras monomodo cada uno. Uno de estos cables no tendrá uso en este proyecto, pero se contempla como aprovisionamiento futuro debido al escaso precio marginal que este supone para el despliegue total del proyecto.

¹³ Disponible en: <https://www.fs.com/es/products/72421.html>

Por último, se van a instalar dos servidores para la inteligencia artificial, uno en Cañizal y otro en Tordesillas. Son dos ya que así se mejoran las prestaciones del servicio, como el balanceo de carga, el rendimiento, la respuesta a picos de demanda o la tolerancia a fallos. La elección del servidor Dell PowerEdge R750xs (Dell, 2024) se ha realizado debido a los siguientes motivos:

- El procesador Intel Xeon Silver 4314, beneficioso para la inferencia y el preprocesamiento de datos debido a que posee múltiples hilos y núcleos.
- RAM suficiente para que el servidor maneje un gran volumen de tareas y datos sin problemas de rendimiento.
- 480GB de SSD, que proporciona tiempos de acceso rápidos y mayor velocidad de lectura/escritura en comparación con los discos duros tradicionales.
- Gran escalabilidad, pudiendo añadir memoria RAM, almacenamiento y GPUs para incrementar la capacidad de procesamiento
- Robustez y fiabilidad en todos los modelos de la empresa Dell EMC, además de ofrecer herramientas de gestión remota y ser eficientes en términos de consumo energético.



Figura 79. Dell PowerEdge R750xs.

También se añadirá una tarjeta gráfica por servidor, la tarjeta de NVIDIA Quadro RTX A6 (Nvidia, 2024). Esta tarjeta permite manejar modelos de IA muy grandes y complejos ya que posee una capacidad de procesamiento paralelo masivo. Posee una memoria de gran capacidad para trabajar con modelos y conjuntos muy grandes y miles de núcleos CUDA para un procesamiento paralelo eficiente.



Figura 80. NVIDIA Quadro RTX A6000.

5 Estudio Económico

Se ha inventado un proveedor, Paco Obras SA, al no disponer de empresas reales dispuestas a presupuestar los servicios presentados anteriormente. Las estimaciones de costes que se han empleado en la realización de los siguientes cálculos se basa en obras públicas de similares características. Además, se ha supuesto la compra, y no cesión, del espectro de frecuencias que se va a emplear. Lo que ayudará a estimar los costes de la realización de un despliegue de infraestructura de estas características. De acuerdo con los materiales, servicios e infraestructura necesaria para este despliegue se han generado unas tablas con su respectivo presupuesto a modo de resumen de todos los costes globales del despliegue.

Esta primera tabla recoge el presupuesto de desplegar una E-UTRAN completa, en la que se estima un coste total de 144.630 € por unidad desplegada.

Despliegue E-Utran Completa				
Componente	Proveedor	Unidades	Coste/u	Coste
Antena	Kathrein	2	1050	2100
RRU	Baicells	2	5000	10000
BBU	Cambium Networks	1	9000	9000
Fuente Alimentación	UniPower	2	1400	2800
PC-control	HP	1	1500	1500
Rack	PcComponentes	1	500	500
Aire-acondicionado	GGGmgastro	1	4500	4500
Generador Emergencia	Hyundai	1	2499	2499
Deposito Exterior		1	1505	1505
Tierra	Paco Obras SA	1	500	500
Acometida	Paco Obras SA	1	8000	8000
Pararrayos	Paco Obras SA	1	4000	4000
Terreno		1	22500	22500
Instalación Torre	Paco Obras SA	1	16500	16500
Acondicionamiento	Paco Obras SA	1	13500	13500
Cimentación Torre	Paco Obras SA	1	7000	7000
Arquetas	Paco Obras SA	2	800	1600
Caseta	Leroy-Merlin	1	800	800
Cerramiento	Paco Obras SA	1	6600	6600
Baliza		1	300	300
Sobrecostes		25%	0	0
Coste TOTAL				144630€

Tabla 4. Tabla con presupuesto de instalación y materiales por E-UTRAN completa.

En la siguiente tabla se recoge el presupuesto de instalación y material de una E-UTRAN en una torre ya existente cuyo propietario es Vodafone. Se estima un coste total de 29.210 € por unidad desplegada.

Despliegue E-Utran Alquiler				
Componente	Proveedor	Unidades	Coste/u	Coste
Antena	Kathrein	2	1050	2100
RRU	Baicells	2	5000	10000
BBU	Cambium Networks	1	9000	9000
Rack	Pcomponentes	1	500	500
PC-control	HP	1	1500	1500
Instalación Equipo		1	2000	2000
Notaria		1	300	300
Sobrecostes		1	15%	3810
Coste TOTAL				29210€

Tabla 5. Tabla con presupuesto de instalación y materiales por E-UTRAN alquiler.

La siguiente tabla completa el presupuesto total para el despliegue de las E-UTRANs necesaria para garantizar una correcta calidad de servicio. El coste de infraestructura es de 2.316.920 € mientras que la supuesta compra del espectro requerido supone el 74,74% del coste total, por un valor de 6.855.165 €.

Despliegue E-UTRANs TOTAL			
Concepto	Unidades	Coste/u	Coste
Adquisición Espectro	1	6855165	6855165
E-UTRAN nueva obra	14	144630	2024820
E-UTRAN alquiler	10	29210	292100
Coste Total			9172085€

Tabla 6. Coste de despliegue E-UTRAN total.

El presupuesto total referido al despliegue del EPC completo, contemplando la creación de franjas para tirar la fibra y llevar la acometida eléctrica, como los equipos correspondientes se encuentra definido en la siguiente tabla:

Despliegue EPC				
Componente	Proveedor	Unidades	Coste/u	Coste
Zanja	Paco Obras SA	74000	30	2220000
OLT	Ubiqui	4	1590	6360
ONT	Ubiqui	24	299	7176
EPC	CISCO	1	78000	78000
Servidor IA	Pcomponentes	2	4128	8256
GPU Nvidia	Pcomponentes	2	6241	12482
Fibra	FS	148000	2,82	417212
Coste Total				2749486€

Tabla 7. Despliegue total añadiendo el espectro.

Debido a las características de este despliegue se debe contemplar el coste de operación. Los cuales incluyen gastos de personal encargado del mantenimiento de equipos, suministros, energía y seguro. El precio de la electricidad empleado ha sido 0,1149 €/kWh. La siguiente tabla presenta el consumo eléctrico estimado por cada unidad de E-UTRAN y por el EPC:

Estudio Consumo E-UTRAN			
Componentes	Unidades	Consumo W/u	Consumo kWh
RRU	2	100	0,2
BBU	1	150	0,15
AIRE AC	1	700	0,7
PC	1	500	0,5
ONT+Extras	1	300	0,3
Coste Luz Anual			1862,07€

Tabla 8. Consumo estimado E-UTRAN.

Estudio Consumo EPC			
Componentes	Unidades	Consumo W/u	Consumo kWh
OLT	4	50	0,2
ONT	24	6	0,144
Server + GPU	1	3500	3,5
EPC	1	2700	2,7
Coste Luz Anual			6586,69€

Tabla 9. Consumo estimado EPC.

Lo que añadidos costos de mantenimiento, vigilancia y costes de arrendamiento, para poder emplear las torres de Vodafone resultan.

E-UTRAN Completa		E-UTRAN Alquiler		Coste Operación EPC	
Concepto	Coste	Concepto	Coste	Concepto	Coste
Vigilancia	2000	Mantenimiento	600	Consumo	6586,69306
Mantenimiento	3000	Consumo Eléctrico	1241,3796	Mantenimiento	6500
Consumo Eléctrico	1862,0694	Arrendamiento	15000	Vigilancia	2000
Coste Unidad	6862,07	Coste Unidad	16841,38	Coste TOTAL	15086,69

Tabla 10. Coste de operación desglosado.

El coste de operación total del despliegue supone alrededor de 320.000 € anualmente, en los que se contemplan imprevistos, recambio de equipos, mantenimiento especializado, seguros, personal de mantenimiento, etc.

Coste Operación Despliegue TOTAL			
Concepto	Unidades	Coste/u	Coste
E-UTRAN Completa	14	6862,0694	96068,9716
E-UTRAN Alquiler	10	16841,3796	168413,796
EPC	1	15086,6931	15086,69306
Seguro + Imprevistos	1	15%	41935,4191
Coste TOTAL			321504,88€

Tabla 11. Coste de operación y despliegue completo.

6 Desarrollo de un prototipo

6.1 Descripción del prototipo

En los capítulos previos, hemos detallado el marco teórico para el desarrollo y funcionamiento de un sistema capaz de permitir a un vehículo obtener y analizar imágenes del entorno que le rodea, e identificar las posibles señales de tráfico presentes. Gracias a ello, será capaz de tomar decisiones de forma automática y ayudar en la conducción de todo tipo de vehículos, creando así un vehículo conectado, que empleará una infraestructura de telefónica móvil 4G e Inteligencia Artificial. De igual modo, partiendo de la creación de este vehículo conectado, se ha propuesto diseñar un vehículo autónomo capaz de responder ante el reconocimiento del entorno y las señales. El control del vehículo podrá ser manual, pero este se controlará de manera autónoma al detectar señales.

En esta sección y secciones posteriores, trataremos de explicar los pasos que se han seguido para desarrollar un prototipo a modo de demostración de la viabilidad y potencial de nuestro sistema. Para ello, emplearemos un coche en miniatura Amazon Deep Racer que dispone de un miniordenador integrado, una cámara con la que capturar imágenes y un módem 4G LTE de Huawei al que ira conectado vía USB. Además, desarrollaremos una infraestructura 4G empleando un sistema SDR bladeRF 2.0, que nos permitirá comunicarnos con el módem instalado en el coche para recibir datos y enviar los correspondientes comandos, para actuar sobre el propio vehículo.

Para el caso del demostrador, se han empleado elementos de juguete como son carreteras de tela y señales de plástico en miniatura, el objetivo es desarrollar el sistema sobre este entorno y escalarlo en un futuro a un entorno real, ya que el demostrador tiene como tarea principal reforzar el estudio realizado sobre la posible implementación del proyecto.

En cuanto a la demostración del funcionamiento del prototipo, montaremos un pequeño circuito en el que se colocarán estratégicamente una serie de señales de tráfico de juguete, en el que el cochecito sea capaz de operar y funcionar en función de los comandos transmitidos desde el núcleo de la red. Todo esto con el añadido de tener cierto nivel de autonomía, al poder controlar temporalmente alguna de las funciones del vehículo como la velocidad o el giro una vez detecte las señales pertinentes en la carretera a través de la cámara.



Figura 81. Dibujo del funcionamiento del sistema.

La idea inicial no es sustituir al piloto por un sistema automatizado, si no dotar al coche de medidas de seguridad como evitar meterse por un lugar donde esté prohibido, frenar en caso de ver un stop o reducir la velocidad en caso de ir demasiado rápido, todo esto con la esperanza de conseguir una conducción más cómoda, segura y tranquila y reducir así los posibles accidentes ocurridos en carretera.

6.2 Descripción de los componentes

A la hora de desarrollar el prototipo, se ha necesitado utilizar una serie de componentes tanto para montar la red 4G como para ejecutar el script de detección, entrenar el modelo y capturar las imágenes.

El PC desde el cual establecemos tanto el *core* como la estación base de la red 4G. Además, desde él ejecutamos los comandos MQTT para manejar el estado del coche y los ficheros Python para realizar la captura de imágenes y el movimiento del coche.



Figura 82. PC del laboratorio

El vehículo conectado será el AWS DeepRacer, un coche en miniatura diseñado para realizar experimentos de vehículo autónomo. Este posee un miniordenador donde configuramos todos los parámetros necesarios para conseguir la conexión con el PC y la puesta a punto para su movilidad, además de un pequeño motor. Ambas partes tienen su batería con su respectivo cargador. En la parte superior está la cámara encargada de grabar el vídeo que luego se tratará y enviará a la red. Graba a 30 fps y el tamaño medio de las imágenes enviadas es de unos 70-90 kB.



Figura 83. Vehículo AWS DeepRacer (izquierda y centro) y cámara empleada (derecha).

El módem Huawei E3372 LTE USB Stick, donde va introducida la tarjeta SIM, que detecta la red 4G y nos conecta a ella. Al introducirlo en un puerto USB, intentará conectarse automáticamente a la red. Tiene un LED con 3 modos para controlar su estado: Verde intermitente significa que está intentando conectarse a una red, Azul intermitente es que está conectado a una red, pero no tiene acceso a Internet y Azul permanente es que está conectado y tiene acceso a Internet.



Figura 84. Módem Huawei E3372 LTE USB Stick.

La BladeRF micro xA9, que es la SDR que empleamos para implementar estación base (eNB) de la red 4G. Esta va enchufada tanto a la corriente como a uno de los puertos USB 3.0 del PC. Tras ejecutar tanto el srseNB como el srsepc y haber configurado la salida hacia Internet del programa (que es la interfaz que da salida a Internet al PC), la BladeRF ofrecerá el servicio de red 4G con acceso a Internet.



Figura 85. BladeRF micro xA9.

El kit de desarrollo del módulo Jetson Orin Nano de 8 GB con GPU de arquitectura Ampere de 1024 núcleos con 32 núcleos Tensor, fabricada y distribuida por NVIDIA con su respectiva placa, cable de alimentación con dos adaptadores y manual de instrucciones (NVIDIA, Jetson Orin Nano Specifications, 2024).



Figura 86. Kit de desarrollador Jetson Orin Nano .

Un router Linksys para poder interconectar la placa Jetson con el ordenador y así poder disponer de internet y saltarnos el inicio de sesión con las credenciales de la Uva.

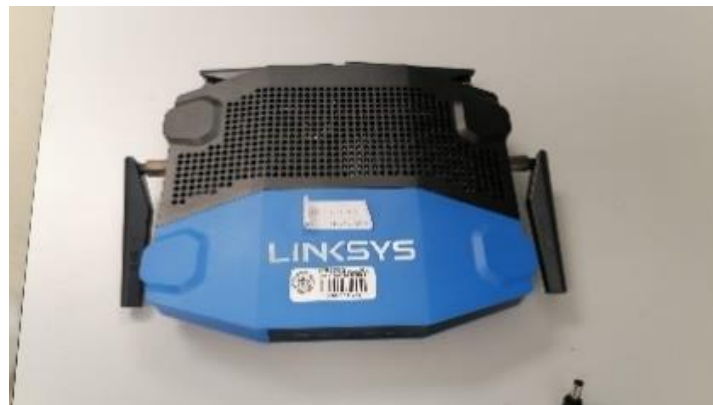


Figura 87. Router LINKSYS empleado.

Cableado variado empleado para desarrollar el prototipo. Los cables son, respectivamente: Adaptador HDMI a VGA para conectar el ordenador del vehículo al monitor, el cargador de la batería del ordenador, un cable de red y un adaptador VGA a HDMI empalmado con otro HDMI Display-in, el cargador de la batería de litio del motor del vehículo y la batería de litio del motor del vehículo.



Figura 88. Cableado variado y batería del motor del vehículo.

6.3 Organización del proyecto

La organización de un proyecto es de gran importancia para una correcta ejecución del mismo, así como el cumplimiento de los plazos. Una buena planificación y gestión permite que todas las tareas se realicen de manera eficiente, se minimicen los riesgos y se optimicen los recursos disponibles. A continuación, se detallan las herramientas que se han utilizado para organizar y gestionar este proyecto, explicando su importancia y función específica:

- **Jira:** Jira es una herramienta de gestión de proyectos ampliamente utilizada en el ámbito del desarrollo de software. Permite a los equipos planificar, rastrear y gestionar tareas y problemas con un alto grado de detalle. Con Jira, hemos podido asignar tareas, añadir descripciones y comentarios, establecer prioridades y seguir el progreso de cada actividad en tiempo real, lo que facilita la coordinación y la comunicación entre los miembros del equipo.
- **Gantt:** El diagrama de Gantt es una herramienta visual que nos permite planificar y programar las diferentes etapas del proyecto. Utilizando diagramas de Gantt, se ha podido establecer un cronograma claro, identificando las dependencias entre tareas y asegurándonos de que el proyecto avance según lo previsto. Esta herramienta resulta de gran ayuda a la hora de gestionar los plazos y anticiparse a posibles retrasos.
- **GitHub:** GitHub es una plataforma de control de versiones que permite gestionar y colaborar en el código de nuestro proyecto. Con GitHub, podemos trabajar de manera colaborativa, compartiendo código, realizando revisiones de cambios y manteniendo un historial de todas las modificaciones realizadas. Esto no solo mejora la calidad del código, sino que también facilita la integración continua y la implementación de nuevas características y versiones.
- **OneDrive:** OneDrive es un servicio de almacenamiento en la nube que se ha utilizado para compartir y almacenar documentos y archivos relevantes para el proyecto. Esta herramienta nos ha permitido acceder a los documentos desde cualquier lugar y en cualquier momento, asegurando que todos los miembros del equipo tengan la información actualizada y puedan colaborar de manera eficiente. Gracias a OneDrive, se ha podido guardar documentos de valor (PDFs, imágenes...) y compartir y modificar en línea ficheros Office como Word, Excel o PowerPoint.
- **Discord:** Discord es una plataforma de comunicación que se ha utilizado para mantener una comunicación fluida y constante entre los miembros del equipo. A través de Discord, se ha podido realizar reuniones virtuales, discutir ideas y resolver problemas. Esta herramienta ha sido esencial para mantener una buena coordinación, fomentar el trabajo en equipo y evaluar el estado del proyecto semanalmente.

La combinación de estas herramientas ha sido fundamental para la organización y el éxito de nuestro proyecto. Cada una de ellas ha aportado funcionalidades clave que han mejorado nuestra capacidad de planificar, gestionar y colaborar de manera efectiva.

Además, se han realizado dos DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades), uno sobre los distintos algoritmos a emplear de cara al modelo de Inteligencia Artificial y otro valorando los distintos softwares 4G a emplear para el despliegue de la red. A continuación podemos ver ambos DAFOS estructurados en una serie de recuadros con sus características.

DAFO-Inteligencia Artificial (Algoritmo)

Debilidades

- Menor documentación, ejemplos y recursos de aprendizaje que otras versiones.
- Funcional solamente con imágenes.
- Menor corrección de bugs y errores que otras versiones más antiguas.
- Mayor riesgo de incompatibilidad con versiones nuevas de software y hardware.

Amenazas

- No disponer de la suficiente información para poder implementarlo.
- No escalable a otros ámbitos distintos del análisis de imágenes.
- Sensible a la resolución de las imágenes y condiciones ambientales.
- Requiere recursos computacionales considerados.

Fortalezas

- Diseñado para analizar imágenes.
- Más rápido, eficiente y preciso que otros modelos de YOLO.
- Modelo compatible con ejecución en tiempo real.
- Detección de múltiples clases simultáneamente frente a detección de solo una clase de otros algoritmos.

Oportunidades

- Implementar un modelo de manera más sencilla y rápida que otros algoritmos.
- Diseño de un sistema basado especialmente en detección de objetos en imágenes.
- Posibilidad de ejecutarlo en entornos como las tarjetas de NVIDIA Jetson.

DAFO-Inteligencia Artificial (Algoritmo)

Comparación de los distintos algoritmos, en esta tabla se recogen una serie de cualidades sobre la que se asigna una serie de puntuaciones (0: no cumple, 1: cumple ligeramente, 2: cumple consistentemente, 3: excelente) en función del nivel de satisfacción de cada algoritmo sobre el requisito y su grado de importancia. Las características para el caso de estudio menos importantes se valorarán x1, las medianamente importantes x2 y las críticas x3, el que mayor puntuación reúna al final será elegido:

Versión	YOLOvX			TensorFlow	Faster R-CNN	SSD
	YOLOv5	YOLOv7	YOLOv9	-		
Existencia de documentación (x3)	3	3	2	3	2	2
Actualizaciones y soporte (x1)	3	1	1	3	2	2
Velocidad y precisión (x3)	1	2	3	2	2	1
Compatibilidad con la familia Jetson (x3)	3	3	3	3	1	1
Capacidad* (x2)	2	2	3	2	1	1
Facilidad de implementación (x3)	2	2	2	1	1	1
TOTAL:	34	35	37	34	22	19

*Capacidad: Posibilidad de emplear el algoritmo para detectar variaciones externas de los objetos a identificar.

DAFO-Software 4G

Debilidades

- SRSRAN se encuentra en desarrollo activo, puede sufrir cambios constantes.
- Requiere de más conocimientos técnicos que softwares como OAI u Open5GS.
- OAI requiere de una mayor cantidad de recursos para poder ser desplegado, seguido de Open5GS y después srSRAN.

Amenazas

- Al ser softwares de código abierto puede estar sujetos a vulnerabilidades.
- SDSRAN y OAI pueden presentar problemas de interoperabilidad.
- OAI solo compatible con Ubuntu 18.04
- OAI solo se enfoca en el desarrollo de entorno completo, Open5GS en core y srsRAN en el despliegue eficiente de la RAN.

Fortalezas

- srsRAN es una plataforma de código abierto, mayor facilidad a la hora de encontrar soluciones en línea y documentación.
- Todos son gratuitos.
- srSRAN tiene una mayor facilidad de implementación que OAI y Open5GS.
- srSRAN es compatible con más versiones de Ubuntu.

Oportunidades

- Instalar y configurar el entorno más rápido que con otros softwares debido a la gran documentación y tutoriales existentes.
- Posibilidad de cambiar la versión de Ubuntu dentro de un cierto límite sin afectar al sistema.

DAFO-Software 4G

Comparación de los distintos softwares, en esta tabla se recogen una serie de cualidades sobre la que se asigna una serie de puntuaciones (0: no cumple, 1: cumple ligeramente, 2: cumple consistentemente, 3: excelente) en función del nivel de satisfacción de cada algoritmo sobre el requisito y su grado de importancia. Las características para el caso de estudio menos importantes se valorarán x1, las medianamente importantes x2 y las críticas x3, el que mayor puntuación reúna al final será elegido:

	SRSRAN	OpenAirInterface	Open5GS
Existencia de documentación (x3)	3	2	1
Actualizaciones y soporte (x1)	3	3	1
Facilidad de implementación (x3)	3	2	1
Compatibilidad (x3)	3	3	1
Prestaciones (x3)	3	3	1
Seguridad (x1)	2	3	1
TOTAL:	41	36	14

Existencia de documentación: Cantidad de documentos, tutoriales, blogs y videos relacionados con el software existentes.

Actualizaciones y soporte: Cantidad de actualizaciones que se publican anualmente para mejorar el rendimiento o incrementar la seguridad y existencia o no de soporte (tanto del desarrollador como de la comunidad).

Facilidad de implementación: Facilidad de implementación en función de opinión de foros y de usuarios de la comunidad que ya lo emplearon.

Compatibilidad: Grado de compatibilidad e interoperabilidad con distintas versiones de S.O., distintos Hardware y distintos programas.

Prestaciones: Nivel de eficiencia,

Seguridad: Posibilidad de sufrir daños ante ataques enemigos.

Para poder ver gráficamente las decisiones tomadas, se ha creado una tabla de puntuaciones en la que asignamos una puntuación a cada característica. Después, multiplicamos esa puntuación por un multiplicador en función de su relevancia dentro del proyecto.

En cuanto a la planificación del proyecto, antes de comenzar a realizar el prototipo y como ya se ha mencionado previamente, se realizó un diagrama Gantt con una planificación inicial del proyecto. Con objeto de reflejar los cambios entre la planificación inicial y la planificación real que se ha seguido, se ha elaborado otro diagrama Gantt que muestra la planificación final del prototipo. Ambos diagramas Gantt pueden observar en el Anexo F-Diagramas Gantt con una mayor resolución. El software Jira incluye una herramienta que genera diagramas Gantt, en la sección Cronograma, mediante la cual se han realizado los siguientes diagramas Gantt.

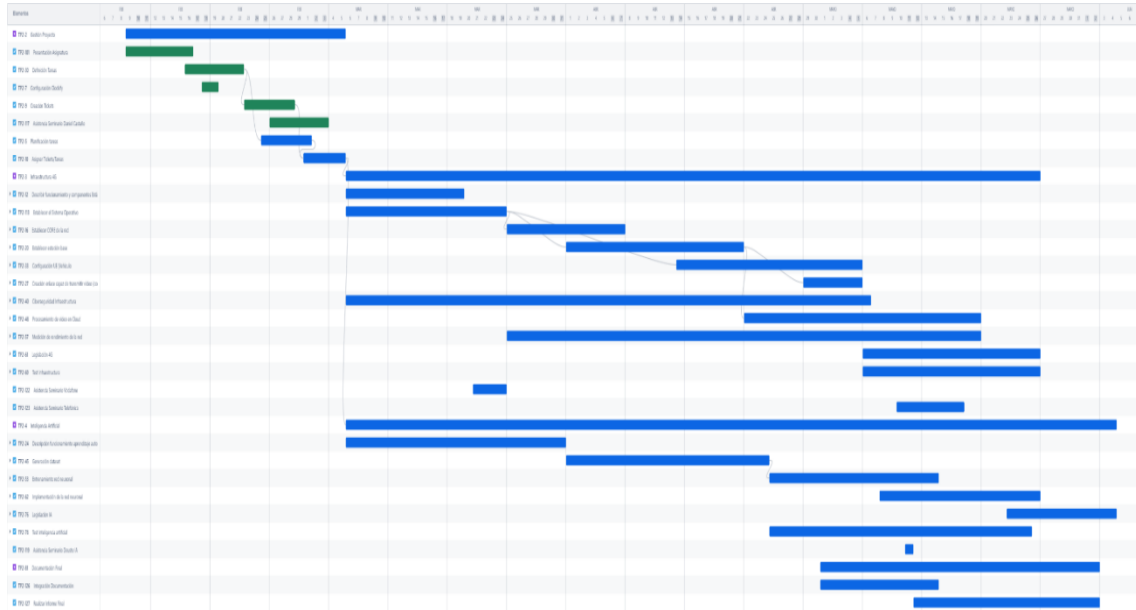


Figura 89. Diagrama Gantt en miniatura.

Finalmente, debemos comentar que en cada tarea y subtarea cada miembro del grupo ha ido documentando todo lo que ocurría durante la realización de una tarea, a modo de cuaderno de Bitácora con el objetivo de tener todo correctamente documentado de cara al futuro.

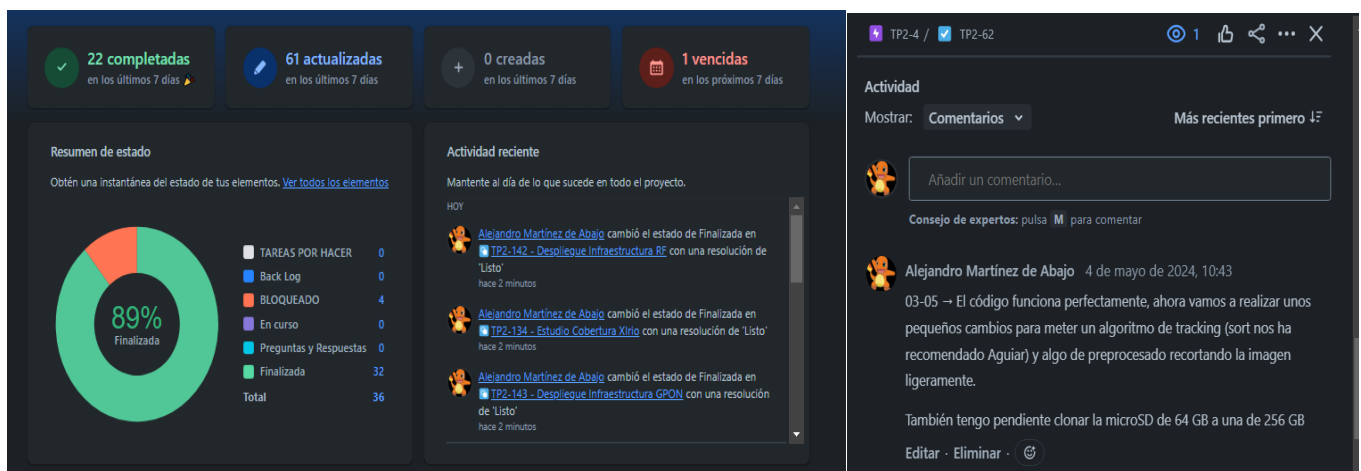


Figura 90. Imagen del panel Jira (izquierda) y un comentario de ejemplo de una de las tareas (derecha)..

6.4 Infraestructura 4G

Antes de comenzar a desarrollar el prototipo, primero tendremos que elegir un software para establecer nuestra red 4G. Como explicamos en el DAFO de apartados anteriores, nuestra elección se decanta por el programa srsRAN 4G.

Tras elegir el software, hay que asegurarse de utilizar un Sistema Operativo compatible tanto con el software como con el resto de los programas y librerías que necesitaremos para que todo funcione correctamente. En nuestro caso elegimos la versión de Ubuntu 22.04 de baja latencia, que es la que instalaremos en el PC del laboratorio.

Para el Sistema Operativo emplearemos una imagen de Ubuntu cargada en un USB para poder arrancarla e instalarla correctamente. Una vez terminada la configuración del Sistema Operativo procedemos a la instalación del software necesario. Además, se han instalado aplicaciones de utilidad variada como es el caso del navegador Chromiun, el editor de texto Visual Studio Code, o el SDK Manager de NVIDIA que se necesitará más adelante para poder empezar a gestionar y arrancar la Jetson.

Para la instalación del software existen dos formas diferentes de hacerlo, una mediante la descarga de paquetes y la otra mediante la compilación del código fuente. Elegimos esta última ya que es la óptima para utilizar un BladeRF. Al proceder de esta manera, antes debemos instalar una serie de dependencias obligatorias incluidas en el siguiente comando.

```
sudo apt-get install build-essential cmake libfftw3-dev  
libmbedtls-dev libboost-program-options-dev libconfig++-dev  
libsctp-dev
```

A mayores, también tenemos que instalar unas dependencias adicionales ya que vamos a utilizar un BladeRF. Es importante tener en cuenta que nuestro BladeRF ya tiene instalada una imagen FPGA por lo que ese paso no hace falta realizarlo.

```
sudo add-apt-repository ppa:nuandllc/bladerf  
sudo apt-get update  
sudo apt-get install bladerf  
sudo apt-get install libbladerf-dev
```

Esta versión de Ubuntu no es compatible con alguna librería del paquete enb de srsRAN que instalaremos posteriormente, por lo que para su correcto funcionamiento también tenemos que instalar los programas SoapySDR, el *plugin* Soapy BladeRF y UHD (USRP Hardware Driver), todos ellos junto a sus dependencias correspondientes.

```

sudo apt-get install cmake g++ libpython3-dev python3-numpy
swig
git clone https://github.com/pothosware/SoapySDR.git
cd SoapySDR
mkdir build
cd build
cmake ..
make -j`nproc`
sudo make install -j`nproc`
sudo ldconfig #needed on debian systems
SoapySDRUtil --info
cd build #use previous build directory, no need to run cmake
again
make -j4
sudo make install

```

```

git clone https://github.com/pothosware/SoapyBladeRF.git
cd SoapyBladeRF
mkdir build
cd build
cmake ..
make
sudo make install

```

```

sudo add-apt-repository ppa:ettusresearch/uhd
sudo apt-get update
sudo apt-get install libuhd-dev uhd-host

```

Todas las dependencias se deben instalar antes de compilar el srsRAN. Incumplir esto puede causar errores al ejecutar o impedir que funcione correctamente. Una vez instaladas todas las dependencias, compilamos el software con las siguientes líneas de código.

```
git clone https://github.com/srsRAN/srsRAN_4G.git
cd srsRAN_4G
mkdir build
cd build
cmake ../
make
```

Tras la correcta compilación y tras haber superado correctamente el test, ya podemos instalar el srsRAN 4G. Además, se copiarán los ficheros de configuración por defecto en la ruta `~/config/srsran`.

```
sudo make install
srsran_install_configs.sh user
make test
```

Para comprobar si el sistema está detectando la BladeRF, ejecutamos el siguiente comando. Este se utiliza para iniciar una sesión interactiva con el dispositivo donde puedes introducir comandos directamente para controlar y configurar el dispositivo en tiempo real. Una vez iniciada, ponemos en la terminal el comando con la opción de *info* para ver los datos del dispositivo.

```
bladeRF-cli -i
```

```
goat@goat-desktop:/etc/mosquitto$ bladerf-cli -i
[INFO @ host/libraries/libbladerf/src/helpers/version.c:103] FPGA version (v0.15.0)
is newer than entries in libbladerf's compatibility table. Please update libbladerf
if problems arise.
bladeRF> info

Board:                Nuand bladeRF 2.0 (bladerf2)
Serial #:             88faf0ad1e074f9c89b82dd02bbda210
VCTCX0 DAC calibration: 0x1eb7
FPGA size:            301 KLE
FPGA loaded:          yes
Flash size:           128 Mbit
USB bus:              4
USB address:          3
USB speed:            SuperSpeed
Backend:              libusb
Instance:             0
```

Figura 91. Información del BladeRF conectado al PC.

Para montar una red LTE necesitamos un eNodeB, un EPC y un UE. El UE no será necesario instalarlo ya que no utilizaremos la máquina como tal, sino un módem que configuraremos posteriormente.

Para instalar el srsENB requerimos una máquina con un SO basado en Linux, un front-end de RF como nuestro BladeRF y el srsRAN correctamente instalado. Cumplidos los requisitos, ejecutamos el código descrito a continuación.

```

sudo add-apt-repository ppa:srslte/releases
sudo apt-get update
sudo apt-get install srsenb
srsran_4g_install_configs.sh user

```

Los mismos requisitos necesitamos para instalar el srsEPC, y su instalación es también prácticamente idéntica.

```

sudo add-apt-repository ppa:srslte/releases
sudo apt-get update
sudo apt-get install srsepc
srsran_4g_install_configs.sh user

```

Para comprobar su funcionamiento correremos tanto el *core* como la estación base en la misma máquina, pero en diferentes terminales. Para su ejecución tenemos que insertar los siguientes comandos, especificando el directorio en el que se encuentran los ficheros de configuración y siempre ejecutando en primer lugar el srsepc.

```

sudo srsepc ~/.config/srsran/epc.conf
sudo srsenb ~/.config/srsran/enb.conf

```

Si la conexión entre el *core* y la estación base se ha realizado correctamente, deberá salir en la terminal donde se ha ejecutado el srsepc los siguientes mensajes.

```

--- Software Radio Systems EPC ---

Reading configuration file /home/goat/.config/srsran/epc.conf...
HSS Initialized.
MME S11 Initialized
MME GTP-C Initialized
MME Initialized. MCC: 0xf901, MNC: 0xff63
SPGW GTP-U Initialized.
SPGW S11 Initialized.
SP-GW Initialized.
Received S1 Setup Request.
S1 Setup Request - eNB Name: srsenb01, eNB id: 0x19b
S1 Setup Request - MCC:901, MNC:63
S1 Setup Request - TAC 7, B-PLMN 0x9f136
S1 Setup Request - Paging DRX v128
Sending S1 Setup Response

```

Figura 92. Conexión realizada entre srsenb y srsepc.

Tras la instalación, tenemos que poner a punto la configuración de ambos para poder establecer la red LTE. Comenzamos configurando el fichero *user_db.csv*, cuya función es servir como base de datos a las UEs que pueden usar la red. Esta base de datos será utilizada por el HSS para comprobar la autenticación del usuario que intenta conectarse a la red. Los parámetros más relevantes que modificar en este fichero son, en orden: el algoritmo de autenticación, el IMSI, la clave, el tipo del código del operador y el código del operador.

Para añadir un UE al fichero, tendremos que dirigirnos al directorio en el que esté guardado y abrirlo con un editor de texto. Iremos directamente a la última línea y ahí añadiremos los datos referentes al UE que queremos que se conecte a nuestra red. En el propio fichero viene la explicación de cada apartado, además de una serie de ejemplos. La siguiente línea es la que introducimos directamente al final del fichero.

```
goat,mil,901630000052123,f75d83f02d34d2404c954d5b84122ce2,opc
,475d93e7358438cf9d86d1e855dfa4b0,9000,000000005cc5,7,dynamic
```

El próximo fichero que tenemos que configurar es el *epc.conf*, encargado del funcionamiento del *core* de la red. En concreto tenemos que configurar el MCC y el MNC (que conforman los primeros 5 dígitos del IMSI) y el nombre del APN que queramos que tenga nuestra red. Los cambios se muestran a continuación.

```
mcc=901
mnc=63
apn=CABRA
```

Por último, configuraremos el *enb.conf*, encargado de configurar la estación base de la red. En este fichero tenemos que cambiar un par de cosas más, comenzando por el MCC y el MNC como en el caso anterior. A mayores modificaremos el número de PRBs, que está directamente relacionado con el ancho de banda, estableciéndolo a 25 (5MHz) ya que tenemos que dividirnos 15 MHz entre los 3 grupos de alumnos del laboratorio para evitar que haya interferencias. También modificaremos el EARFCN de bajada, ya que entre este valor y la banda de frecuencias en la que nos encontremos podemos fijar la frecuencia central de la red. En nuestro caso queremos fijar la frecuencia en 2662.5 MHz, estando en la banda 7 y necesitaremos un valor del EARFCN de 3175.

```
mcc=901
mnc=63
n_prb = 25
dl_earfcn = 3175
```

Al terminar de configurar todo lo referente al UE, lanzamos el *srsepc* y el *srsenb* y enchufamos el módem a un ordenador. Este intenta conectarse automáticamente a la red y si la autenticación es correcta saldrá por la ventana de la terminal donde está ejecutándose el *srsepc* los siguientes mensajes. A continuación, pueden verse los resultados mencionados:

```

S1 Setup Request - eNB Name: srsenb01, eNB id: 0x19b
S1 Setup Request - MCC:901, MNC:63
S1 Setup Request - TAC 7, B-PLMN 0x9f136
S1 Setup Request - Paging DRX v128
Sending S1 Setup Response
Initial UE message: LIBLTE_MME_MSG_TYPE_ATTACH_REQUEST
Received Initial UE message -- Attach Request
Attach request -- IMSI: 901630000052123
Attach request -- eNB-UE S1AP Id: 1
Attach request -- Attach type: 2
Attach Request -- UE Network Capabilities EEA: 11110000
Attach Request -- UE Network Capabilities EIA: 11110000
Attach Request -- MS Network Capabilities Present: false
PDN Connectivity Request -- EPS Bearer Identity requested: 0
PDN Connectivity Request -- Procedure Transaction Id: 1
PDN Connectivity Request -- ESM Information Transfer requested: false
Downlink NAS: Sending Authentication Request
UL NAS: Received Authentication Response
Authentication Response -- IMSI 901630000052123
UE Authentication Accepted.

```

Figura 93. Autenticación exitosa del UE con la red 4G.

Para terminar con la configuración de la red tenemos que especificar la interfaz que conecta la máquina a Internet ya que, si no lo hacemos, la conexión se establecerá, pero no dará servicio. Para ello tenemos que ejecutar el siguiente comando, donde `enp0s25` es la interfaz que nos conecta a Internet en nuestra máquina de laboratorio.

```
sudo srsepc_if_masq enp0s25
```

Por último, instalaremos un servidor MQTT para realizar el control de estado del vehículo conectado. A mayores, necesitaremos un cliente MQTT para enviar los mensajes con los tópicos deseados para controlar el estado del coche. Los comandos para instalar ambos servicios son los siguientes:

```
apt install mosquitto
apt install mosquitto-clients
```

Para configurar este servidor, accedemos al directorio `/etc/mosquitto/conf.d` y generaremos un fichero extra de configuración donde escribiremos estas líneas de código.

```
listener 1883
allow_anonymous true
```

Para poner en funcionamiento el servicio MQTT, primero tenemos que ejecutar el servidor con los siguientes comandos, especificando el fichero de configuración deseado.

```
cd /etc/mosquitto
sudo mosquitto -c mosquitto.conf
```

Posteriormente mandaremos las órdenes a partir del siguiente comando, en el que indicamos la IP del servidor, el tópico para enviar mensajes al coche (ID/command) y el contenido del mensaje que le dirá al vehículo lo que debe realizar.

```
mosquitto_pub -h 192.168.73.3 -t 1/command -m "AM-Cloud"
```

Por último, para establecer la conexión procedemos a configurar el vehículo DeepRacer. En concreto, abriremos el fichero *vehicle.conf* y cambiaremos las direcciones IP y el puerto tanto del servidor MQTT como del servidor de Cloud. Para saber la dirección IP, simplemente ejecutamos el comando `ip addr` para ver la dirección del PC.

```
[mqtt]
vehicle_id = 1
mqtt_server_ip = 192.168.73.3
mqtt_server_port = 1883
[cloud_autonomous_driving]
cloud_server_ip = 192.168.73.3
cloud_server_port = 20001
[steering_calibration]
max = 1790000
mid = 1500000
min = 1250000
```

Es importante tener en cuenta que el ordenador situado en el coche puede detectar el módem como una unidad de almacenamiento. Para que lo reconozca como un módem, hay que ejecutar una línea de código representada en el siguiente bloque de comandos. Tras ejecutarlo, introduciremos el comando para conectar el coche tanto al Cloud como al MQTT. Si todo funciona correctamente, la luz trasera del vehículo se pondrá en amarillo.

```
sudo usb_modeswitch -v 12d1 -p 1f01 -J
sudo systemctl restart ARTEMIS.service
```

6.5 Inteligencia Artificial

En capítulos previos, hemos abordado los conceptos teóricos relacionados con el desarrollo de la parte de Inteligencia Artificial del proyecto. Por ende, a continuación, se analizan los distintos pasos seguidos para desarrollar dichos conceptos teóricos.

En primer lugar, tal y como se ha mencionado en el capítulo relacionado con la Inteligencia Artificial, debemos desarrollar un dataset con imágenes de los objetos que se desean analizar y clasificar. Además, emplearemos un sistema basado en *Computer Vision* (Visión por ordenador) con el que lograremos a través de imágenes extraer los datos necesarios relativos a cada señal de tráfico y asignarles una etiqueta que definirá el comportamiento del vehículo.

6.5.1 Desarrollo de los datasets

Con el objetivo de desarrollar una demostración lo más parecida posible al entorno real del proyecto, trataremos de generar dos dataset. Uno de ellos empleando señales de tráfico reales extraídas de internet y de distintas bases de datos, y un segundo dataset empleando las imágenes de las señales en miniatura tomadas por el propio coche.

La generación del primer dataset se han tomado y etiquetado imágenes de señales de tráfico reales, como ya se ha explicado en detalle los puntos a seguir en apartados previos carece de interés repetir los mismos procedimientos (En caso de querer ver los pasos seguidos consulte el apartado 3.3 Generación de un dataset).

No obstante, la generación del segundo dataset, basado en señales en miniatura requiere de un punto adicional. Debido a que las imágenes de entrenamiento deben de ser lo más similares posibles al caso real, desarrollamos un script en Python capaz de acceder a la cámara del coche y obtener imágenes de su entorno directamente desde ella, imágenes que más adelante serán empleadas para entrenar y validar el modelo. A continuación, se puede apreciar parte del script desarrollado para la captura de vídeo por parte del vehículo, el código completo puede consultarse en el Anexo A al final del documento (ver Anexo A-Scripts).

```
#Esta función permite capturar varias imágenes seguidas, es decir,
un video.
def capture_video():
    #Iniciamos la cámara.
    cam = cv2.VideoCapture(0)

    #Establecemos el codec.
    fourcc = cv2.VideoWriter_fourcc(*"XVID")

    #20.0 indica los frames por segundo, y 640x480 la resolución
    obtenida de "artemis_autonomus_car.py" dentro de
    "servicios_cloud_deep_racer" en el campus.
    video_out = cv2.VideoWriter(os.path.join("Videos_capturados",
"Video_" + time.strftime("%Y-%m-%d_%H-%M-%S") + ".avi"), fourcc,
20.0, (640, 480))

    #Bucle infinito que saca fotos hasta que se pulsa la tecla q del
    teclado.
    while True:
        #Capturamos y guardamos.
        retval, frame = cam.read()
        video_out.write(frame)

        if cv2.waitKey(500) & 0xFF == ord('q'):
            break

    #Liberamos recursos para no consumir.
    cam.release()
    video_out.release()
```

Código 1. Función capaz de capturar video desde la cámara del vehículo.

Gracias a este script hemos sido capaces de obtener distintos fotogramas a partir de la cámara del vehículo en distintos entornos, con diferentes señales y diferente iluminación, todo con el objetivo de lograr una mayor variedad en las imágenes. A continuación, se muestra una de las imágenes capturadas con el propio cochecito.



Figura 94. Ejemplo de fotograma capturado con el propio vehículo.

Una vez capturadas una gran cantidad de imágenes, a través de la plataforma Roboflow procederemos a su etiquetado, pero antes debemos definir las distintas posibles clases a las que puede pertenecer las señales detectadas, debido a que disponemos de una menor cantidad de señales en miniatura en nuestro modelo, optaremos por crear menos clases y centrarnos solo en las señales más importantes.

	OBLIGATORIO-CONTINUAR-RECTO
	OBLIGATORIO-GIRAR-DERECHA
	OBLIGATORIO-GIRAR-IZQUIERDA
	PROHIBIDO
	STOP
	VELOCIDAD-MAX-30
	VELOCIDAD-MAX-90

Figura 95. Clases creadas para el desarrollo del dataset con señales en miniatura.

Tras etiquetar todas las imágenes, seleccionamos las mismas características para el preprocesado y la aumentación que el dataset de señales de tráfico reales. Esto se debe a que se ha tomado la decisión de generar cada dataset de la forma más parecida al otro con el fin de poder compararlos de cara a un futuro. (En caso de querer ver los valores de preprocesado y aumentación consulte el apartado 3.3 Generación de un dataset).



Figura 96. Estado del dataset generado con imágenes de señales en miniatura.

En caso de querer comprender como emplear Roboflow, puede consultar el Anexo C (ver Anexo C-Tutorial Roboflow) para obtener más información y los pasos a seguir para repetir el proceso de creación de un dataset de igual forma que durante el desarrollo del prototipo.

6.5.2 Entrenamiento del modelo

Siguiendo el DAFO realizado en capítulos previos referente a Inteligencia Artificial, optaremos por emplear el algoritmo YOLOvX, ampliamente utilizado en sistemas basados en *Computer Vision*. Mas concretamente, emplearemos la versión YOLOv9 que destaca por su rapidez y precisión frente a otras versiones como YOLOv5 o YOLOv7. Para poder exportar el dataset a través de la API de Roboflow, podemos emplear el servicio web y seleccionar “Exportar dataset” en la pestaña de generación para que nos faciliten el código necesario agregar a nuestro script para poder añadir el dataset.

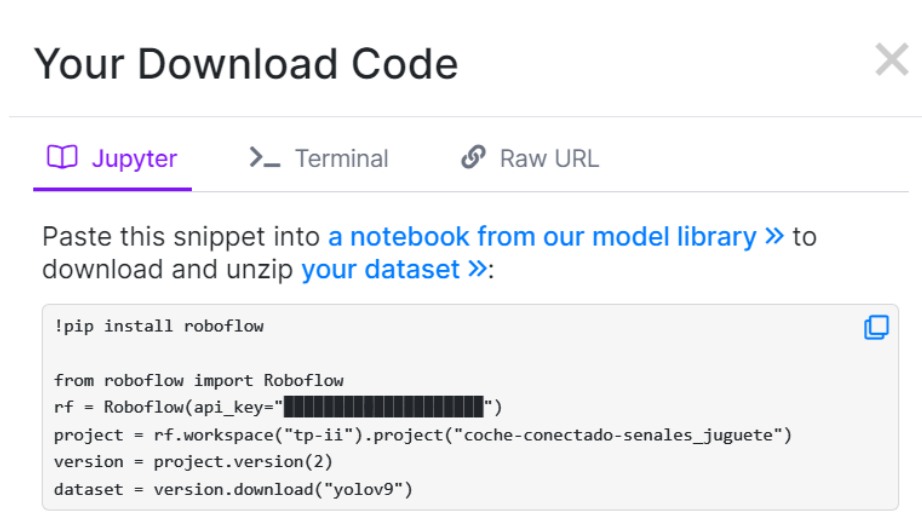


Figura 97. Empleo de la API de Roboflow para poder exportar el dataset a un entorno Jupyter Notebook.

Una vez desarrollados ambos datasets, continuaremos con el entrenamiento de los dos modelos, para ello, al igual que se ha explicado en capítulos previos, trataremos de entrenar ambos de la misma manera con los recursos ofrecidos en Google Collab. Debemos abrir un cuaderno de

Jupyter y añadir una serie de celdas para descargarnos los requisitos para entrenar el modelo, agregar el dataset a través de la API de Roboflow, sincronizar una carpeta en Google Drive (para poder guardar resultados) y mostrar los resultados obtenidos por pantalla.

A continuación, podemos ver la celda encargada de cargar el dataset a través de la API y la celda encargada de entrenar el modelo. Para poder cargar el dataset debemos de emplear la API_KEY (secreta y única para cada usuario), el nombre del *Workspace* (en nuestro caso “tp-ii”) y el identificador del proyecto (en este caso “coche-conectado-senales-juguete”), el código completo puede verse en el Anexo A (ver Anexo A-Scripts).

```
#A continuación mostramos los dos datasets posibles a cargar, en la práctica
solo deberá elegirse una de las opciones para entrenar el modelo cada vez.
%cd {HOME}/
!pip install roboflow

# Opción A: Cargamos roboflow y añadimos el dataset de señales en miniatura.
from roboflow import Roboflow
rf = Roboflow(api_key="*****") #La API_KEY es secreta.
project = rf.workspace("tp-ii").project("coche-conectado-senales_juguete")
version = project.version(2)
dataset = version.download("yolov9")

# Opción B: Cargamos roboflow y añadimos el dataset de señales reales.
from roboflow import Roboflow
rf = Roboflow(api_key="*****") #La API_KEY es secreta.
project = rf.workspace("tp-ii").project("coche-conectado-senales-reales")
version = project.version(5)
dataset = version.download("yolov9")
```

Código 2. Fragmento de código modificado para cargar los datasets extraído del Jupyter Notebook.

```
# Entrenamos el modelo con cualquiera de los dos dataset.
%cd {HOME}/yolov9

!python train.py \
--batch 16 --epochs 20 --img 640 --device 0 --min-items 0 --close-
mosaic 15 \
--data {dataset.location}/data.yaml \
--weights {HOME}/weights/gelan-c.pt \
--cfg models/detect/gelan-c.yaml \
--hyp hyp.scratch-high.yaml
```

Código 3. Fragmento de código para entrenar el modelo extraído del Jupyter Notebook.

Completar el entrenamiento demora un periodo de tiempo relativamente largo, puede ser mayor o menor en función del valor asignado a la variable `epochs`. A mayor valor asignado, más tiempo de entrenamiento necesario. En nuestro caso, y empleando los recursos de computación en paralelo de Google Collab nos demoró ente 30-45min para ser completado. El resultado generado es un fichero de pesos denominado `best.pt` que emplearemos más adelante para generar una inferencia.

6.5.3 Resultados obtenidos

Una vez entrenados el modelo con el dataset de señales de tráfico reales y el modelo con el dataset de señales en miniatura, debemos obtener los resultados en cuanto a rendimiento y precisión obtenidos para poder validar, cambiar o reconfigurar alguno de los pasos realizados previamente. Para ello, optaremos por añadir una celda a mayores al Jupyter Notebook con la que podremos mostrar por pantalla las gráficas obtenidas, la matriz de confusión y las detecciones realizadas en las imágenes de validación.

```
# Mostramos la imagen de las gráficas resultado, la imagen de la matriz
de confusión y la imagen con los resultados de validación del modelo.
!pip install matplotlib
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

graficas = mpimg.imread(f"{HOME}/runs/train/exp/results.png")
matriz = mpimg.imread(f"{HOME}/runs/train/exp/confusion_matrix.png")
validation = mpimg.imread(f"{HOME}/runs/train/exp/val_batch0_pred.jpg")

plt.imshow(graficas)
plt.imshow(matriz)
plt.imshow(validation)

plt.axis('off') # Desactivar los ejes si no son necesarios
plt.show()
```

Código 4. Fragmento de código para extraer los resultados del modelo extraído del Jupyter Notebook.

Con este fragmento de código podemos ser capaces de extraer resultados a partir de cualquiera de los modelos generados con cada uno de los dataset. Debemos de tener en cuenta que el dataset de señales en miniatura está completamente entrenado y validado con imágenes de señales en miniatura, sin embargo, el dataset de señales reales ha sido entrenado con señales reales y validado con una mezcla de señales reales y en miniatura.

El objetivo de esto es confirmar y demostrar como los modelos son muy sensibles al entorno, la cámara y los objetos empleados para entrenar el modelo, es decir, a pesar de ser señales de tráfico en ambos casos, podremos comprobar como el dataset de señales reales presenta un rendimiento más bajo debido a que ha sido preparado para reconocer señales de tráfico reales y no de juguete. Una forma de evitar esto sería entrenar el dataset de imágenes reales con imágenes de señales de juguete para que también aprenda a identificarlas.

A continuación, tenemos una comparación de las matrices de confusión obtenidas con cada uno de los datasets (ver siguiente página). A simple vista se puede observar que la matriz de confusión tiene una parte con ligera incertidumbre en la zona céntrica, es decir, tiende a confundir las clases de PROHIBIDO-GIRAR-DERECHA y PROHIBIDO-GIRAR-IZQUIERDA. Además, hay una mayor cantidad de falsos negativos repartidos por toda la matriz.

Es interesante destacar como en muchas ocasiones ambos modelos tienden a confundir el entorno o el fondo de la imagen como una señal, se puede apreciar esto en la última columna de cada matriz, donde aparecen predicciones de señales que realmente eran el fondo del escenario.

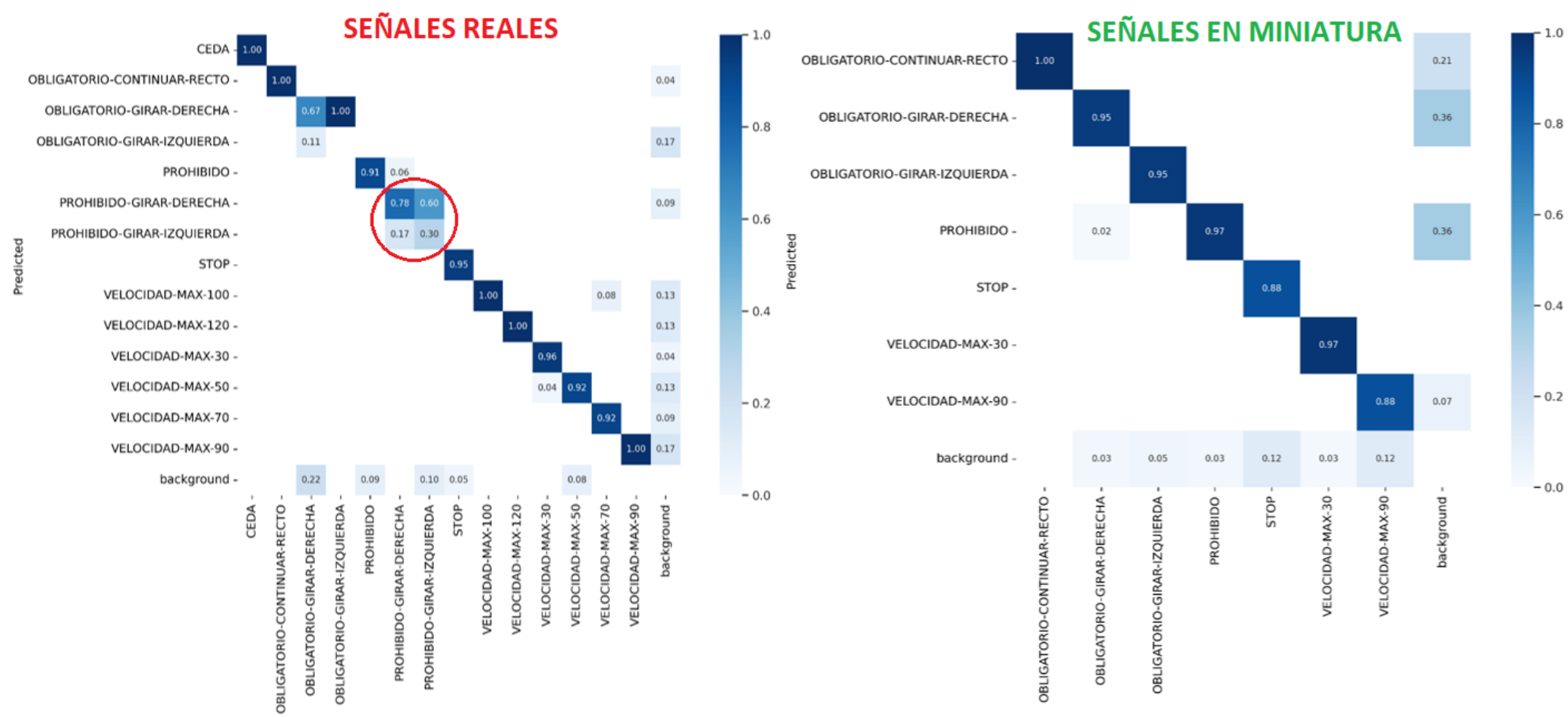


Figura 98. Matrices de confusión del dataset de señales de tráfico reales (izquierda) y del dataset de señales en miniatura (derecha).

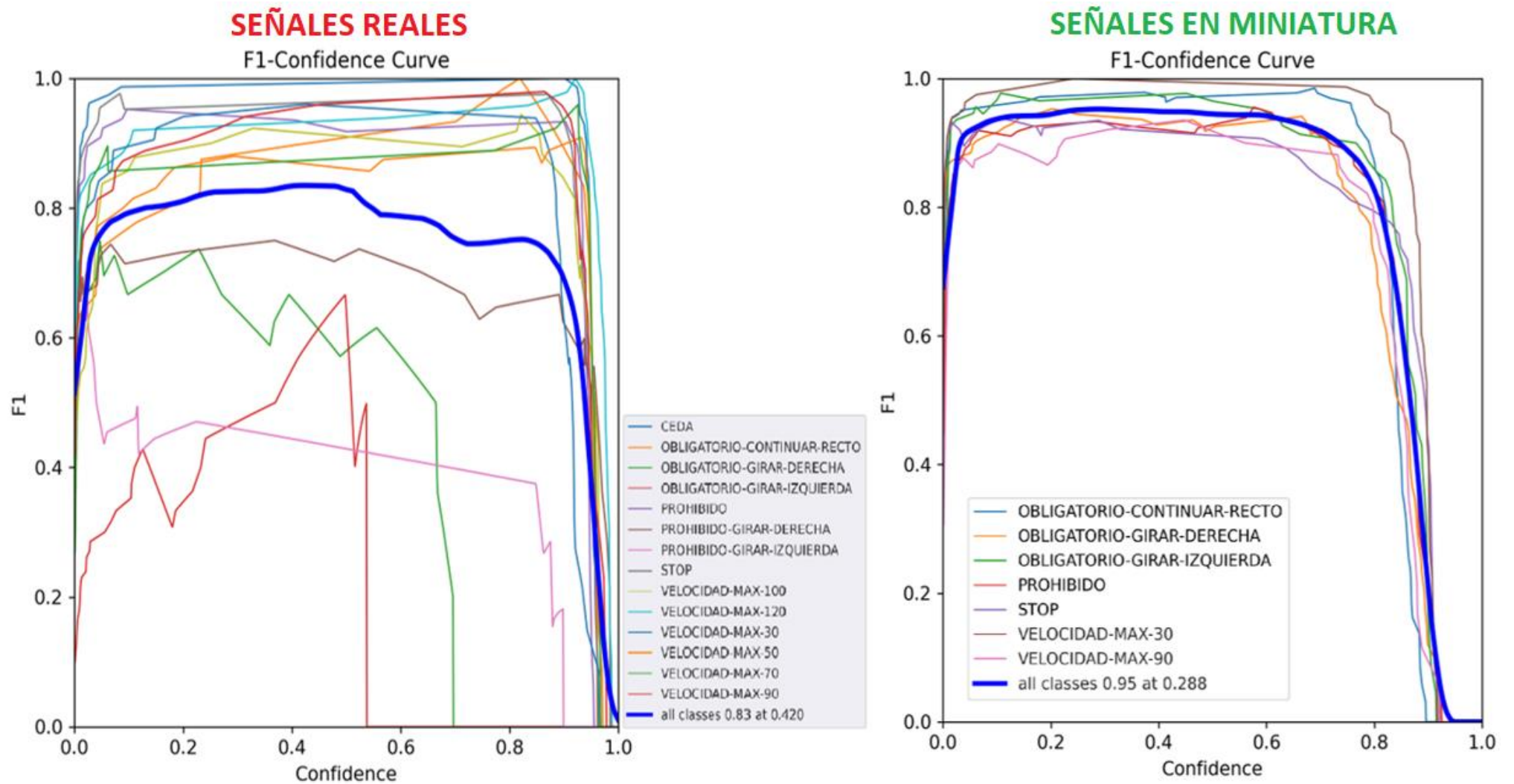


Figura 99. Gráficas F1-Confiianza del dataset de señales de tráfico reales (izquierda) y del dataset de señales en miniatura (derecha).

Además de las matrices de confusión, disponemos también de las gráficas F1, curvas de precisión-confianza, precisión-*recall*, *recall*-confianza y las propias imágenes sometidas al proceso de validación. Las gráficas F1-Confianza (ver Figura 99), representan la relación ente dos métricas de gran importancia a la hora de evaluar sistemas basados en detección de objetos en imágenes. La puntuación F1, que evalúa de manera indirecta la precisión y el *recall* mediante una sencilla fórmula (ver Ecuación 4) y el umbral de confianza que define el punto a partir del cual se decide si una predicción llega al puntaje suficiente para ser considerada de una clase o no.

Debido a la estructura de la ecuación para la puntuación de F1, el hecho de variar o mover el valor del umbral de confianza afecta a la precisión y el *recall* por lo que, también varía la puntuación de F1. Como podemos observar en el caso del dataset generado a partir de imágenes de señales reales las gráficas son mucho más erráticas e inestables que para el caso del dataset con señales en miniatura. Además, cuanto mayor sea la puntuación de F1 (esto es más cercano a 1.0) mejor resultados tendrá el modelo, y como se puede observar el dataset entrenado y validado solamente con imágenes de señales en miniatura todas las clases tienen puntuaciones muy altas y estables llegando a superar todas, el valor de 0.8 de puntuación (a excepción claro de valores del intervalo de confianza próximos a 1.0).

Finalmente, cabe destacar como las clases del dataset generado con señales reales se mantienen durante un intervalo de umbral de confianza mayor, ya que como se puede apreciar en la propia figura, las gráficas de cada clase comienzan a decaer en puntuación en torno a valores de umbral de confianza de 0.93-0.95, sin embargo, en el caso del dataset de señales en miniatura, la puntuación asignada a las clases comienza a decaer en torno a 0.8 de umbral de confianza.

Habiendo comparado los sistemas de medición de resultados más relevantes, comentaremos ahora otros resultados obtenidos de menor importancia, ya que un valor alto en estos resultados no implica necesariamente buenos resultados en el modelo. Entre estos resultados podemos observar las curvas de precisión-confianza y *recall*-confianza:

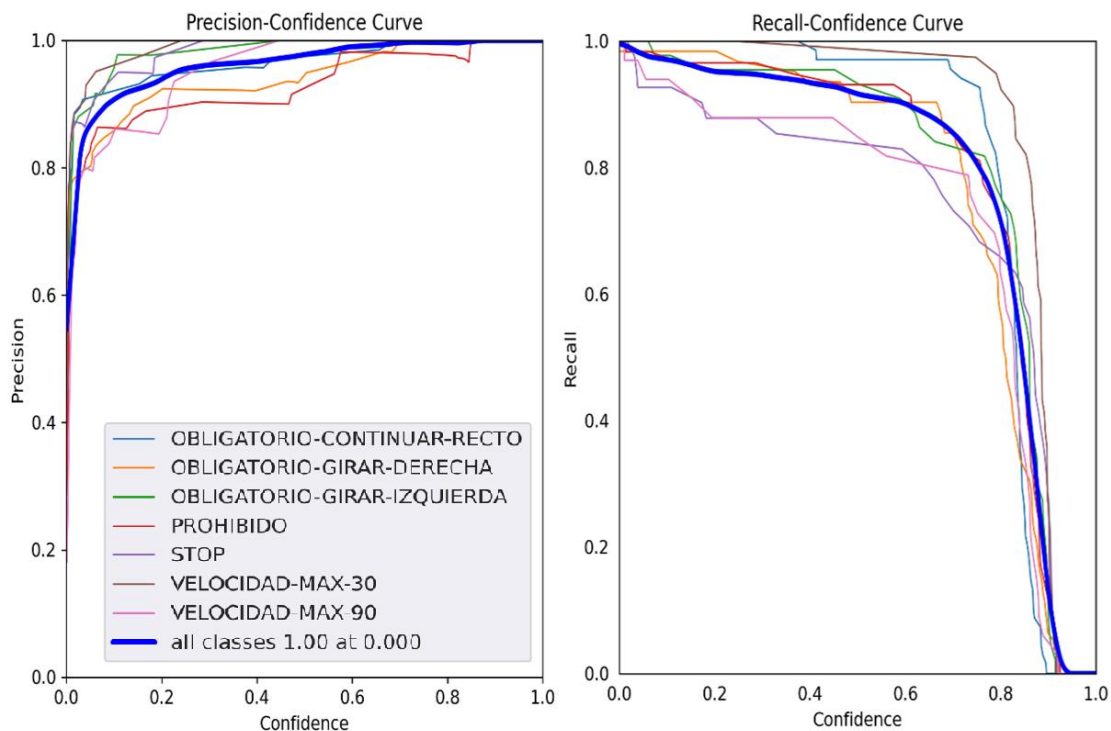


Figura 100. Curvas precisión-confianza y *recall*-confianza del dataset generado con señales en miniatura.

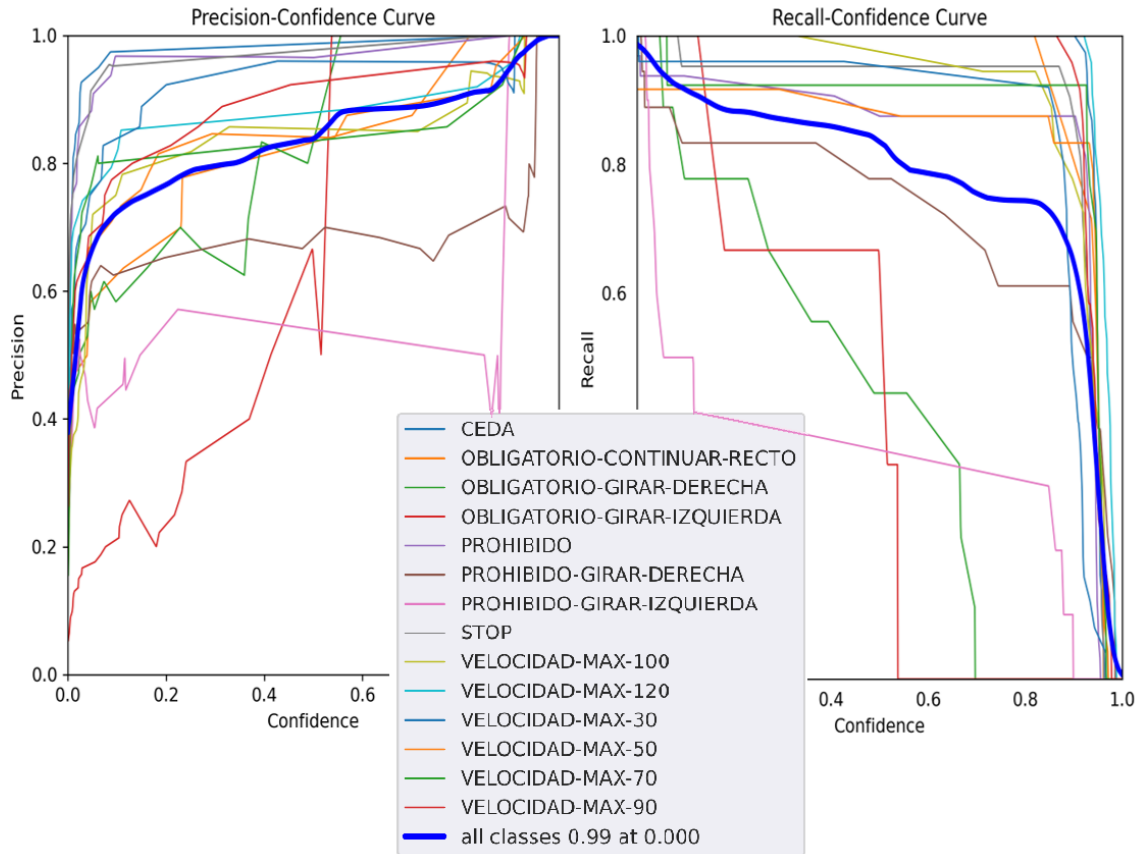


Figura 101. Curvas precisión-confianza y recall-confianza del dataset generado con señales reales.

Al igual que ocurría con los resultados de las gráficas anteriores, en este caso ocurre algo similar, los resultados del dataset de señales en miniatura presentan una mejor estructura y estabilidad que los datos del dataset de señales reales, además de alcanzar mejores puntuaciones, como podemos ver se pueden apreciar dos cosas comparando las gráficas:

- En el caso de la precisión, las puntuaciones del dataset de señales en miniatura rondan valores superiores al 0.8 en casi todo el intervalo del umbral de confianza, lo cual indica que la cantidad de predicciones correctas es casi idéntica a la cantidad de predicciones realizadas, es decir que de todas las predicciones que el modelo realiza casi todas son correctas. No obstante, en el caso del dataset de señales reales, podemos ver como la media de dichos valores ronda el 0.65-0.7 lo cual indica que acierta en torno a 3 de cada 4 predicciones realizadas.
- En el caso del *recall*, vemos como las puntuaciones de ambos disminuyen progresivamente conforme aumenta el umbral de confianza, esto se debe a que cuanto mayor sea este umbral más puntuación necesitará una señal para pertenecer a la clase correcta, por lo que solo se consideran como correctas aquellas de las que se esté realmente seguro (en umbrales altos), no obstante en el caso del dataset de señales en miniatura vemos que el decrecimiento es menos acelerado que en el caso del dataset con señales reales.

Las conclusiones que podemos sacar son que una buena puntuación de precisión indica que nuestro modelo acierta muchas de las predicciones que realiza, pero si la puntuación de *recall* es baja el modelo es malo, debido a que hay muchas identificaciones correctas que el modelo no está siendo capaz de ver.

En el caso de las curvas de precisión-*recall*, debemos de intentar obtener el máximo área debajo de la curva, por lo que cuanto más se aproxime la curva a los extremos superior-derecho, mejor será el modelo (siendo la esquina superior derecha lo ideal). A continuación, podemos comparar las curvas precisión-recall de ambos datasets.

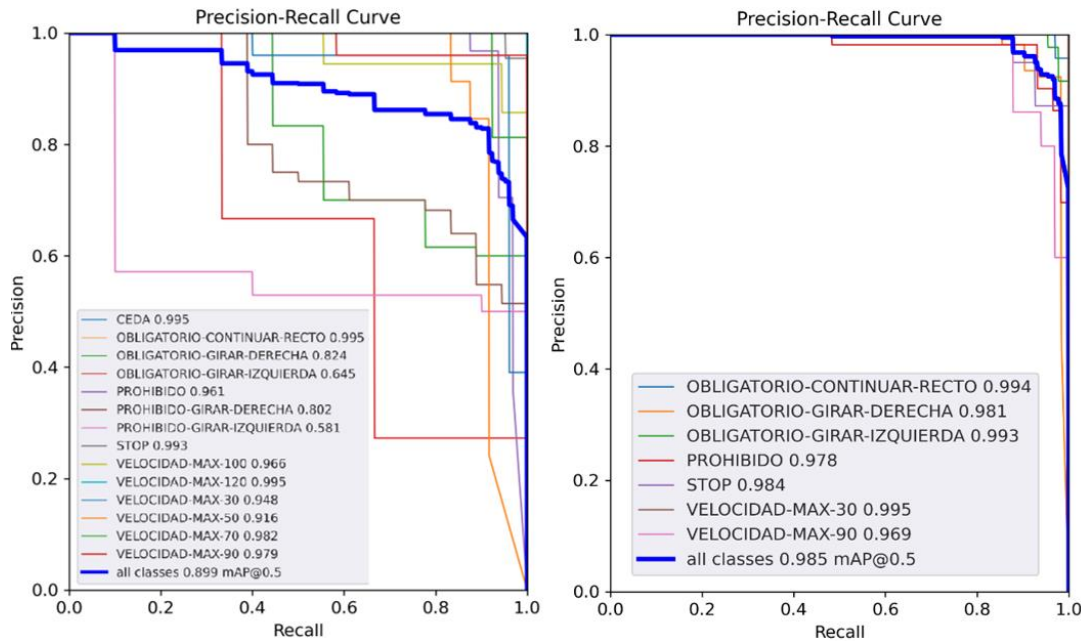


Figura 102. Gráficas Precisión-recall del dataset de señales de tráfico reales (izquierda) y del dataset de señales en miniatura (derecha).

A simple vista se puede apreciar la enorme superioridad del modelo generado con el dataset de señales en miniatura, que obtiene una mayor área bajo la curva a simple vista sin la necesidad de realizar ningún cálculo a mayores.

Finalmente, antes de observar los resultados en las propias imágenes de validación, observaremos las gráficas de *box_loss* y *cls_loss* tanto para entrenamiento como para validación, debemos de tener en cuenta lo siguiente (Huang, Tian, Tan, Dong, & Xu, 2023):

- **Criterio *box_loss*:** métrica asociada a la predicción de las coordenadas de las *bounding boxes* de los objetos detectados en la imagen. Consiste en medir la diferencia en cuanto a posición y tamaño de los cuadros predichos y los reales generados durante el proceso de etiquetado.
- **Criterio *cls_loss*:** métrica asociada a la predicción de la clase de cada objeto detectado en la imagen, es decir, mide como de bien se ajusta la etiqueta predicha a la real. Este tipo de métricas son importantes para que el modelo no solo sea capaz de encontrar objetos sino también de identificarlos correctamente.



Figura 103. Ejemplo de resultados modelo con problemas de *cls_loss* (izquierda), sin problemas (centro) y con problemas de *box_loss* (derecha) (Autoría propia).

A continuación, vamos a ver diferentes gráficas de pérdidas de tipo `box_loss` y de tipo `cls_loss` tanto para la etapa de entrenamiento como para la de validación para ambos datasets:

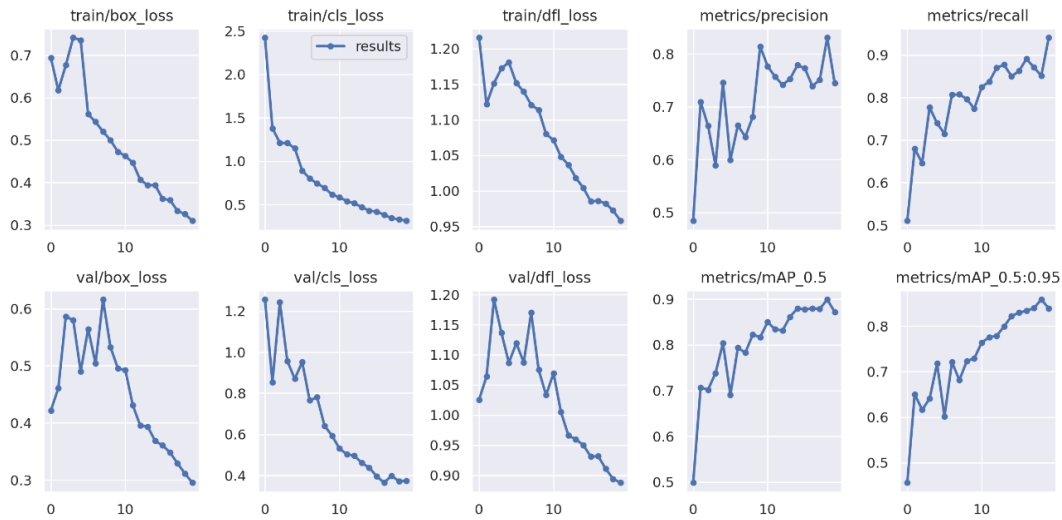


Figura 104. Gráficas `box_loss` y `cls_loss` del dataset de señales de tráfico reales.

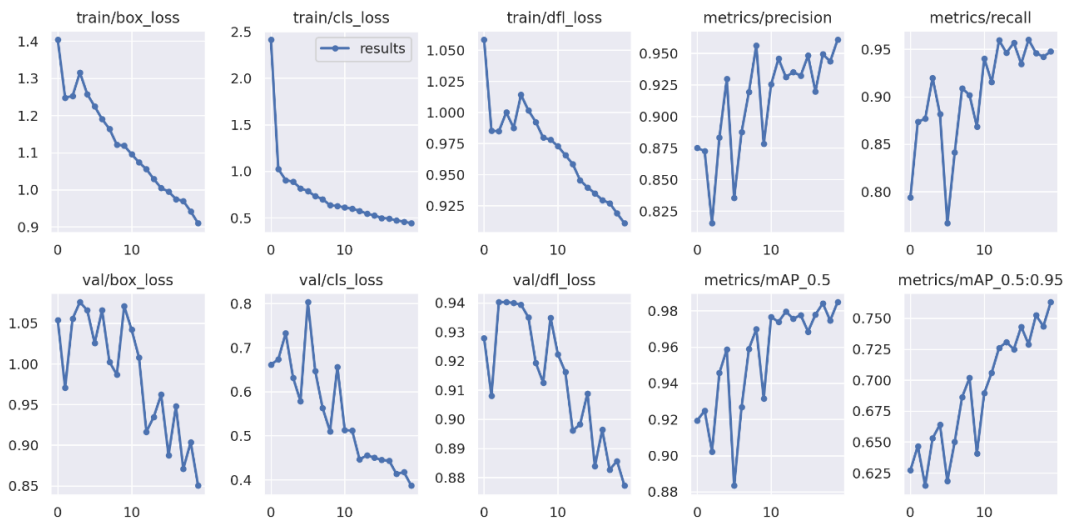


Figura 105. Gráficas `box_loss` y `cls_loss` del dataset de señales de tráfico en miniatura.

Como se puede ver, el dataset de señales en miniatura para el caso de la métrica `class_loss`, presenta unas pérdidas más bajas en la etapa de validación, llegando apenas a 0.8 mientras que el dataset de señales reales alcanza los 1.25 puntos de pérdidas. No obstante, resulta curioso destacar como en los términos de `box_loss` es, al contrario, es el dataset de señales en miniatura el que presenta unos peores resultados con un valor de pérdidas de en torno a 1.10 frente al 0.65 del dataset de señales reales.

Otra característica a tener en cuenta es que las gráficas de entrenamiento de ambas métricas son bastante similares en forma y puntuación, la explicación a esto es que ambos dataset se han entrenado con imágenes del mismo ámbito (por lo que los resultados son parecidos), pero en el caso de la validación el dataset de señales reales también contenía imágenes de señales en miniatura para validar, lo que explica esas notables diferencias entre la etapa de entrenamiento y la de validación.

Por último, vamos a analizar alguna de las imágenes sometidas a la etapa de validación, a continuación, podemos ver dos collages de cada uno de los datasets. Uno con los resultados de las predicciones y otro con las etiquetas reales que se asignaron a dichos objetos (puede verlas con un mayor tamaño en el Anexo D-Imágenes en gran escala):

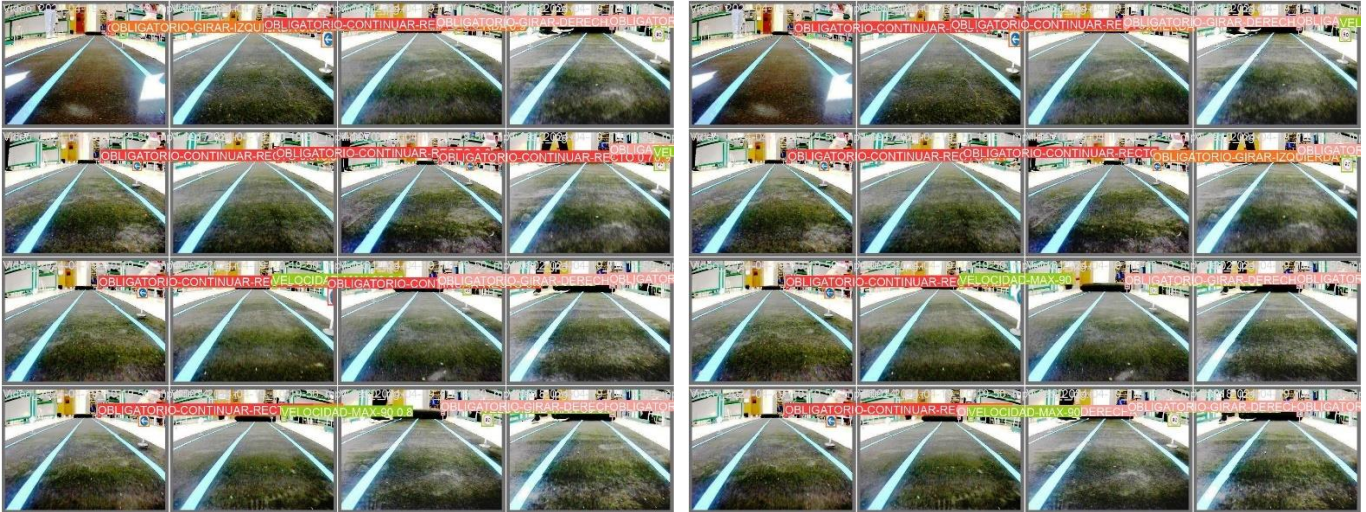


Figura 106. Collage con las predicciones sobre las imágenes (izquierda) y las etiquetas reales (derecha) del dataset de señales en miniatura.

Como se puede apreciar, el modelo funciona realmente bien, pues es capaz de predecir y acertar señales que ni siquiera etiquetamos nosotros, puede observar la primera imagen superior a la izquierda donde el modelo es capaz de predecir una señal de OBLIGATORIO-GIRAR-IZQUIERDA correctamente sin haberla etiquetado nosotros.

Por otro lado, el dataset con imágenes de señales reales:



Figura 107. Collage con las predicciones sobre las imágenes (izquierda) y las etiquetas reales (derecha) del dataset de señales reales.

En este caso, podemos ver como la 3era imagen de la fila inferior, hay un error ya que el modelo predice la etiqueta OBLIGATORIO-GIRAR-DERECHA y el caso real es OBLIGATORIO-GIRAR-IZQUIERDA. Demostrando nuestro análisis previo realizado sobre las gráficas de datos donde veíamos que el modelo entrenado con señales reales era ligeramente peor que el modelo entrenado con señales en miniatura.

Con todo esto, queda demostrado la sensibilidad de los modelos de detección de objetos a los datos, hemos podido comprobar como al meter imágenes de señales en miniatura para la validación del modelo de señales reales, los resultados empeoran notablemente. Es por ello, que debemos tener muy presente que, para entrenar correctamente un modelo, debemos emplear imágenes del caso real donde se va a implementar, y no imágenes obtenidas de un caso simulado en un laboratorio.

6.6 Implementación del modelo

Una vez entrenado el modelo y analizados sus resultados, llegó el momento de desplegarlo, para poder desplegarlo debemos crear una inferencia, es decir, una aplicación práctica del modelo que pretendemos desplegar. La inferencia de un modelo, creada a partir de los pesos obtenidos tras su entrenamiento, nos permitirá introducir nuevos datos en él y obtener así un resultado.

Vamos a tratar de desplegar el modelo de 3 formas distintas, cada una más compleja que la anterior. En primer lugar, trataremos de desplegarlo empleando las propias herramientas de Roboflow, en segundo lugar, emplearemos un ordenador que actuará de servidor y en tercer lugar la Jetson Orin Nano de NVIDIA, diseñada específicamente para ejecutar inferencias con una gran rapidez y precisión. Nota: para el caso del prototipo solo desplegaremos el dataset referente a señales en miniatura.

6.6.1 Implementación en Roboflow

El primer paso, por tanto, será desplegar nuestro modelo empleando las herramientas brindadas por el servicio web de Roboflow, para ello debemos seguir los siguientes pasos (Roboflow, Train a Model in Roboflow, 2024):

- 1- Una vez generado el dataset, al seleccionarlo podemos ver la opción de entrenarlo con Roboflow, esto llevará un tiempo, pero permitirá entrenar un modelo. Debemos tener en cuenta que, si empleamos la versión gratuita, nos consumirá uno de los 3 créditos disponibles gratuitos que nos regalan al crear un proyecto. Debemos seleccionar *Train with Roboflow*.

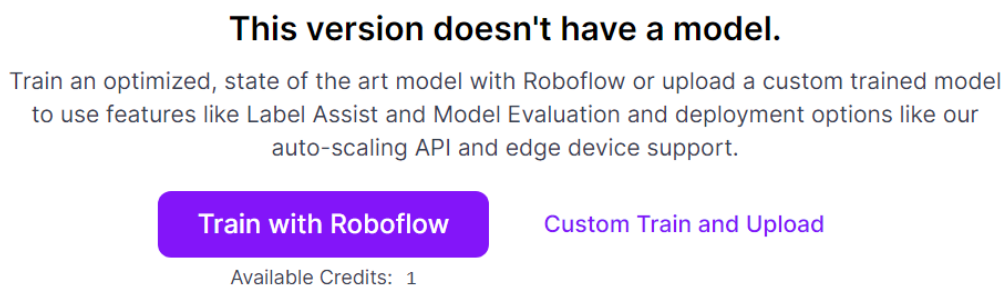


Figura 108. Opción de entrenar el modelo con Roboflow.

- 2- Una vez tengamos el modelo entrenado, en el menú lateral podemos seleccionar la opción dentro de *Models* -> *Visualize* y en la pestaña desplegable *Switch Model*: elegimos el modelo que acabamos de entrenar.

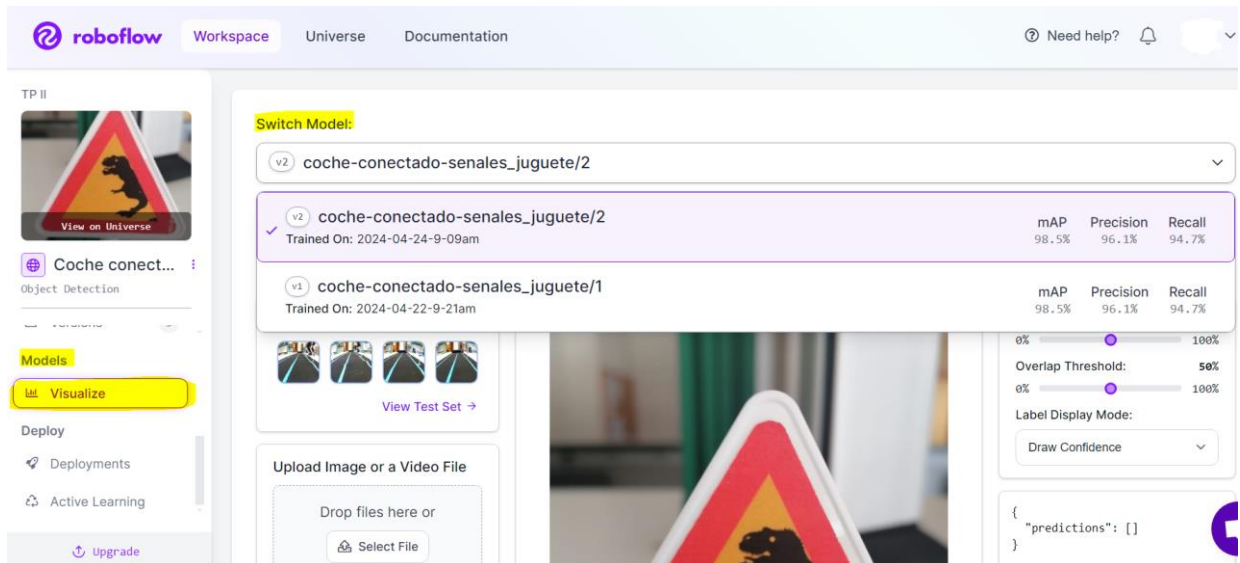


Figura 109. Desplegar modelo entrenado con Roboflow con su servicio web.

- 3- Una vez se haya desplegado podemos cargar cualquier foto que queramos para que nos devuelva los resultados estimados. Para ello, podemos elegir una de las imágenes de test, cargar nosotros nuestras propias imágenes y vídeos o cargar un enlace de un vídeo de YouTube o una imagen. Para elegir las imágenes de test podemos hacer click en cualquiera de las imágenes en miniatura debajo de *Samples from Test Set*. Si, por el contrario, queremos meter nuestras propias imágenes, podemos buscarlas desde el explorador de archivos o arrastrarlas directamente debajo de *Upload Image or a Video file*. Y, por último, en el caso de querer emplear un vídeo de YouTube o una imagen de Google, podemos añadir el URI de dicho vídeo debajo de *Paste YouTube or Image URL*.

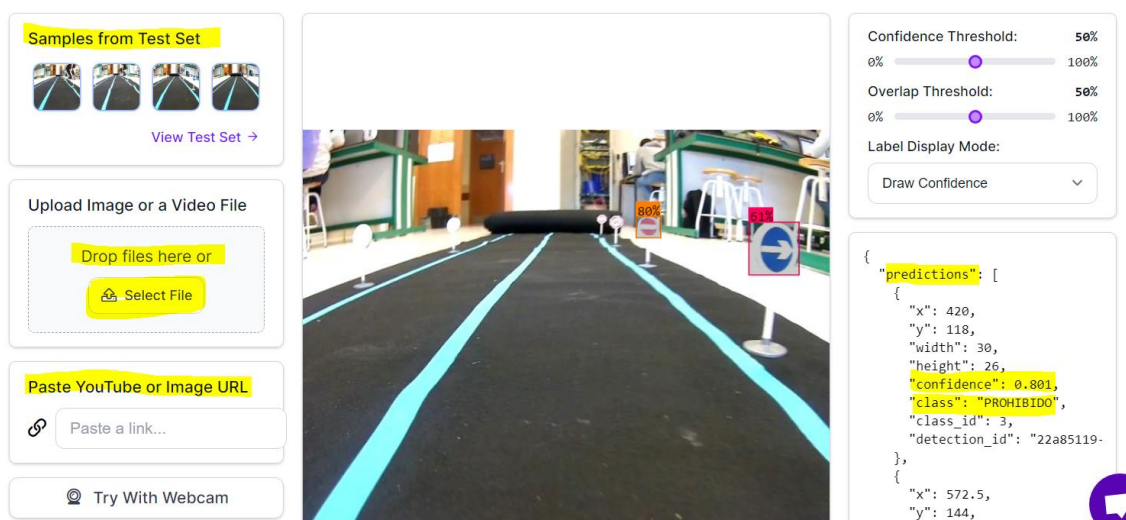


Figura 110. Cargar una imagen para obtener sus resultados.

- 4- Finalmente, a la derecha, se pueden apreciar las predicciones realizadas, con el nombre de la clase asociada y la puntuación de confianza obtenida. Además, en el centro aparecerá la imagen con los *bounding boxes* de cada detección realizada y su puntuación en tanto por ciento.

Nota: por motivos ajenos a nosotros, Roboflow a fecha de abril de 2024, está experimentando problemas a la hora de cargar vídeos ya sea a mano o mediante el uso de URIs.

6.6.2 Implementación en un servidor

La segunda manera de desplegar la inferencia consiste en emplear los pesos generados por el entrenamiento realizado en Google Collab y ejecutar un script en el ordenador para crear dicha inferencia. Para ello, tenemos dos opciones, desplegarlo empleando la API de Roboflow, lo que conlleva disponer de conexión a internet o desplegarlo de manera completamente local sin necesidad de tener conexión a internet (aunque sí que debemos tener descargado Python y las librerías correspondientes).

La primera opción, empleando la API de Roboflow. Para ello, debemos de ejecutar el siguiente script de código, en él debemos incluir la imagen o vídeo a procesar y los datos referentes a nuestro proyecto de Roboflow (Roboflow, Roboflow Inference: Effortless Local Computer Vision Deployment with Python, 2023).

```
#Script desarrollado para el despliegue online de una inferencia empleando la API de Roboflow.
!pip install inference
!pip install opencv-python

from inference.models.utils import get_roboflow_model
import cv2

#Metemops el modelo de Roboflow, la API_KEY es secreta.
model_name = "coche-conectado-senales_juguete"
model_version = "2"
model = get_roboflow_model(model_id="{} / {}".format(model_name, model_version), api_key="*****")

#Cargamos la imagen con OpenCV
frame = cv2.imread("/test/imagen_prueba")
results = model.infer(image = frame, confidence = 0.5, iou_threshold = 0.5)

#Mostramos la imagen.
if results[0]:
    bounding_box = results[0][0]
    print(bounding_box)
    x0, y0, x1, y1 = map(int, bounding_box[:4])
    cv2.rectangle(frame, (x0, y0), (x1, y1), (255,255,0), 10)

cv2.imshow("Image Frame", frame)
cv2.waitKey(0) #Espera a que una tecla sea pulsada.
cv2.destroyAllWindows() #Destruimos para no consumir recursos.
```

Código 5. Despliegue de una inferencia empleando la API de Roboflow.

A continuación, podemos ver los resultados obtenidos:



Figura 111. Frame resultado extraído de un vídeo sometido a la inferencia del modelo (primer método).

Como podemos ver los resultados son los mismos que en el caso de emplear Roboflow, no obstante, ambas opciones tienen una pega, requieren de conexión a internet en todo momento para poder funcionar, por ende, vamos a desplegar la inferencia de manera local sin necesitar de una conexión a internet (se entiende que las librerías y requerimientos necesarios están ya cargadas).

Para ello, podemos emplear el fichero *detect.py* proporcionado en los requerimientos del GitHub de YOLOV9. Para ello debemos de ejecutar la siguiente línea con el fichero *detect.py*, los pesos obtenidos de haber entrenado el modelo, y la imagen o vídeo que se quiere someter a la inferencia para obtener resultados.

```
!python /content/yolov9/detect.py --source /content/video.mp4 /
--weights /content/best.pt
```

De nuevo los resultados obtenidos son los esperados:



Figura 112. Frame resultado extraído de un vídeo sometido a la inferencia del modelo (segundo método).

6.6.3 Implementación en una Jetson

Finalmente, implementaremos el modelo de una tercera manera, empleando una de las herramientas más potentes para realizar diseños y desarrollo de modelos basados en Inteligencia Artificial, *Deep Learning* o cualquier otro sistema que requiera de gran computación en paralelo de datos. Para ello, emplearemos uno de los miniordenadores de NVIDIA, concretamente el modelo Jetson Orin Nano.

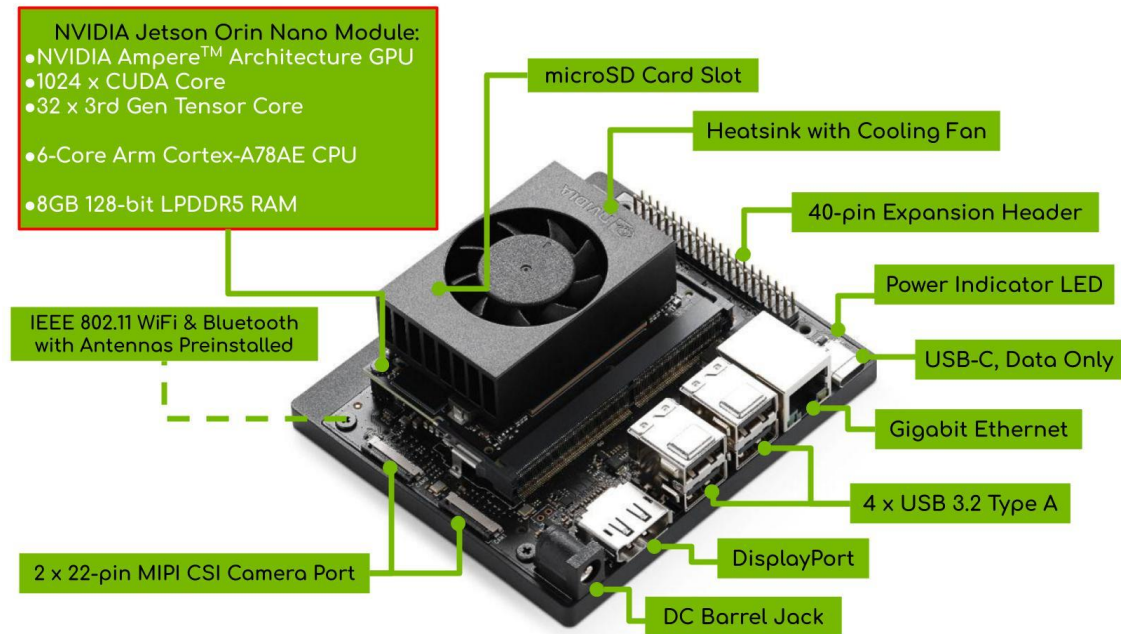


Figura 113. Microordenador Jetson Orin Nano de la familia NVIDIA con sus principales periféricos (NVIDIA).

Este microordenador presenta antenas de Wifi y Bluetooth ya instaladas, una CPU con 6 núcleos de tipo ARM Cortex-A78AE, memoria LPDDR5 integrada, un chip GPU con la arquitectura Ampere, una toma de corriente, un puerto de conexión tipo *Display-in*, 4 puertos tipo USB 3.2, un puerto de conexión por cable Ethernet, un puerto tipo C de datos, un cabezal con 40 pines para expandir sus periféricos, un ventilador integrado y una ranura para una microSD donde cargar el S.O., la imagen y todas las herramientas de software necesarias (NVIDIA, Jetson Orin Nano Specifications, 2024).

Antes de proceder, asumiremos que ya tenemos correctamente instalado en la microSD el S.O, las herramientas de desarrollo y la imagen para poder trabajar con la Jetson. En caso de querer consultar este proceso de instalación, consulte el Anexo E para más información (ver Anexo E-Tutorial Jetson).

Una vez la Jetson esté completamente operativa, trataremos de emplear las funcionalidades de ultralytics, esta empresa, pone a nuestra disposición herramientas de todo tipo para facilitar la adopción de los modelos de detección de objetos (sobre todo YOLOvX). Entre estas herramientas disponemos de una gran cantidad de documentación, herramientas para mejorar el rendimiento y funciones para poder desplegar los modelos (Ultralytics, 2024).

Entonces, emplearemos las funciones de ultralytics para desarrollar un pequeño script capaz de detectar objetos en imágenes, encerrarlos en una *bounding box* con una etiqueta asociada y mostrar el resultado por pantalla. A continuación, podemos ver el script desarrollado (en caso de querer ver el script completo, consulta Anexo A-Scripts):

```
#Imágenes en el directorio
image_files = [os.path.join(directory, file) for file in os.listdir(directory) if
file.endswith(('.jpg', '.jpeg', '.png'))]

for image_file in image_files:
    img = cv2.imread(image_file)
    height, width, _ = img.shape
    img = img[:, width // 2:, :]

    results = model(img)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    old_maxboxsize = 0 #Variable para detectar el bounding
    predictions = []

    #Coordenadas
    for r in results:
        boxes = r.boxes
        for box in boxes:
            # bounding box
            x1, y1, x2, y2 = box.xyxy[0]
            x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2) #Convertimos en int.

            #Añadimos el bounding box, el grado de confianza y la etiqueta de la clase.
            cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 255), 3)
            confidence = math.ceil((box.conf[0] * 100)) / 100
            print("Confidence --->", confidence)
            cls = int(box.cls[0])
            print("Class name -->", classNames[cls])

            # object details
            org = [x1, y1]
            font = cv2.FONT_HERSHEY_SIMPLEX
            fontScale = 1
            color = (255, 0, 0)
            thickness = 2

            prediction = []
            prediction.append(x1, x2, y1, y2, confidence)

            cv2.putText(img, classNames[cls], org, font, fontScale, color, thickness)

            #Trackeamos las señales.
            if (abs(x1-y2) > old_maxboxsize):
                old_maxboxsize = abs(x1-y2)
                box_prioritario = classNames[cls]

            predictions.append(prediction)

    track_results = mot_tracker.update(np.array(predictions))
    print(str(track_results))
    print("El bounding box de más a la derecha es: " + box_prioritario)

    fin2 = time.time()
    print("tiempo modelo: " + str(fin2-inicio2))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    cv2.imshow('Image', img)
    fin1 = time.time()
    print("tiempo ejecución completa: " + str(fin1-inicio1)) #Para medir los tiempos.
    cv2.waitKey(0)

cv2.destroyAllWindows()
```

Figura 114. Fragmento de código del script para desplegar la inferencia de forma local.

Los resultados de emplear este script los podemos ver a continuación:

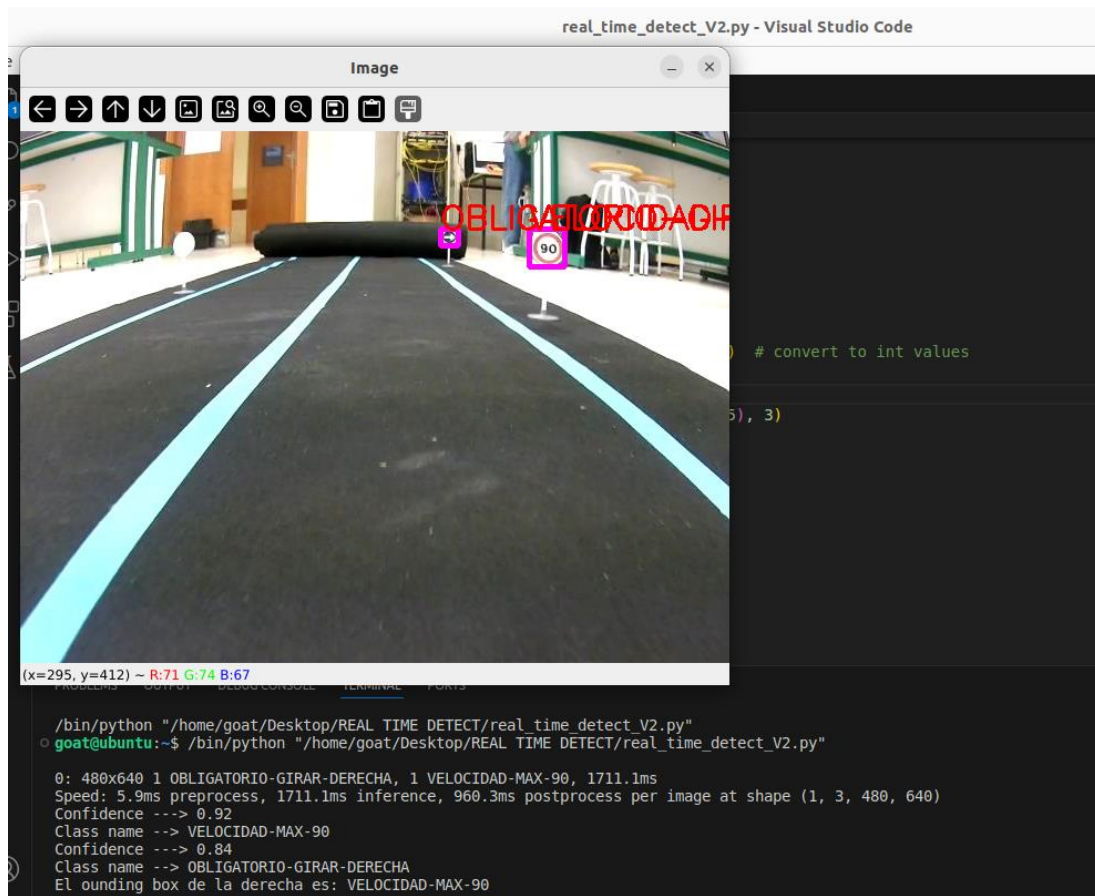


Figura 115. Resultado ejecución script despliegue modelo local.

Con este último resultado damos por finalizada la parte de implementación, el punto restante consistiría en implementarlo de forma conjunta con el resto del sistema.

6.7 Control del vehículo

Como se ha visto, el objetivo es realizar un prototipo de vehículo conectado. Para ello, bastaría con conectar el coche a la red 4G y que este sea capaz de transmitir el vídeo captado por la cámara y de ser controlado manualmente. No obstante, se ha querido realizar un prototipo de vehículo autónomo que sea capaz de manejarse solo al reconocer las señales.

Para el correcto funcionamiento del prototipo como vehículo autónomo, se necesitan implementar las siguientes funcionalidades:

- Manejo manual del vehículo
- Grabación de vídeo del coche
- Trasmisión de vídeo desde el coche al PC
- Análisis del vídeo/imágenes
- Etiquetado de las señales
- Control autónomo del vehículo en función de las señales detectadas

Con este objetivo, se han desarrollado varios ficheros Python con los que podremos manejar el coche. Algunos de estos ficheros han sido íntegramente elaborados por el grupo, mientras que otros han sido entregados por el profesorado algunos de los cuales han sido modificados para lograr la funcionalidad deseada para el prototipo.

A continuación, se van a ver cada uno de estos ficheros Python, así como explicar detalladamente la función que cumplen. El código de estos ficheros puede verse en el Anexo A (ver Anexo A-Scripts).

- ***artemis_autonomous_car.py***: fichero Python que define la clase *artemis_autonomous_car* para el control del vehículo autónomo. La clase contiene métodos para procesar imágenes, calcular parámetros de control y ejecutar el algoritmo de Stanley para el seguimiento de trayectorias. Inicializa parámetros como la resolución de la imagen, matrices de transformación para la perspectiva, y ajustes de control longitudinal y lateral. También contiene métodos para manejar la parada del vehículo, controlar el nivel de la batería y procesar datos del LiDAR. Asimismo, se ha modificado el fichero para que almacene todos los frames del vídeo recibido del coche en un directorio llamado '*images*'. Con el objetivo de no sobrescribir las imágenes y llamar a cada una con un nombre diferente, las imágenes son nombradas en función de una marca de tiempo con el instante en el que fueron creadas. Esta marca temporal, tiene el formato: AÑO-MES-DIA-HORA-MINUTO-SEGUNDO.
- ***car1_cloud_control_server.py***: fichero que crea un servidor UDP para el coche autónomo, recibiendo y procesando datos en tiempo real. Se configura la dirección de escucha con la IP y puerto del coche. Recibe datos, que pueden ser imágenes, datos LiDAR, o información de batería. Las imágenes recibidas se decodifican y se procesan en la clase *artemis_autonomous_car* para obtener comandos de control de giro y aceleración, los cuales se envían de vuelta. Los datos LiDAR y de batería se procesan según corresponda para mantener el funcionamiento del coche autónomo.
- ***car1_cloud_control_server_real_time_control.py***: fichero con la misma estructura que el fichero anterior. De nuevo, se define una dirección de servidor y un socket para recibir datos, que pueden ser imágenes, datos LiDAR, o información de batería y las imágenes se procesan llamando a la clase *artemis_autonomous_car*. La diferencia con el fichero anterior radica en que este se encarga de controlar el coche en tiempo real a mediante la introducción de comandos por teclado. Los comandos para controlar el vehículo son los siguientes:
 - 'a': avanzar
 - 'z': parar
 - '1': girar a la izquierda en el próximo cruce
 - '2': recto en el próximo cruce
 - '3': girar a la derecha en el próximo cruce
- ***car1_manual_control_server.py***: fichero similar a los dos anteriores que define una dirección de servidor y un socket para recibir y enviar datos. En este caso, nos permite controlar manualmente el coche a través del teclado. Para ello, utiliza la biblioteca *pynput* que permite capturar eventos del teclado y asociarlos a valores de control de acelerador o giro del coche. El control del vehículo se lleva a cabo con las teclas:

- 'w': marcha hacia delante
- 's': marcha hacia atrás
- 'a': girar a la izquierda
- 'd': girar a la derecha
- '2': marcha hacia delante a gran velocidad (no recomendado)
- 'x': marcha hacia atrás a gran velocidad (no recomendado)

Además, muestra una ventana de visualización en tiempo real para la imagen del vehículo y un mapa LiDAR utilizando OpenCV.

- ***real_time_detect_V3.py***: fichero encargado de realizar la detección y seguimiento de señales de tráfico en las imágenes utilizando el modelo YOLO y un *tracker* de objetos. Continuamente carga y procesa las imágenes almacenadas en el directorio '*images*'. A estas imágenes se les aplica el modelo YOLO para detectar las señales de tráfico y se dibuja un cuadro delimitador alrededor de ellas. Las predicciones de señales son filtradas y priorizadas por el tamaño del cuadro delimitador más grande, de esta manera se selecciona la señal más cercana que es la señal más relevante para el control del coche. La clase de la señal detectada es almacenada en un fichero de texto (*señales.txt*) que usaremos para controlar el vehículo
- ***control_vehiculo_autonomo.py***: script que se ocupa del control autónomo del vehículo. En primer lugar, activa el movimiento del vehículo haciendo que este se mueva hacia delante a una velocidad moderada. Tras esto, almacena las imágenes del vídeo recibidas del coche en un directorio llamando a la clase *artemis_autonomous_car.py*. Seguidamente, leerá las señales detectadas leyendo el contenido del fichero *señales.txt*. Cuando recibe una señal, el script se encarga de controlar el vehículo en función de la señal detectada. El control del vehículo se lleva a cabo mediante las variables *control_acelerador* y *control_giro* mediante las cuales se define la velocidad del coche y la rotación de las ruedas respectivamente. El vehículo es capaz de reaccionar ante las señales para las que se entrenó el modelo, realizando la siguiente función para cada una de ellas:
 - OBLIGATORIO-CONTINUAR-RECTO: establece que no se permita girar las ruedas durante un cierto tiempo.
 - OBLIGATORIO-GIRAR-DERECHA: hace que el coche gire a la derecha durante un cierto tiempo.
 - OBLIGATORIO-GIRAR-IZQUIERDA: hace que el coche gire a la izquierda durante un cierto tiempo.
 - PROHIBIDO: el coche se detiene.
 - STOP: el coche se detiene.
 - VELOCIDAD-MAX-30: fija la velocidad del coche a una velocidad inferior.
 - VELOCIDAD-MAX-90: fija la velocidad del coche a una velocidad superior.

Para el control autónomo del coche es necesario la ejecución simultánea de los ficheros *real_time_detect_V3.py* y *control_vehiculo_autonomo.py*. Gracias a estos, se podrá almacenar las imágenes grabadas por el coche, detectar y clasificar las señales y, por último, controlar el vehículo en función de las señales detectadas.

6.8 Análisis de rendimiento

6.8.1 Rendimiento de la red

Una vez establecida la red 4G de nuestro prototipo, se ha sometido la red a un análisis de rendimiento. Mediante este análisis, se pretende evaluar la eficiencia y calidad del servicio, así como estimar la capacidad requerida por nuestro modelo. En este apartado, se presentan los resultados obtenidos del análisis de rendimiento llevado a cabo.

Con este propósito, se han tomado medidas de los parámetros de rendimiento de una red explicados en el apartado 2.3 mediante diversas herramientas.

El primero de las medidas tomadas ha sido uno de los numerosos tests de velocidad¹⁴ que pueden encontrarse en la web. Concretamente, esta prueba fue realizada en el coche para tomar las medidas como si fuera nuestro propio ordenador. En la Figura 116 se muestra las medidas de velocidad que se tomaron mediante dicha esta prueba online:



Figura 116. Resultado de test de velocidad online en el coche.

Se observa que conseguimos una velocidad de descarga (*down-link*) de 8.66 Mbps (Megabits por segundo) y una velocidad de subida (*up-link*) de 9.45 Mbps. Estos valores son bastante bajos para redes 4G que están ente 40 y 300 Mbps. No obstante, estas velocidades, concretamente la de subida, pueden ser suficientes para la transmisión del vídeo capturado por el coche hasta el PC.

Estos tests de velocidad en ocasiones pueden no resultar fiables, por lo que se han realizado otras mediciones tanto en el coche como en el PC para evaluar de forma verídica el rendimiento de la red.

Los siguientes experimentos han sido realizados mediante *iperf3*. *iperf3* es una herramienta que nos permite medir el rendimiento de redes. Permite monitorizar el ancho de banda, la tasa de bits y la latencia entre dos puntos. Para ello, es necesario crear un servidor y un cliente. En nuestro caso, el enlace sería el formado entre el coche y el PC por lo que el coche será el cliente, mientras

¹⁴ Concretamente realizamos el test disponible en: <https://www.speedtest.net/>

que el PC será el servidor. Con la opción `-s` podremos configurar el servidor y con la opción `-c` configuraremos el cliente, especificando la IP del servidor. Además, con la opción `-p` fijaremos el puerto deseado.

Con el siguiente comando configuramos el PC como servidor iperf3 en el puerto 8000:

```
iperf3 -s -p 8000
```

La configuración del coche como cliente iperf3 en el puerto 8000 es la siguiente, siendo 172.16.0.1 la dirección IP utilizada por el PC para el servidor iperf3:

```
iperf3 -c 172.16.0.1 -p 8000
```

De esta manera, podemos establecer un enlace entre el PC y el coche y medir parámetros de rendimiento de la red. En la Figura 117 se muestra los resultados obtenidos de las mediciones tomadas en el servidor iperf3, esto es el PC:

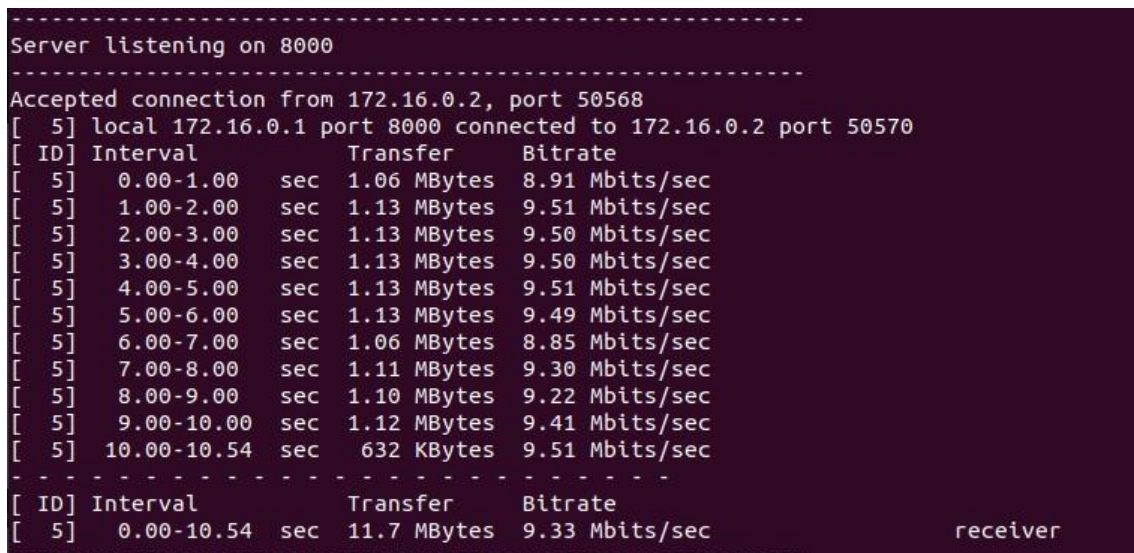


Figura 118. Medidas de rendimiento en el PC con iperf3 (servidor)

Podemos observar que el PC (servidor) es capaz de recibir del coche (cliente) datos a una tasa de bit (*bitrate*) media de 9.33 Mbps. Se puede comprobar que se obtiene un valor muy similar al obtenido en el test de velocidad online (véase Figura 116).

En cuanto al cliente iperf3 (coche), se han realizado diferentes pruebas para evaluar su rendimiento en detalle. Para ello, se va a utilizar la herramienta ping la cual nos permite probar la conectividad de red entre dos dispositivos mediante el envío de paquetes de peticiones de eco ICMP (*Internet Control Message Protocol*) y el retorno de las respuestas de eco. Gracias a ping, podremos ver la latencia de la red en ese momento observando el tiempo de ida y vuelta de los paquetes ICMP. El comando utilizado ha sido el siguiente:

```
ping -c 10 google.com
```

Así, se realiza un ping al host google.com y mediante la opción `-c` se fija el número de peticiones ICMP generadas.

La primera de las medidas en el cliente se ha llevado a cabo sin ningún tipo de conexión mediante iperf3 ni generación de tráfico. Esto incluye el tráfico TCP o UDP generado por iperf3 desde el PC o el envío de vídeo por parte del coche. Por tanto, únicamente se ha realizado un ping, en la Figura 118 se muestra el resultado:

```
deepracer@amss-x3h0:~$ ping -c 10 google.com
PING google.com (142.250.200.142) 56(84) bytes of data.
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=1 ttl=113 time=43.2 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=2 ttl=113 time=42.3 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=3 ttl=113 time=41.2 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=4 ttl=113 time=39.7 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=5 ttl=113 time=38.1 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=6 ttl=113 time=36.8 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=7 ttl=113 time=36.4 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=8 ttl=113 time=34.8 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=9 ttl=113 time=33.4 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=10 ttl=113 time=31.8 ms

--- google.com ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 901ms
rtt min/avg/max/mdev = 31.868/37.817/43.277/3.653 ms
```

Figura 119. Resultado de ping en el coche sin ninguna conexión.

Se puede observar que, sin ningún tipo de conexión ni tráfico adicionales, el enlace tiene una latencia media de 37.817 ms (parámetro *time* de cada petición ICMP). Se trata de un buen valor de latencia para redes 4G la cual suele oscilar entre 30 y 50 ms. Para este tiempo también habría que tener en cuenta la latencia entre el PC y el host google.com.

```
goat@goat-desktop:~$ ping -c 10 google.com
PING google.com (142.250.184.174) 56(84) bytes of data.
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=1 ttl=115 time=3.40 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=2 ttl=115 time=3.42 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=3 ttl=115 time=3.35 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=4 ttl=115 time=3.39 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=5 ttl=115 time=3.19 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=6 ttl=115 time=3.31 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=7 ttl=115 time=3.20 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=8 ttl=115 time=3.25 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=9 ttl=115 time=3.27 ms
64 bytes from mad07s23-in-f14.1e100.net (142.250.184.174): icmp_seq=10 ttl=115 time=3.37 ms

--- google.com ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 901ms
rtt min/avg/max/mdev = 3.189/3.313/3.418/0.078 ms
```

Figura 120. Resultado de ping en el PC

La latencia media entre el PC y el host google.com es 3.313 ms. Por lo tanto, si sustraemos esta latencia a la obtenida en la Figura 118, obtendremos la latencia base real del enlace, entre el coche y el PC, concretamente 34.504 ms.

La siguiente de las pruebas ha sido creando un flujo TCP con iperf3 mediante los comandos vistos anteriormente. En la Figura 120 se muestra la creación del cliente iperf3 en el coche en la que se puede observar que tenemos un ancho de banda aproximado de 10 Mbps.


```
eepracer@amss-x3h0:~$ iperf3 -c 172.16.0.1 -p 8000
Connecting to host 172.16.0.1, port 8000
4] local 192.168.8.105 port 50570 connected to 172.16.0.1 port 8000
ID] Interval          Transfer      Bandwidth      Retr  Cwnd
4]  0.00-1.00    sec  1.24 MBytes  10.4 Mbits/sec    0   116 KBytes
4]  1.00-2.00    sec  1.23 MBytes  10.3 Mbits/sec    0   174 KBytes
4]  2.00-3.00    sec  1.19 MBytes  9.96 Mbits/sec    0   232 KBytes
4]  3.00-4.00    sec  1.37 MBytes  11.5 Mbits/sec    0   290 KBytes
4]  4.00-5.00    sec  1.05 MBytes  8.77 Mbits/sec    0   348 KBytes
4]  5.00-6.00    sec  1.21 MBytes  10.2 Mbits/sec    0   406 KBytes
4]  6.00-7.00    sec  1.12 MBytes  9.39 Mbits/sec    0   460 KBytes
4]  7.00-8.00    sec  1.27 MBytes  10.6 Mbits/sec    0   518 KBytes
4]  8.00-9.00    sec  1.43 MBytes  12.0 Mbits/sec    0   573 KBytes
4]  9.00-10.00   sec  1.20 MBytes  10.0 Mbits/sec    0   631 KBytes
---
ID] Interval          Transfer      Bandwidth      Retr
4]  0.00-10.00   sec  12.3 MBytes  10.3 Mbits/sec    0
4]  0.00-10.00   sec  11.7 MBytes  9.84 Mbits/sec
perf Done.
```

Figura 121. Medidas de rendimiento en el coche con iperf3 (cliente)

Tras esto, se ha realizado un ping (exactamente igual que en el caso anterior) para observar la latencia cuando hay una conexión TCP a mayores. En la Figura 121 se muestra el resultado del ping.

```
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=1 ttl=113 time=200 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=2 ttl=113 time=248 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=3 ttl=113 time=292 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=4 ttl=113 time=348 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=5 ttl=113 time=409 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=6 ttl=113 time=454 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=7 ttl=113 time=498 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=8 ttl=113 time=536 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=9 ttl=113 time=256 ms
64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=10 ttl=113 time=35.0 ms

--- google.com ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9005ms
rtt min/avg/max/mdev = 35.042/328.024/536.212/144.878 ms
```

Figura 122. Resultado de ping en el coche con una conexión TCP.

En este caso, la latencia media es de 328.024 ms. Comparando con el caso anterior en el que no había ningún tipo de conexión, la latencia ha aumentado prácticamente 300 ms al existir una conexión TCP. Este valor de latencia es un valor alto de latencia para redes 4G, sin embargo, es suficiente para acciones básicas como navegar por la web.

Alternativamente a las mediciones tomadas teniendo en cuenta una conexión TCP, se ha realizado otro experimento creando una conexión UDP entre el coche y el PC mediante iperf3. En este caso, el comando para una conexión UDP desde el cliente tiene algunas diferencias:

```
iperf3 -c 172.16.0.1 -p 8000 -u -b 15 MB -l 40 KB
```

Con la opción `-u` se fija que la conexión se mediante el protocolo UDP, la opción `-b` sirve para configurar el ancho de banda de la conexión (15 Mbps) y la opción `-l` para el tamaño del búfer (40 KB, KiloBytes). El valor de 15 Mbps de ancho de banda nos permite sobrepasar la capacidad de canal vista en la Figura 117 de 9.33 Mbps.

A continuación, se ha realizado el ping para evaluar la latencia del enlace. En la Figura 122, se muestra el resultado de este ping:

```
deeperacer@amss-x3h0:~$ ping -c 10 google.com
PING google.com (142.250.178.174) 56(84) bytes of data.
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=1 ttl=113 time=1480 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=2 ttl=113 time=1549 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=3 ttl=113 time=1552 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=6 ttl=113 time=1589 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=7 ttl=113 time=1937 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=8 ttl=113 time=2056 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=9 ttl=113 time=1056 ms
64 bytes from mad41s08-in-f14.1e100.net (142.250.178.174): icmp_seq=10 ttl=113 time=49.5 ms

--- google.com ping statistics ---
10 packets transmitted, 8 received, 20% packet loss, time 9092ms
rtt min/avg/max/mdev = 49.517/1409.005/2056.724/586.207 ms, pipe 3
```

Figura 123. Resultado de ping en el coche con una conexión UDP.

Como era de esperar, ha habido pérdida de paquetes ya que hemos sobrepasado la capacidad de canal. Concretamente, se han perdido el 20% de ellos, esto es, 2 paquetes ya que hemos enviado 10 y se han recibido 8. En cuanto a la latencia, en este caso la latencia media han sido unos 1409.005 ms. Obviando los 3.313 ms de latencia entre el PC y el host, queda una latencia media de 1405.692 ms, una latencia muy alta que impediría el correcto funcionamiento del prototipo.

El último de los experimentos realizados ha sido realizando mediciones mientras se transmite el vídeo captado por la cámara, desde el coche hasta el PC. De esta manera, se podrá comprobar el funcionamiento y el rendimiento de la red para el caso de uso de nuestro proyecto en el que el coche manda continuamente vídeo hacia el PC a través de la red 4G.

En la Figura 123 se muestra el resultado de realizar un ping mientras se transmite vídeo:

```
deeperacer@amss-x3h0:~$ ping -c 10 google.com
PING google.com (142.250.200.78) 56(84) bytes of data.
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=1 ttl=113 time=2258 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=2 ttl=113 time=2598 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=3 ttl=113 time=2697 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=4 ttl=113 time=1846 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=5 ttl=113 time=2178 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=6 ttl=113 time=2093 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=7 ttl=113 time=1942 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=8 ttl=113 time=1841 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=9 ttl=113 time=1781 ms
64 bytes from mad07s24-in-f14.1e100.net (142.250.200.78): icmp_seq=10 ttl=113 time=1599 ms

--- google.com ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 10919ms
rtt min/avg/max/mdev = 1599.587/2083.762/2697.531/337.920 ms, pipe 3
deeperacer@amss-x3h0:~$
```

Figura 124. Resultado de ping en el coche mientras se transmite vídeo hacia el PC.

Se puede comprobar que la latencia media en este caso es de 2083.762 ms, esto es, aproximadamente 2 segundos. Se trata de un valor de latencia muy alto que puede condicionar el correcto funcionamiento de la red 4G de nuestro prototipo ya que la transmisión de vídeo desde el coche al PC ocupa todo el ancho de banda disponible del grupo.

6.8.2 Rendimiento del PC

Más allá del rendimiento del enlace entre el coche y el PC formado por la red 4G, se ha evaluado también el rendimiento del propio PC. Observando los procesos que se están ejecutando en el PC, se puede medir el porcentaje de utilización de la CPU o de la memoria del PC. Estos parámetros de rendimiento del PC también son importantes ya que el procesamiento de datos o vídeo/imágenes recibidas del coche pueden llegar a ser limitantes

En las siguientes figuras se muestra el rendimiento del PC para diferentes situaciones gracias al comando `top`. El comando `top` muestra en tiempo real los procesos activos y el uso de recursos del sistema en sistemas Unix y Linux.

En la figura 124 se muestra el rendimiento del PC sin ninguna transmisión de datos, únicamente ejecutando el eNodeB (`srsebn`) y el *core* (`srsepc`).

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3272	root	-51	0	1637092	646660	45920	S	34,6	11,0	22:04.15	srsebn
3252	root	20	0	238092	67456	6272	S	4,3	1,1	2:28.17	srsepc

Figura 125. Rendimiento del PC sin ninguna conexión.

Este caso consiste en la situación base del rendimiento del PC en el que se ejecutan el eNodeB y *core* de la red 4G. El proceso `srsebn` que se encarga de ejecutar el eNodeB ocupa un porcentaje de la CPU del 34.6% y un 11% de la memoria. Por su parte, el proceso `srsepc` que ejecuta el *core* EPC, utiliza un 4.3% de la CPU y un 1.1%. Los porcentajes de utilización de memoria son bajos en ambos casos. En cuanto al porcentaje de uso de la CPU del eNodeB es más alto que el resto, pero está en un valor eficiente y seguro en el que no debería haber problemas de rendimiento.

En la figura 125 se muestra el rendimiento del PC con una conexión TCP.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3272	root	-51	0	1637092	646660	45920	S	57,3	11,0	23:45.07	srsebn
3252	root	20	0	238092	67456	6272	S	6,3	1,1	2:40.48	srsepc

Figura 126. Rendimiento del PC con una conexión TCP.

Al existir una conexión TCP, los porcentajes de utilización de la CPU suben sustancialmente, mientras que los de memoria no varían. El porcentaje de utilización del proceso del eNodeB es de 57.3% y el del *core* EPC de 6.3%. Un porcentaje de uso de la CPU de 57.3% es una carga moderadamente alta, pero es manejable y entra dentro de lo normal.

En la Figura 126 se muestra el rendimiento del PC cuando hay transmisión de vídeo desde el coche hasta el PC.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
5269	goat	20	0	873188	119872	79668	R	79,7	2,0	0:16.67	python3
3272	root	-51	0	1637092	646660	45920	S	61,8	11,0	31:34.29	srsebn

Figura 127. Rendimiento del PC mientras el coche transmite video al PC.

Este es el caso que realmente nos interesa ya que nuestro prototipo tiene como objetivo transmitir vídeo desde el coche al PC. Tenemos un nuevo proceso llamado *python3*, que se trata de un script de Python encargado de recibir el vídeo proveniente del coche y mostrarlo por pantalla. Este proceso consume un 79.7% de utilización de la CPU y un 2%. Por otro lado, la utilización de la CPU del proceso del eNodeB también ha aumentado, teniendo un porcentaje de 61.8%. Ambos porcentajes de uso de la CPU son altos y podrían afectar al rendimiento del PC, especialmente si se ejecutan más tareas que demanden rendimiento por parte de la CPU. Como anotación, es interesante comentar que la suma de estos porcentajes puede superar el 100% de CPU ya que contamos con un sistema de múltiples núcleos y cada núcleo puede manejar hasta el 100%, y comando `top` muestra el uso por proceso en relación con un único núcleo.

A continuación, se ha analizado el rendimiento del PC mientras se ejecuta el script de control (autónomo) del coche (*control_vehiculo_conectado.py*). El rendimiento en este caso se muestra en la Figura 127.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
8082	root	-51	0	1637092	618288	45724	S	49,5	10,5	22:21.74	rsenb
10303	goat	20	0	946024	117620	76024	S	20,3	2,0	0:02.18	python3

Figura 128. Rendimiento del PC mientras se ejecuta el script de control manual del coche.

El porcentaje de CPU utilizados por el proceso del eNodeB y del script de Python no son muy elevados y son manejables para nuestro proyecto.

Las Figuras 128 y 129 son para el caso en el que se ejecuta el script de detección y clasificación de señales (*real_time_detect_V3.py*) que se vio en el apartado 6.7. En la Figura 128 se muestra la ejecución de este, en la que se muestra el tiempo que tarda en clasificar y en la Figura 129 el rendimiento del PC en ese momento.

```
goat@goat-desktop:~/servicios_cloud_deep_racer$ python3 real_time_detect_V3.py

[W NNPack.cpp:61] Could not initialize NNPACK! Reason: Unsupported hardware.
0: 416x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 VELOCIDAD-MAX-30, 1156.1ms
Speed: 3.4ms preprocess, 1156.1ms inference, 784.9ms postprocess per image at shape (1, 3, 416, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1230.2ms
Speed: 2.4ms preprocess, 1230.2ms inference, 1.2ms postprocess per image at shape (1, 3, 480, 640)

0: 416x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 VELOCIDAD-MAX-30, 1083.6ms
Speed: 7.3ms preprocess, 1083.6ms inference, 1.5ms postprocess per image at shape (1, 3, 416, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1201.7ms
Speed: 8.8ms preprocess, 1201.7ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-DERECHA, 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1266.4ms
Speed: 4.3ms preprocess, 1266.4ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1245.8ms
Speed: 8.0ms preprocess, 1245.8ms inference, 1.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1200.4ms
Speed: 6.5ms preprocess, 1200.4ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1197.0ms
Speed: 6.3ms preprocess, 1197.0ms inference, 1.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1192.5ms
Speed: 7.2ms preprocess, 1192.5ms inference, 1.7ms postprocess per image at shape (1, 3, 480, 640)
```

Figura 129. Ejecución y tiempos del script de detección y clasificación de señales.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
10394	goat	20	0	4344900	844740	273536	R	317,3	14,4	0:46.46	pt_main_thread
8082	root	-51	0	1637092	618288	45724	S	55,5	10,5	24:04.22	srsenb
10361	goat	20	0	945840	116612	75180	S	13,3	2,0	0:11.11	python3

Figura 130. Rendimiento del PC mientras se ejecuta el script de detección y clasificación de señales.

Se puede observar que la detección y clasificación de cada imagen lleva aproximadamente un tiempo de 1200 ms. Este tiempo, superior a 1 segundo, es un tiempo considerablemente elevado que puede condicionar el correcto control del vehículo de manera autónoma y en tiempo real. Esto se refleja en el que rendimiento de la CPU del proceso del hilo principal de Python (*pt_main_thread*) se incrementa hasta el 317.3% lo que nos hace ver que este fichero demanda una alta capacidad de procesamiento.

Para analizar el caso de uso de nuestro prototipo, se ha analizado el rendimiento de la red mientras se ejecutan tanto el script de control del coche (*control_vehiculo_conectado.py*) como el script de detección y clasificación de señales (*real_time_detect_V3.py*). En la Figura 130 se muestra el tiempo de clasificación de señales y en la Figura 131 se muestra el rendimiento del PC.

```
0: 480x640 1 OBLIGATORIO-GIRAR-DERECHA, 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1815.3ms
Speed: 8.2ms preprocess, 1815.3ms inference, 3.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1908.4ms
Speed: 5.5ms preprocess, 1908.4ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1952.1ms
Speed: 2.5ms preprocess, 1952.1ms inference, 3.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 2140.9ms
Speed: 9.2ms preprocess, 2140.9ms inference, 8.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1755.9ms
Speed: 2.5ms preprocess, 1755.9ms inference, 3.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1755.5ms
Speed: 4.1ms preprocess, 1755.5ms inference, 1.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 STOP, 1814.6ms
Speed: 7.6ms preprocess, 1814.6ms inference, 1.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-DERECHA, 1 VELOCIDAD-MAX-90, 1810.7ms
Speed: 2.4ms preprocess, 1810.7ms inference, 1.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-DERECHA, 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1805.1ms
Speed: 2.6ms preprocess, 1805.1ms inference, 3.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1814.5ms
Speed: 3.6ms preprocess, 1814.5ms inference, 1.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1846.6ms
Speed: 2.2ms preprocess, 1846.6ms inference, 1.8ms postprocess per image at shape (1, 3, 480, 640)
```

Figura 131. Ejecución y tiempos del script de detección y clasificación de señales.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
9941	goat	20	0	4410024	908996	273920	R	337,5	15,4	0:40.23	pt_main_thread
8082	root	-51	0	1637092	618288	45724	S	55,1	10,5	17:20.66	srsenb

Figura 132. Rendimiento del PC mientras se ejecuta el script de control de vehículo y el de detección y clasificación de señales.

En este caso, el tiempo de clasificación de cada imagen asciende aproximadamente a 1800 ms y el rendimiento de la CPU del proceso del hilo principal de Python a 337.5%. Un tiempo cercano

a los 2 segundos por cada imagen analizada es un tiempo bastante elevado lo que imposibilita una conducción autónoma en tiempo real. Es por ello por lo que se va a utilizar la Jetson para reducir la capacidad de cómputo que tiene que hacerse en el PC, lo veremos con mayor profundidad en el siguiente apartado.

6.9 Incorporación de la Jetson

Como ya se ha mencionado en el apartado anterior, el ordenador del laboratorio carecía de los recursos de computación necesarios para lograr ejecutar todos los scripts de forma correcta y fluida y que así el sistema pueda ofrecer el mejor servicio posible.

Para solucionar este problema, se planteó la posibilidad de emplear un router para poder conectar el ordenador del laboratorio donde estarían corriendo los scripts de control del coche y la Jetson donde se estarían ejecutando los scripts relacionados con la inferencia capaz de detectar señales de imágenes. Todo ello a través de cables de red al router para disponer de la conexión entre dispositivos y conexión a internet (conectando el router al puerto Ethernet donde estaba previamente conectado el PC).

A continuación, podemos ver un esquema del conexionado realizado:

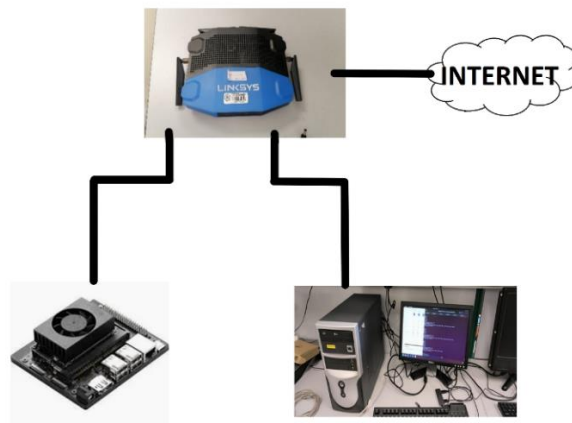


Figura 133. Montaje del router.

Para poder ver los contenidos de uno de los dispositivos en el otro, agregamos ambos dispositivos a la misma subred dándoles IP distintas y conocidas. Concretamente asociamos las IPs de la siguiente manera:

ELEMENTO	IPs y máscaras
ROUTER	192.168.73.1
JETSON	192.168.73.2
PC LAB	192.168.73.3
GATEWAY	192.168.73.1
DNS	192.168.73.1
MÁSCARA	255.255.255.0

Tabla 12. IPs asociadas para el montaje.

De esta manera, solo debemos asociar las IPs correctamente en la configuración del PC y de la Jetson y realizar correctamente el conexionado. Para comprobar que se haya realizado todo correctamente se puede emplear la herramienta ping para comprobar si ambas máquinas son alcanzables desde la otra.

A continuación, podemos ver los resultados de configurar correctamente PC:

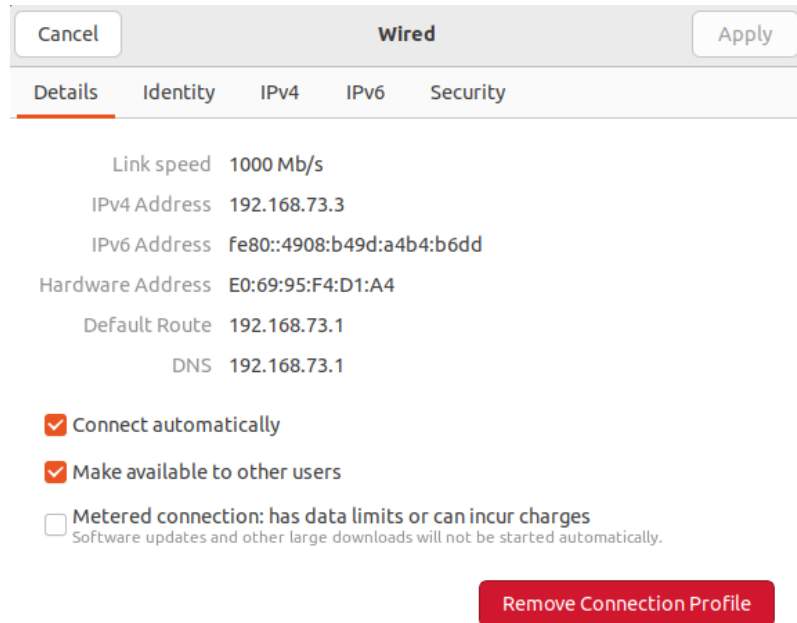


Figura 134. Resultados configuración IP en el PC.

Una vez hecho esto, debemos de crear un servidor NFS (*Network File System*) y conectar los directorios que nos interesen en ambos dispositivos para ello debemos ejecutar los siguientes comandos en el PC:

```
sudo apt install nfs-kernel-server
sudo nano /etc/exports
```

Y añadir la siguiente línea al fichero */etc/exports*:

```
/home/goat/servicios_cloud_deep_racer 192.168.73.2
(rw,sync,no_subtree_check)
```

Luego en el fichero */etc/fstab* añadimos la siguiente línea para automatizar el sistema de ficheros a montar:

```
192.168.73.2:/home/goat/compartido /home/goat/servicios_cloud
_deep_racer nfs default 0 0
```

Y finalmente ejecutando el siguiente comando para reiniciar el servicio:

```
sudo systemctl restart nfs-kernel-server
```

De esta forma quedaría preparado el PC, ahora en la Jetson debemos de ejecutar los siguientes comandos:

```
sudo apt install nfs-common
sudo nano /etc/exports
```

Y añadir la siguiente línea al fichero `/etc/exports`:

```
/home/goat/compartido 192.168.73.3(rw,sync,no_subtree_check)
```

Luego en el fichero `/etc/fstab` añadimos la siguiente línea para automatizar el sistema de ficheros a montar:

```
192.168.73.3:/home/goat/servicios_cloud_deep_racer
/home/goat/compartido nfs default 0 0
```

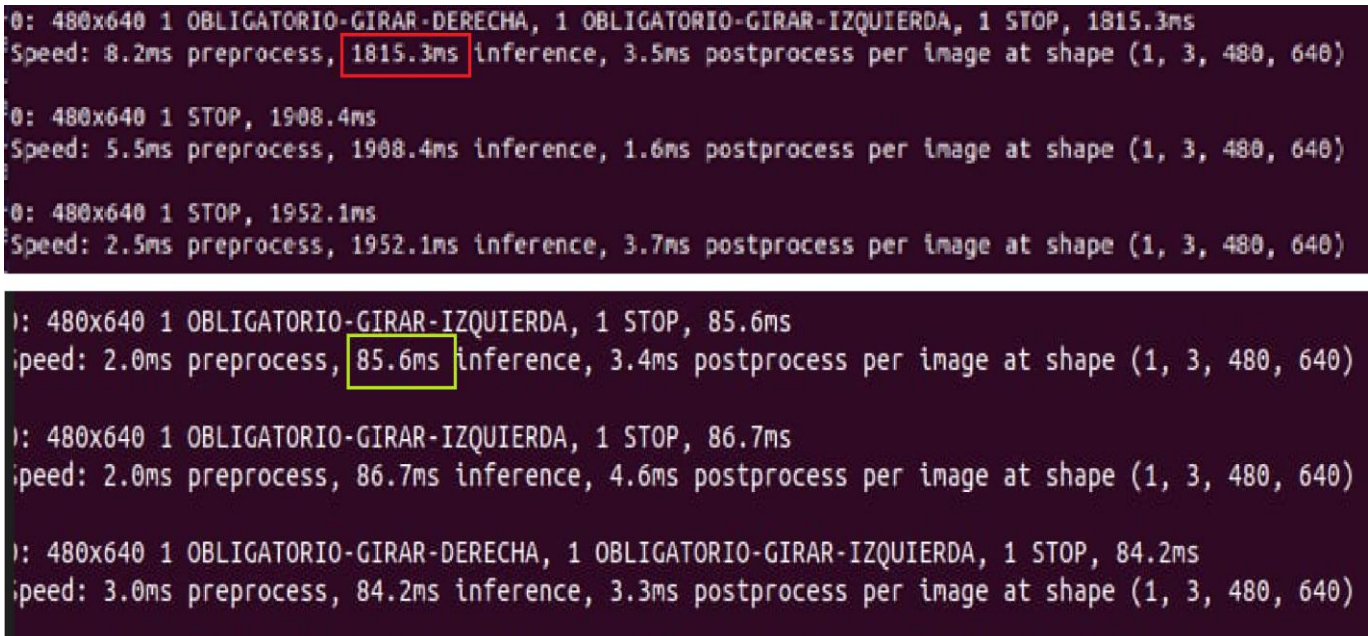
Y finalmente ejecutando el siguiente comando para reiniciar el servicio:

```
sudo systemctl restart nfs-kernel-server
```

Finalmente ejecutando el siguiente comando en cualquiera de los dos ordenadores tendríamos los directorios compartidos que hemos mencionado anteriormente:

```
sudo mount -a
```

Repitiendo la prueba de ejecutar los scripts, pero ahora en máquinas separadas, podemos observar como el tiempo en realizar el procesamiento de la inferencia ha disminuido notablemente de 1900 ms de media a tan solo 85 ms.



The figure consists of two terminal screenshots. The top screenshot shows the results of a single-machine inference test, with the inference time highlighted in a red box as 1815.3ms. The bottom screenshot shows the results of a distributed inference test, with the inference time highlighted in a green box as 85.6ms.

Top Screenshot (Single Machine):

```
0: 480x640 1 OBLIGATORIO-GIRAR-DERECHA, 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 1815.3ms
Speed: 8.2ms preprocess, 1815.3ms inference, 3.5ms postprocess per image at shape (1, 3, 480, 640)
0: 480x640 1 STOP, 1908.4ms
Speed: 5.5ms preprocess, 1908.4ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)
0: 480x640 1 STOP, 1952.1ms
Speed: 2.5ms preprocess, 1952.1ms inference, 3.7ms postprocess per image at shape (1, 3, 480, 640)
```

Bottom Screenshot (Distributed):

```
0: 480x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 85.6ms
Speed: 2.0ms preprocess, 85.6ms inference, 3.4ms postprocess per image at shape (1, 3, 480, 640)
0: 480x640 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 86.7ms
Speed: 2.0ms preprocess, 86.7ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)
0: 480x640 1 OBLIGATORIO-GIRAR-DERECHA, 1 OBLIGATORIO-GIRAR-IZQUIERDA, 1 STOP, 84.2ms
Speed: 3.0ms preprocess, 84.2ms inference, 3.3ms postprocess per image at shape (1, 3, 480, 640)
```

Figura 135. Resultados de latencia en la inferencia comparativa con valores antiguos (superior) y nuevos (inferior).

Además, podemos emplear el comando `jtop` para ver el uso de la GPU al ejecutar nuestro script y así poder comprobar que la Jetson está empleando correctamente los recursos de la GPU. A continuación, podemos ver gráficamente dicho resultado:

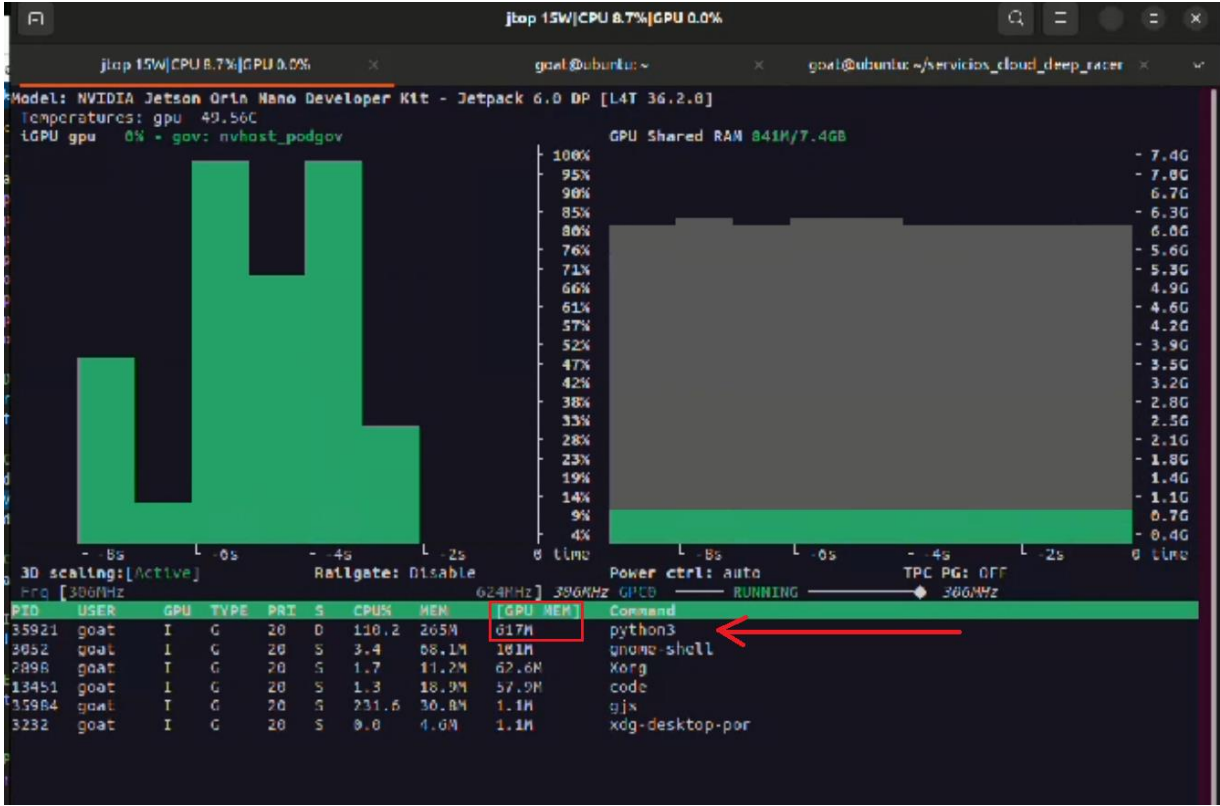


Figura 136. Uso de la GPU al ejecutar nuestro script.

Podemos comprobar que el resultado es muy satisfactorio por lo que la solución es realmente buena y ha sido correctamente implementada.

6.10 Legislación

De acuerdo con la legislación vigente en España recogida en la Ley General de Telecomunicaciones (BOE, Ley 11/2022, de 28 de junio, 2022). Y teniendo en consideración el artículo 48 recogido en el Real Decreto 123/2017, de 24 de febrero (BOE, Real Decreto 123/2017, 2017). Se ha cumplimentado la correspondiente solicitud para realizar una prueba de 6 meses, ocupando el espectro radioeléctrico de banda 7, cedidas por Vodafone S.A.U, asignada para la realización de las pruebas de laboratorio.

En el ANEXO G-Documentos Legislación, se presenta la totalidad de la documentación necesaria para realizar la solicitud de nuevas redes radioeléctricas del servicio móvil y fijo de banda estrecha. Desde la administración se ofrece un tutorial para realizar correctamente dicha solicitud, en el cual se detalla todos los pasos necesarios para generar el fichero XML (SEDE, 2024).

7 Conclusiones y líneas futuras

En el presente informe se ha propuesto el diseño de un vehículo conectado en el tramo de autovía A62 desde el kilómetro 158 hasta el kilómetro 231. El servicio ofrecido a los vehículos de este tramo de autovía consiste en el envío de vídeo grabado por los coches a través de la red 4G, el cual será procesado a fin de detectar las señales de tráfico. Para ello, se ha desarrollado un diseño de red 4G, incluyendo el despliegue de la infraestructura y el estudio del entorno; así como el sistema de Inteligencia Artificial capaz de detectar las señales.

Se ha realizado un sobredimensionamiento de la infraestructura requerida por razones estratégicas, operativas y para futuros negocios. Al contemplar la necesidad de construcción, se ha decidido realizar un aprovisionamiento futuro mayor. Puesto que el coste marginal de optar por un mejor equipo para realizar dicho despliegue no resulta muy elevado. Adquirir y desplegar la infraestructura en una sola fase será más rentable que hacerlo en múltiples fases en el futuro, reduciendo costos de instalación y actualización. Lo que dotará a la parte interesada una posición estratégica en el sector de los vehículos conectados. Aparte de la obvia ventaja competitiva futura, se garantizará una calidad de servicio mayor, siendo capaz de adaptarse fácilmente a nuevas tecnologías y servicios, sin necesidad de modificaciones significativas y permitirá gestionar riesgos asociados con un crecimiento imprevisto de la demanda o cambios en la tendencia del sector de vehículo conectado.

Además, se ha podido comprobar las posibilidades de los sistemas basados en Inteligencia Artificial, la importancia de la calidad de los datos y la similitud con las condiciones del caso real y las posibles aplicaciones que pueden tener este tipo de sistemas en todas las áreas de nuestro entorno cotidiano. También, se han podido comprender y analizar las distintas métricas empleadas para medir la calidad de un modelo de Inteligencia Artificial y los resultados obtenidos para poder valorar la viabilidad del sistema.

El parque de vehículos conectados, futuros vehículos autónomos, utilizando 4G ofrece ventajas como disponibilidad extensa, fiabilidad y menores costos debido a su infraestructura ya existente, pero presenta limitaciones en velocidad de subida, latencia, ancho de banda y límite en la cantidad de usuarios. En contraste, la tecnología 5G proporciona velocidades de datos significativamente mayores, menores latencias cruciales para la conducción autónoma y una mayor capacidad para soportar numerosos dispositivos conectados. Sin embargo, esta tecnología enfrenta desafíos como cobertura limitada inicialmente, mayores costes de implementación y posibles problemas de compatibilidad con dispositivos antiguos.

Con objeto de demostrar la viabilidad y el potencial del sistema de vehículo conectado propuesto en el presente informe. Utilizando un coche en miniatura Amazon Deep Racer equipado con un miniordenador, una cámara y un módem 4G LTE, junto con una infraestructura 4G basada en un sistema SDR bladeRF 2.0, se ha creado un prototipo funcional que opera en un circuito de miniatura. Este prototipo ha sido capaz de reaccionar a las señales de tráfico de juguete, mostrando un nivel básico de autonomía y seguridad en la conducción. La experiencia adquirida y los resultados obtenidos en este entorno controlado permiten vislumbrar la implementación a escala real, con el objetivo de mejorar la seguridad y eficiencia en la conducción de vehículos. La demostración ha reforzado el estudio inicial y sienta las bases para futuros desarrollos en la creación de vehículos autónomos conectados, que prometen transformar significativamente el panorama del transporte.

Como líneas futuras, se podría profundizar aún más el estudio del proyecto mediante el cumplimiento de alguno de los puntos citados a continuación:

- Instalar un segundo EPC en Tordesillas sería muy interesante ya que proporcionaría redundancia en caso de fallos, balanceo de carga y una mayor flexibilidad y capacidad de crecimiento de la red en caso de aumentar la zona de servicio.
- Actualización de la red a 5G para mejorar considerablemente todas sus prestaciones y poder utilizar el servicio en zonas con mayor frecuencia de señales de tráfico o con más obstáculos como una zona urbana.
- El servicio está pensado para un tramo de carretera donde no existe señalización que requiera una acción instantánea como un Stop, un Prohibido circular o un Ceda el paso (estos sí existen, pero solamente para incorporación, el servicio se otorga a los vehículos del interior de la carretera). Esto provoca que se pueda permitir una latencia más elevada que de costumbre al no requerir una acción inmediata. Por lo tanto, si se quisiese emplear en zonas urbanas, a mayores de actualizar a 5G habría que optimizar la latencia de la red para detectar señales y reaccionar con más rapidez, incluyendo el tiempo de procesamiento de las imágenes.
- Aumentar el número de usuarios simultáneos por antena, mediante una actualización próxima de su software y ofrecer una modulación mínima mayor en todo el tramo.
- Mejorar el dataset con una mayor cantidad de imágenes y escenarios y entrenar un modelo más preciso y ligero, empleando IA ligeras, otro tipo de algoritmos u otros productos del fabricante NVIDIA.
- Añadir al dataset imágenes de personas y vehículos (solo la parte frontal), para poder emplear el sistema para actuar en caso de que se cruce un peatón o un vehículo de frente en dirección contraria, y así poder reducir los accidentes en carretera y ciudad.
- Expandir nuestro sistema a más tramos de la autovía o a más autovías.
- Incorporar un algoritmo de detección de carril para mejorar la autonomía del vehículo, además de mejorar la autonomía del vehículo evitar salidas accidentales de la carretera.

Estos son solo algunos de los puntos en los que se podría trabajar si bien la lista podría ser infinita en función de la expansión que se le quiera dar al proyecto realizado.

8 Bibliografía

- 3GPP. (2006). *Evolved Universal Terrestrial Radio Access (E-UTRA) and Evolved Universal Terrestrial Radio Access Network (E-UTRAN); Overall description; Stage 2*.
- ACM. (2022). Obtenido de https://dl.acm.org/doi/full/10.1145/3485767?casa_token=w4c5noeJkk0AAAAA%3AQvAsSJacvS6gtmSFGpiGXXUfdxqsUWM_DgFbCfjdNAOuihw41r8bTNg5a_ZPDc86LN7iEF4-XKU
- ADLittle. (2013). Obtenido de https://www.adlittle.com/sites/default/files/viewpoints/ADL_LTE_Spectrum_Network_Strategies.pdf
- AEMET. (s.f.). Obtenido de <https://www.ingesco.com/es/noticias/normativas-proteccion-contra-rayo>
- AEMET. (2019). *Climatología de descargas eléctricas y de días de tormenta en España*. Obtenido de Climatología de descargas eléctricas y de días de tormenta en España : https://www.aemet.es/documentos/es/conocermas/recursos_en_linea/publicaciones_y_estudios/publicaciones/Climatologia_de_descargas_electricas/Climatologia_de_descargas_electricas.pdf
- AEPD Agencia Española de Protección de Datos. (2020). *Adecuación al RGPD de tratamientos de datos que incorporan Inteligencia Artificial*. Obtenido de Adecuación al RGPD de tratamientos de datos que incorporan Inteligencia Artificial.: <https://www.aepd.es/guias/adecuacion-rgpd-ia.pdf>
- AESA. (2018). *Guía de señalamiento e iluminación de obstáculos*. Obtenido de Guía de señalamiento e iluminación de obstáculos.: https://www.seguridadareia.gob.es/sites/default/files/SSAA-16-GUI-121-A01_Se%C3%B1alamiento%20e%20iluminaci%C3%B3n%20de%20obst%C3%A1culos_1.0.pdf
- Agusti-Comes, R., Bernardo-Álvarez, F., Casadevall-Palacio, F., Ferrús-Ferre, R., Pérez-Romero, J., & Sallent-Roig, O. (s.f.). *LTE: Nuevas tendencias en comunicaciones móviles*.
- Arcos, Á. (May de 2015). *GitHub: Dataset de señales de tráfico de España*. Obtenido de GitHub: Dataset de señales de tráfico de España: <https://daus-lab.github.io/spanish-traffic-sign-dataset/>
- AT3. (2012). *Principales normativas sobre protección contra el rayo*. Obtenido de Principales normativas sobre protección contra el rayo.: <https://at3w.com/blog/principales-normativas-sobre-proteccion-contra-el-rayo/>
- Authors, V. (2022). *Sustainable Development in Engineering and Technology*. En V. Authors, *Sustainable Development in Engineering and Technology*. 3ciencias, ÁREA DE INNOVACIÓN Y DESARROLLO, S.L.
- Baicells. (2024). *NOVA 436Q*. Obtenido de NOVA 436Q: <https://www.doubleradius.com/site/stores/baicells/baicells-nova-436q-outdoor-tdd-enodeb-datasheet.pdf>
- BOE. (2017). *Real Decreto 123/2017*. Obtenido de Real Decreto 123/2017: <https://www.boe.es/eli/es/rd/2017/02/24/123>

- BOE. (2022). *Ley 11/2022, de 28 de junio*. Obtenido de Ley 11/2022, de 28 de junio: <https://www.boe.es/eli/es/l/2022/06/28/11>
- Cambium_Network. (2019). *cn Ranger Sierra 800 Baseband Unit (BBU)*. Obtenido de cn Ranger Sierra 800 Baseband Unit (BBU): https://brandcentral.cambiumnetworks.com/m/1835d06757cd7f8d/original/Cambium_Networks_data_sheet_cnRanger_Sierra_800_BBU.pdf
- Cayetano, M. R. (2023-2024). *Cloud Computing y Virtualización: Introducción a la computación en la nube y a la virtualización*. Valladolid: Apuntes propios.
- CISCO. (2024). *EPC-Datasheet*. Obtenido de https://www.cisco.com/c/dam/en/us/td/docs/wireless/asr_5000/12_2/OL-25550_Prod_Overview1.pdf
- Cisco-GPL. (2024). *itprice*. Obtenido de <https://itprice.com/cisco-gpl/asr%205000>
- Conector. (s.f.). Obtenido de https://www.fs.com/es/products/84700.html?country=ES¤cy=EUR&languages=Espa%C3%B1ol&paid=google_shopping&utm_country=1005546&gad_source=1&gclid=Cj0KCQjwpNuyBhCuARIsANJqL9MxbyNcdU1_6QgsP9_gL5gWPPojnAwPJgft27GFOpdhgRLolCiDGnEaAu6qEALw_wcB
- Dahlman, E., Parkvall, S., & Sköld, J. (2011). *4G LTE/LTE-Advanced for Mobile Broadband*.
- Dahlman, E., Parkvall, S., & Sköld, J. (2011). *4G: LTE/LTE-Advanced for Mobile Broadband*. Academic Press Inc.
- de Miguel, I., & Durán, R. (2023-2024). *Fundamentos de Aprendizaje Automático para las TIC*. Valladolid: Apuntes propios.
- Dell. (2024). *Dell*. Obtenido de https://i.dell.com/sites/csdocuments/Product_Docs/en/R750xs-specsheet.pdf
- DGT. (2022). *Anuario Estadístico General DGT*. Obtenido de Anuario Estadístico General DGT: https://www.dgt.es/export/sites/web-DGT/.galleries/downloads/dgt-en-cifras/publicaciones/Anuario_Estadistico_General/Anuario-Estadistico-General-2022-v2.pdf
- DGT. (2023). *cifras tráfico*. Obtenido de cifras tráfico: <https://www.dgt.es/inicio/>
- EjePrime. (2022). *Precio del suelo urbano*. Obtenido de Precio del suelo urbano: <https://www.ejeprime.com/mercado/el-suelo-urbano-encara-un-2023-sin-sobresaltos-y-con-la-estadistica-del-ministerio-en-tela-de-juicio#:~:text=Tomando%20como%20referencia%20la%20cifra,interanual%20del%205%2C2%25>.
- Eloy_Gonzalez. (2022). *Despliegue de una red*. Obtenido de Despliegue de una red: https://oa.upm.es/40010/1/PFC_ELOY_GONZALEZ_HERNANDEZ.pdf
- Erik Dahlman, S. P. (2011). *4G LTE/LTE-Advanced for Mobile Broadband*.
- Girshick, R. (2015). *Fast R-CNN*. Microsoft.
- Gobierno de España, M. d. (2014). *Señalización Vertical Norma 8.1-IC*. Valladolid.

- Gobierno de España, M. d. (2024). *Mapa de tráfico de la DGC*. Obtenido de Mapa de tráfico de la DGC: <https://www.transportes.gob.es/carreteras/trafico-velocidades-y-accidentes-mapa-estimacion-y-evolucion/mapas-de-trafico/serie-historica-mapas>
- Google. (Last Checked: May de 2024). *Google Collab*. Obtenido de Google Collab: https://colab.research.google.com/notebooks/basic_features_overview.ipynb
- Google. (Last Checked: May de 2024). *Introducción a Google Collab*. Obtenido de Introducción a Google Collab: https://colab.research.google.com/?hl=es#scrollTo=GJBs_flRovLc
- Hayes, J. (2016). *FOA*. Obtenido de <https://www.thefoa.org/tech/ref/basic/design.html>
- Higuera, P. C. (2024). *Vehículo conectado y autónomo*. Laboratorio de Procesado de Imagen.
- Huang, P., Tian, S., Tan, W., Dong, Y., & Xu, w. (2023). IA-CIOU: An Improved IOU Bounding Box Loss Fucntion for SAR Ship Target Detection Methods. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*.
- Hull, I. (2020). *Machine Learning for Economics and Finance in Tensorflow 2*. apress.
- IEEE. (2021). *Band Study*. Obtenido de 10.1109/AICT52120.2021.9628960
- INE. (2024). Obtenido de <https://www.ine.es/varipc/verVariaciones.do?idmesini=1&anyoini=2011&idmesfin=5&anyofin=2020&ntipo=2&enviar=Calcular>
- Ingesco. (2020). *Normativas de protección contra los rayos*. Obtenido de Normativas de protección contra los rayos: <https://www.ingesco.com/es/noticias/normativas-proteccion-contra-rayo>
- IOP. (2018). Obtenido de <https://iopscience.iop.org/article/10.1088/1757-899X/252/1/012096/meta>
- Jose, A. (2024). Obtenido de <https://itprice.com/cisco-gpl/asr%205000>
- Kelleher, J. D. (2019). *DEEP LEARNING*. Cambridge, Masachusetts.
- Lyntia. (2024). *Lyntia*. Obtenido de <https://www.lyntia.com/red-fibra-optica/>
- Mártin, R. (2015). *Proyecto Despligue Merida*. Obtenido de chrome-extension://efaidnbmninnibpcajpcgclclefindmkaj/<https://descargas.merida.es/contratante/general/268-fibra-optica-presupuesto.pdf>
- MastDatabase. (2021). Obtenido de <https://mastdatabase.co.uk/blog/2021/07/24/lte-bands-and-duplex-modes/>
- Mauri Isorna, S. (2017). *biblus*. Obtenido de <https://biblus.us.es/bibing/proyectos/abreproy/12388/fichero/PFC+SANTIAGO+MAURI+ISORNA.pdf>
- MiFleet, D. (2024). *MiFleet Dash Board Cameras Hardware Datasheet*.
- MINISTERIO DE ASUNTOS ECONÓMICOS Y TRANSFORMACIÓN DIGITAL. (2022). *20220919_Resumen_detallado_Reglamento_IA.pdf*. Obtenido de https://portal.mineco.gob.es/es-es/digitalizacionIA/sandbox-IA/Documents/20220919_Resumen_detallado_Reglamento_IA.pdf

- MITYC. (10 de Noviembre de 2011). *Subasta*. Obtenido de https://avancedigital.mineco.gob.es/es-es/ResultadosSubasta2/Resultados_segunda_subasta_10112011.pdf
- Netel. (2022). *Cobertura España*. Obtenido de <https://netelcomunicaciones.es/cuales-son-las-bandas-de-frecuencia-de-cobertura-en-espana/>
- NVIDIA. (2024). *Developer Kits*. Obtenido de Developer Kits: <https://developer.nvidia.com/embedded-computing>
- NVIDIA. (2024). *Familia Jetson*. Obtenido de Familia Jetson : <https://www.nvidia.com/es-es/autonomous-machines/embedded-systems/>
- NVIDIA. (2024). *Jetson Orin Nano Specifications*. Obtenido de Jetson Orin Nano Specifications: <https://www.nvidia.com/es-es/autonomous-machines/embedded-systems/jetson-orin/>
- NVIDIA. (2024). *Key Features in Jetpack 6.0*. Obtenido de Key Features in Jetpack 6.0: <https://developer.nvidia.com/embedded/jetpack>
- Nvidia. (2024). *Nvidia*. Obtenido de [https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/quadro-product-literature/proviz-print-nvidia-rtx-a6000-datasheet-us-nvidia-1454980-r9-web%20\(1\).pdf](https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/quadro-product-literature/proviz-print-nvidia-rtx-a6000-datasheet-us-nvidia-1454980-r9-web%20(1).pdf)
- Peiyuan Jiang, D. E. (2021). A Review of Yolo Algorithm Developments. En D. E. Peiyuan Jiang, *A Review of Yolo Algorithm Developments*. Elsevier.
- Pertíñez López, I. J. (2015). *Planificación radioeléctrica de una red de*.
- Reintel. (2024). *Reintel*. Obtenido de https://www.reintel.es/sites/webreintel/files/REINTEL_Iberico_WEB.pdf
- Roboflow. (2023). *Roboflow Inference: Effortless Local Computer Vision Deployment with Python*. Obtenido de Roboflow Inference: Effortless Local Computer Vision Deployment with Python: <https://blog.roboflow.com/inference-python/>
- Roboflow. (Last Checked: May de 2024). *Roboflow*. Obtenido de Roboflow - Principios: Available at: <https://blog.roboflow.com/getting-started-with-roboflow/>
- Roboflow. (Febrero de 2024). *Train a Model in Roboflow*. Obtenido de Train a Model in Roboflow: <https://docs.roboflow.com/train/train>
- SEDE. (2024). *Servicio móvil y servicio fijo*. Obtenido de Servicio móvil y servicio fijo: https://sede.serviciosmin.gob.es/es-ES/procedimientosselectronicos/Documents/SE%20Telecomunicaciones/tramitacion-redes-radioelectricas-movil-fijo-banda-estrecha/manual_usuario_SM_GenXML.pdf
- Ubiquiti. (2024). *Ubiquiti*. Obtenido de https://dl.ubnt.com/ds/uf_gpon.pdf
- UE. (2023). *El Consejo adopta un nuevo marco para impulsar la implantación de sistemas de transporte inteligentes*. Obtenido de El Consejo adopta un nuevo marco para impulsar la implantación de sistemas de transporte inteligentes: <https://www.consilium.europa.eu/es/press/press-releases/2023/10/23/council-adopts-new-framework-to-boost-the-roll-out-of-intelligent-transport-systems/>
- Ultralytics. (2024). *Esto es Ultralytics*. Obtenido de Esto es Ultralytics: <https://www.ultralytics.com/es/about>

- UniPower. (2022). *ASPIRO 1U M35 MODELO M00035 Datasheet*. Obtenido de ASPIRO 1U M35 MODELO M00035 Datasheet: <https://unipowerco.com/products/45-amp-dc-power-system-aspiro1u/>
- Vigo, F. d. (2008). Obtenido de <https://www.farodevigo.es/gran-vigo/2008/11/06/operadoras-pagan-12-000-euros-17990081.html>
- Yacouby, R., & Axman, D. (2020). *Probabilistic Extension of Precision, Recall, and F1 Score for More*.

ANEXO A-SCRIPTS

SCRIPT CAPTURAR IMÁGENES Y VÍDEO (captura_imagen.py)

```
# -*- coding: utf-8 -*-
#Primera versión del código para capturar imágenes con la cámara del coche.
#Created by: The_GOAT_Projects. S.A

import cv2
import os
import time

print(cv2.__version__) #Imprimimos la versión de la biblioteca cv2.

#Creamos las carpetas donde se van a guardar las fotos y videos en caso de
que no existan ya.
if not os.path.exists("Fotos_capturadas"):
    os.makedirs("Fotos_capturadas")

if not os.path.exists("Videos_capturados"):
    os.makedirs("Videos_capturados")

#main, aquí se introducirán las funciones a realizar.
def main():
    print("Menú de selección: \n")
    print(" 1. Capturar una Imagen.")
    print(" 2. Capturar un video. (pulse q para detenerlo)")
    print(" 3. Obtener USB cámara. \n")

    opcion = input("Introduzca una opción: ")

    if opcion == '1':
        capture_imagen()
    elif opcion == '2':
        capture_video()
    elif opcion == '3':
        encontrar_camara()
    else:
        print("Opción no válida")
```

Código 6. Primer fragmento script capaz de capturar imágenes y vídeos con la cámara del coche (Autoría propia).

```

#Esta función permite capturar varias imágenes seguidas, es decir, un video.
def capture_video():
    #Iniciamos la cámara.
    cam = cv2.VideoCapture(0)

    #Establecemos el codec.
    fourcc = cv2.VideoWriter_fourcc(*"XVID")

    #20.0 indica los frames por segundo, y 640x480 la resolución obtenida de
    "servicios_cloud_deep_racer" en el campus.
    video_out = cv2.VideoWriter(os.path.join("Videos_capturados", "Video_" +
time.strftime("%Y-%m-%d_%H-%M-%S") + ".avi"), fourcc, 20.0, (640, 480))

    #Bucle infinito que saca fotos.
    while True:
        #Capturamos y guardamos.
        retval, frame = cam.read()
        video_out.write(frame)
        if cv2.waitKey(500) & 0xFF == ord('q'):
            break
    #Liberamos recursos.
    cam.release()
    video_out.release()

#Esta función permite capturar una sola imagen, para sacar una foto.
def capture_imagen():
    #Iniciamos la cámara.
    cam = cv2.VideoCapture(0)
    #Capturamos y guardamos en la carpeta.
    retval, frame = cam.read()
    cv2.imwrite(os.path.join("Fotos_capturadas", "Fotograma_" + time.strftime("%Y-%m-
%d_%H-%M-%S") + ".jpg"))

    #Limberamos recursos.
    cam.release()

##Esta función permite obtener el número del USB cámara en caso de no conocerlo.
def encontrar_camara():
    for i in range(10):
        #Probamos hasta que consiga abrirse la cámara correctamente.
        cam = cv2.VideoCapture(i)
        if( cam.isOpened()):
            print("La cámara se encuentra en el USB número: " + str(i))

if __name__ == "__main__":
    main()

```

Código 7. Segundo fragmento script capaz de capturar imágenes y videos con la cámara del coche (Autoría propia).

SCRIPT ENTRENAMIENTO MODELO EN JUPYTER NOTEBOOK

```

#Comprobamos las versiones que se están empleando y solicitamos GPU.
import sys
import torch
print(f"Python version: {sys.version}, {sys.version_info} ")
print(f"Pytorch version: {torch.__version__} ")
!nvidia-smi #Solicitamos GPU

#Clonamos el repositorio de github y descargamos los requerimientos.
!git clone https://github.com/SkalskiP/yolov9.git
%cd /content/yolov9
!pip3 install -r requirements.txt -q

#Creamos un Directorio HOME (Opcional).
import os
%cd /content/yolov9
HOME = os.getcwd()
print(HOME)

#Descargamos los ficheros de pesos.
!mkdir -p {HOME}/weights
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/yolov9-c.pt
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/yolov9-e.pt
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/gelan-c.pt
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/gelan-e.pt

# Cargamos roboflow y añadimos el dataset.
!pip install roboflow
from roboflow import Roboflow
rf = Roboflow(api_key="*****") #La API_KEY es secreta.
project = rf.workspace("tp-ii").project("coche-conectado-senales_juguete")
version = project.version(2)
dataset = version.download("yolov9")

# Conectamos con Google Drive
from google.colab import drive
drive.mount('/content/gdrive')

# NOTA: CADA CONJUNTO DE LÍNEAS PRECEDIDA POR UN COMENTARIO REPRESENTA UNA CELDA DEL
CUADERNO DE JUPYTER NOTEBOOK DE GOOGLE COLLAB

```

Código 8. Primer fragmento del código para entrenar el modelo desarrollado en el Jupyter Notebook de Google Collab.

```

# Creamos un directorio en drive cada iteración.
import os
def generar_directorio():
    # Directorio base donde se generarán los directorios.
    root = "/content/gdrive/MyDrive/Colab_Notebooks/Outputs/"

    # Verificar si el directorio base existe y si no existe lo crea.
    if not os.path.exists(root):
        os.makedirs(root)

    # Obtener la lista de directorios existentes.
    directorios = os.listdir(directorio_base)

    # Si no hay directorios existentes, empieza desde el 1.
    if not directorios:
        indice = 1
    else:
        # Obtener el último número utilizado y sumar 1
        ultimo_indice = max(directorios)
        indice = int(ultimo_indice) + 1

    # Crear el nuevo directorio.
    nuevo_directorio = os.path.join(root, str(indice))
    os.makedirs(nuevo_directorio)

    print("Se ha creado el directorio: " + str(nuevo_directorio))

    # Devolver el nombre del directorio creado
    print("Nombre del directorio creado:", nuevo_directorio)
    return nuevo_directorio

# Entrenamos el modelo con 20 épocas.
!python train.py \
--batch 16 --epochs 20 --img 640 --device 0 --min-items 0 --close-mosaic 15 \
--data {dataset.location}/data.yaml \
--weights {HOME}/weights/gelan-c.pt \
--cfg models/detect/gelan-c.yaml \
--hyp hyp.scratch-high.yaml

# Guardamos en drive los resultados del entrenamiento:
if os.path.exists('/content/yolov9/runs'):
    nombre_directorio = generar_directorio()
    !cp -r /content/yolov9/runs nombre_directorio

```

Código 9. Segundo fragmento del código para entrenar el modelo desarrollado en el Jupyter Notebook de Google Collab.

```
# Mostramos la imagen de las gráficas resultado, la imagen de la matriz
de confusión y la imagen con los resultados de validación del modelo.
!pip install matplotlib
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

graficas = mpimg.imread(f"{HOME}/runs/train/exp/results.png")
matriz = mpimg.imread(f"{HOME}/runs/train/exp/confusion_matrix.png")
validation = mpimg.imread(f"{HOME}/runs/train/exp/val_batch0_pred.jpg")

plt.imshow(graficas)
plt.imshow(matriz)
plt.imshow(validation)

plt.axis('off') # Desactivar los ejes si no son necesarios
plt.show()
```

Código 10. Tercer fragmento del código para entrenar el modelo desarrollado en el Jupyter Notebook de Google Collab

SCRIPT DESPLIEGUE DE LA INFERENCIA DE MODO LOCAL

```

import os
import cv2
import math
from ultralytics import YOLO
import matplotlib.pyplot as plt
import numpy as np

#Añadimos los ficheros con el algoritmo de tracking sort.
from sort import *

#Análisis de rendimiento.
import time
inicio1 = time.time()

#Directorio de imágenes
directory = "/home/goat/Desktop/REAL_TIME_DETECTOR/images"

#Modelo y clases.
model = YOLO("/home/goat/Desktop/REAL_TIME_DETECTOR/best.pt")
classNames = ["OBLIGATORIO-CONTINUAR-RECTO", "OBLIGATORIO-GIRAR-DERECHA",
              "OBLIGATORIO-GIRAR-IZQUIERDA", "PROHIBIDO", "STOP",
              "VELOCIDAD-MAX-30", "VELOCIDAD-MAX-90"]

#Creamos la instancia de SORT basado en MOT.
mot_tracker = Sort()

while(1):
    # lista de imágenes en el directorio
    image_files = [os.path.join(directory, file) for file in os.listdir(directory) if
file.endswith(('.jpg', '.jpeg', '.png'))]

    #Recorremos todos los ficheros del directorio.
    for image_file in image_files:
        img = cv2.imread(image_file)

        height, width, _ = img.shape
        img = img[:, width // 2:, :]

        results = model(img)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        old_maxboxsize = 0 #Variable para detectar el bounding

        predictions = []

        print("\nIter: " + str(image_file))

        #Coordenadas
        inicio2 = time.time()
        for r in results:
            boxes = r.boxes

```

Código 11. Primer fragmento del código para desplegar el modelo de forma local.

```

for box in boxes:

    # bounding box
    x1, y1, x2, y2 = box.xyxy[0]
    x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2) #Lo convertimos
a valores int.

    #Añadimos el bounding box.
    cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 255), 3)

    #Agregamos el valor de confianza.
    confidence = math.ceil((box.conf[0] * 100)) / 100
    print("Confidence --->", confidence)

    # Agregamos la etiqueta.
    cls = int(box.cls[0])
    print("Class name -->", classNames[cls])

    org = [x1, y1]
    font = cv2.FONT_HERSHEY_SIMPLEX
    fontScale = 1
    color = (255, 0, 0)
    thickness = 2

    prediction = []
    prediction.append(x1)
    prediction.append(x2)
    prediction.append(y1)
    prediction.append(y2)
    prediction.append(confidence)

    cv2.putText(img, classNames[cls], org, font, fontScale, color,
thickness)

    #Trackeamos las señales.
    if (abs(x1-y2) > old_maxboxsize):
        old_maxboxsize = abs(x1-y2)
        box_prioritario = classNames[cls]

    predictions.append(prediction)

    track_results = mot_tracker.update(np.array(predictions))
    print(str(track_results))
    print("El bounding box de más a la derecha es: " + box_prioritario)

    fin2 = time.time()
    print("tiempo modelo: " + str(fin2-inicio2))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    cv2.imshow('Image', img)
    fin1 = time.time()
    print("tiempo ejecucción completa: " + str(fin1-inicio1))
    cv2.waitKey(0)

cv2.destroyAllWindows()

```

Código 12. Segundo fragmento del código para desplegar el modelo de forma local.

SCRIPT artemis_autonomous_car.py

```

import imutils
import cv2
import time
import numpy as np
import math, cmath
import os #NUESTRO
import datetime #NUESTRO

#Clase principal
class artemis_autonomous_car:

    def __init__(self, path, steering_calibration_param=0):

        #NUESTRO
        # Marca de tiempo con la fecha para nombres diferentes
        #timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
        # Crea el directorio con el nombre con la fecha
        self.output_dir = os.path.join('images')
        # Asegura que el directorio existe
        if not os.path.exists(self.output_dir):
            os.makedirs(self.output_dir)
        #NUESTRO
        # Img resolution

        self.img_width = 640
        self.img_height = 480
        self.img_width_center = int(self.img_width/2)
        self.img_height_center = int(self.img_height/2)

        # Transformation matrix definition
        source =
np.float32([[0,200],[self.img_width,200],[self.img_width,self.img_height],[0,s
elf.img_height]])

        dest =
np.float32([[0,80],[self.img_width,80],[self.img_width_center+90,self.img_heig
ht],[self.img_width_center-90,self.img_height]])
        self.transform_matrix=cv2.getPerspectiveTransform(source,dest)

```

Código 13. Primer fragmento del código artemis_autonomous_car.py.


```

# Logitudinal control
self.stop = 0 # At 1 vehicle stopped
self.throttle_s = 0.04 # Throttle sensitivity
self.throttle_o = 0.80 # Throttle offset
self.max_throttle = 0.53 # Maximum throttle
self.min_longitude_obstacle = 0.4 # Minimum distance to an
obstacle
self.lidar_throttle_control = 0
self.battery_level = 10

# Lateral control
self.k_cross_track_error = 2.5 # Route offset (Stanley
modification). It calibrates the point at which the vehicle turns when it
sees a curve.
self.k_stanley = 1 # Stanley constant (Setting to avoid
oscillation)
self.minimum_contour_area = 350 # Minimum area (in px) for
interpreting line point
self.contour_junction_param = (15,15) # Parameter for joining
close contours and avoiding failure due to noise (Higher number = less
noise and less precision).
self.pixels_in_meter = 1000.0 # Number of pixels of the
transformed image equivalents to a meter.
self.max_larger_width_contour = 200 # Maximum width in pixels of
a contour to consider its centre point reliable.
self.contour_cut_x_position_1 = int(self.img_width/3)
self.contour_cut_x_position_2 = int(2*self.img_width/3)
self.steering_calibration_param = steering_calibration_param

# Color definition:
self.lower_color = (80,21,101) # Color range
self.upper_color = (114,255,255)

self.frames_counter_switch_fork=15 # Number of frames in which
multiple routes must not be observed to consider that a fork has been
taken.
self.path = path # Vector of numbers defining the route
to follow
self.counter_path = 0
self.switch_fork=0
self.counter_switch_fork=self.frames_counter_switch_fork

```

Código 14. Segundo fragmento del código artemis_autonomous_car.py.

```

# Set stop variable
def set_stop(self, stop):
    self.stop = stop

def set_battery_level(self, battery_level):
    self.battery_level = battery_level

# Function that process lidar information [In progress]

def proceso_lidar(self, ranges, show_img):
    min_range=min(ranges[340:360]+ranges[0:20])
    print(min_range)
    throttle_control=self.throttle_s*min_range+self.throttle_o
    if min_range < self.min_longitude_obstacle:
        self.lidar_throttle_control = 0
        print("PARO")
    elif throttle_control > self.max_throttle:
        self.lidar_throttle_control = self.max_throttle
    else:
        self.lidar_throttle_control=throttle_control

    if show_img == True:
        points=[]
        img_lidar = np.zeros((480,640,3), np.uint8)
        img_car = cv2.imread("/home/artemis/Artemis/servicio-
servidor/demos/car60.png")
        img_lidar[210:210+img_car.shape[0],307:307+img_car.shape[1]]=img_car
        i=0
        for range in ranges:
            Z=cmath.rect(range,math.radians(-i-90))
            i = i+1
            if Z.real != float('inf') and Z.real != float('-inf'):
                x=int(Z.real*50)+self.img_width_center
            else:
                x=0
            if Z.imag= float('inf') and Z.imag != float('-inf'):
                y=int(Z.imag*50)+self.img_height_center
            else:
                y=0
            cv2.circle(img_lidar,(x,y), radius=2, color=(0,0,255),thickness=2)
            cv2.putText(img_lidar, "Obstacle ahead: "+str(round(min_range*100,3)) +
"cm", (5,20), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
            cv2.imshow("Mapa LIDAR",img_lidar)
            cv2.waitKey(1)

```

Código 15. Tercer fragmento del código artemis_autonomous_car.py.

```

if Z.imag != float('inf') and Z.imag != float('-inf'):
    y=int(Z.imag*50)+self.img_height_center
    else:
        y=0
        cv2.circle(img_lidar,(x,y), radius=2, color=(0,0,255),thickness=2)
        cv2.putText(img_lidar, "Obstacle ahead: "+str(round(min_range*100,3)) +
"cm", (5,20), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
        cv2.imshow("Mapa LIDAR",img_lidar)
        cv2.waitKey(1)

# Global function that performs all steps of image processing and calculation of
control parameters.
# Input:
#   - img:      Original image
#   - show_info: If true, windows with real-time information are displayed.
#   - real_time_control: 0 = (Default) Follow the path set in the path argument, 1
= Take the left route, 2 = Take the middle route, 3 = Take the right route, 4 = Left
line change, 5 = Right line change
# Output:
#   - steering_control: Vehicle steering control parameter.
#   - throttle_control: Vehicle throttle control parameter.
#   - trajectory_not_found: At 1 if trajectory not found

def proceso_fotograma(self, img, show_info, real_time_control=0):

    # Perspective transformation
    transformed_img = self.perspective_transformation(img)

    # Calculate the heading error and the cross track error
    (heading_error,cross_track_error,trajectory_not_found,filtered_img,imgLinea)=se
lf.calculate_trajectory(transformed_img,real_time_control,show_info)

    # If trajectory not found
    if trajectory_not_found == 0:

        # Implement Stanley Algorithm
        target_angle =
self.calculo_stanley(heading_error,cross_track_error,0.52,self.k_St Stanley)
        steering_control = self.rad2control(target_angle)

```

Código 16. Cuarto fragmento del código artemis_autonomous_car.py.

```

if (self.stop == 1):
    throttle_control=0
else:
    throttle_control=self.lidar_throttle_control
    if show_info == True:
        cv2.putText(imgLinea, "Heading error: "+str(round(heading_error,3)) +
"rad", (5,20), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
        cv2.putText(imgLinea, "Cross track error:
"+str(round(cross_track_error*100,3)) + "cm", (5,50),cv2.FONT_HERSHEY_SIMPLEX, 0.7,
(255, 255, 255), 1, cv2.LINE_AA)
        cv2.putText(imgLinea, "Target steering angle:
"+str(round(target_angle,3))+ "rad", (5,80), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255,
255), 1, cv2.LINE_AA)
        if real_time_control==0:
            cv2.putText(imgLinea, "Going to:
"+str(self.path[self.counter_path]), (5,110),cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255,
255), 1, cv2.LINE_AA)
        else:
            cv2.putText(imgLinea, "Going to: "+str(real_time_control),
(5,110),cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
            cv2.putText(img, "Battery level: "+str(self.battery_level),
(5,20),cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 0), 1, cv2.LINE_AA)
            cv2.putText(img, "Throttle: "+str(throttle_control),
(5,50),cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 0), 1, cv2.LINE_AA)
            cv2.imshow("Original frame", img)
            cv2.imshow("Filtred frame", filtered_img)
            cv2.imshow("Transformed frame", imgLinea)
            cv2.waitKey(1)

# If trajectory found:
elif trajectory_not_found == 1:
    steering_control=0
    throttle_control=0
    if show_info == True:
        cv2.imshow("Original frame", img)
        cv2.imshow("Filtred frame", filtered_img)
        cv2.imshow("Transformed frame", imgLinea)
        #NUESTRO: llama a la función save_image nuestra para guarda la imagen
        self.save_image(img,"frame")
        #NUESTRO
        cv2.waitKey(1)

return (steering_control,throttle_control,trajectory_not_found)

```

Código 17. Quinto fragmento del código artemis_autonomous_car.py.

```

#Function that transforms perspective according to specified transformation matrix
# Input:
#   - img: Original image
# Output:
#   - img: Transformed image

def perspective_transformation(self,img):
    img=cv2.warpPerspective(img,self.transform_matrix,(self.img_width,self.img_height))
    return img

#Function that calculates the centre of the contours of an image and the number of
contours found.
# Input:
#   - img: Filtered image
#   - offset: height offset in pixels to be added to the calculated points
# Output:
#   - num_contours: Number of contours found.
#   - centers: Vector of points [[x,y],[x,y],...] indicating the centre of the
contours exceeding the margin defined by self.minimum_contour_area
#   - larger_width_contour: Returns de the width in the X axis of the larger
def calculate_center_contours(self,img,offset):
    num_contours=0
    centers=[]
    contours=cv2.findContours(img.copy(),
cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
    contours=imutils.grab_contours(contours)
    contours=sorted(contours, key=cv2.contourArea, reverse=True)[:5]
    for contour in contours:
        if num_contours > 2:
            break
        if cv2.contourArea(contour)<self.minimum_contour_area:
            break
        M=cv2.moments(contour)
        center=[int(M["m10"]/M["m00"]),int(M["m01"]/M["m00"])+offset]
        centers.append(center)
        num_contours = num_contours + 1
    centers.sort()
    if num_contours>0:
        larger_width_contour=cv2.boundingRect(contours[0])[2]
    else:
        larger_width_contour=0
    return num_contours,centers,larger_width_contour

```

Código 18. Sexto fragmento del código artemis_autonomous_car.py.

```

    #Function that calculates the angle and deviation of the trajectory from a bird's-
    eye view image.
    #   - img: bird's eye view image or front image with perspective transform
    #   - real_time_control: 0 = (Default) Follow the path set in the path argument, 1
    = Take the left route, 2 = Take the middle route, 3 = Take the right route, 4 = Take
    left line for line changing, 5 = Take right line for line changing
    #   - show_info: If true, image with points detected and chosen trajectory is
    # Output:
    #   - heading_error: Angle of the trajectory with respect to the vehicle
    #   - cross_track_error: Transversal deviation in metres of the vehicle with
    respect to the route
    #   - trajectory_not_found: 0 = Trajectory found, 1 = Trajectory not found
    #   - filtered_img: Filtered image
    #   - img: Input img modified with points detected and chosen trajectory.
    def calculate_trajectory(self, img, real_time_control=0, show_info=True):

        cropped_img=[]
        num_contours=[0,0,0]
        points=[[[]],[[]],[[]]]
        trajectory_not_found = 0
        heading_error=0
        cross_track_error=0
        point2=[0,0]

        if real_time_control == 0:
            fork=self.path[self.counter_path]
#Function that transforms perspective according to specified transformation matrix
    # Input:
    #   - img: Original image
    # Output:
    #   - img: Transformed image
    def perspective_transformation(self, img):
        img=cv2.warpPerspective(img, self.transform_matrix, (self.img_width, self.img_heig
ht))

        return img

    #Function that calculates the centre of the contours of an image and the number of
    contours found.

```

Código 19. Séptimo fragmento del código artemis_autonomous_car.py.


```

# First points raw
num_contours[0],points[0],larger_width_contour=self.calculate_center_contours(c
ropped_img[0],380)
if num_contours[0] == 0: # Zero points detected
    point1=[320,430] # Take default point
if num_contours[0] == 1: # One point detected
    point1=points[0][0]
if num_contours[0] == 2: # Two points detected
    if fork == 1: # Take left path
        point1 = points[0][0]
    if fork == 2: # Take center path: cetermost point
        if abs(points[0][0][0]-self.img_width_center)<abs(points[0][1][0]-
self.img_width_center):
            point1 = points[0][0]
        else:
            point1 = points[0][1]
    if fork == 3: # Take right path
        point1 = points[0][1]
if num_contours[0] == 3: # Three points detected
    point1 = points[0][fork-1] # Take correponding point

# Second points raw
num_contours[1],points[1],larger_width_contour=self.calculate_center_contours(c
ropped_img[1],250)
if larger_width_contour > self.max_larger_width_contour: # Width of contour
very large: Unreliable recovered points
    # Make 2 cuts to the cropped img for contour splitting and reliable points
recovery
    cv2.line(cropped_img[1],(self.contour_cut_x_position_1,0),(self.contour_cut
_x_position_1,100),(0,0,0),6,cv2.LINE_AA)
    cv2.line(cropped_img[1],(self.contour_cut_x_position_2,0),(self.contour_cut
_x_position_2,100),(0,0,0),6,cv2.LINE_AA)
    num_contours[1],points[1],larger_width_contour=self.calculate_center_contou
rs(cropped_img[1],250)
print("larger_width_contour = "+str(larger_width_contour))
if num_contours[1] == 0: # Zero points detected
    trayectory_not_found=1
if num_contours[1] == 1: # One point detected
    point2=points[1][0]
    if real_time_control==0:
        if self.switch_fork == 1:
            self.counter_switch_fork = self.counter_switch_fork - 1
            if self.counter_switch_fork < 1:
                self.switch_fork = 0
                self.counter_path = self.counter_path + 1
                if self.counter_path == len(self.path)-1:
                    self.stop=1

```

Código 20. Octavo fragmento del código artemis_autonomous_car.py.

```

if num_contours[1] == 2:    # Two points detected

    if fork == 1 or fork == 4:    # Take left path
        point2 = points[1][0]
    if fork == 2:                # Take center path: cetermost point
        if abs(points[1][0][0]-self.img_width_center)<abs(points[1][1][0]-
self.img_width_center):
            point2 = points[1][0]
        else:
            point2 = points[1][1]
    if fork == 3 or fork == 5:    # Take right path
        point2 = points[1][1]
    if real_time_control==0:
        self.counter_switch_fork = self.frames_counter_switch_fork
        self.switch_fork = 1
if num_contours[1] == 3:    # Three points detected
    if fork < 4:                # No lane change
        point2 = points[1][fork-1] # Take corresponding point
    elif fork == 4:              # Left line change: Take left path
        point2 = points[1][0]
    elif fork == 5:              # Right line change: Take right path
        point2 = points[1][2]
    if real_time_control==0:
        self.counter_switch_fork = self.frames_counter_switch_fork
        self.switch_fork = 1

    # Third points raw
    num_contours[2],points[2],larger_width_contour=self.calculate_center_contours(c
ropped_img[2],120)
    if (fork > 3) and (num_contours[2] > 1): # Points only used in case of lane
change
    trayectory_not_found=0
    if num_contours[2] == 2: # Two points detected
        if fork == 4:        # Left lane change: Take left path
            point2 = points[2][0]
        if fork == 5:        # Right lane change: Take right path
            point2 = points[2][1]
    if num_contours[2] == 3: # Three points detected
        if fork == 4:        # Left lane change: Take left path
            point2 = points[2][0]
        if fork == 5:        # Right lane change: Take right path
            point2 = points[2][2]
    point1=[point2[0],430]    # Replace point1 for point parallel to vehicle
axis in function of point2

```

Código 21. Noveno fragmento del código artemis_autonomous_car.py.

```

# Print data

print("NUM_CONTOURS: "+str(num_contours))
print("POINTS: "+str(points))
print("Trajectory not found: "+ str(trajetory_not_found))
print("\n")
#cv2.imshow("Corte 1",cropped_img[0])
#cv2.imshow("Corte 2",cropped_img[1])
#cv2.imshow("Corte 3",cropped_img[2])

# Calculate heading error and cross track error
if trajetory_not_found == 0:
    heading_error=np.arctan((float(point1[0])-point2[0])/(point1[1]-point2[1]))
    cross_track_error=-(point2[0]-320.0+((point1[0]-
point2[0])*self.k_cross_track_error))/self.pixels_in_meter#,point2[1]+(point1[1]-
point2[1])*self.k_cross_track_error)/1000.0#-(point1[0]-320.0)/1000.0

# Add info to img
if show_info == True:
    for points_in_cut in points:
        for point in points_in_cut:
            cv2.circle(img, (point[0],point[1]), radius=2,
color=(0,0,255),thickness=5)

    if trajetory_not_found==0:
        cv2.line(img, (point1[0],point1[1]), (point2[0],point2[1]), (0,255,255),6,
cv2.LINE_AA)

    return (heading_error,cross_track_error,trajetory_not_found,filtered_img,img)

#Function that converts angle in radians to the vehicle steering control parameter
# Input:
#   - angle_radians: Angle in radians
# Output:
#   - steering_control: Vehicle steering control parameter

def rad2control(self,angle_radians):
    steering_control=angle_radians/(np.pi/6.0)+self.steering_calibration_param
    if steering_control>1:
        steering_control=1
    if steering_control<-1:
        steering_control=-1
    return steering_control

```

Código 22. Décimo fragmento del código artemis_autonomous_car.py.

```

#Function that implements the Stanley algorithm

# Input:
#   - heading_error
#   - cross_track_error
#   - speed
#   - k_stanley
# Output:
#   - delta: Steering angle target
def
calculo_stanley(self, heading_error, cross_track_error, speed, k_stanley):
    if speed==0:
        if cross_track_error>0:
            delta=heading_error+np.pi/2.0
        if cross_track_error<0:
            delta=heading_error-np.pi/2.0
        if cross_track_error == 0:
            delta=heading_error
    else:
        delta=heading_error+np.arctan((k_stanley*cross_track_error)/sp
eed)

    return delta

#NUESTRO: Función para guardar las imágenes
def save_image(self, img, filename):
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    # Nombre de la imagen a guardar (diferente por la fecha)
    unique_filename = f"{filename}_{timestamp}.jpg"
    filepath = os.path.join(self.output_dir, unique_filename)
    # Guarda la imagen
    cv2.imwrite(filepath, img)

```

Código 23. Undécimo fragmento del código artemis_autonomous_car.py.

SCRIPT car1_cloud_control_server.py

```

import imutils
import socket
import struct
import pickle
import cv2
import artemis_autonomous_car
import time

server_address = ('192.168.73.3', 20001)
show_info=True

def send_control(control_giro,control_acelerador,address):
    sock.sendto(struct.pack('c',bytes('C','ascii'))+struct.pack('d',round(control_giro,
3))+struct.pack('d',round(control_acelerador,3)),address)

if __name__ == "__main__":
    cuenta = 0
    received_payload=b''
    auto_utils=artemis_autonomous_car.artemis_autonomous_car([2,3,2,3,2,2,2,2,0],0)#[2,
3,1,3,2,2,0][2,3,2,3,2,2,2,2,0]
    #Generación de socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    sock.bind(server_address)
    #try:
        # Receive the data in small chunks and retransmit it
    while True:
        #print("Esperando dato")
        data, address = sock.recvfrom(99999)
        #print("Dato recibido")
        data_type = struct.unpack('c',bytes([data[0]]))[0]
        #print("Tipo de dato: ",data_type)
        received_payload=bytes(data[1:])
        data = pickle.loads(received_payload,encoding='latin1')
        if data_type == b'I':
            img=cv2.imdecode(data,1)
            control_giro,control_acelerador,trayectoria_not_found=auto_utils.proceso_fot
ograma(img,show_info)
            send_control(control_giro,control_acelerador,address)
        if data_type == b'D':
            pass
        if data_type == b'L':
            #print("Información LIDAR recibida!")
            auto_utils.proceso_lidar(data,False)
        if data_type == b'B':

```

Código 24. Primer fragmento de código de car1_cloud_control_server.py.

SCRIPT car1_cloud_control_server_real_time_control.py

```

import socket
import struct
import pickle
import cv2
import artemis_autonomous_car
import time

from pynput import keyboard

auto_utils=0
real_time_control=2
stop=0
def on_press(key):
    global real_time_control
    global auto_utils
    if key==keyboard.KeyCode.from_char('1'):
        real_time_control=1
    if key==keyboard.KeyCode.from_char('2'):
        real_time_control=2
    if key==keyboard.KeyCode.from_char('3'):
        real_time_control=3
    if key==keyboard.KeyCode.from_char('4'):
        real_time_control=4
    if key==keyboard.KeyCode.from_char('6'):
        real_time_control=5
    if key==keyboard.KeyCode.from_char('z'):
        auto_utils.set_stop(1)
    if key==keyboard.KeyCode.from_char('a'):
        auto_utils.set_stop(0)
listener = keyboard.Listener(
    on_press=on_press)
listener.start()
server_address = ('192.168.73.3', 20001)
show_info=True

def send_control(control_giro,control_acelerador,address):
    sock.sendto(struct.pack('c',bytes('C','ascii'))+struct.pack('d',round(control_giro,
3))+struct.pack('d',round(control_acelerador,3)),address)

if __name__ == "__main__":
    cuenta = 0
    received_payload=b''
    auto_utils=artemis_autonomous_car.artemis_autonomous_car([1,2,3],0)#[1,1,1,1,0][2,2
,2,1,2,1,2,1,1,0]

```

Código 25. Primer fragmento de código de car1_cloud_control_server_real_time_control.py.


```

#try:
    # Receive the data in small chunks and retransmit it
    while True:
        #print("Esperando dato")
        data, address = sock.recvfrom(99999)
        #print("Dato recibido")
        data_type = struct.unpack('c',bytes([data[0]]))[0]
        #print("Tipo de dato: ",data_type)
        received_payload=bytes(data[1:])
        data = pickle.loads(received_payload,encoding='latin1')
        if data_type == b'I':
            img=cv2.imdecode(data,1)
            control_giro,control_acelerador,trayectoria_not_found=auto_util
s.proceso_fotograma(img,True,real_time_control)
            send_control(control_giro,control_acelerador,address)
        if data_type == b'D':
            pass
        if data_type == b'L':
            #print("Informacion LIDAR recibida!")
            auto_utils.proceso_lidar(data,False)
        if data_type == b'B':
            auto_utils.set_battery_level(data)
    #except Exception:
    #    pass
    #finally:
    #    # Clean up the connection
    #    pass#connection.close()

```

Código 26. Segundo fragmento de código de carl_cloud_control_server_real_time_control.py.

SCRIPT car1_manual_control_server.py

```

import socket
import struct
import pickle
import cv2
#import artemis_autonomous_car
import time
from pynput import keyboard
import numpy as np
import cmath, math

server_address = ('192.168.73.3',20001)
control_acelerador=0
control_giro=0

def on_press(key):
    global control_acelerador
    global control_giro
    if key==keyboard.KeyCode.from_char('2'):
        control_acelerador=1
    if key==keyboard.KeyCode.from_char('w'):
        control_acelerador=0.6
    if key==keyboard.KeyCode.from_char('s'):
        control_acelerador=-0.5
    if key==keyboard.KeyCode.from_char('x'):
        control_acelerador=-0.9
    if key==keyboard.KeyCode.from_char('a'):
        control_giro=1
    if key==keyboard.KeyCode.from_char('d'):
        control_giro=-1

def on_release(key):
    global control_acelerador
    global control_giro
    if key==keyboard.KeyCode.from_char('2'):
        control_acelerador=0
    if key==keyboard.KeyCode.from_char('w'):
        control_acelerador=0
    if key==keyboard.KeyCode.from_char('s'):
        control_acelerador=0
    if key==keyboard.KeyCode.from_char('x'):
        control_acelerador=0
    if key==keyboard.KeyCode.from_char('a'):
        control_giro=0.25
    if key==keyboard.KeyCode.from_char('d'):
        control_giro=0.25

```

Código 27. Primer fragmento de código de car1_manual_control_server.py.

```

listener = keyboard.Listener(
    on_press=on_press,
    on_release=on_release)
listener.start()

def send_control(control_giro, control_acelerador, address):
    sock.sendto(struct.pack('c', bytes('C', 'ascii'))+struct.pack('d', round(control_giro, 3))+struct.pack('d', round(control_acelerador, 3)), address)

if __name__ == "__main__":

    cuenta = 0
    received_payload=b''
    #auto_utils=artemis_autonomous_car.artemis_autonomous_car([0])
    #Generacion de socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    sock.bind(server_address)
    #try:
    # Receive the data in small chunks and retransmit it
    while True:
        #print("Esperando dato")
        data, address = sock.recvfrom(99999)
        #print("Dato recibido")
        data_type = struct.unpack('c', bytes([data[0]]))[0]
        #print("Tipo de dato: ", data_type)
        received_payload+=bytes(data[1:])
        data = pickle.loads(received_payload, encoding='latin1')
        if data_type == b'I':
            img=cv2.imdecode(data,1)
            cv2.imshow("Coche ARTEMIS", img)
            cv2.waitKey(1)
            send_control(control_giro, control_acelerador, address)
        if data_type == b'D':
            pass
        if data_type == b'L':

            img_lidar = np.zeros((480, 640, 3), np.uint8)

            i=0
            ranges=data

```

Código 28. Segundo fragmento de código de car1_manual_control_server.py.

```

for range in ranges:
    Z=cmath.rect(range,math.radians(-i-90))
    i = i+1
    if Z.real != float('inf') and Z.real != float('-inf'):
        x=int(Z.real*70)+320
    else:
        x=0
    if Z.imag != float('inf') and Z.imag != float('-inf'):
        y=int(Z.imag*70)+240
    else:
        y=0
    cv2.circle(img_lidar,(x,y), radius=2,
color=(0,0,255),thickness=2)

    cv2.imshow("Mapa LIDAR",img_lidar)
    cv2.waitKey(1)
    pass
except Exception:
    # pass
finally:
    # Clean up the connection
    # pass#connection.close()

```

Código 29. Tercer fragmento de código de car1_manual_control_server.py.

SCRIPT real_time_detect_V3.py

```

def analizar():
    import os
    import cv2
    from ultralytics import YOLO # Ensure you have this package correctly
    installed and imported
    import numpy as np
    import math

    if os.path.exists("/home/goat/compartido/images"):
        directory = "/home/goat/compartido/images"

        if os.path.exists("/home/goat/servicios_cloud_deep_racer/best.pt"):
            model = YOLO("/home/goat/servicios_cloud_deep_racer/best.pt")
            classNames = ["OBLIGATORIO-CONTINUAR-RECTO", "OBLIGATORIO-
GIRAR-DERECHA", "OBLIGATORIO-GIRAR-IZQUIERDA", "PROHIBIDO", "STOP",
"VELOCIDAD-MAX-30", "VELOCIDAD-MAX-90"]

            # Define the output file path
            output_file_path = "/home/goat/compartido/senales.txt"

            # Para que me borre el contenido inicial
            with open(output_file_path, 'w') as file:
                pass # Just opening and closing in 'w' mode clears the
file

            box_prioritario = None # Initialize as None to handle cases
where no boxes meet criteria

            while True:
                image_files = [os.path.join(directory, file) for file in
os.listdir(directory) if file.endswith(('.jpg', '.jpeg', '.png'))]
                image_files.sort() # Sort the list alphabetically
                for image_file in image_files:
                    img = cv2.imread(image_file)
                    results = model(img)
                    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                    old_maxboxsize = 0
                    predictions = []

```

Código 30. Primer fragmento de código de real_time_detect_V3.py.

```

for r in results:
    boxes = r.bboxes
    for box in boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cv2.rectangle(img, (x1, y1), (x2, y2), (255,
0, 255), 3)

        confidence = math.ceil((box.conf[0] * 100))
/ 100

        cls = int(box.cls[0])
        prediction = [x1, y1, x2, y2, confidence,
cls]

        predictions.append(prediction)

        if (abs(x2-x1) > old_maxboxsize):
            old_maxboxsize = abs(x2-x1)
            box_prioritario = classNames[cls]

    if box_prioritario:
        # Write to file if box_prioritario is not None
        with open(output_file_path, 'a') as file:
            file.write(f"{box_prioritario}\n")

    cv2.destroyAllWindows()
    break # Remove or modify this line based on how you
want to handle the loop

    return box_prioritario

# Call the function
analizar()

```

Código 31. Segundo fragmento de código de real_time_detect_V3.py.

SCRIPT control_vehiculo_conectado.py

```

import socket
import warnings
import struct
import pickle
import cv2
import artemis_autonomous_car
import time
from pynput import keyboard
import numpy as np
import cmath, math
import real_time_detect_V2
import os
import multiprocessing
import threading

#Configuramos la Ip y Puerto de nuestro socket
server_address = ('192.168.73.3',20001)

#nuevo
show_info=True
#nuevo

warnings.simplefilter("ignore")

#Definimos variables de control
control_acelerador=0.41
acc=0.6
control_giro=0

def set_control_giro(value, duration):
    global control_giro
    control_giro = value
    print(f"Variable establecida a {value} durante {duration} segundos.")
    timer = threading.Timer(duration, reset_control_giro)
    timer.start()

def reset_control_giro():
    global control_giro
    control_giro = 0
    print("Variable restablecida a 0.")

```

Código 32. Primer fragmento de código de control_vehiculo_conectado.py.

```

def set_control_acelerador(value, duration):
    """
    Establece la variable global `control_acelerador` a `value` durante
    `duration` segundos.

    :param value: int, el valor al que se establecerá la variable
    :param duration: int, la duración en segundos durante la cual la
    variable se mantendrá en el valor dado
    """
    global control_acelerador, ignore_keyboard
    control_acelerador = value
    print(f"Variable establecida a {value} durante {duration} segundos.")
    ignore_keyboard = True
    timer = threading.Timer(duration, reset_control_acelerador)
    timer.start()

def reset_control_acelerador():
    """
    Restablece la variable global `control_acelerador` a 0.
    """
    global control_acelerador, ignore_keyboard
    control_acelerador = 0
    ignore_keyboard = False
    print("Variable restablecida.")
    listener.start()

def leer_ultima_linea(nombre_fichero):
    """
    Lee la última línea de un fichero de texto.

    :param nombre_fichero: Nombre del fichero de texto.
    :return: La última línea del fichero.
    """
    ultima_linea = ''

    with open(nombre_fichero, 'r', encoding='utf-8') as archivo:
        for linea in archivo:
            ultima_linea = linea.strip()

    return ultima_linea

```

Código 33. Segundo fragmento de código de control_vehiculo_conectado.py.

```

#Función envío de controles a Vehículo
def send_control(control_giro,control_acelerador,address):
    sock.sendto(struct.pack('c',bytes('C','ascii'))+struct.pack('d',round(c
ontrol_giro,3))+struct.pack('d',round(control_acelerador,3)),address)

if __name__ == "__main__":

    # Inicializar
    control_acelerador=0.41
    acc=0.6
    print (control_giro,control_acelerador)
    control_giro = 0
    ignore_keyboard = True

    cuenta = 0
    received_payload=b''
    auto_utils=artemis_autonomous_car.artemis_autonomous_car([2,3,2,3,2,2,2
,2,0],0)#[2,3,1,3,2,2,0][2,3,2,3,2,2,2,2,0]
    print (control_giro,control_acelerador)
    #Generacion de socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    print (control_giro,control_acelerador)
    sock.bind(server_address)
    #try:
        # Receive the data in small chunks and retransmit it

    aux = ''
    aux2 = control_acelerador
    aux3= control_giro
    cont_linea = 1
    print (control_giro,control_acelerador)
    while True:
        #print("Esperando dato")
        data, address = sock.recvfrom(99999)
        #print("Dato recibido")
        data_type = struct.unpack('c',bytes([data[0]]))[0]
        print (control_giro,control_acelerador)
        #print("Tipo de dato: ",data_type)
        received_payload=bytes(data[1:])
        print (control_giro,control_acelerador)
        data = pickle.loads(received_payload,encoding='latin1')
        if data_type == b'I':
            img=cv2.imdecode(data,1)
            cv2.imshow("Coche ARTEMIS",img)

```

Código 34. Tercer fragmento de código de control_vehiculo_conectado.py.

```

#nuevo
    aux2,aux3,trayectory_not_found=auto_utils.proceso_fotograma(img
,show_info)

    send_control(control_giro,control_acelerador,address)
    #nuevo

    senal=leer_ultima_linea('senales.txt')

    if senal!=aux:
        # IF majestuoso
        if aux == "OBLIGATORIO-CONTINUAR-RECTO":
            set_control_giro(0, 1.5) # Función para girar no poder
girar durante un tiempo
        elif aux == "OBLIGATORIO-GIRAR-DERECHA":
            set_control_giro(-1, 1.5) # Función para girar a la
derecha durante un tiempo
        elif aux == "OBLIGATORIO-GIRAR-IZQUIERDA":
            set_control_giro(1, 1.5) # Función para girar a la
izquierda durante un tiempo
        elif aux == "PROHIBIDO":
            set_control_acelerador(0, 1.5) # Función para parar
durante un tiempo
        elif aux == "STOP":
            set_control_acelerador(0, 1.5) # Función para parar
durante un tiempo
        elif aux == "VELOCIDAD-MAX-30":
            control_acelerador=0.45
            print('Velocidad máxima 30')

        elif aux == "VELOCIDAD-MAX-90":
            control_acelerador=0.5
            print('Velocidad máxima 30')
        else:
            control_giro = 0

    # Nuestro
    #Almacenamos la señal anterior
    aux=senal

    cv2.waitKey(1)
    send_control(control_giro,control_acelerador,address)
    if data_type == b'D':
        pass

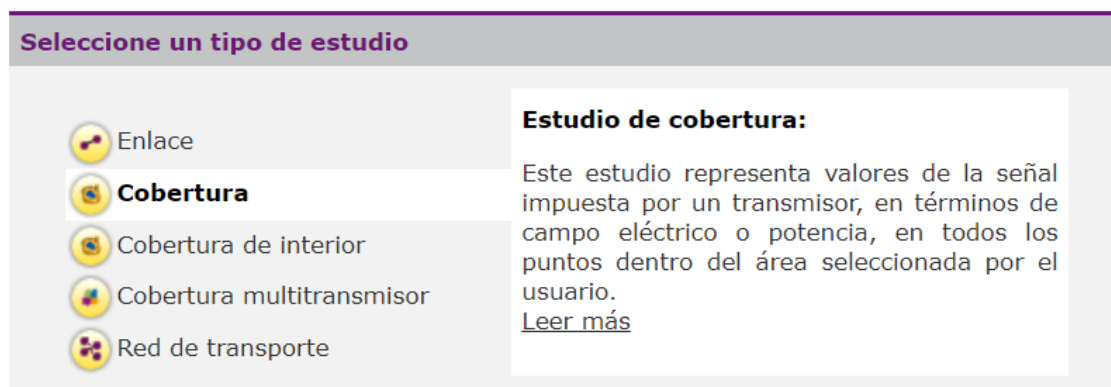
```

Código 35. Cuarto fragmento de código de control_vehiculo_conectado.py.

ANEXO B-TUTORIAL XIRIO

Realización de un estudio empleando la herramienta Xirio-Online (Tutorial)

En la herramienta Xirio-Online se pueden realizar de forma gratuita una serie de estudios. Entre los cuales encontramos: estudios de enlace, de red de transporte, cobertura simple, cobertura de interior y cobertura multitransmisor. A continuación, se va a ejemplificar como realizar a modo de tutorial un estudio de cobertura simple.



Seleccione un tipo de estudio

- Enlace
- Cobertura**
- Cobertura de interior
- Cobertura multitransmisor
- Red de transporte

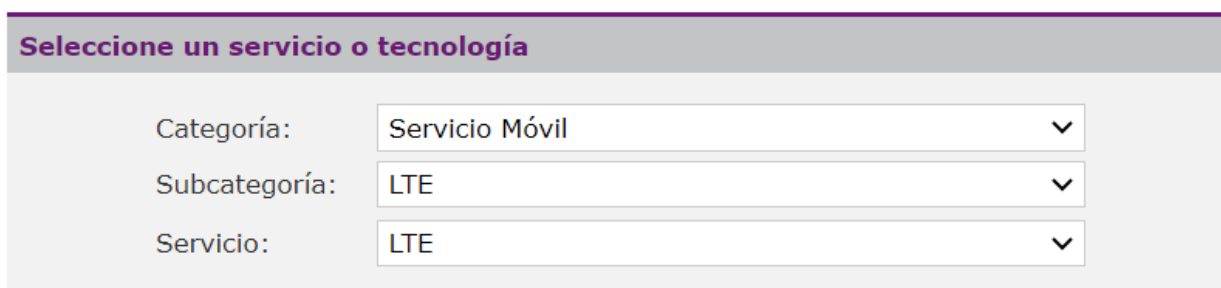
Estudio de cobertura:

Este estudio representa valores de la señal impuesta por un transmisor, en términos de campo eléctrico o potencia, en todos los puntos dentro del área seleccionada por el usuario.

[Leer más](#)

Figura 137. Selección del estudio.

Después de haber realizado la selección del servicio sobre el cual se va a realizar el estudio de cobertura, haremos click sobre la opción de modo asistente la cual permitirá facilitar la creación de dicho estudio.



Seleccione un servicio o tecnología

Categoría: Servicio Móvil ▼

Subcategoría: LTE ▼

Servicio: LTE ▼

Figura 138. Selección del servicio.

Una vez se abre el asistente es necesario asignar un nombre al estudio, así como una breve descripción sobre el mismo. La banda de frecuencia que se ha de emplear debe estar definida con anterioridad o, por el contrario, crearla desde cero. Para la continuación del tutorial se empleará la banda de frecuencia LTE 7 según se ha definido en la licitación.

Figura 139. Descripción del estudio.

Una vez se ha definido el estudio a realizarse, es necesario realizar una sectorización sobre el mismo y, para ello, hace falta asignar un nombre, así como el emplazamiento en el cual se colocarán las antenas correspondientes. La ubicación de este emplazamiento estará previamente definida o, por el contrario, podrá seleccionarse de forma manual, siempre y cuando se conozcan sus coordenadas. También es posible definir su ubicación mediante el visualizador incorporado en Xirio-Online.

Figura 140. Creación de un sector.

Definida la posición y el nombre del sector, se puede realizar una sectorización del transmisor. Esto permite emplear la misma posición de un emplazamiento para colocar más de una antena, generando un estudio de cobertura individual para cada uno de los sectores deseados.

La aplicación concreta para la que se han realizado estos tutoriales requiere emplear un par de sectores por estación base, de forma que cada uno de los sectores se haga cargo de cubrir un lado de la calzada por la cual circularán los vehículos. Se definirá, por tanto, una antena sectorial direccional y podrá crearse o recoger del catálogo si esta está previamente definida. Cada uno de esos sectores ha de recibir un nuevo nombre puesto que serán estudios de cobertura simples nuevos, individuales e independientes.

Sectorización

☒ Sectorizar estudio

Permite definir sectores en la misma estación. Esta opción crea un estudio de cobertura para cada sector, hasta un máximo de ocho.

Número de sectores

Azimut del primer sector ° [Recalcular](#)

Antena sectorial      

Seleccione la antena que se utilizará para la simulación de cada uno de los sectores.

Sectores

Nombre	Tx	Azimut	Inclinación mecánica	Inclinación eléctrica	
Nueva cobertura_S1	Nuevo sector_S1	180	0	0	 
Nueva cobertura_S2	Nuevo sector_S2	0	0	0	 

Si los valores de acimut e inclinación por defecto no son los que necesita personalice cada uno de ellos.

[< Anterior](#) [> Siguiente](#) [> Cancelar](#)

Figura 141. Sectorización del transmisor.

Una vez se ha creado cada uno de los sectores correspondientes es necesario realizar una configuración de sus parámetros. Para ello, se ha definido el tipo de sistema, la altura de la antena, las frecuencias de transmisión oportunas para el sector, su polarización y la potencia de radiación de la antena.

Parámetros del sector [Ayuda ?](#)

Los parámetros de radio del transmisor se han configurado automáticamente para el servicio radioeléctrico del estudio. Es conveniente que revise y personalice los mismos.

Tipo sistema:

Altura antena: m

Referencia de alturas de antenas

Alturas respecto a:

Usar altura de edificio:

Altura edificio: m

Figura 142. Configuración sector.

Frecuencias de transmisión

Frecuencias	Canal
2665.000 MHz	1'

Polarización: Vertical

Feeder:

Longitud del feeder: 0 m

Pérdidas del feeder: 0.00 dB

Pérdidas pasivos: 0 dB

Potencia: 45.051 dBm

RSRP: W

Figura 143. Configuración de frecuencias de un sector.

Definida de la configuración de los parámetros de la antena que servirá a la estación base, es necesario también configurar los parámetros del terminal UE para los cuales se han definido una altura de 1,5m, polarización vertical y frecuencia de transmisión, para el canal de Up-link, a 2545 MHz.

Parámetros del terminal

Los parámetros de recepción se han configurado automáticamente para el servicio radioeléctrico del estudio. Es conveniente que revise y personalice los mismos.

Antena: 3G/4G/5G 2 dBi Omni

Altura antena: 1.5 m

Frecuencias de transmisión:

Frecuencias	Canal
2545.000 MHz	1

Polarización: Vertical

Figura 144. Parámetros terminal.

Es importante elegir un método de cálculo apropiado para la aplicación que vaya a estudiarse. En nuestro caso se ha empleado el método de propagación Rec. UIT-R P.526-15, que ofrece la herramienta Xirio de manera gratuita. Este método, puede emplearse tanto en entornos rurales como en entornos urbanos y ofrece una resolución suficiente para nuestro estudio de planificación.

El cálculo gratuito de este servicio ofrece la configuración de parámetros básicos como es el factor K y el margen de desvanecimiento. Sin embargo, estos parámetros no han sido modificados y se han dejado por defecto. También ofrece modificar la permitividad relativa, así como la conductividad del terreno, pero de nuevo estos valores no han sido modificados. Tampoco se ha contemplado la penetración de edificios para el caso de este estudio, en el que se estudia una calzada, se ha entendido que no se encontrará con este tipo de edificaciones.

Método de cálculo

Nombre: Nuevo Rec. UIT-R P.526-15

Método de propagación: Rec. UIT-R P.526-15 ▼

 Método determinístico basado en difracción. Válido para frecuencias mayores de 30 MHz. Empleado en todos los servicios radioeléctricos en entornos rurales y mixtos siempre que se disponga de cartografía de media o alta resolución.

Resolución: 100 m/pixel

☐ Limitar la distancia máxima del cálculo

Límite: 0 m

Parámetros básicos

☐ Cálculo automático de factor K

Factor K: 1.333 

☐ Usar margen de desvanecimiento desde clutter

Margen de desvanecimiento: 0 dB

Figura 145. Método de cálculo.

Propiedades de Rec. UIT-R P.526-15

Permitividad relativa: 5

Conductividad: 0.005 S/m

Penetración en edificios

☒ No aplicar

☐ No calcular en edificios

☐ Calcular sólo en azoteas

☐ Pérdidas fijas en edificios

☐ Receptor ubicado en el interior del edificio

Pérdidas fijas: 10 dB

Tipo de cálculo: Pérdidas menos restrictivas ▼

Pérdidas horizontales: 0.6 dB/m

Pérdidas verticales: 5 dB/piso

Metros por piso: 3 m

Receptor ubicado en el piso: 0

Figura 146. Otras propiedades de cálculo.

Para realizar el estudio, y que de forma visual sea fácilmente reconocible, es necesario definir las propiedades del rango de colores que estableceremos a nuestro gusto. Para nuestra aplicación se ha definido el siguiente rango.

Rango de colores ★

Nombre:

Rangos

Color	Rango	Descripción
	$[-136.00, -103.00)$	Muy mala
	$[-103.00, -91.00)$	Buena
	$[-91.00, -85.00)$	Muy Buena
	$[-85.00, \text{Infinity})$	Ideal

Visualizar niveles de señal ▼

Figura 147. Definición rango colores.

Una vez se ha llevado a cabo toda esta configuración, se habrá creado un nuevo estudio de cobertura. Aquí aún será necesario definir el área donde el estudio se vaya a llevar a cabo. También será posible realizar alguna modificación de la configuración previa. Una vez se esté contento se realizará un click en la opción “calcular estudio”, tras la cual se ofrecerá el cálculo del estudio de forma gratuita y de baja resolución. Se ofrecen otros servicios de pago, pero estos no han sido contemplados para la realización del estudio.

Acciones

- Configurar estudio
- Centrar visor en el estudio
- Ajustar área automáticamente
- Duplicar estudio
- Sectorizar estudio
- Mostrar en 3D
- Eliminar estudio

Calcular estudio

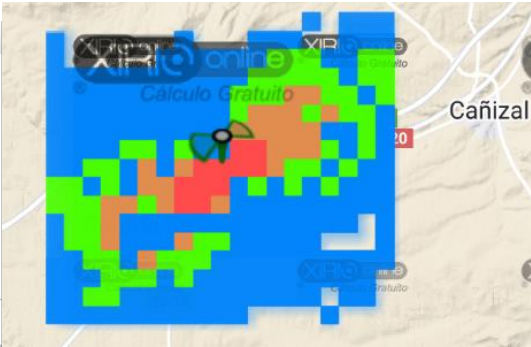


Figura 148. Resultado del estudio simple.

Una vez realizado el estudio individual para cada una de las coberturas simples, se podrá realizar una cobertura multi transmisor. Donde se procederá a la creación del estudio de manera similar a como se ha visto anteriormente.

Seleccione un tipo de estudio

-  Enlace
-  Cobertura
-  Cobertura de interior
-  **Cobertura multitransmisor**
-  Red de transporte

Estudio de cobertura multitransmisor:

Este estudio permite determinar la cobertura radioeléctrica que alcanza una red de estaciones. Consiste en una combinación de estudios de coberturas simples.

[Leer más](#)

Figura 149. Selección tipo estudio.

De acuerdo con la Figura 148, esta vez se escogerá la opción de cobertura multitransmisor, en la cual no existe un modo asistente. Se desplegará un menú en el cual podrán añadirse las coberturas individuales al estudio de forma que se importarán las coberturas simples definidas y calculadas con anterioridad.

Añadir coberturas al estudio:

Filtro por área:

☒ Intersección con área

☐ Intersección con transmisor

Esquina NorOeste

Esquina SurEste

Latitud

Longitud

41°38'17.27"N

005°47'34.17"W

40°41'13.17"N

004°48'54.15"W

Resultados 1-16 de 16 (0 seleccionados)

Buscar

Dupl.	Nombre	Cobertura multitransmisor	Modificado
☐			
☒	7_churches_S2	TheGoatProyects_4G_	2024-05-22 - 08:47:46
☒	7_iglesias_2_S1	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Alaejos_S1	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Alaejos_S2	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Castrillo_S1	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Castrillo_S2	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Moriscos_S1	TheGoatProyects_4G_	2024-05-22 - 08:48:35
☒	Pajares_1_S1	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Pajares_1_S2	TheGoatProyects_4G_	2024-05-21 - 14:03:49
☒	Pedernales_1	TheGoatProyects_4G_	2024-05-21 - 14:03:49

Aceptar Cancelar

Figura 150. Importación de cobertura.

Una vez se hayan importado las coberturas simples al estudio multitransmisor. De nuevo, será tan sencillo cómo calcular el estudio de forma gratuita, siguiendo el procedimiento definido en el apartado anterior. En este caso, el cálculo del estudio empleará un mayor tiempo y el servicio de alta resolución repercutirá en un mayor coste. La siguiente imagen muestra el resultado del estudio inicial con los establecimientos definidos según el ministerio del gobierno.



Figura 151. Estudio Multitransmisor.

ANEXO C-TUTORIAL ROBOFLOW

En el siguiente Anexo se recoge un breve tutorial sobre el uso de la herramienta Roboflow, ampliamente utilizada durante el desarrollo de este proyecto. Este Anexo se centra en explicar a modo de tutorial los pasos seguidos para la generación de un dataset desde cero.

En primer lugar, debemos acceder a la plataforma de Roboflow¹⁵ y crearnos una cuenta para poder acceder a sus herramientas. Una vez dentro debemos seleccionar + *Create New Project* a la derecha de la interfaz.

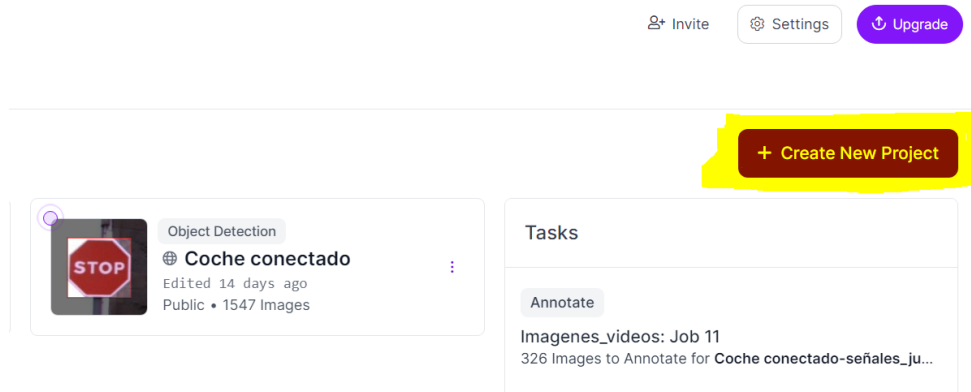


Figura 152. Creación de un nuevo proyecto con Roboflow.

Nos aparecerá una ventana emergente donde debemos darle un nombre a nuestro proyecto y especificar los objetos que se van a detectar (porque en este caso estamos elaborando un modelo que detecta objetos), finalmente, con pulsar la opción *Object Detection* dejando el resto de las opciones por defecto podremos continuar.

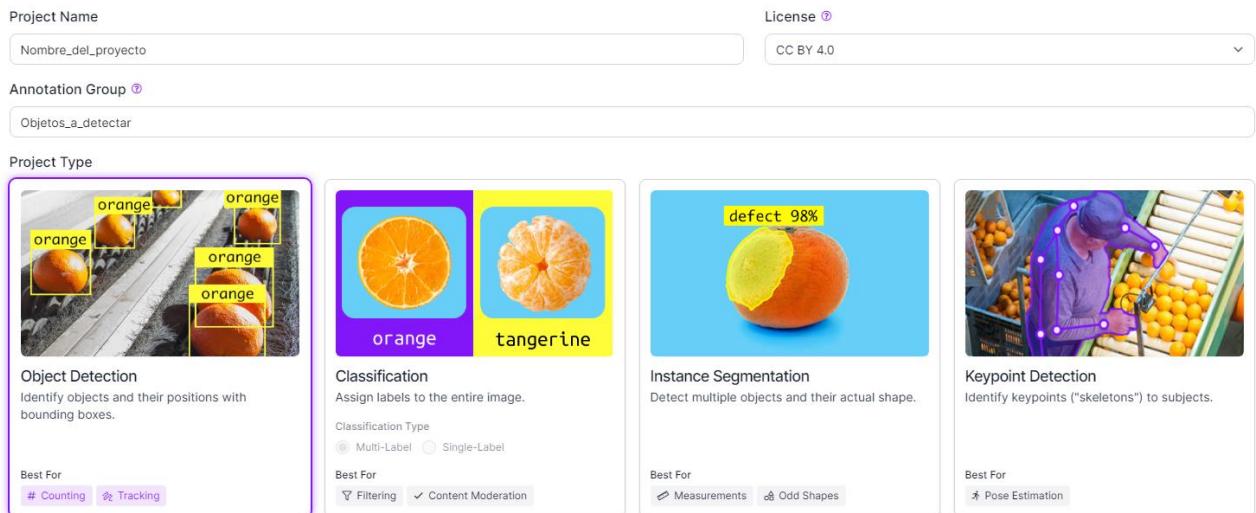


Figura 153. Características del proyecto.

Una vez creado nuestro proyecto nos debería de aparecer en el *Workspace*. Ahora, podemos empezar a elaborar el dataset, para ello, debemos seleccionar el proyecto que acabamos de crear y en el menú izquierdo debemos elegir la opción de *Classes* y en la ventana elegimos la opción de *Add Classes*, rellenamos tantas clases como queramos tener en nuestro algoritmo y elegimos *Save and Continue* para terminar.

¹⁵ Disponible en: <https://app.roboflow.com>

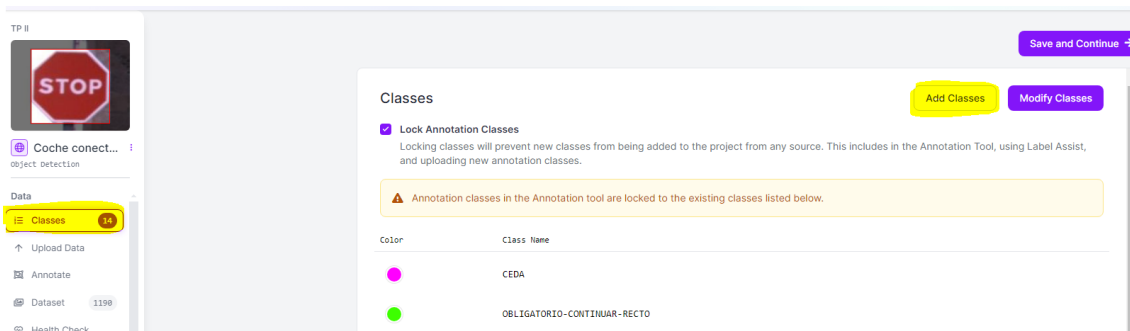


Figura 154. Creación de nuevas clases.

Ahora se nos abrirá una ventana donde podremos cargar arrastrando o mediante el explorador de archivos las imágenes que queremos usar para entrenar, validar y testear nuestro modelo. Añadimos todas las imágenes que deseemos y elegimos *Save and Continue*.

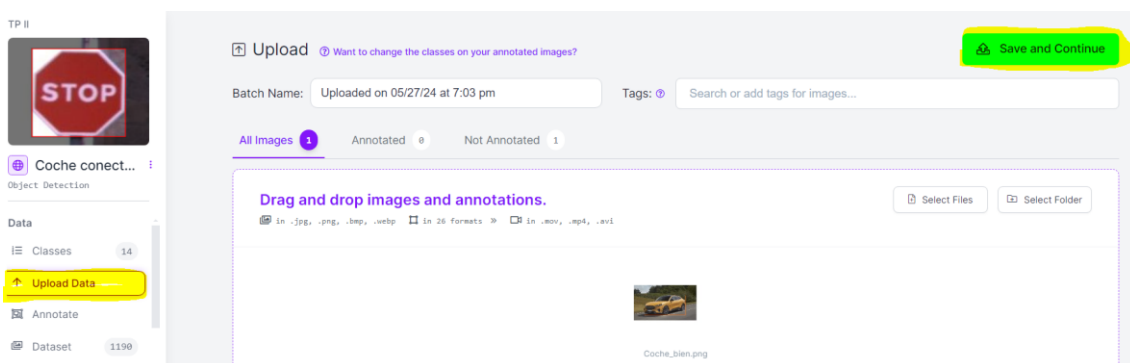


Figura 155. Añadimos las imágenes al dataset.

En la siguiente opción del menú, la opción de *Annotate* podemos empezar a anotar las imágenes haciendo click en la opción *Annotate Images* de las imágenes que acabamos de añadir. Al hacerlo se nos abrirá una ventana donde podremos ir etiquetando las imágenes una a una.

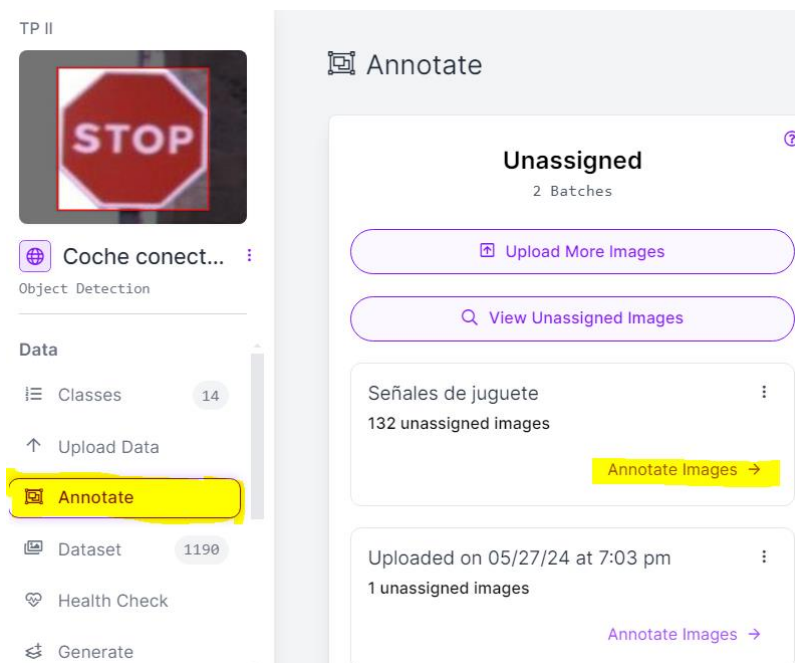


Figura 156. Elegimos la opción de Anotar Imágenes.



Figura 157. Entorno de etiquetado con descripción de sus componentes.

Como podemos apreciar en la Figura 156, en el entorno de etiquetado tenemos un visor donde poder ver las imágenes y etiquetarlas, un cuadro de información, un cuadro con herramientas variadas y un selector de las clases a etiquetar.

Una vez hayamos etiquetado todas las imágenes que deseemos volvemos al proyecto y dentro de *Annotate*, elegimos *Add X images to Dataset*.

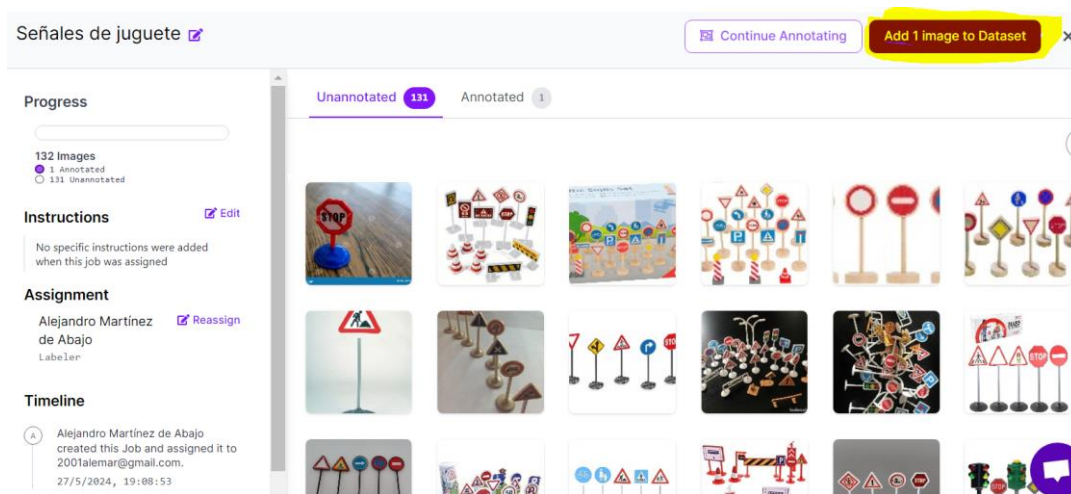


Figura 158. Añadimos las imágenes al dataset.

Podemos comprobar el dataset en la opción del menú de la izquierda *Dataset*:

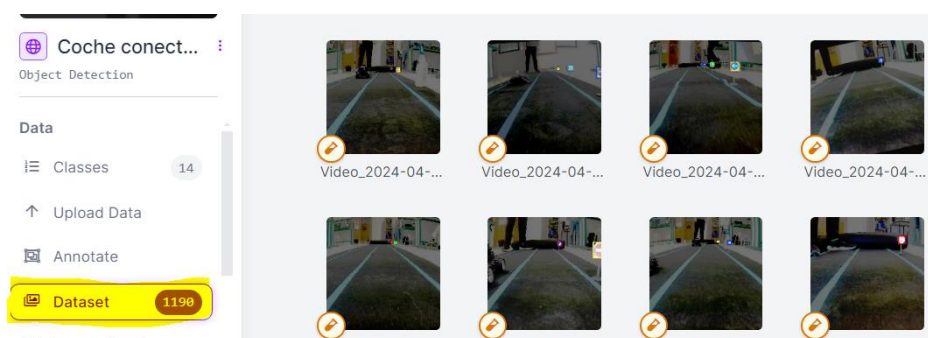


Figura 159. Comprobación del Dataset.

A la hora de generar el dataset podemos elegir la cantidad de imágenes que se añadirán al conjunto de datos de entrenamiento, de validación y de test, para ello simplemente podemos emplear el propio *slider* de Roboflow para elegir los porcentajes.



Figura 160. Elección de los porcentajes de cada grupo de imágenes.

Además, podemos consultar los parámetros que se han ido observando a lo largo del informe en la opción *Health Check* del menú de la izquierda. En él, disponemos de estadísticas como el tamaño medio de imágenes, el número de imágenes en el dataset o la cantidad de anotaciones realizadas.

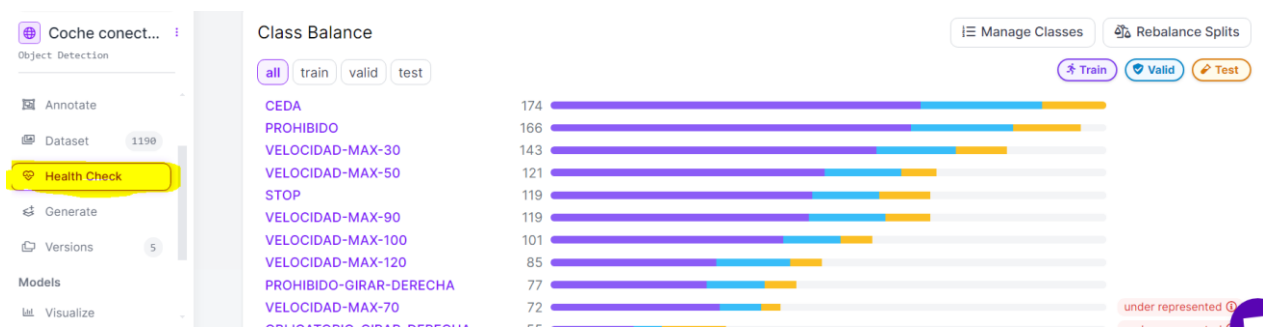


Figura 161. Health Check del dataset.

Finalmente, también tenemos la opción de aplicarle distintos factores de preprocesado y de augmentación al dataset, pero como ya se ha mencionado y explicado en el capítulo de Inteligencia Artificial carece de sentido repetirlo.

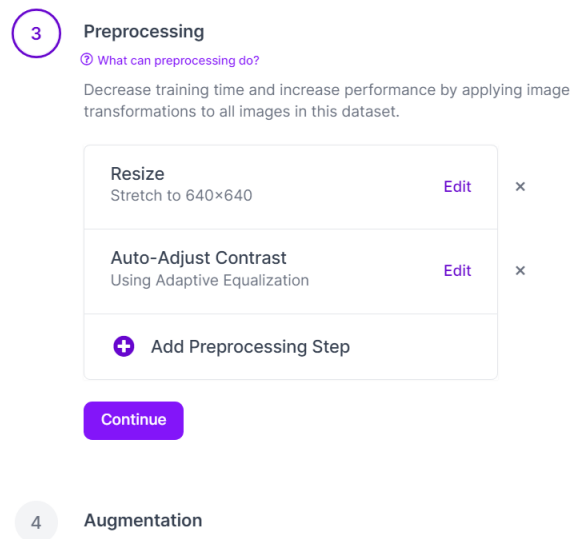


Figura 162. Opciones de preprocesado y augmentación.

Tras haber seguido estos pasos, deberás haber sido capaz de repicar nuestro dataset con las imágenes que más gustes.

ANEXO D-IMÁGENES EN GRAN ESCALA

Resultados validación dataset señales en miniatura (ampliadas):

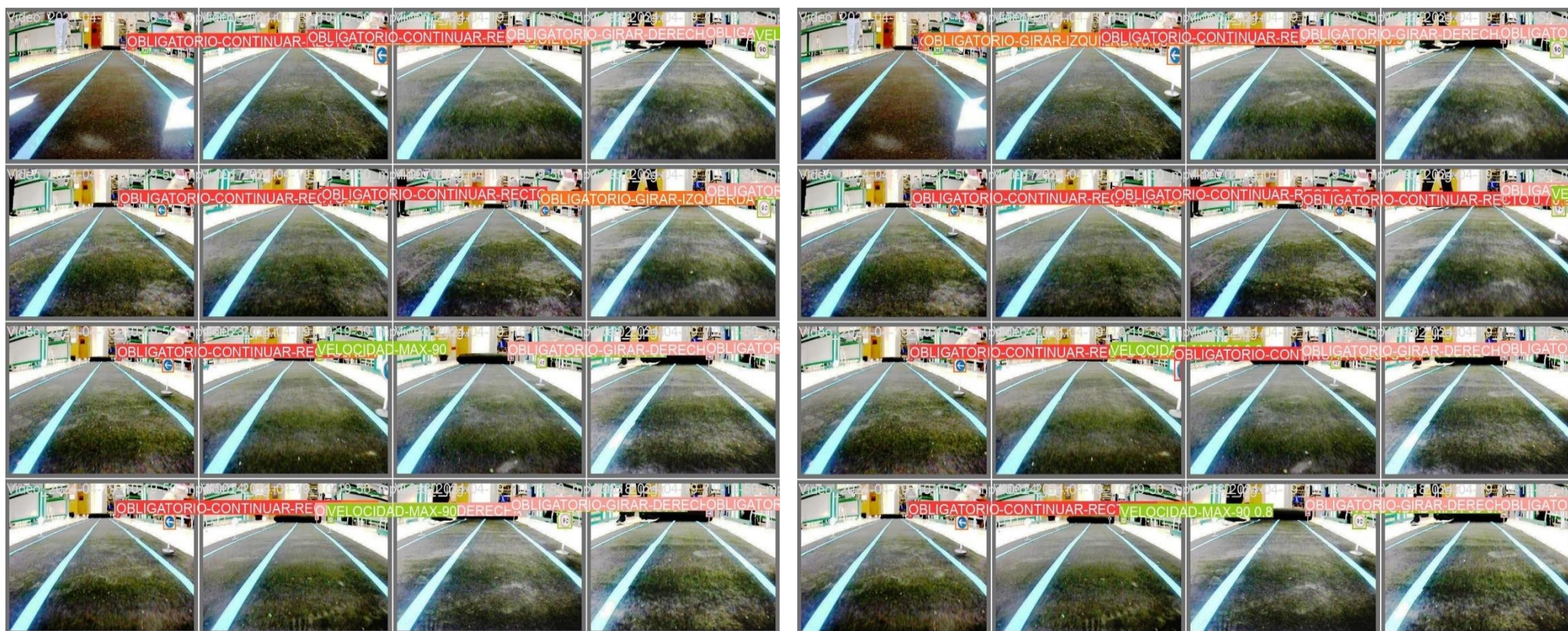


Figura 163. Collage con las predicciones sobre las imágenes (izquierda) y las etiquetas reales (derecha) del dataset de señales en miniatura a gran escala y mayor resolución.

Resultados validación dataset señales reales (ampliadas):



Figura 164. Collage con las predicciones sobre las imágenes (izquierda) y las etiquetas reales (derecha) del dataset de señales reales a gran escala y mayor resolución.

ANEXO E-TUTORIAL JETSON

En el siguiente Anexo, se tratará en profundidad a modo de tutorial los pasos seguidos para instalar por primera vez de forma correcta las herramientas necesarias para poder trabajar con la Jetson.

Debemos de tener en cuenta que para poder comenzar debemos instalar el S.O en una microSD de la suficiente capacidad¹⁶ para poder cargar la imagen y las herramientas de desarrollo de software adecuadas. En primer lugar, debemos de hacernos con el software de NVIDIA SDK Manager¹⁷, ya que si tratamos de instalar la imagen directamente en la microSD al iniciar la Jetson no terminará de arrancar por que le falta un gestor de arranque. Es importante destacar que debemos disponer de un ordenador con Ubuntu 20.04 o 22.04 instalado para poder bajarnos la versión de Jetpack 6.0 (compatible con la Jetson Orin Nano y con YOLOv9). Podemos comprobar la tabla de compatibilidades en el mismo lugar de donde hemos obtenido el software SDK Manager (debemos tener una cuenta de NVIDIA).

NVIDIA SDK	SDK Version	Supported Host Operating System												
		Ubuntu				CentOS			Red Hat Enterprise Linux				Rocky Linux	Debian
		16.04	18.04	20.04	22.04	7.6	8	8.2	7.6	8	8.2	8.6	8.6	10.8
JetPack	JetPack 4.x	✓	✓											
	JetPack 5.x		✓	✓										
	JetPack 6.x			✓	✓									

Figura 165. Fragmento de tabla de compatibilidades de los distintos softwares SDK de NVIDIA (NVIDIA).

Jetpack 6.0 consiste en un conjunto de herramientas de desarrollo y bibliotecas de software que han sido diseñadas específicamente para la familia de productos Jetson, este conjunto de herramientas incluye componentes como (NVIDIA, Key Features in Jetpack 6.0, 2024):

- **Kits de desarrollo de software (SDKs):** Herramientas y bibliotecas para desarrollar aplicaciones para la plataforma Jetson.
- **Librerías de procesamiento de Datos:** Librerías ideadas para optimizar el uso de los recursos de CPU y GPU al máximo.
- **Entorno de desarrollo (IDE):** Herramientas para la depuración, desarrollo y análisis de rendimiento de nuestros diseños.

Es por ello, por lo que es tan importante de disponer de todo esto en la Jetson.

Una vez hayamos descargado el software del instalador en un ordenador con los requisitos de S.O especificados, ejecutaremos el programa. Nota: en lugar de bajarse el ejecutable desde Internet, podemos ejecutar directamente el siguiente comando.

```
sudo apt install /home/goat/sdkmanager_2.1.0-11669_arm64.deb
```

Una vez instalado el software SDK Manager, podemos ejecutarlo con el comando:

```
./home/sdkmanager
```

¹⁶ El propio fabricante NVIDIA recomienda una microSD de 64 GB UHS-1 o más. Información disponible en: <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit#intro>

¹⁷ Software SDK Manager y tabla de compatibilidades oficial de NVIDIA disponible en: <https://developer.nvidia.com/sdk-manager>

Nos aparecerá una primera ventana donde debemos realizar los siguientes puntos para completar el paso 1:

- Primero debemos elegir *Jetson* en la opción *Product Category*.
- Marcamos la opción d *Host Machine*, enchufamos nuestra Jetson Orin Nano al ordenador a través de un cable tipo c-USB y elegimos nuestra placa, la Jetson Orin Nano en la opción de *Target Hardware*.
- Elegimos la opción de *Jetpack 6.0* en la opción de *SDK Version*
- Marcamos la opción *DeepStream* de *Additional SDKS*
- Finalmente, elegimos *Continue* para pasar al paso 2.

Debería de quedarnos de la siguiente forma:

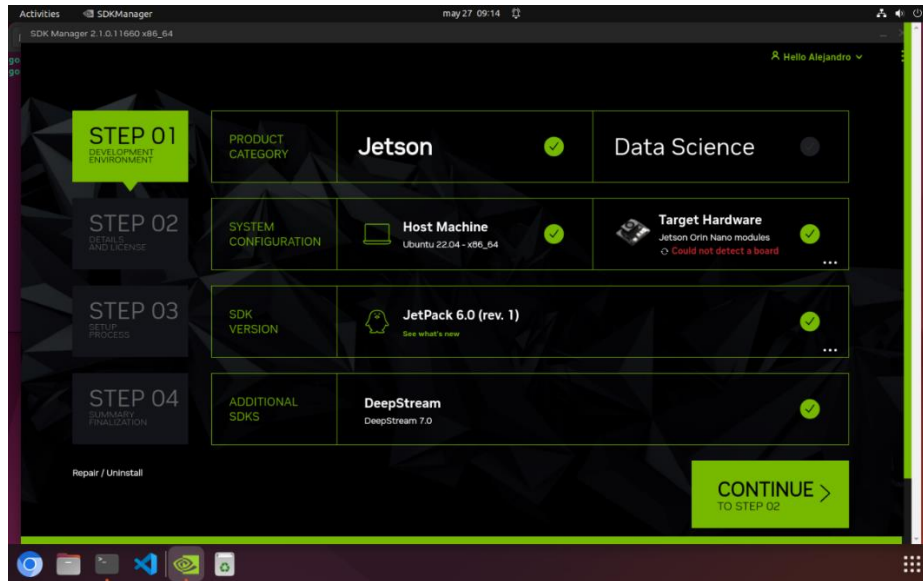


Figura 166. Paso 1 SDK Manager NVIDIA.

A continuación, en el paso 2 debemos marcar todas las opciones que nos ofrecen y crear dos carpetas donde guardaremos temporalmente algunos archivos (en caso de no crearla se nos crearán automáticamente, pero recomendamos seleccionar el directorio para tenerlo controlado).

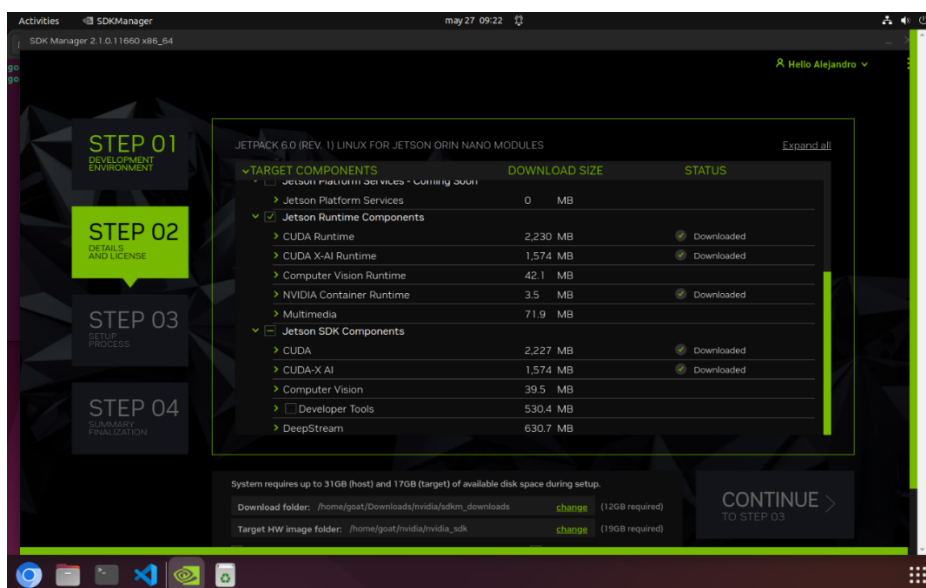


Figura 167. Paso 2 SDK Manager NVIDIA.

Una vez marcado todo marcamos haber leído la política de privacidad en el *checkbox* de abajo del todo y elegimos *Continue* para poder pasar el paso 3.

En este paso 3, al ser la primera vez que cargamos el software en la placa, debemos de disponer de un pequeño *jumper* o cable con el que poder unir dos pines de la Jetson. En primer lugar, debemos elegir la opción *Manual Setup – Jetson Ori Nano [8GB developer kit version]* en el desplegable de *Recovery mode setup*.

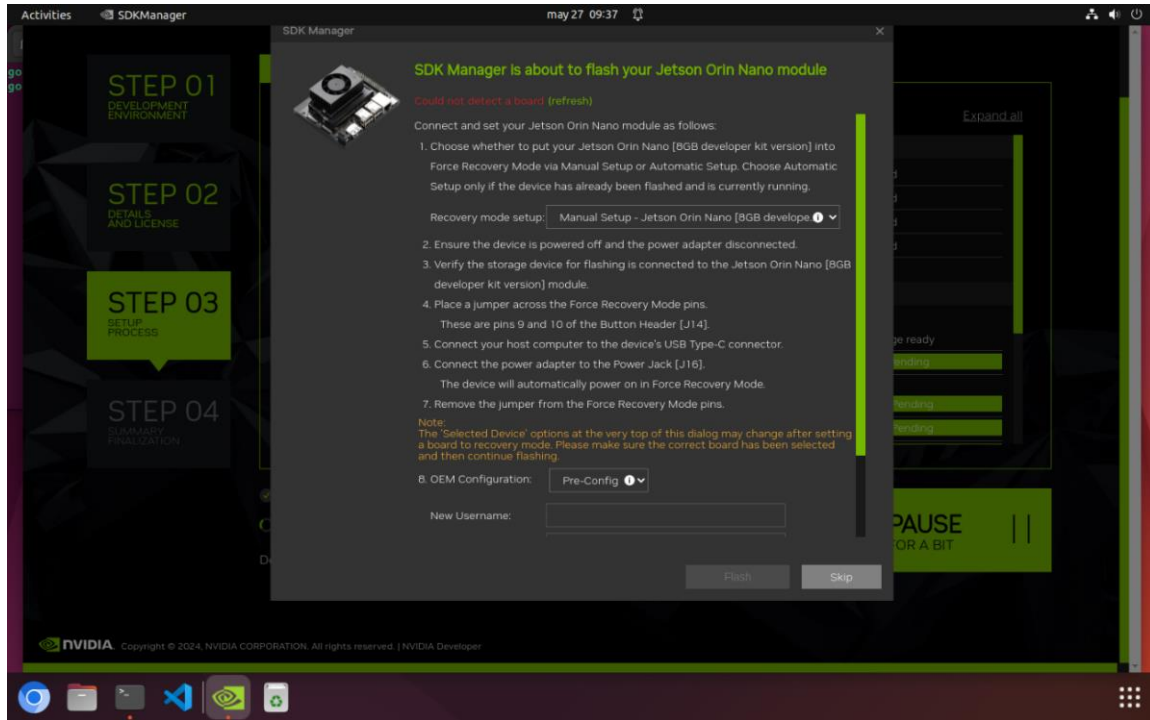


Figura 168. Paso 3 primera configuración SDK Manager NVIDIA.

Una vez elegido, debemos de colocar el jumper entre los pines 9 y 10 del lateral de la Jetson, es muy importante que sean esos pines y no los pines del conjunto de 40 pines de expansión. Para reconocerlos, son una serie de 12 pines situados debajo del ventilador en vertical, al lado de la microSD (Nota: la Jetson debe estar conectada al ordenador, pero no a la corriente). Debe quedarse de la siguiente manera:

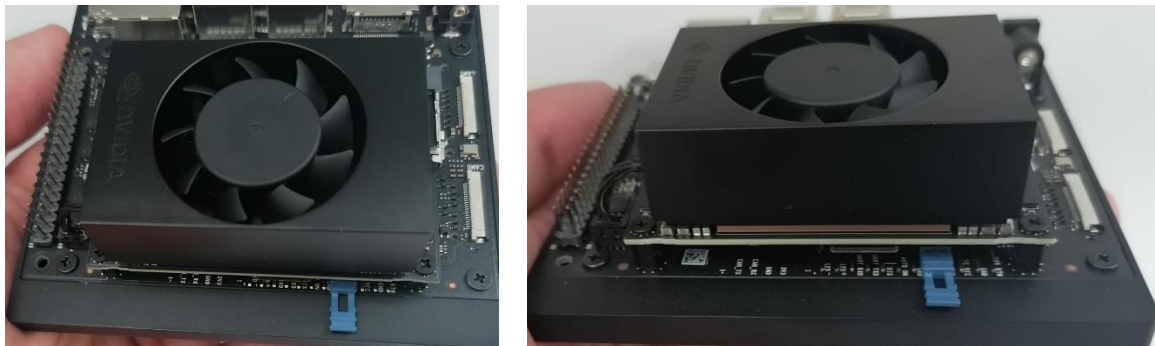


Figura 169. Vistas superior y lateral de la colocación del jumper en la Jetson.

A continuación, debemos elegir *Pre-config* en la opción *OEM Configuration* y elegir el usuario y la contraseña que nosotros queramos, tenga en cuenta que se emplearán más adelante en la Jetson, así que se recomienda apuntarla. Finalmente, con elegir *SD card* en la opción *Storage Device* y tener una microSD válida metida dentro de la Jetson podemos elegir *Flash* (Nota: la microSD se borrará por completo, se recomienda emplear una microSD vacía).

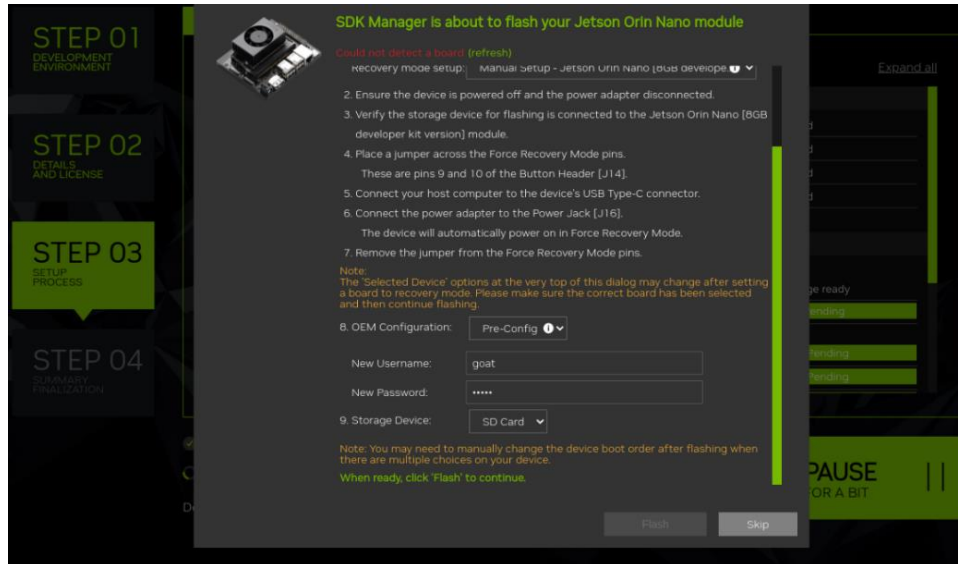


Figura 170. Paso 3 flasheo de la microSD con SDK Manager NVIDIA.

Cuando el proceso haya finalizado, podemos retirar el *jumper* de la placa y avanzar al siguiente paso, nos debería haber aparecido una ventana emergente donde deberemos rellenar los siguientes campos:

- Deberemos elegir *Ethernet* en la opción de *Connection*.
- En *IP Address* elegimos *IPv4* en el desplegable e introducimos la IP 192.168.11.1
- Introducimos el nombre de usuario y contraseña que elegimos en el paso previo.
- Dejamos la opción *Do not set proxy* por defecto.

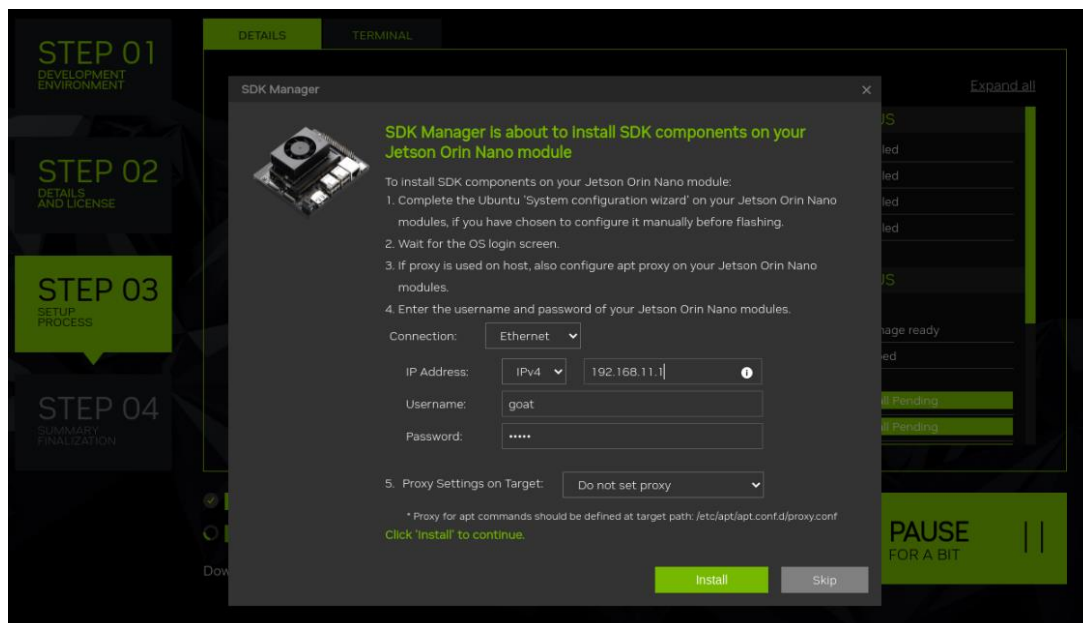


Figura 171. Paso 3 complementos de SDK Manager de NVIDIA.

Lo dejamos que termine y si todo ha salido exitosamente debería aparecernos lo siguiente:

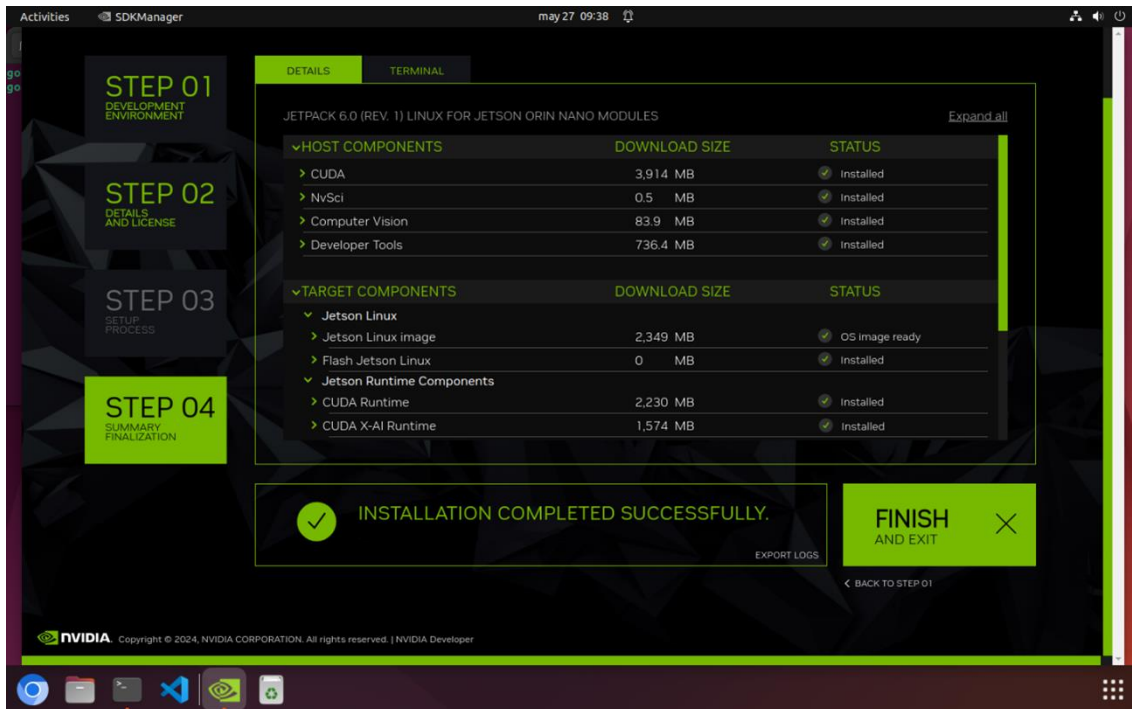


Figura 172. Paso 4 resumen de SDK Manager NVIDIA.

Con todo esto al conectar la Jetson a la corriente ya funcionará correctamente y podrá ver la pantalla del S.O con el logo de NVIDIA.

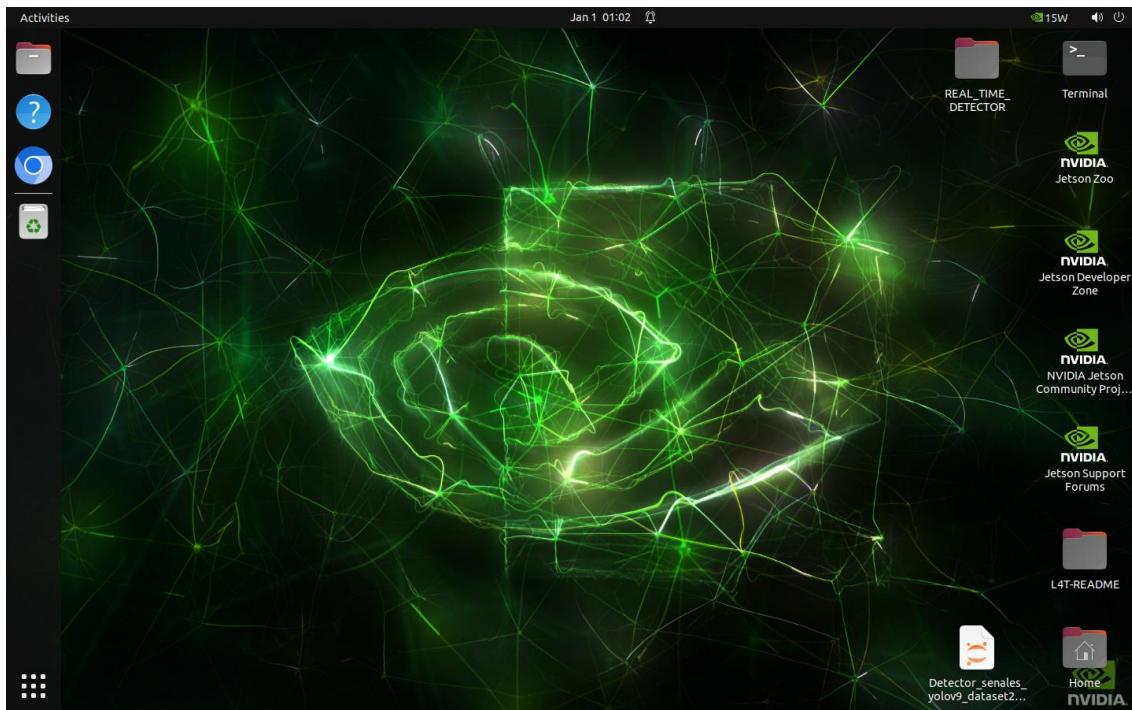


Figura 173. Escritorio de la Jetson Orin Nano.

ANEXO F-DIAGRAMAS GANTT

PLANIFICIACIÓN INICIAL

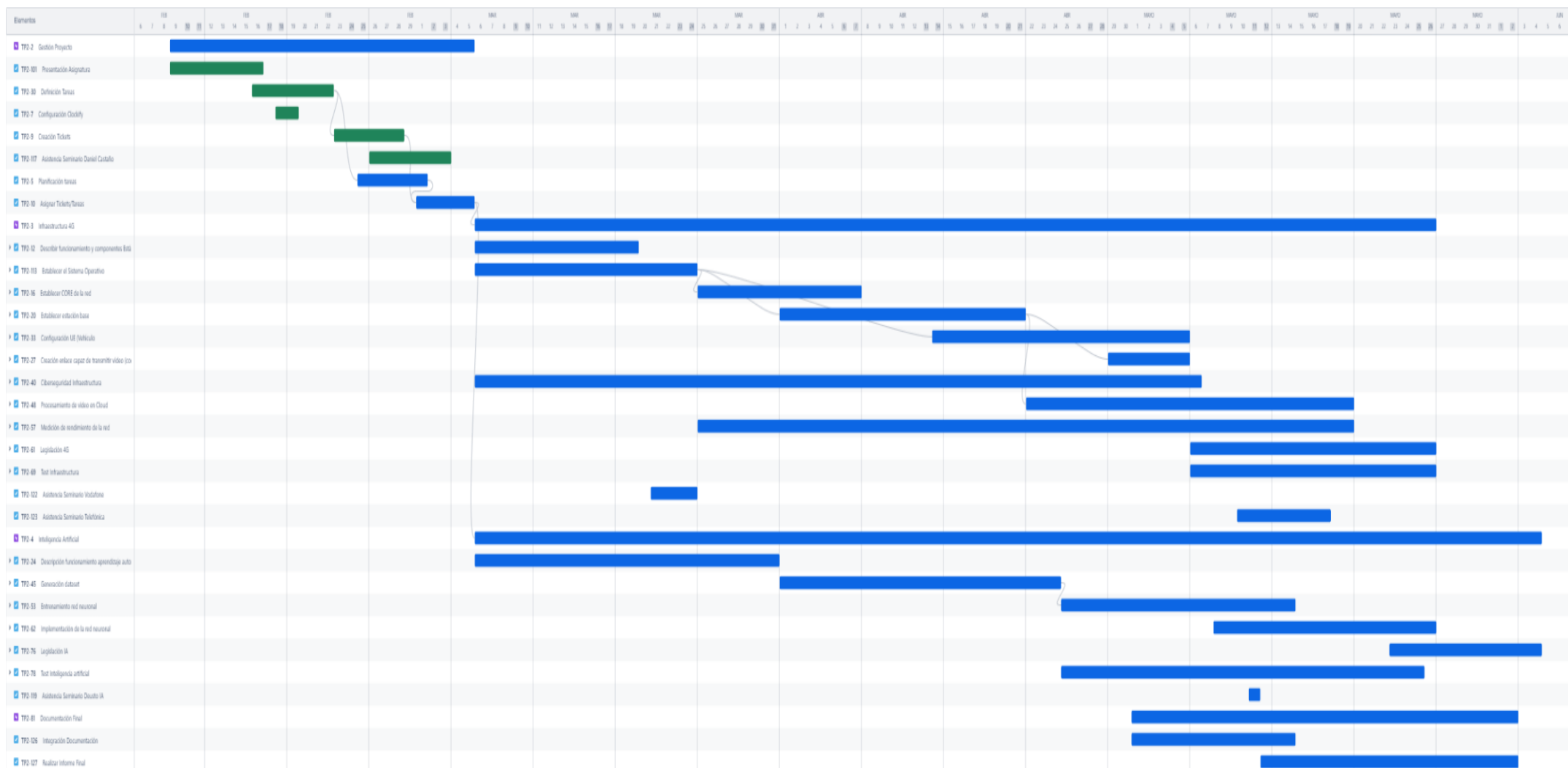


Figura 174. Diagrama Gantt con la planificación inicial.

PLANIFICIACIÓN FINAL

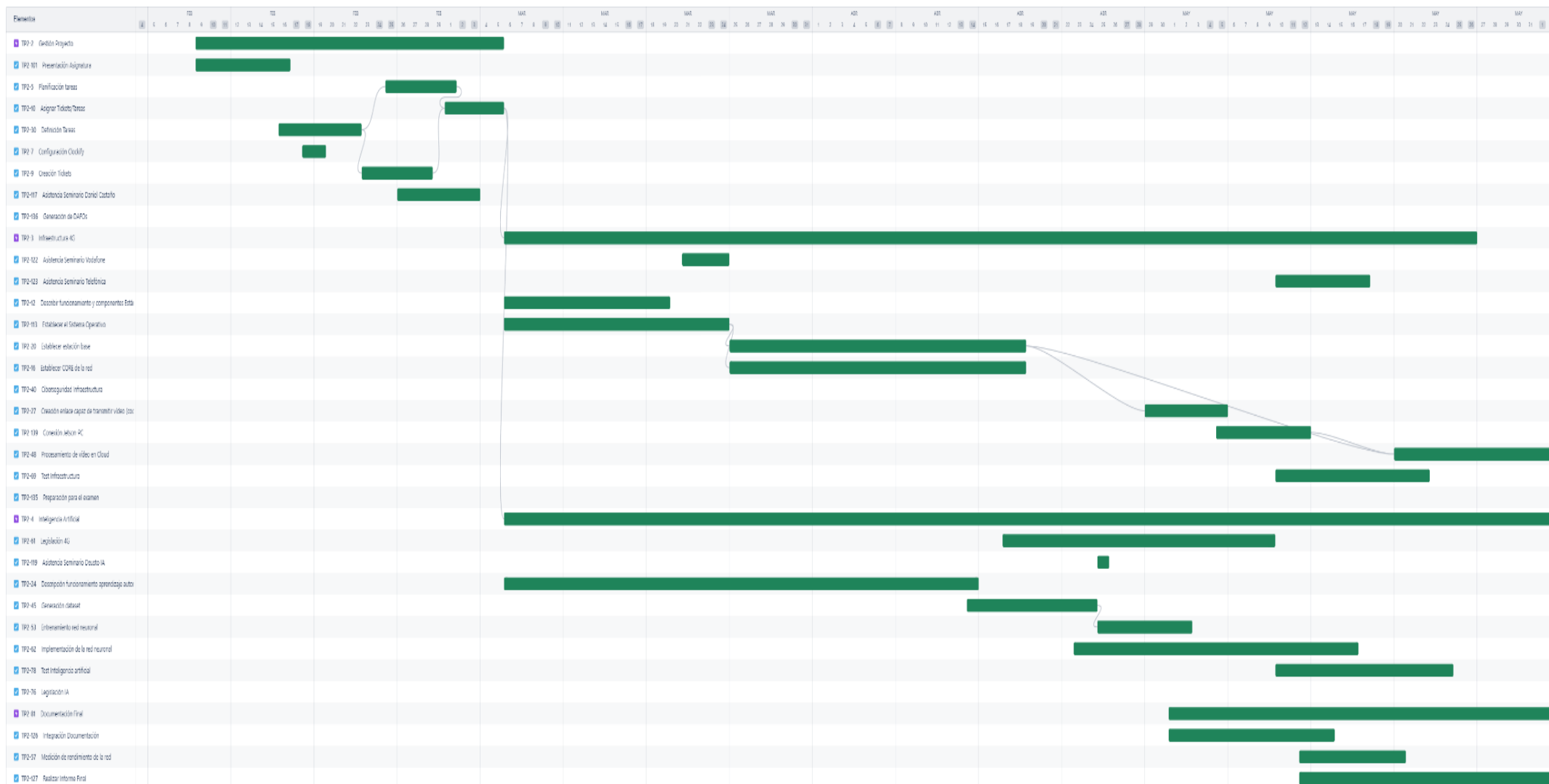


Figura 175. Diagrama Gantt con la planificación final.

ANEXO G-DOCUMENTOS LEGISLACIÓN



**REDES
RADIOELÉCTRICAS**

SOLICITUD DE NUEVAS REDES RADIOELÉCTRICAS DEL SERVICIO MÓVIL Y FIJO DE BANDA ESTRECHA

¿La persona física que presenta la solicitud es el titular?

- ☐ Sí
☒ No

La persona que presenta la solicitud acredita la representación del titular:

- ☒ Adjuntando poderes de representación
☐ Mediante inscripción previa en el Registro Electrónico de Apoderamientos (REA)

1.1 DATOS DE LA PERSONA QUE PRESENTA LA SOLICITUD

NIF/NIE *	Nombre *	Primer Apellido *	Segundo Apellido
71183109H	ALEJANDRO	MARTINEZ	DE ABAJO

Indique a continuación una dirección de correo electrónico donde se remitirá un aviso informándole de la puesta a disposición en la sede electrónica de las notificaciones relacionadas con esta solicitud. Este aviso no constituirá en ningún caso la notificación, que en todo caso deberá producirse mediante comparecencia en la sede.

thegoatprojects@gmail.com	Correo electrónico *
thegoatprojects@gmail.com	Confirmar Correo electrónico *

1.2 DATOS DEL TITULAR

CIF *	Razón social *				
A-38100014	The Goat Projects S.A.				
Tipo vía *	Nombre vía *	Número *	Escalera	Piso	Puerta
Camino	Buen Pastor	7			
Provincia *	Municipio *				
Valladolid	Valladolid				
Localidad *	Código postal *	Fax *	Teléfono		
Valladolid	470014				

Haga clic en el botón **EXAMINAR** para seleccionar el fichero (.pdf o .xsig) que contiene **los poderes de representación otorgados por el titular** y después haga clic en el botón **ANEXAR**. Tamaño máximo permitido 1MB. Para el nombre del fichero solo se permitirán caracteres del alfabeto tradicional.

NINGÚN ARCHIVO SELECCIONADO
  Anexar

Haga clic en el botón **EXAMINAR** para seleccionar, en su caso, el fichero (.pdf o .xsig) que contiene **otras autorizaciones al presentador por parte del titular o su representante** y después haga clic en el botón **ANEXAR**. Tamaño máximo permitido 1MB. Para el nombre del fichero solo se permitirán caracteres del alfabeto tradicional.

NINGÚN ARCHIVO SELECCIONADO
  Anexar

2. DATOS DE LA SOLICITUD

Tipo de título *

AFECTACIÓN "ADMINISTRACIÓN AUTONÓMICA O LOCAL" ▼

Haga clic en el botón **EXAMINAR** para seleccionar el fichero (.xsig) **que contiene las características técnicas de la estación o de la red radioeléctrica (fichero XML firmado electrónicamente)**, y después haga clic en el botón **ANEXAR**. Tamaño máximo permitido 8MB. Para el nombre del fichero solo se permitirán caracteres del alfabeto tradicional.

 NINGÚN ARCHIVO SELECCIONADO


Uso de la red y Datos adicionales *

En caso de afectaciones que cumplan los requisitos exigibles para acceder a la exención de la tasa por reserva del dominio público radioeléctrico establecidos en el Anexo I, apartado 3.7 de la Ley 9/2014, de 9 de mayo, de Telecomunicaciones, desarrollada por el artículo 29 del Real Decreto 123/2017, de 24 de febrero, por el que se aprueba el Reglamento sobre el uso del dominio público radioeléctrico, se podrá solicitar dicha exención cumplimentando el impreso al efecto que está disponible en la página web del Ministerio para la Transformación Digital y de la Función Pública, adjuntando, en su caso, la documentación acreditativa del cumplimiento de dichos requisitos.

¿Solicitar exención del pago de la tasa por Reserva del Dominio Público Radioeléctrico?

☒ No ☐ Si

Recepción de las notificaciones relativas a la solicitud

De acuerdo con lo establecido en el artículo 43.1 de la Ley 39/2015, de 1 de octubre, del procedimiento administrativo Común de las Administraciones Públicas, las notificaciones correspondientes a esta solicitud se realizarán por medios electrónicos mediante comparecencia del interesado o su representante debidamente identificado en la sede electrónica del Ministerio para la Transformación Digital y de la Función Pública.

IR A FIRMAR >>

Gobierno de España. Ministerio para la Transformación Digital y de la Función Pública.
Dirección General de Telecomunicaciones y Ordenación de los Servicios de Comunicación Audiovisual (E04796207)
P. de la Castellana 162, C.P. 28046 Madrid, España





MINISTERIO
DE ASUNTOS ECONÓMICOS
Y TRANSFORMACIÓN DIGITAL

SECRETARÍA DE ESTADO DE TELECOMUNICACIONES
E INFRAESTRUCTURAS DIGITALES

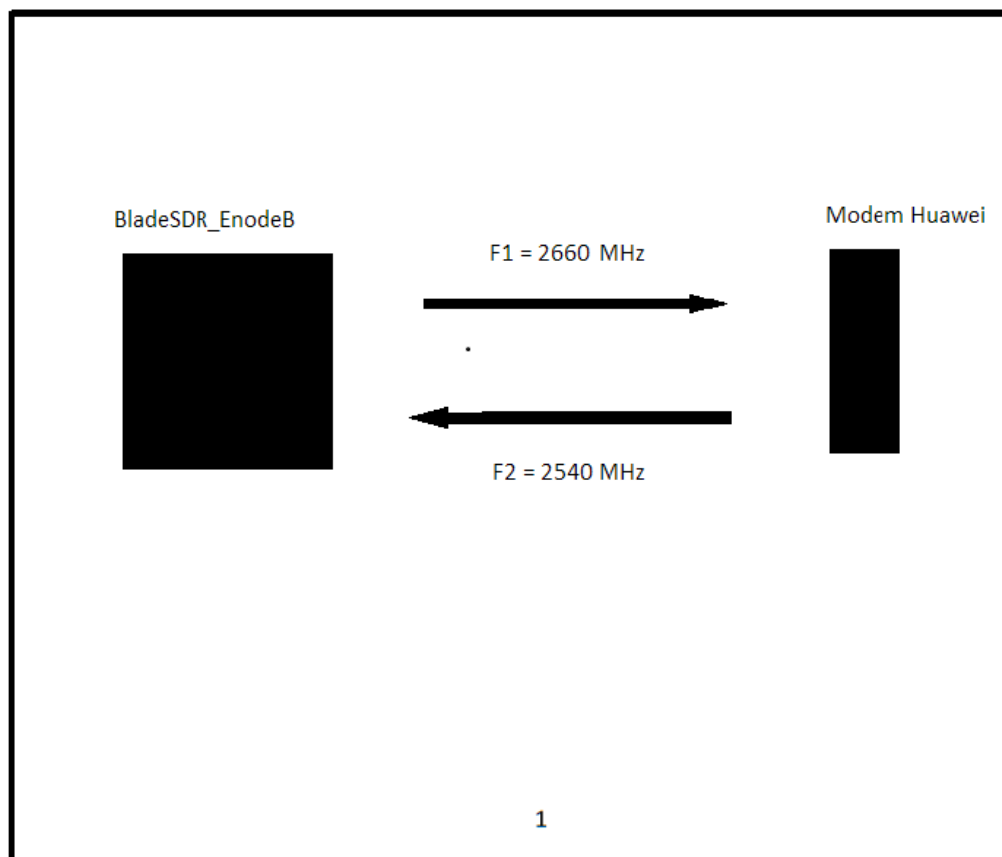
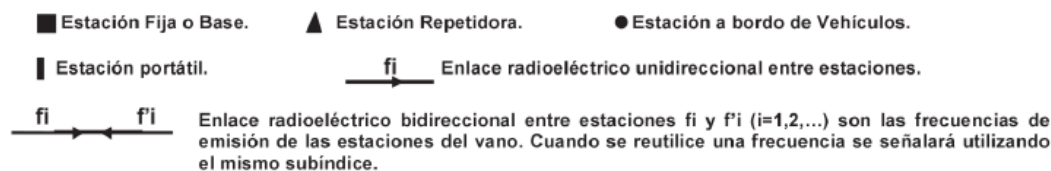
DIRECCIÓN GENERAL DE TELECOMUNICACIONES
Y ORDENACIÓN DE LOS SERVICIOS DE
COMUNICACIÓN AUDIOVISUAL

SUBDIRECCIÓN GENERAL DE PLANIFICACIÓN
Y GESTIÓN DEL ESPECTRO RADIOELÉCTRICO

DATOS ADICIONALES NO ESTRUCTURADOS DEL PROYECTO PARA SOLICITUDES DE TÍTULO HABILITANTE PARA EL USO DEL DOMINIO PÚBLICO RADIOELÉCTRICO. SERVICIO MÓVIL Y FIJO DE BANDA ESTRECHA

Dibújese el esquema completo de la estación radioeléctrica o de la red de estaciones para la que se realiza la solicitud. Deberán designarse a las estaciones con los mismos nombres que se utilizan para las mismas en el fichero XML descriptivo de dicha red radioeléctrica. Si se trata de una modificación de una red ya existente, indíquese claramente cuál es la parte de la red que se amplía o modifica.

En ese esquema se deberán indicar las diferentes estaciones y los enlaces entre las estaciones utilizando los símbolos siguientes.





MINISTERIO
DE ASUNTOS ECONÓMICOS
Y TRANSFORMACIÓN DIGITAL

SECRETARÍA DE ESTADO DE TELECOMUNICACIONES
E INFRAESTRUCTURAS DIGITALES

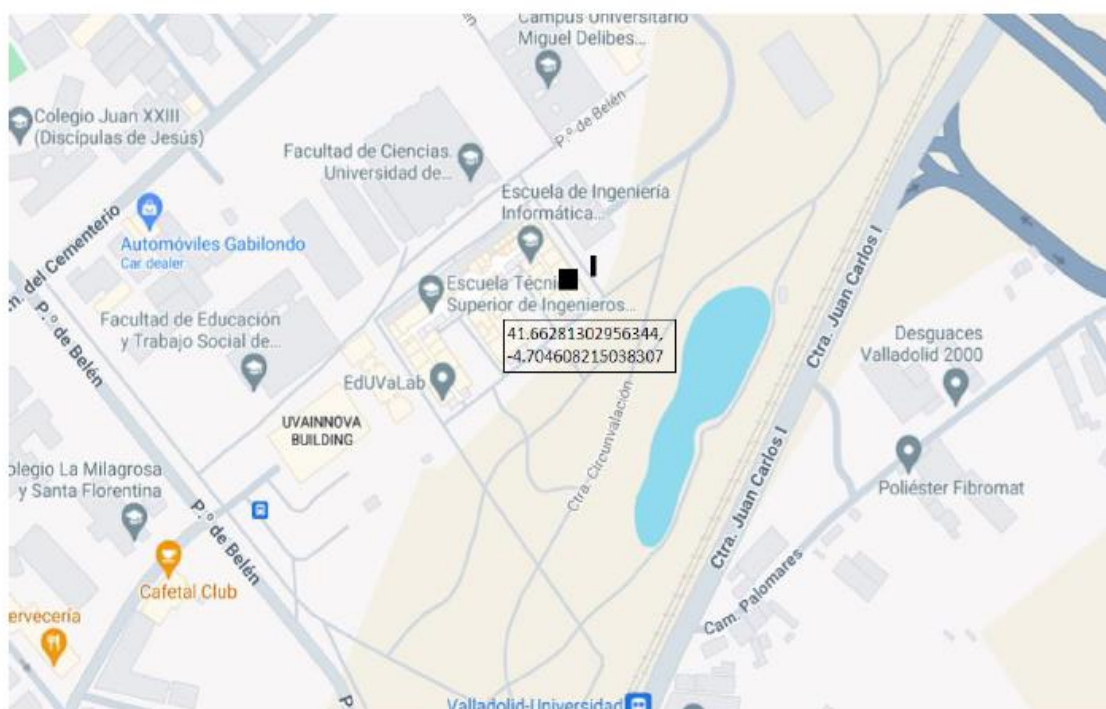
DIRECCIÓN GENERAL DE TELECOMUNICACIONES
Y ORDENACIÓN DE LOS SERVICIOS DE
COMUNICACIÓN AUDIOVISUAL

SUBDIRECCIÓN GENERAL DE PLANIFICACIÓN
Y GESTIÓN DEL ESPECTRO RADIOELÉCTRICO

En hoja u hojas aparte adjúntese el plano topográfico a escala adecuada en el que se señalen los emplazamientos de las estaciones fijas de las que conste la red, con indicación marginal de las coordenadas geográficas de las estaciones.

Adjúntese también en hoja u hojas aparte cualquier tipo de información que se considere necesaria para la descripción técnica de la red radioeléctrica (fotos, imágenes, croquis, etc.).

Describa brevemente a continuación la finalidad de la red radioeléctrica para la que se realiza la solicitud, indicando la localización de la red, tipo de transmisión (voz, datos, TV, etc.), bandas de frecuencia que se solicitan y posibles frecuencias alternativas a las solicitadas según el rango de funcionamiento de los equipos.





MINISTERIO
DE ASUNTOS ECONÓMICOS
Y TRANSFORMACIÓN DIGITAL

SECRETARÍA DE ESTADO DE TELECOMUNICACIONES
E INFRAESTRUCTURAS DIGITALES

DIRECCIÓN GENERAL DE TELECOMUNICACIONES
Y ORDENACIÓN DE LOS SERVICIOS DE
COMUNICACIÓN AUDIOVISUAL

SUBDIRECCIÓN GENERAL DE PLANIFICACIÓN
Y GESTIÓN DEL ESPECTRO RADIOELÉCTRICO

La experimentación con vehículos conectados e infraestructura 4G LTE implica la integración de tecnología de comunicación móvil de alta velocidad en automóviles para mejorar la seguridad y la eficiencia del transporte. Esta combinación permite la transmisión de datos entre vehículos y la infraestructura vial. Por ello, en el máster de la Universidad de Valladolid se realiza un ejercicio en pequeña escala con fin de experimentación. Tratándose de una infraestructura de red 4G LTE en las frecuencias 2660Mhz y 2540MHz, bajada y subida respectivamente alquiladas a la empresa Vodafone donde el tipo de transmisión requerida es la transmisión de datos de video.

Los dispositivos que se van a emplear para el experimento demostrador es la siguiente:



Ordenador



Blade RF 2.0 micro xA9



Modem



Deep Racer AWS

(Firma electrónica)





MINISTERIO
DE ASUNTOS ECONÓMICOS
Y TRANSFORMACIÓN DIGITAL

SECRETARÍA DE ESTADO DE TELECOMUNICACIONES
E INFRAESTRUCTURAS DIGITALES

DIRECCIÓN GENERAL DE TELECOMUNICACIONES
Y ORDENACIÓN DE LOS SERVICIOS DE
COMUNICACIÓN AUDIOVISUAL

SUBDIRECCIÓN GENERAL DE PLANIFICACIÓN
Y GESTIÓN DEL ESPECTRO RADIOELÉCTRICO

**DECLARACIÓN RESPONSABLE EN RELACIÓN CON EL PROYECTO TÉCNICO O
MEMORIA TÉCNICA ADJUNTO A LA SOLICITUD DE TÍTULO HABILITANTE PARA USO
DEL ESPECTRO RADIOELÉCTRICO**

D./Dña TheGOATProyects con NIF/NIE 12345678H, domiciliado en Camino del Buen Pastor, 7 C.P. 47014, población Valladolid, teléfono de contacto 123456789 y dirección de correo electrónico thegoatprojects@gmail.com

A) CASO DE QUE EL PROYECTO TÉCNICO DEBA SER FIRMADO OBLIGATORIAMENTE POR UN TÉCNICO COMPETENTE EN MATERIA DE TELECOMUNICACIONES, DECLARA

1. Que posee el título universitario de.....

.....otorgado por la Universidad de.....

2. Que cumple los requisitos establecidos en la normativa legal sobre el ejercicio de la profesión y que no existe ningún impedimento para la firma del proyecto/memoria técnica adjunto a la presente solicitud.

B) CASO DE QUE LA MEMORIA TÉCNICA NO REQUIERA LA FIRMA DE UN TÉCNICO COMPETENTE EN MATERIA DE TELECOMUNICACIONES, DECLARA

Que teniendo en cuenta que se trata de una solicitud de autorización de una red radioeléctrica para fines experimentales/evento de corta duración y por lo tanto, la memoria técnica de la misma no precisa ser firmada por un técnico competente en materia de telecomunicaciones, el fichero XML descriptivo de la red radioeléctrica lo firma el titular de la red o su representante legal.

(Firma electrónica)





C.I.F. 52801502B

CENTRO GESTOR
Secretaría de Estado de Telecomunicaciones
e Infraestructuras Digitales

TASA de telecomunicaciones

Modelo

CÓDIGO **041****790**

Identificación

Espacio reservado para la etiqueta del sujeto pasivo. Si no dispone de etiquetas, consigne los datos que se solicitan en las líneas inferiores.		Devengo		Ejercicio	
		Nº DE JUSTIFICANTE 1289-xxxx-21-aa			
N.I.F. o C.I.F.		Nombre y apellidos o razón social			
71183109H		ALEJANDRO MARTINEZ DE ABAJO			
Calle/ Piza/ Avda.	Nombre de la vía pública	Número	Esc.	Piso	Puerta
Camino✓	Buen Pastor	7			
Municipio		Provincia			Cod. Postal
Valladolid		[SELECCIONE UNA PROVINCIA] ✓			

Autoliquidación

EXPEDIENTE:		EUROS	
TASA DE TELECOMUNICACIONES.- Ley 11/ 2022, de 28 de junio, General de Telecomunicaciones, Disposición transitoria séptima			
Cant.			
1	☐ Por la expedición de certificaciones registrales	1	43.80
2	☐ Por la expedición de certificaciones de presentación a la administración de las telecomunicaciones del proyecto técnico de infraestructuras comunes de telecomunicaciones, el acta de replanteo, el boletín de instalación y el protocolo de pruebas y, en su caso, el certificado de fin de obra y sus anexos	2	43.80
3	☐ Por la expedición de certificaciones de cumplimiento de especificaciones técnicas de equipos de telecomunicación	3	345.65
4	☐ Por cada acto de inspección previa o comprobación técnica efectuado	4	363.42
5	☐ Por la presentación de cada certificación expedida por técnico competente sustitutiva del acto de inspección previa	5	90.67
6	☒ 1 Por la tramitación de concesión demanial o autorización para el uso privativo o de autorización general para el uso especial del dominio público radioeléctrico	6	70.53
7	☐ Por la tramitación de la autorización individual para el uso especial del dominio público radioeléctrico	7	114.36
8	☐ Por la presentación a los exámenes de capacitación para operar estaciones de radioaficionado	8	23.67
9	☐ Por inscripción en el registro de empresas instaladoras de telecomunicación	9	107.72
10	☐ Por la solicitud y emisión del dictamen técnico de evaluación de la conformidad de equipos de telecomunicación	10	356.30
11	☐ REGULARIZACION del importe por cambio del importe de la tasa en la Ley de Presupuestos Generales del Estado	11	
CUOTA TOTAL TRIBUTARIA A INGRESAR EN EUROS		70.53	0,00

DECLARANTE

Jueves 09 de mayo de 2024

Firma:

THE GOAT PROJECTS

INGRESO

Ingreso a favor del Tesoro Público, cuenta restringida de la AEAT para la recaudación de TASAS

Importe Euros | 70,53 0,00

Código IBAN

Cod. País	D.C. IBAN	Entidad	Oficina	D.C.	Núm. de cuenta
ES	ES00	9999	8888	aa	xxxxxxxxxx

Enviar

Cancelar